



TEAMCENTER

Workflow Handlers

Software Version 2606

This Documentation contains trade secrets or otherwise confidential information owned by Siemens Industry Software Inc. or its affiliates (collectively, "Siemens"), or its licensors. Access to and use of this Documentation is strictly limited as set forth in Customer's applicable agreement(s) with Siemens. This Documentation may not be copied, distributed, or otherwise disclosed by Customer without the express written permission of Siemens, and may not be used in any way not expressly authorized by Siemens.

This Documentation is for information and instruction purposes. Siemens reserves the right to make changes in specifications and other information contained in this Documentation without prior notice, and the reader should, in all cases, consult Siemens to determine whether any changes have been made.

No representation or other affirmation of fact contained in this publication shall be deemed to be a warranty or give rise to any liability of Siemens whatsoever.

If you have a signed license agreement with Siemens for the product with which this Documentation will be used, your use of this Documentation is subject to the scope of license and the software protection and security provisions of that agreement. If you do not have such a signed license agreement, your use is subject to the Siemens Universal Customer Agreement, which may be viewed at <https://www.sw.siemens.com/en-US/sw-terms/base/uca/>, as supplemented by the product specific terms which may be viewed at <https://www.sw.siemens.com/en-US/sw-terms/supplements/>.

SIEMENS MAKES NO WARRANTY OF ANY KIND WITH REGARD TO THIS DOCUMENTATION INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT OF INTELLECTUAL PROPERTY. SIEMENS SHALL NOT BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, CONSEQUENTIAL OR PUNITIVE DAMAGES, LOST DATA OR PROFITS, EVEN IF SUCH DAMAGES WERE FORESEEABLE, ARISING OUT OF OR RELATED TO THIS DOCUMENTATION OR THE INFORMATION CONTAINED IN IT, EVEN IF SIEMENS HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

TRADEMARKS: The trademarks, logos, and service marks (collectively, "Marks") used herein are the property of Siemens or other parties. No one is permitted to use these Marks without the prior written consent of Siemens or the owner of the Marks, as applicable. The use herein of third party Marks is not an attempt to indicate Siemens as a source of a product, but is intended to indicate a product from, or associated with, a particular third party. A list of Siemens' Marks may be viewed at: <https://www.sw.siemens.com/en-US/trademarks/>. The registered trademark Linux® is used pursuant to a sublicense from LMI, the exclusive licensee of Linus Torvalds, owner of the mark on a world-wide basis.

About Siemens Digital Industries Software

Siemens Digital Industries Software is a global leader in the growing field of product lifecycle management (PLM), manufacturing operations management (MOM), and electronic design automation (EDA) software, hardware, and services. Siemens works with more than 100,000 customers, leading the digitalization of their planning and manufacturing processes. At Siemens Digital Industries Software, we blur the boundaries between industry domains by integrating the virtual and physical, hardware and software, design and manufacturing worlds. With the rapid pace of innovation, digitalization is no longer tomorrow's idea. We take what the future promises tomorrow and make it real for our customers today. Where today meets tomorrow. Our culture encourages creativity, welcomes fresh thinking and focuses on growth, so our people, our business, and our customers can achieve their full potential.

Support Center

Log in to Support Center to access software downloads and updates, product and release documentation, Knowledge Base articles, videos, other self-service resources, and to open support cases: <https://support.sw.siemens.com>.

If you do not have a Support Center login, register here: <https://support.sw.siemens.com/register>.

To send feedback on documentation: https://support.sw.siemens.com/doc_feedback_form.

Table of Contents

Workflow handlers.....	1
What are workflow handlers?.....	1
Executing workflow handlers.....	2
Updating your task templates to use the new handler and argument names.....	4
Renaming your custom handlers and arguments.....	4
Handler argument values.....	11
Syntax for handler arguments and values.....	11
Keywords as argument values.....	11
Lists of values as argument values.....	21
Differentiating between classes and types.....	23
Specifying relations.....	24
Debugging handler data.....	26
Action handlers.....	27
Action Handlers.....	27
AI-export-AH.....	28
AI-process-import.....	31
AI-process-export.....	32
AMX-auto-assign-task-assignees-from-assignment-matrix.....	34
AR-mark-archive.....	44
ASBUILT-attach-physical-components	46
ASBUILT-release-asbuilt-structure.....	48
ASMAINTAINED-attach-physical-components	50
ASMAINTAINED-release-asmaintained-structure.....	52
BC-perform-export.....	54
CAE-attach-related-cae-folder-objects.....	56
CAE-decode-token-and-update-model-attributes.....	58
CAE-mark-up-to-date.....	59
CAE-simulation-process-launch-handler.....	60
CAE-structuremap-execution-handler.....	63
CFG0-attach-allocations.....	65
CFG0-attach-constraint-rules.....	67
CFG0-attach-families.....	70
CFG0-attach-familygroups.....	72
CFG0-attach-rule-variability.....	75
CFG0-find-constraint-conflict.....	78
CM-Approve-ECO-Markup-Handler.....	80
CM-baseline-solution-item-revisions-on-change-notice.....	82
CM-cancel-change-notice-handler.....	83
CM-Cancel-ECO-Markup-Handler.....	84
CM-concurrent-notification.....	84
CM-create-multisite-change-notice-objects.....	86
CM-derive-change.....	88
CM-inactivate-edit-context.....	90
CM-merge-changes.....	91

CM-promote-change-notice.....	97
CM-propagate-change-notice-effectivity.....	102
CM-show-concurrent-warning-on-task.....	103
CM-transfer-multisite-parent-structure.....	104
CM-validate-ECO-Markup-Handler.....	105
CONFMGMT-cut-back-effectivity.....	106
CONTMGMTS1000D-increment.....	111
CONTMGMTS1000D-setQAStatus.....	112
CPD-collect-related-items.....	113
CPD-update-item-realization.....	115
CPD-where-used-item-revision.....	116
CPDWORKSET_collect_impacted_worksets_handler.....	117
CSI-propagate-folder-contents.....	119
DOCMGTAPP-apply-pdf-control.....	121
DOCMGTAPP-insert-pdf-cover-page.....	123
DOCMGT-render-document-revision.....	125
DOCMGT-update-docprop-logicalobject.....	127
DOCMGT-update-document-property.....	129
EPM-adhoc-signoffs.....	131
EPM-apply-digital-signature.....	147
EPM-assign-responsible-party-dynamic-participant.....	148
EPM-assign-signoff-dynamic-participant.....	156
EPM-assign-team-selector.....	164
EPM-attach-related-objects.....	170
EPM-auto-assign.....	181
EPM-auto-assign-rest.....	187
EPM-auto-check-in-out.....	193
EPM-change-all-started-to-pending.....	195
EPM-change-group-owner.....	196
EPM-change-ownership.....	197
EPM-change-target-group.....	199
EPM-change-target-group-owner.....	200
EPM-check-signoff-comments.....	202
EPM-create-form.....	204
EPM-create-relation.....	207
EPM-create-status.....	209
EPM-create-sub-process.....	210
EPM-debug.....	225
EPM-demote.....	226
EPM-demote-on-reject.....	227
EPM-display-form.....	229
EPM-end-bypass.....	232
EPM-execute-follow-up.....	234
EPM-fill-in-reviewers.....	236
EPM-inherit.....	244
EPM-invoke-system-action.....	246
EPM-late-notification.....	253
EPM-move-attached-objects.....	258
EPM-notify.....	260

EPM-notify-report.....	270
EPM-notify-signoffs.....	281
EPM-remove-objects.....	284
EPM-request-PKI-authentication.....	289
EPM-require-authentication.....	290
EPM-run-external-command.....	291
EPM-set-condition.....	305
EPM-set-duration.....	309
ERP-set-form-value-AH.....	311
EPM-set-job-protection.....	312
EPM-set-owning-project-to-task.....	313
EPM-set-parent-result.....	314
EPM-set-property.....	315
EPM-set-rule-based-protection.....	327
EPM-set-status.....	330
EPM-set-task-result-to-property.....	335
EPM-start-bypass.....	338
EPM-suspend-on-reject.....	340
EPM-system.....	341
EPM-trigger-action.....	342
EPM-trigger-action-on-related-process-task.....	344
ERP-att-logfile-as-dataset-RH.....	349
ERP-attach-targets-AH.....	350
ERP-delete-log-dataset-AH.....	351
ERP-download-AH.....	352
ERP-post-upload-AH.....	353
ERP-set-pathnames-in-logds-AH.....	355
ERP-transform-AI-contents-AH.....	356
FSO-release-haraanalysis-objects.....	357
GMIMAN-invoke-subscription-event-on-item.....	359
HRN-revise.....	360
HRN-set-reject-state.....	365
HRN-set-release-state.....	368
IM1-update-session-for-machine-bom.....	371
IMD0-attach-basic-di-to-target.....	372
IMD0-attach-itemsku-to-target.....	373
IMD0-attach-parents-to-target.....	374
IMD0-check-di-prop-release.....	375
IMD0-export-to-tcxmfile.....	377
IMD0-validate-eudi-record-prop.....	379
Imp0LabelDesignApprove.....	381
Imp0SendToTcif.....	382
IPM-Add-Reviewers.....	383
IPM-attach-objects-from-query.....	383
IPM-get-all-related-objects.....	384
IPM-Get-Event-To-Process.....	386
IPM-send-alert.....	386
IPM-set-completeness.....	387
IPM-set-conditional-status-false.....	388

IPM-set-conditional-status-true.....	388
IPM-set-due-date.....	389
IPM-Update-Event.....	390
IPM-Update-Maturity.....	390
ISSUEMGT-check-review-decision.....	392
ISSUEMGT-update-issue-status.....	394
LDF-create-object.....	396
LDF-set-task-result-to-property.....	400
SUBSCMPL-List-Vendor-parts-and-vendors.....	403
MDL-attach-changes-to-baselines.....	404
MDL-attach-subset-definition-changes.....	405
MDL-promote-objects-to-history.....	406
MDL-snapshot-baseline-revisions.....	407
MDT-attach-related-assignment-matrix.....	409
MDT-attach-related-deliverables.....	410
MDT-process-content-load-request	411
MDT-set-release-status-on-deliverable-instances	412
MDT-set-transmittal-sender-property.....	414
MDT-update-progress-from-roc.....	415
ME-create-mirror-mbom-AH.....	416
ME-create-revision-change-XML-AH.....	421
ME-mbom-resolve-AH.....	422
ME-stamp-ids-AH.....	426
ME-update-mirror-mbom-AH.....	428
MES-Update3DPDFReports.....	431
MultiSite-offline-export.....	432
MultiSite-offline-import.....	434
OBJIO-restore-target-objects.....	435
OBJIO-archive-target-objects.....	436
OBJIO-release-and-replicate.....	437
OBJIO-send-target-objects.....	439
PARTITION-activate-or-inactivate.....	441
SUBSCMPL-Perform-Compliance-Check.....	442
PIE-export-to-plmxmlfile.....	443
PMAO_generate_dbom_handler.....	445
PMAO_generate_part_ebom_handler.....	446
PMAO_generate_product_ebom_handler.....	447
PMAO_update_dbom_handler.....	449
PMAO_update_ebom_handler.....	450
PMAO_align_dbom_and_ebom_handler.....	451
PMAO_align_design_revs_and_part_revs_in_context_handler.....	452
SUBSCMPL-Populate-Material-Substance-Declaration-Form.....	453
PROJ-assign-members.....	454
PROJ-update-assigned-projects.....	458
PS-attach-assembly-components.....	461
PS-make-mature-design-primary.....	466
PS-occ-effectivity-cutback.....	467
PS_attach_occurrence_revisions.....	468
PUBR-publish-target-objects.....	469

PUBR-unpublish-target-objects.....	470
QC-Release-Checklist-Specification.....	471
QFM0FMAREALIZATION-release-fmea-structure.....	472
RDV-delete-ugcgm-markup.....	473
RDV-generate-image.....	474
RDV-generate-ugcgm-drawing.....	475
RDV-tessellation-handler.....	477
SUBSCMPL-Review-Substance-Declaration.....	480
RM-attach-SM-tracelink-requirement.....	481
RM-attach-tracelink-requirement.....	483
SAP-set-valid-date-AH.....	485
SAP-upload-AH.....	486
SCHMGT-approve-timesheetentries.....	487
SCHMGT-reject-timesheetentries.....	488
SCHMGT-revise-timesheetentries.....	489
SCHMGT-sync-schedule-attachments.....	490
SERVICEFORECASTING-approve-ma-extension.....	491
SERVICEPROCESSING-approve-service-structure.....	492
SFD0-add-mpwshuttle-objects-as-targets.....	493
SFD0-check-mpwmaskset-attached.....	494
SMCO-create-solution-variants.....	495
SMCO-create-solution-variants2.....	497
SMCO-update-solution-variants.....	499
SMCO-update-solution-variants2.....	500
SMP-auto-relocate-file.....	502
TCRS-auto-approve-first-step.....	503
TCRS-create-form.....	504
TCRS-Create-Print-Requests.....	506
TCRS-create-snapshot.....	508
TCRS-Create-Translation-Request.....	509
TCRS-delete-dataset.....	511
TCRS-delete-log-datasets.....	513
TCRS-export-signoff-data.....	515
TCRS-export-to-tcxmfile.....	518
TCRS-IRM-cleanfields.....	520
TCRS-purge-dataset.....	521
TCRS-release-previous-itemrevs.....	522
TCRS-remove-targets-with-status.....	525
TCRS-set-bom-precise.....	526
TCRS-setstatus-EngOrder-folder.....	527
TCRS-store-review-data.....	529
TCRS-generate-pdf.....	532
TCRS-bom-plmxml-export.....	535
TCRS-trigger-approve-first-step.....	537
TSTK-CreateTranslationRequest.....	538
SUBSCMPL-Update_Compliance_Result.....	539
VAL-approve-result-overrides.....	540
VAL-reject-result-overrides.....	541
VAL-set-condition-by-check-validation-result.....	542

VAL-set-condition-result-overrides.....	544
Rule handlers.....	544
Rule Handlers	544
ASBUILT-validate-for-checkedout-physicalpartrevision.....	545
ASBUILT-validate-for-physicalpartrevision.....	546
ASBUILT-validate-missing-structure.....	547
ASMAINTAINED-validate-for-checkedout-physicalpartrevision.....	548
ASMAINTAINED-validate-for-latest-asmphysicalpartrevision.....	549
ASMAINTAINED-validate-for-unserviceable-physicalpartrevision.....	550
ASMAINTAINED-validate-missing-asmaintained-structure.....	551
AUTOSCHEDULING-person-reassign-validate.....	552
CM-validate-concurrent-change-exists.....	553
EPM-assert-signoffs-target-read-access.....	554
EPM-assert-targets-checked-in.....	555
EPM-check-action-performer-role.....	556
EPM-check-condition.....	559
EPM-check-item-status.....	561
EPM-check-object-properties.....	563
EPM-check-related-objects.....	567
EPM-check-responsible-party.....	571
EPM-check-signoff.....	572
EPM-check-status-progression.....	573
EPM-check-target-attachments.....	577
EPM-check-target-object.....	583
EPM-debug-rule.....	585
EPM-disallow-adding-targets.....	586
EPM-disallow-removing-targets.....	588
EPM-disallow-reviewers.....	589
EPM-hold.....	591
EPM-invoke-system-rule.....	593
EPM-signoff-team-validation.....	599
EPM-validate-target-objects.....	600
EPM-verify-digital-signature.....	602
ERP-check-effective-date-RH.....	603
ERP-check-target-status-RH.....	604
ERP-validate-data-RH.....	605
ICS-assert-target-classified.....	606
IM1-check-bomchild-release-status.....	607
IMD0-check-related-eudi-record.....	608
IMD0-Validate-DI-Record.....	609
IPM-check-attachments.....	609
IPM-check-duplicate-objects.....	610
IPM-compare-object-properties.....	611
LDF-sync-ldf-status.....	613
MDT-deliverable-validation-for-instances.....	618
MDT-hard-errors-validation.....	620
MDT-reference-deliverables-validation.....	621
MDT-validate-transmittal-response-for-del-instances	622
MESINTEG_ValidateReleaseAndExport.....	623

MFG-invoke-customized-validations.....	626
MROCORE-validate-for-class.....	628
PMA0_validate_change.....	629
PS-check-assembly-status-progression.....	630
PS-check-occ-notes.....	635
QFM0FMEAREALIZATION-validate-for-class.....	636
SAP-check-forms-attached-RH.....	637
SAP-check-forms-to-download-RH.....	639
SFD0-check-violations-exists.....	640
SFD0-mpwshuttle-objects-commit-review.....	641
SFD0-mpwshuttle-objects-production-review.....	642
TCRS-check-approver.....	643
TCRS-check-bom-precise.....	645
TCRS-check-bomchild-statuslist.....	646
TCRS-check-comps-against-pattern.....	649
TCRS-check-datasets.....	651
TCRS-check-itemrev-status.....	653
TCRS-check-jobowner.....	655
TCRS-check-prev-itemrev-status.....	657
TCRS-check-signoff.....	659
TCRS-check-status.....	660
TCRS-has-target-drawing.....	663
VAL-check-validation-result.....	664
VAL-check-validation-result-with-rules.....	666
validateIObject.....	668

Workflow handlers

What are workflow handlers?

Handlers are the lowest-level building blocks in workflow. They are small ITK programs used to extend and customize tasks. There are two kinds of handlers:

- Action handlers extend and customize task actions. They perform such actions as displaying information, retrieving the results of previous tasks (inherit), notifying users, setting object protections and launching applications.
- Rule handlers integrate workflow business rules into EPM workflow processes at the task level. They attach conditions to an action. Rule handlers confirm that a defined rule has been satisfied. If the rule is met, the handler returns the **EPM_go** command, allowing the task to continue. If the rule is not met, it returns the **EPM_nogo** command, preventing the task from continuing. If there are multiple targets for a single rule handler, all targets must satisfy the rule for **EPM_go** to be returned (**AND** condition).

Many conditions defined by a rule handler are binary (that is, they are either true or false). However, some conditions are neither true nor false. EPM allows two or more rule handlers to be combined using logical **AND/OR** conditions. When several rule handlers are combined using a logical **Or** condition, rule handler quorums specify the number of rule handlers that must return **EPM_go** for the action to complete.

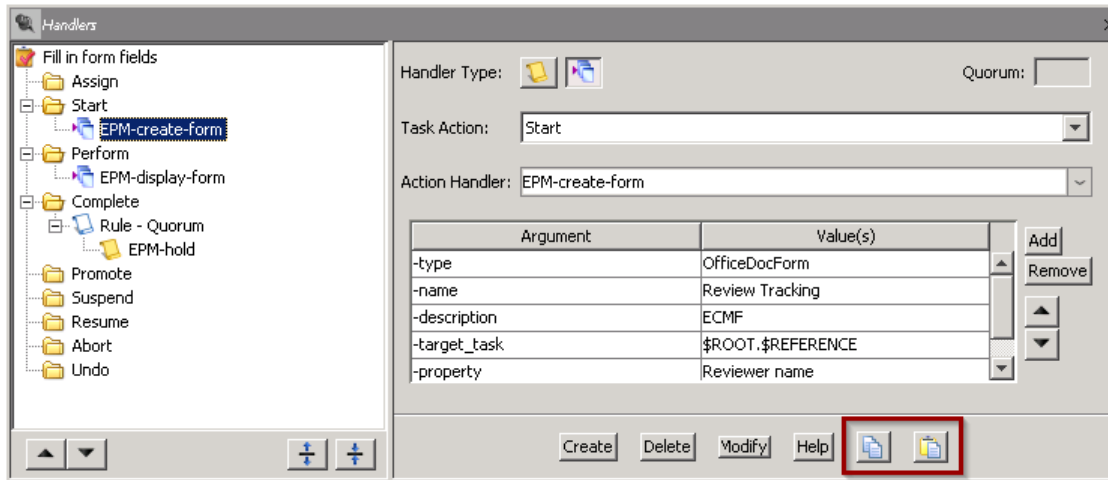
Action and rule handlers in the **Handlers** panel can be copied:

- From one action to another action in a task.
- From one task to another task in the same template.
- From a task in one template to a task in another template.

For the selection in the action tree, click **Performs a Copy action** or **Performs a Paste action** as desired.

Note

For **Performs a Paste action**, the process template must be in **Edit**  mode.



- To paste on another task in the same template, select the target task in the task hierarchy tree.
- To paste on a task in another template, select the target template from the **Process Template** list.

Executing workflow handlers

Handlers that require access execute according to either the default (**regular**) access or **system** access. The preference **WRKFLW_access_level_for_handlers_execution** indicates the access level to be used. This preference value takes effect on all handlers collectively (not individually).

- **Regular** access: In a typical user session, there are several factors that control access. Some of those factors include access rules, ACLs, and handler logic. Allowing these factors to determine access without overriding them is referred to as **regular** access. Regular access allows handlers to execute with their default access. This is the default preference setting.
- **System** access: When a handler executes with system access, the handler's default access will be overridden and the handler will be granted system access. This means that in a session where the preference is set to **system**, a handler would be granted access; whereas in a session where the preference is set to **regular**, the same handler would be denied access.

The following examples demonstrate the difference in handler behavior according to the preference setting. The **EPM-attach-related-objects** handler is used in these examples.

Consider the scenario where object access is denied and **EPM-attach-related-objects** is attempting to attach that object to a workflow.

- If the value for **WRKFLW_access_level_for_handlers_execution** is set to **regular**, then the handler will not be allowed to attach the object and an error will occur.
- If the value for **WRKFLW_access_level_for_handlers_execution** is set to **system**, then the default access will be overridden and access to the object will be granted, and the handler will be allowed to attach the object without error.

Working with Access Manager

The **EPM-set-rule-based-protection** handler indicates that an ACL will be passed to Access Manager. Access Manager will then apply access as defined by the ACL. Although **EPM-set-rule-based-protection** will indicate an ACL, it does not apply the ACL. Access Manager picks up the ACL and applies and enforces it. The ACL set by **EPM-set-rule-based-protection** is not exposed to Access Manager until a task's state has been set to Started. A task's state does not transition to Started until all the handlers on the Start action execute successfully. This means that other handlers that are located on the same Start action as **EPM-set-rule-based-protection** will not execute under the access indicated by **EPM-set-rule-based-protection**. In order for other handlers to adhere to the access indicated by **EPM-set-rule-based-protection**, they can either be placed on the Complete action of the current task or the Start action of a successor task. It is important to understand this concept because some handlers rely on access, and therefore a proper configuration is required to ensure the intended access is being applied when these handlers execute.

For example, a desired configuration may be to have all the following happen on a single task:

1. **EPM-set-rule-based-protection** indicates an ACL.
2. Access Manager picks up the ACL and applies the access.
3. **EPM-attach-related-objects** executes based on the access of the ACL that was indicated by **EPM-set-rule-based-protection**.

Since the ACL will not be applied until the task starts, to achieve the desired behavior, the **EPM-attach-related-objects** should be placed on the Complete action.

In this example, the configuration should look like this:

- Task Start handler: **EPM-set-rule-based-protection**
- Task Complete handler: **EPM-attach-related-objects**

With this configuration the processing will execute as follows:

1. Current task state is Pending.
2. Task is triggered to start.
3. Handlers on the Start action will execute, which in this example is the **EPM-set-rule-based-protection** handler, and **EPM-set-rule-based-protection** will indicate an ACL.
4. After the handlers on the Start action execute successfully, the task state is set to Started. The indicated ACL will now be applied (and this access will remain until a different access is set).
5. Handlers on the Complete action will execute, which in this example is the **EPM-attach-related-objects** handler.
6. The **EPM-attach-related-objects** executes under the intended ACL, which is the ACL indicated by **EPM-set-rule-based-protection** in step 3.

Updating your task templates to use the new handler and argument names

Starting with Teamcenter version 10.1, many of the workflow handlers, their arguments, and accepted argument values were changed to make them more consistent. The effect of the renaming depends on your situation:

- If you did not have an installation of Teamcenter prior to version 10.1, the renaming has no effect for you.
- If your installation was upgraded from a Teamcenter version prior to 10.1 to the current version, the **migrate_wf_handlers** utility was run during the upgrade and the handlers and arguments provided by Teamcenter were automatically renamed.
- If you are importing templates from a Teamcenter version prior to 10.1 to the current version, you must run the **migrate_wf_handlers** utility after importing the templates to rename the handlers and arguments.

When you import templates from a Teamcenter version prior to 10.1, do not select the **Apply template changes to all active workflow processes** check box in the rich client or use the **-apply_template** argument in the **plmxml_import** utility. If you do, Teamcenter does not successfully import the template.

- If you have custom handlers, you can use the **migrate_wf_handlers** utility and a custom mapping file to rename your custom handlers and arguments.

Renaming your custom handlers and arguments

You can use a custom XML mapping file and the **migrate_wf_handlers** utility to rename your custom handlers and arguments to make them consistent with the Teamcenter handlers and arguments. The elements of the mapping file are:

Element	Attributes	Usage
<Mapping>	None.	The <Mapping> element is the root level element in the XML file.
<Remove>	None.	Removes a handler or handler argument depending on where it is placed and its child elements. If <Remove> is the top level element, it may only have a <Handler> element as a child. If <Remove> is the child of a <Handler> element, it may only have one or more <Argument> elements as children. If an Argument value is specified, the Argument is removed only if the Argument value in the mapping file is a subset of the actual Argument

Element	Attributes	Usage
<Replace>	None.	Value in the system. If an Argument value is not specified, the Argument is removed, ignoring whether it has any value or not. Replaces a handler with more than one handler as specified by subsequent <Add> elements. The <Handler> child element of the <Replace> names the handler to be replaced. The arguments that need to be copied over to the new handlers (for example, see arg3 below) should be explicitly identified. If an argument from the old handler is not explicitly defined to be copied over, it is not added to a new handler, unlike the update/rename handler case.
<Update>	None.	For replacing one handler with another single handler, use the <Update> element. Changes a handler's name and/or arguments.
<Argument>	- name (optional) The current name of an argument. - va lue (optional) The current value of an argument. - newName (optional) The new name to be given to an argument. - newVa lue (optional) The new value to be given to an argument. - index (optional) Position of the argument in the handler. The index and name attributes are mutually exclusive.	Specifies the current and possibly new names and values for arguments of a handler.
<Handler>	- name The current name of a handler. - newName (optional)	Specifies the current and possibly new name of a handler.

Element	Attributes	Usage
	The new name to be given to a handler. <code>transformAssignees=<to-be-argname></code> (optional) Use this attribute when your existing handler has any number of users, groups, roles, address lists, and/or resource pools as arguments where they are not already specified in the form of a <code><-argname=argvalue></code> pair (such as <code>-participant=Smith</code>).	
<code><Criteria></code>	<code>match</code> (optional) <code>false</code>—the result of the criteria should be negated. <code>true</code>—default value.	Specifies restrictions on the <code><Handler></code> element in which it is embedded. The action specified by the <code><Handler></code> element is only applied if the criteria evaluate to true. <code><Criteria></code> may have two child elements: <code><Template name="<template-name>" /></code>, and <code><Argument name="<arg-name>" value="<arg-value>" /></code> that may be specified alone or together. The <code><template-name></code> is compared to the name of template containing the handler. The <code><arg-name></code> and <code><arg-value></code> are compared to the list of handler arguments. If both <code><Template></code> and <code><Argument></code> are specified, a handler must match both of the respective attributes.
<code><Template></code>	<code>name=<template-name></code> Compared to the name of the template containing the handler.	Restricts the <code><Criteria></code> element in which it is embedded to the specified template. If both <code><Template></code> and <code><Argument></code> are specified, a handler must match both of the respective attributes.
<code><Argument></code>	<code>name=<arg-name></code> <code>value=<arg-value></code> Compared to the list of handler argument names.	Restricts the <code><Criteria></code> element in which it is embedded to the specified argument name and value. If both <code><Template></code> and <code><Argument></code> are specified, a handler must match both of the respective attributes.

Element	Attributes	Usage
	Compared to the list of handler argument values.	
<Add>	None.	Adds a handler or handler argument. Unlike the <Remove> element, <Add> is never a top level element, but is always a child of a <Handler> element.
<Modify>	None.	Modifies a handler argument.
<Split>	- name An argument name. - newName An argument value. - Delimiter (optional) For splitting two delimited values existing only in the handler name field. For example, values delimited by two colons (::).	Splits any handler argument <old-name>=<old-value> pair into separate arguments <name1>=<old-name> and <name2>=<old-value>. A wildcard may be used for the name to match <old-name>. For example, <Split name="*" newName="- source, -decision" /> splits Cond1=Checked and Cond2=true into -source=Cond1, -decision=Checked, and -source=Cond2, -decision=true. Because handler arguments with the same name are combined into a single argument, this finally results in -source=Cond1,Cond2 and -decision=Checked,true.

Note

For any handler matched and processed by the **migrate_wf_handlers** utility, arguments having the same name are combined into a single argument with a resulting value composed of a comma-separated list.

Here is a full example of a mapping file:

```
<Mapping>
  <Remove>
    <!-- Remove all instances of Handler -->
    <Handler name="old-handler-name">
    </Handler>
  </Remove>

  <Update>
    <Handler name="old-handler-name" newName="new-handler-name">
      <Remove>
        <!-- if value is specified, remove the argument only if arg4
        has value val4 -->
        <Argument name="arg4" value="val4"/>
      </Remove>
    </Handler>
  </Update>
</Mapping>
```

```

        <!-- if value is not specified, remove argument irrespective
        of its value -->
        <Argument name="arg5" />
    </Remove>
</Handler>

different
<Replace>
    <Handler name="old-handler-name">
        <Add>
            <Handler name="new-handler1">
                <!-- copy value from arg1 to new-arg1 -->
                <Argument name="arg1" newName="new-arg1" />

                <!-- if arg2 has val2 (substring match) on old handler,
                add new argument new-arg2, copy over the value and
                replace the substring to new-sub-value2 -->
                <Argument name="arg2" value="sub-val2-1" newName="new-arg2"
                newValue="new-sub-val2-1" />

                <!-- The same argument can be repeated multiple times for
                substring value -->
                <Argument name="arg2" value="sub-val2-2" newName="new-arg2"
                newValue="new-sub-val2-2" />

                <!-- if arg3 is defined on old handler, add it to new handler
                and copy its value from old handler -->
                <Argument name="arg3" />

                <!-- add new argument with new value -->
                <Argument newName="new-arg6" newValue="new-val6"/>
            </Handler>

            <Handler name="new-handler2">
                <Argument newName="new-arg5" newValue="new-val5"/>

                <!-- copy value from arg1 to new-arg1 -->
                <Argument name="arg1" newName="new-arg1"/>
            </Handler>
        </Add>
    </Replace>

    <Update>
        <!-- Rename the old handler, as well as removing, adding and
        modifying
        its arguments. -->
        <!-- If any handler argument names are not mentioned in remove/modify
        sections, they are copied over to new handler. -->
        <Handler name="old-handler-name" newName="new-handler-name">
            <Remove>
                <!-- if value is specified, remove the argument only if arg4
                has
                val4 -->
                <Argument name="arg4" value="val4"/>
            </Remove>
        </Handler>
    </Update>

```

```

its      <!-- if value is not specified, remove argument irrespective of
value    value -->
        <Argument name="arg5"/>
        </Remove>

        <Add>
        <Argument name="new-arg6" value="new-val6"/>

value    <!-- if value is not specified or is empty, set the argument
        to empty -->
        <Argument name="new-arg7" value=""/>
        </Add>

        <Modify>
        <Argument name="arg1" value="val1" newName="new-arg1"
        newValue="new-val1"/>

to      <!-- if newValue is not specified, copy the old argument value
        new argument -->
        <Argument name="arg2" value="val2" newName="new-arg2" />
        </Modify>
        </Handler>
        </Update>

        <Remove>
        <!-- Remove Handler if the criteria matches (arg1 exists with value
        val1 and arg2 exists) -->
        <Handler name="old-handler-name">
        <Criteria>
        <Argument name="arg1" value="val1"/>
        <Argument name="arg2"/>
        </Criteria>
        </Handler>
        </Remove>

        <Update>
        <!-- Rename Handler if arg3 does not exist on the handler -->
        <Handler name="old-handler-name" newName="new-handler-name">
        <!-- If match set to 'false', the result of the criteria should be
        negated. (!) -->
        <Criteria match="false">
        <Argument name="arg3"/>
        </Criteria>
        </Handler>
        </Update>

        <Update>
        <!-- Add one or more handler arguments -->
        <Handler name="old-handler-name">
        <Add>
        <Argument name="new-arg1" value="new-val1"/>
        <Argument name="new-arg2" value="new-val2"/>

```

```

        </Add>
    </Handler>
</Update>

<Update>
    <Handler name="old-handler-name">
        <Modify>
            <Argument name="arg1" value="val1" newName="new-arg1"
            newValue="new-val1"/>

            <!-- if newValue is not specified, copy over the old argument
value to
            new argument -->
            <Argument name="arg2" value="val2" newName="new-arg2"/>

            <!-- if newValue is empty, clear the value for new argument. If
val3 is
            a substring of original value, special care should be taken in
            removing ',' -->
            <Argument name="arg3" value="val3" newName="new-arg3" newValue=""/>

            <!-- if new argument name is not specified, do not rename the
argument,
            but modify the argument value -->
            <Argument name="arg8" value="val8" newValue="new-val8" />

            <!-- Rename Handler Argument, keeping/copying-over the value -->
            <Argument name="arg9" newName="new-arg9" />

            <!-- Irrespective of the name of the argument, rename it to new-
arg1
            and copy the argument name as value of the new argument. If the
new
            argument name is already defined/added on the handler, append the
            value to existing value of that argument with delimiter set in the
            preference. -->
            <Argument name="*" newName="new-arg11" newValue="$ARGNAME"/>

            <!-- Replace the argument value by another value which includes
the
                    original value. If value is a comma separated list,
                    the new value will be a comma separated list
                    with the static string (user:) added
                    to each value in the list. -->
            <Argument name="user" newName="-assignee"
newValue="user:$ARGVALUE"/>

            <!-- index attribute will mention the arguments sequence in the
            handler. name and index are mutually exclusive. -->
            <Argument index="1" newName="year" newValue="$ARGNAME"/>
            <Argument index="2" newName="week" newValue="$ARGNAME"/>
        </Modify>
    </Handler>
</Update>

<Update>

```

```

    <!-- Rename Handler example. Rename "old-handler-name" handler to
         "new-handler-name" for all instances of "old-handler-name"
handler -->
    <Handler name="old-handler-name" newName="new-handler-name">
    </Update>
</Mapping>

```

Handler argument values

Syntax for handler arguments and values

Define handler arguments and values using the **Handlers** dialog box.

When you select a handler name, the existing arguments and values for the selected handler populate the argument table. You can enter additional arguments by typing argument and value data into the table cells. To assign multiple values to a single argument, separate the values with commas or the character specified by the **EPM_ARG_target_user_group_list_separator** preference. For example:

Argument	Values
-relation	IMAN_specification
-type	UGMASTER, UGPART
-att_type	target

Note

- Handler values are case-sensitive and must be accurate to the letter.
- If an argument calls for the name of an object, attribute, or property defined in the Business Modeler IDE, it must use the actual name, not its display name.
- In **Assignee** and **Recipient** fields, if an argument value has a comma in its name, you must use the **EPM_ARG_target_user_group_list_separator** preference to specify another separator for multiple values.

For example, if you have a **Recipient 1**, **Recipient 2**, and **Recipient 3** group and use it as an argument value, you must change the preference to use a different separator character, such as a semicolon (;).

Keywords as argument values

What are handler keywords?

Keywords are special arguments that extract values from the system, inserting the data into the handler's argument values in place of the keyword. Keyword syntax is the dollar sign (\$) followed by the keyword

name. For example, **\$USER** extracts the logon ID of the current user and inserts that value into the handler argument.

Some keywords are **common keywords**. You can use common keywords with many Teamcenter handlers. You can use some common keywords with custom handlers by using the **EPM_substitute_keyword** and **EPM_substitute_task_keyword** ITK functions. Use of these functions is illustrated within some of the sample workflow handlers delivered in the **sample** directory.

Other keywords are **handler-specific keywords**. You can use handler-specific keywords only with specific handlers. The documentation for each handler lists any handler-specific keywords that you can use with that handler.

Common keywords

The following table lists common keywords that you can use with many Teamcenter handlers and with custom handlers by using the **EPM_substitute_keyword** ITK function.

Keyword	Description
\$USER	Extracts the user ID of the current user.
\$GROUP	Extracts the group ID of the current user.
\$ROLE	Extracts the role of the current user.

The following table lists common keywords that you can use with many Teamcenter handlers and with custom handlers by using the **EPM_substitute_task_keyword** ITK function.

Keyword	Description
\$PROCESS_OWNER	Extracts the user ID of the owner of the current workflow process.
\$PROCESS_GROUP	Extracts the group ID of the owner of the current workflow process.
\$TARGET_OWNER[[Class]Type]	Extracts the user ID of the owner of the current workflow process's target. You can define an optional type or bracketed class in square brackets to specify the type or class of target object from which to extract the owner ID. If you do not define a class or type, the system uses the class of ItemRevision by default. If the system finds more than one object, it returns the owner ID from the first object.

Keyword	Description
	For example, \$TARGET_OWNER[(Dataset)] extracts the owning user ID from the first dataset target found, and \$TARGET_OWNER[UGMASTER] extracts the owning user ID from the first UGMASTER target found.
\$TARGET_GROUP[(Class) Type]	Extracts the group ID of the owner of the current workflow process's target. Only the first owner is returned.
	As with \$TARGET_OWNER , you can provide a type or bracketed class in square brackets to specify the type or class of target object from which to extract the owning group ID.
\$TARGET_OWNERS[(Class) Type1[,Type2,...]]	Extracts the user IDs of the owners of the current workflow process's targets. Only the first owner is returned.
	This keyword works the same as \$TARGET_OWNER , except that it returns a unique comma-separated list of the different owning user IDs from all specified target types.
\$TARGET_GROUPS[(Class) Type1[,Type2,...]]	Extracts the group IDs of the owners of the current workflow process's targets.
	This keyword works the same as \$TARGET_OWNERS , except it returns group IDs.
\$ROLE_IN_GROUP	Extracts the user's current logged-on group ID and role in the format of a resource string, for example, <code><group>::<role></code> .

Handler-specific keywords

The following table lists keywords that you can only use with specific handlers.

The documentation for each [action handler](#) and [rule handler](#) lists any handler-specific keywords that you can use with that handler. You can search the handler documentation for a particular handler-specific keyword to find all handlers that accept that keyword and to read a description of its functionality.

Keyword	Handlers
\$ANALYST	EPM-adhoc-signoffs EPM-auto-assign EPM-auto-assign-rest EPM-assign-team-selector

Keyword	Handlers
\$CHANGE_IMPLEMENTATION_BOARD	EPM-fill-in-reviewers
	EPM-notify-report
	EPM-notify
	EPM-adhoc-signoffs
	EPM-fill-in-reviewers
\$CHANGE_REVIEW_BOARD	EPM-notify-report
	EPM-notify
	EPM-adhoc-signoffs
	EPM-fill-in-reviewers
	EPM-notify-report
\$CHANGE_SPECIALIST1	EPM-notify
	EPM-adhoc-signoffs
	EPM-auto-assign
	EPM-auto-assign-rest
	EPM-assign-team-selector
\$CHANGE_SPECIALIST2	EPM-fill-in-reviewers
	EPM-notify-report
	EPM-notify
	EPM-adhoc-signoffs
	EPM-auto-assign
	EPM-auto-assign-rest
	EPM-assign-team-selector
	EPM-fill-in-reviewers

Keyword	Handlers
\$CHANGE_SPECIALIST3	EPM-notify-report
	EPM-notify
	EPM-adhoc-signoffs
	EPM-auto-assign
	EPM-auto-assign-rest
	EPM-assign-team-selector
	EPM-fill-in-reviewers
\$CURRENT_DATE	EPM-notify-report
	EPM-notify
	EPM-set-property
\$OWNER	EPM-check-action-performer-role
	EPM-late-notification
\$PROCESS	EPM-notify
	EPM-notify-signoffs
\$PROJECT_ADMINISTRATOR	EPM-adhoc-signoffs
	EPM-auto-assign
	EPM-auto-assign-rest
	EPM-assign-team-selector
	EPM-fill-in-reviewers
	EPM-notify-report
	EPM-notify
\$PROJECT_AUTHOR	EPM-adhoc-signoffs
	EPM-fill-in-reviewers
	EPM-notify-report
	EPM-notify

Keyword	Handlers
\$PROJECT_MEMBER	EPM-notify
	EPM-adhoc-signoffs
	EPM-fill-in-reviewers
	EPM-notify-report
\$PROJECT_TEAM_ADMINISTRATOR	EPM-notify
	EPM-adhoc-signoffs
	EPM-auto-assign
	EPM-auto-assign-rest
	EPM-assign-team-selector
	EPM-fill-in-reviewers
	EPM-notify-report
\$PROPOSED_RESPONSIBLE_PARTY	EPM-notify
	EPM-adhoc-signoffs
	EPM-auto-assign
	EPM-auto-assign-rest
	EPM-assign-team-selector
	EPM-fill-in-reviewers
	EPM-notify-report
\$PROPOSED_REVIEWERS	EPM-notify
	EPM-adhoc-signoffs
	EPM-fill-in-reviewers
	EPM-notify-report
	EPM-notify

Keyword	Handlers
\$REFERENCE	EPM-attach-related-objects
	EPM-create-form
	EPM-create-relation
	EPM-display-form
	EPM-remove-objects
	EPM-set-property
	EPM-notify
	EPM-notify-signoffs
\$RELEASE_STATUS	EPM-create-form
	EPM-create-relation
	EPM-display-form
\$RESPONSIBLE_PARTY	EPM-notify-report
	EPM-check-action-performer-role
	EPM-late-notification
	EPM-notify
\$REQUESTOR	EPM-adhoc-signoffs
	EPM-auto-assign
	EPM-auto-assign-rest
	EPM-assign-team-selector
	EPM-fill-in-reviewers
	EPM-notify-report
	EPM-notify
	EPM-fill-in-reviewers
\$REVIEWERS	

Keyword	Handlers
\$SIGNOFF	EPM-notify-report
	EPM-late-notification
	EPM-notify
	EPM-create-form
\$TARGET	EPM-create-relation
	EPM-display-form
	EPM-attach-related-objects
	EPM-check-target-attachments
	EPM-create-form
	EPM-create-relation
	EPM-display-form
	EPM-remove-objects
	EPM-set-property
	EPM-notify
\$UNDECIDED	EPM-notify-signoffs
	EPM-notify-report
	EPM-late-notification
	EPM-notify

Use keywords to implement dynamic participants in handlers

You can use the following keywords to invoke dynamic participants:

\$ANALYST	\$PROJECT_ADMINISTRATOR
\$CHANGE_SPECIALIST1	\$PROJECT_TEAM_ADMINISTRATOR
\$CHANGE_SPECIALIST2	\$PROJECT_AUTHOR

\$CHANGE_SPECIALIST3	\$PROJECT_MEMBER
\$CHANGE_REVIEW_BOARD	\$REQUESTOR
\$CHANGE_IMPLEMENTATION_BOARD	

If you want to use your custom dynamic participants, follow these steps:

Procedure

1. In Business Modeler IDE, create a child of the **Participant** business object.
2. For each child you create, associate a keyword in Business Modeler IDE.
3. In Workflow Designer, use the keyword you associated with a **Participant** business object child in a handler.
The handler associates the keyword with the dynamic participant defined in Business Modeler IDE and users with the specified role.

Configuring assigning participants automatically

You can configure your workflow to automatically assign participants with a set of Business Modeler IDE constants that have conditions as values. You can also use assign participants by adding workflow handlers that use properties that have participants as values.

Workflow constants

A set of constants is provided in the form:

`<<prefix>><participant-name>>AssignableCondition`

The variable `<<prefix>>` is the Business Modeler IDE template prefix and `<<participant-name>>` is an existing participant name.

Note

If the participant name also has a template prefix, the prefix appears twice.

For example, if the prefix is **Fnd0** and the participant name is **PROPOSED RESPONSIBLE PARTY**, the constant is **Fnd0ProposedResponsiblePartyAssignableCondition**.

The constants are for item revisions and change item revisions.

Workflow conditions

The values of the constants are conditions in the form:

is<<participant-name>>Assignable

For example, if the participant name is **PROPOSED RESPONSIBLE PARTY**, the condition is **isProposedResponsiblePartyAssignable**.

This is used while assigning dynamic participants. Teamcenter gets the value of the <<prefix><participant-name>>**AssignableCondition** constant to get the condition name to evaluate before assigning the participant.

Search for condition names

You can search for the constant name given an object type and participant type using pattern matching.

For example, to find a constant associated with an item revision and the **Fnd0MyNewParticipant** participant, search for a constant that ends with **Fnd0MyNewParticipantAssignableCondition**. The actual constant name is **Fnd0Fnd0MyNewParticipantAssignableCondition**.

If there are multiple matches, choose the one which has the same prefix as the prefix of the participant name.

Creating constants and conditions

If you have your own participant types, you must create your own constants and conditions for them.

For example, if your template prefix is **CUS1** and the new participant name is **MyParticipant**:

1. Create a participant named **CUS1MyParticipant**.
2. Create a constant named **CUS1CUS1MyParticipantAssignableCondition** with a value of **isMyParticipantAssignable**.

The participant creation code looks up the constant and corresponding condition and evaluates it.

Assigning participants with workflow handlers

You can use the following workflow handlers when automatically assigning participants:

- [EPM-assign-responsible-party-dynamic-participant](#)
- [EPM-assign-signoff-dynamic-participant](#)

The following handlers can be used to get assignees from a property value:

- [EPM-adhoc-signoffs](#)
- [EPM-assign-team-selector](#)
- [EPM-auto-assign](#)

- [EPM-auto-assign-rest](#)
- [EPM-fill-in-reviewers](#)

You can use the **user:PROP::<property_name>**, **resourcepool:PROP::<property_name>**, or **allmembers:PROP::<property_name>** values for the **-assignee** argument to get the name of the assignee from a property of the target, reference, or schedule task.

You can find the object type with the **-include_related_type**, **-exclude_related_type**, **-include_type**, **-exclude_type**, **-from_relation**, and **-from_attach** arguments.

For more information, see the full handler description.

Lists of values as argument values

Using lists of values (LOVs) in handler arguments

Some handlers have the ability to work on many objects, or may require many pieces of information to fully define what it is required of them. In these cases, it is cumbersome to supply all of the information as arguments or to add the handler several times to the same task, defining multiple arguments each time.

In cases when a handler is placed several times in a workflow process on different tasks (or in different workflow processes), adding many arguments to each instance of the handler is time consuming. If arguments later need to be modified, they may need to be changed in every instance of the handler, which is also time consuming.

Using LOVs as handler arguments is an efficient alternative. Standard LOVs supply a list of possible values to form attributes. LOVs used in handler arguments are created in the same way, using the Business Modeler IDE; however they do not need to be attached to any attributes. Each line in the LOV supplies configuration information relevant to the specific handler it is used for and in the format required by the handler.

LOV syntax

Any handler using an LOV accepts the **-lov=<lov-name>** argument, which specifies the LOV to be used.

The format of the data in a handler LOV is dependent on the information required by the handler, therefore, it is not the same across all handlers that accept LOV arguments. Where similar types of information are required, however, a consistent format is used. For example, when multiple fields of information are required in an LOV line, the fields are separated by tildes (~). The individual handler documentation describes the LOV line format required for that handler.

Note

The name of an LOV used with a handler can be anything, but the Business Modeler IDE may enforce a particular naming convention, for example, an **M4_** prefix. You can add the handler name as a suffix to help identify LOVs used by handlers.

Defining multilevel object paths

With some handlers, you can specify a multilevel path for locating objects using relation type/object type pairs, or relation type/class pairs. Typically, you use this method when working with **LOVs**.

The general syntax is:

```
<relation>.{<type>[,<type>]}(<class>)[!<type>]} . <relation> .{<type>[,<type>]}(<class>)[!<type>]}
```

You specify multiple types in a comma-separated list. For any relation or type field in the path, you can use either an asterisk (*) or **ALL** as a wildcard to mean any relation, type, or class.

You can specify target and reference relations within a workflow process using the **\$TARGET** and **\$REFERENCE** keywords.

For example, use multilevel object paths to find forms of a specific type attached to revisions within revisions. Consider this scenario:

A change item revision is currently in a change process. The change object contains item revisions with the **Solution Items** relation. Each of these solution revisions contain an **Affected Item Form** type in a reference relation that needs to be attached to the change process. You can identify these forms using this syntax:

```
$TARGET.(ItemRevision).CMHasSolutionItem.(ItemRevision)
.Reference.Affected Item Form
```

The previous example uses three relation pairs, as follows:

Pair	Description
\$TARGET.(ItemRevision)	Finds objects of the class ItemRevision attached as workflow process targets.
CMHasSolutionItem.(ItemRevision)	For each of the revisions found by the first pair, the system searches the CMHasSolutionItem relation to find objects of the class ItemRevision .
Reference.Affected Item Form	For each of the revisions found by the second pair, the system searches the Reference relations to find objects of the type Affected Item Form .

The individual handler documentation indicates which handlers accept this syntax.

LOV syntax example

This LOV example can be used with the [EPM-attach-related-objects](#) handler. Each line is a separate value in the LOV.

Argument	Values
-lov	M4_EPM_attach_objects

The **M4_EPM_attach_objects** LOV contains this data:

Value	Description
\$TARGET.(ItemRevision).Specification.*	Attach all objects in target revision Specification relation
\$TARGET.(ItemRevision).Specification.(Dataset).Form.(Form)!UGPartAttr	Attach all forms attached to datasets in target revision Specification relation
\$TARGET.(ItemRevision).PSBOMViewRevision.*	Attach all BOM View Revisions in target revision
\$TARGET.(ItemRevision).Manifestation.(Form)	Attach all forms in target revision Manifestation relation

Differentiating between classes and types

The purpose of many handlers is to locate and/or act on specified *types* or *classes*. Specifying a type directs the system to identify an object type. But specifying a class directs the system to identify *any* of the many types within that class. Therefore, it can be difficult to distinguish between types and classes.

For example, in the case of item revisions, some handlers perceive **ItemRevision** as a class of item revisions, making it difficult to designate the **ItemRevision** type.

Some handlers have the ability to distinguish between a class and type definitively. These handlers accept syntax that uses round brackets () to specify a class. For example, **(ItemRevision)** specifies the class and **ItemRevision** specifies the type. When this bracket notation is accepted, an exclamation point (!) can be used to exclude specific types, using this format:

```
(Class)[!Type1[!Type2[!...]]]
```

For example, given the four item types defined:

- **Item**
- **Document**
- **Design**
- **Software**

then:

(Item)	Matches any object of the Item class.
(Item) ! Software	Matches any object of the Item class except for the type Software .
(Item) ! Document ! Item	Matches any object of class Item except for the Document and Item types.
Design	Matches only the Design type.

The individual handler documentation indicates which handlers accept this syntax.

Specifying relations

Some relations for certain objects cannot be specified with standard generic relationship management (GRM) relation types. For example, you cannot specify to select all the revisions in an item. The following table lists available types of relations, including GRM relations and special relations.

Class	Relation	Description
Item	Any GRM relation	Identifies any GRM-related objects attached to items.
	Revisions	For example: (Item).IMAN_reference Identifies all revisions from items.

Class	Relation	Description
		For example, to find all the datasets in the IMAN_specification relation of all revisions in any items found: (Item).Revisions.*.IMAN_specification. (Dataset) Note The type of revision is not relevant as there is only one type of revision in any item; therefore, an asterisk (*) is used to specify any type.
	PSBOMView or BV	Identifies all BOM views from items. For example, to select all BOM views: (Item). PSBOMView Select only the <i>view</i> BOM views: (Item).BV.BOMView Revision
Revision	Any GRM relation	Identifies any GRM-related objects attached to revisions. For example, to identify all reference objects from revisions: (ItemRevision).IMAN_reference Identifies all specification objects in document revisions that are attached as requirements to design revisions: Design Revision.IMAN_requirement.Document Revision.IMAN_specification.*
Dataset	PSBOMViewRevision or BVR Any GRM relation	Identifies all BOM view revisions from revisions. Identifies any GRM-related objects attached to datasets. For example: (Dataset).IMAN_Rendering

Class	Relation	Description
	Any reference	Identifies any objects attached as references to datasets, such as UGPART-ATTR forms attached to UGMASTER and UGPART datasets. For example: (Dataset).UGPART-ATTR
Folder	*	Identifies objects in folders. For example, to identify all revisions in a folder: (Folder).*(ItemRevision)
Job	\$TARGET or Targets	Identifies targets attached to a job. For example: (Job).\$TARGET
	\$REFERENCE or References	Identifies targets attached to a job. For example: (Job).\$REFERENCE

Debugging handler data

The following handlers offer debugging functionality, enabled through the **TC_HANDLERS_DEBUG** environment variable:

- [EPM-check-target-object](#)
- [EPM-validate-target-objects](#)
- [EPM-check-target-attachments](#)
- [EPM-attach-related-objects](#)
- [EPM-remove-objects](#)

The debugging data displays in the system log file. Use the debugging information to solve small usability issues, such as incorrect argument usage. You can also submit the data in incident reports to customer service.

You can enable debugging functionality for all the above handlers and their subfunctions by setting the **TC_HANDLERS_DEBUG** environment variable to **ALL**.

Alternatively, you can enable debugging functionality for specific handlers by entering one or more of the above handler names as the value.

Action handlers

Action Handlers

Action handlers extend and customize task actions. They perform such actions as displaying information, retrieving the results of previous tasks (inherit), notifying users, setting object protections and launching applications.

AI-export-AH

Description

This handler has two modes of operation, depending on whether the required **type** argument is used with or without additional arguments.

- When **type** is the only argument:
 - When there already is an **AIOBJECT** in the reference attachments, this handler does nothing.
 - When there is initially no **AIOBJECT** in the reference attachments, this handler creates a new **AIOBJECT** of the specified type and a new **CCOBJECT** of type **CCOBJECT** and name **ERPOBJECT**. The handler creates a **StructureContext** for each **ItemRevision** found in the target attachments. The **Latest Working** revision rule is used in the **StructureContext** that is attached to the **CCOBJECT**.
- When **type** is specified with at least one of the available optional arguments:
 - Exports the objects found in target attachments to one or more **AIOBJECTS**, based on the settings of the optional arguments.
 - Searches the reference attachments for an **AIOBJECT** of the type specified by the **type** argument.
 - ◊ When an **AIOBJECT** is found, it is used. Otherwise this handler creates an **AIOBJECT** of the specified type.

The objects attached to the targets attachments can be filtered by the list of types specified by **targetTypes** argument.

The types listed must be one of the following supported types:

- **ItemRevision**
- **Item**
- **PSBOMView**
- **PSBOMViewRevision**
- **CCOBJECT**
- **AppearanceGroup**

If a **targetTypes** value is not provided, then all types are included.

- If the **multipleAI** value is equal to **1**, the handler creates an **AIOBJECT** for each object in the target attachments.
- If the **multipleAI** value is equal to **0** and **createRequests** is equal to **1**, the handler creates a single **AIOBJECT** with a new **RequestObject** for each object in the target attachments.
- If **createCC** is equal to **1**, the handler creates a **CCOBJECT** of the type specified by the **ccType** argument for non CC/SC objects in the target attachments, and exports the **CCOBJECT**.

Syntax

```
AI-export-AH -type=<ai-object-type> [-multipleAI= <0 | 1>] [-createCC= <0 | 1> ]
[-ccType= <cc-object-type>] [-createRequests= <0 | 1> ]
[-targetTypes= <delimited list of object types by which to filter target attachments>]
```

Arguments

-type

The type of **AIOBJECT** to search for in the reference attachments or, if none are found, the type of **AIOBJECT** to be created. The created **AIOBJECT** is attached to the root task.

-multipleAI

If equal to **0**, creates a single **AIOBJECT**. This is the default value.

If equal to **1**, creates an **AIOBJECT** for each object found in the target attachments.

-createCC

If set equal to **1**, creates a **CCOBJECT** with the type specified in the **-ccType** argument. The default value is **0**.

-ccType

The type of **CCOBJECT** to be created.

-createRequests

If **-multipleAI** is equal to **0** and **-createRequests** is equal to **1**, this handler creates a single **AIOBJECT** with a new **REQUESTOBJECT** for each object in target attachments. The default value is **0**.

-targetTypes

Uses a delimited list of object types for filtering target attachments. The types listed must be of the following supported types: **ItemRevision**, **Item** | **PSBOMView** | **PSBOMViewRevision** | **CCOBJECT** | **AppearanceGroup**.

The delimiter can be a colon (:) or a comma (,).

If no types are provided, all types are considered without filtering.

Placement

This handler can be placed on any task.

Restrictions

None.

Examples

Select an **ItemRevision** and submit to a workflow with this handler. This handler creates and exports the **AIOBJECT**, and then attaches it to the root task.

Argument	Values
-type	NX_AI
-createCC	1
-ccType	CCOBJECT

AI-process-import

Description

Imports the PLM XML associated with the target **RequestObject** objects.

RequestObject objects are contained within **ApplicationInterface** (AI) objects.

Syntax

AI-process-import

Arguments

None.

Placement

Requires no specific placement.

Restrictions

The attachments must be placed under the root task.

Examples

To import the PLM XML associated with a new **RequestObject** object created by any client application under an existing AI object, use a workflow template containing this handler. Initiate the workflow against the AI and select one or more **RequestObject** objects as target attachments, including the new **RequestObject**. Optionally, also select an **ICRevision** object as a reference attachment. The structure is updated with the contents of the PLM XML contained within the **RequestObject** object.

AI-process-export

Description

Creates a new **RequestObject** object under the target **ApplicationInterface** (AI) object without changing the base references of the AI object.

An AI object is a persistent workspace object that is the repository for the import and export transactions between Teamcenter and an external application for a predefined and configured structure. It contains:

- An ordered list of request objects.
- The transfer mode (import or export).
- The root or top-level object of the structures to exchange. This can be any object that is valid to export from Teamcenter using PLM XML, for example, a structure context, item, or BOM view revision.
- Tracking information to allow updates of changed data (*deltas*).

Use this handler in workflows containing at least one AI object as a target, and containing reference attachments such as **StructureContext** or **CollaborationContext** objects, or objects accepted by PLM XML export (such as BOM views, BOM view revisions, items, and item revisions).

Note

Without a **StructureContext** or **CollaborationContext** object, the PLM XML cannot export a structure, because there is no configuration; only the **workspaceObject** is exported. Typically, a **StructureContext** or **CollaborationContext** object is used as a reference attachment.

Syntax

AI-process-export

Arguments

None.

Placement

Requires no specific placement.

Restrictions

The attachments must be placed under the root task.

Examples

To share an existing **CollaborationContext** object with another application using PLM XML format, use a workflow template containing this handler. Initiate the workflow against an AI object, selecting the AI object as the target attachment and the **CollaborationContext** object as the reference attachment. The workflow creates a new **RequestObject** object. The AI can now be shared with another application.

AMX-auto-assign-task-assignees-from-assignment-matrix

Description

Assigns tasks in the workflow process to the specified user defined on entries of a related assignment matrix (AMX).

Note

All targets undergo evaluation during the assessment of assignment matrix rows.

Syntax

AMX-auto-assign-task-assignees-from-assignment-matrix

```
[-target_type=<object-type>]
[-application_context=<assignment-matrix-app-context>]
[-assignment_type=<role-of-assignee>]
[ -property=<property-name> ]
{-relation=<relation-name> }
[-secondary_type=<secondary object-type>]
[-relationS2P=<relation-name>]
[-primary_type=<primary object type name>]
[-partial_match_option=<All | First Match | Notify Only>]
[-assignment_option=<skip_if_exist | merge_if_exist | override_if_exist>]
[-participant_name=<participant-name>]
[-exclude_type=<object-type>}
[-target_task=<multilevel_task_path>]
```

Arguments

-target_type=<object-type>

(Required) Specifies a comma-separated list of the real name of the business object type (subtype of workspace object) on which decisive properties are identified.

-application_context=<assignment-matrix-app-context>

(Required) Specifies the application context of an assignment matrix

-assignment_type=<role-of-assignee>

(Required) Specifies the role of the assignees (for example, **Responsible**, **Reviewer**, **Notifeye**, **Acknowledger**).

-relation=<relation-name>

Specifies the relation which is used to retrieve the secondary objects using the target object. These secondary objects are used to locate an assignment matrix associated with them via the **Amx0AssignmentMatrixRel** relation.

Relation Type	Valid relation name
Manifestation	IMAN_manifestation
Specification	IMAN_specification
Requirement	IMAN_requirement
Reference	IMAN_reference

Note

You cannot use this argument with the **-property** and **-relationS2P** arguments.

-secondary_type=<secondary-object-type>

(Optional) This is a dependent argument which can be specified with the **-relation** argument. It specifies the type name which is used to filter the secondary objects which are retrieved using the **-relation** argument value.

Filtered secondary objects are used to locate an assignment matrix associated with them via the **Amx0AssignmentMatrixRel** relation. If this argument is not specified, all secondary objects are used to locate assignment matrices.

-relationS2P=<relation-name>

Specifies the relation which is used to retrieve the primary objects using the target object. These primary objects are used to locate an assignment matrix associated with them via the **Amx0AssignmentMatrixRel** relation.

Relation Type	Valid relation name
Manifestation	IMAN_manifestation
Specification	IMAN_specification
Requirement	IMAN_requirement
Reference	IMAN_reference

Note

You cannot use this argument with the **-property** and **-relation** arguments.

-primary_type=<primary-object-type>

(Optional) This is a dependent argument which can be specified with the **-relationS2P** argument. It specifies the type name which is used to filter the primary objects which are retrieved using the **-relationS2P** argument value. Filtered primary objects are used to locate an assignment matrix associated with them via the **Amx0AssignmentMatrixRel** relation. If this argument is not specified, all primary objects are used to locate assignment matrices.

-property=<property-name>

(Optional) Specifies a comma-separated list of the property names which are used to retrieve the secondary objects using the target object. These secondary objects are used to locate an assignment matrix associated with them via the **Amx0AssignmentMatrixRel** relation.

The property name specified must be the **typed reference** or **untyped reference** type. Singular as well as an array type of **property** can be specified.

There is a 1:1 correlation between the **-target_type** argument value and the **-property** argument value.

If both lists are specified:

1. If the sizes of both the **-target_type** and **-property** argument value list are greater than 1, then they must match. Otherwise, an error is raised.
2. If the size of the **-target_type** list is greater than 1 and the **-property** argument value list is 1, then same **-property** value will be used for all the **-target_type** names specified in the **-target_type** argument list. If the **-property** value is not a valid value on any of the **-target_type** present in the list, an error is raised.

You cannot use this argument with the **-relation** and **-relationS2P** arguments.

Assignment matrix discovery

The handler locates assignment matrices in the following priority order:

1. Objects retrieved using the **-relation**, **-relationS2P**, or **-property** arguments (if configured).
1. Workflow target objects (if the arguments in step 1 are not configured).
1. Workflow reference objects (if no assignment matrix is found in step 2).

If no assignment matrix is found, the handler does not perform any action and does not send a notification to the user.

-partial_match_option

(Optional) Identifies the set of assignees in case there are multiple assignment matrix rows qualified based on the assignment matrix configuration and target object content. Values include the following:

- **All** (default)

Assignees from all matching assignment matrix rows are added on the task.

- **First Match**

Only assignees from the first matched assignment matrix row are used for task assignment.

- **Notify Only**

If more than one assignment matrix rows qualify based on the target object, an error is raised, and the user is notified with the appropriate error message and no assignments are made.

-assignment_option

(Optional) Segregates/skips/overrides the assignees collected from various mechanisms, such as from any other handler and with this handler. Values include the following:

- **merge_if_exist** (default)

This argument merges assignees from any existing handler and assignees from the assignment matrix and assigns them to a task.

- **override_if_exist**

This argument overrides existing assignees with assignees from the assignment matrix and assigns them to a task.

- **skip_if_exist**

This argument checks for any existing assignees from any other handler. If none are found, then it will get assignees from the assignment matrix and assign them to a task.

Note

To restore the legacy behavior when **merge_if_exist** is not your desired default, explicitly specify **-assignment_option=override_if_exist** in your handler configuration.

-participant_name

(Optional) Specifies the keyword of the dynamic participant that you want to assign participants to. The following are some valid values:

- **PROPOSED_REVIEWERS**
- **CHANGE_REVIEW_BOARD**
- **CHANGE_IMPLEMENTATION_BOARD**
- **PROPOSED_RESPONSIBLE_PARTY**
- **ANALYST**
- **CHANGE_SPECIALIST1**

-exclude_type

(Optional) Defines the type of objects that should not be processed using this handler. You can define more than one type by using commas. This argument accepts valid Teamcenter object types, such as **ItemRevision**, **UGMASTER**, and **UGPART**.

If this argument is specified as **ItemRevision**, any type of item revision (for example, **DocumentRevision**, and so on, and any custom item revision types) is disallowed.

-target_task

(Optional) Specifies the multilevel task path identifying the task to which the assignees are added. The path is from the root task to the subtask with the path levels separated with colons (:).

For example: **Change Request Review:QA Review:perform-signoff**

If this argument is not specified, the assignees are added to the task where the handler is located. If the **-target_task** value does not point to a valid task in the workflow, an invalid handler configuration error occurs.

Placement

No specific placement is required. The handler is typically placed on the **Start** action of the task.

Restrictions

1. Only one of the **-property**, **-relation**, **-relationS2P** argument can be specified. All of them are mutually exclusive.
2. **-target_type** and **-property** arguments have correlation between them.
 - a. If both argument values are specified, then their sizes must match.
 - b. If sizes of both **-target_type** and **-property** argument value list is greater than 1, then they must match. Otherwise, an error is raised.
 - c. If size of **-target_type** list is greater than 1 and **-property** argument value list is 1, then same **-property** value will be used for all the **-target_type** names specified in the **-target_type** argument list. If **-property** value is not a valid value on either of the **-target_type** present in the list, an error is raised.
 - d. The property value specified at a specific index in the **-property** list must be available on corresponding **-target_type** name value specified at same index in **-target_type** argument value list. If it is not present, an error will be raised.
3. The property name specified in the **-property** argument must be a typed reference or untyped reference type. Both singular and array type properties are supported. If a property of any other type is specified, an error occurs

Examples

Example 1

This example finds an assignment matrix attached with the target type object **ChangeRequestRevision**; if not found then search in workflow references and finds **All** assignees and assigns tasks in the workflow process to the specified user defined on entries of related assignment matrix, based on **General** application context and **Responsible** assignment type:

Argument	Values
-target_type	ChangeRequestRevision
-application_context	General
-assignment_type	Responsible

Example 2

This example includes a Problem Report revision as the target object, as well as another business object type (attached as a secondary object) that is attached with the relation **IMAN_reference**.

The secondary objects are used to locate an assignment matrix associated via the **Amx0AssignmentMatrixRel** relation. In this example, the secondary business object is related to an instance of **Amx0AssignmentMatrix** type with the **Amx0AssignmentMatrixRel** relation.

The instance of **Amx0AssignmentMatrix** type is fetched by the handler and is used to evaluate assignees.

Argument	Values
-target_type	ProblemReportRevision
-application_context	General
-assignment_type	Reviewer
-relation	IMAN_reference
-partial_match_option	All

Example 3

This example demonstrates the default **merge_if_exist** behavior when the AMX handler is used alongside the **EPM-adhoc-signoffs** handler. The handler sequence is important: EPM-adhoc-signoffs executes first, followed by the AMX handler. The AMX handler looks for an assignment matrix related to the **-target_type** object **ProblemReportRevision**. It then reads all assignees from that assignment matrix (for the **General** application context and the **Reviewer** assignment type) and assigns the workflow tasks to those users.

Because the **-assignment_option** argument is not specified, the default **merge_if_exist** behavior applies. This merges assignees from the EPM-adhoc-signoffs handler with assignees from the assignment matrix.

In this scenario, the EPM-adhoc-signoffs handler identifies User_1 and User_2, and the AMX handler identifies User_3 and User_4 from the assignment matrix. As a result, all four reviewers (User_1, User_2, User_3, and User_4) are assigned to the task.

Argument	Values
-target_type	ProblemReportRevision
-application_context	General
-assignment_type	Reviewer
-relation	IMAN_reference
-partial_match_option	All

Note

If the **-assignment_option** argument is omitted, the default **merge_if_exist** behavior ensures that existing ad hoc reviewers are preserved and combined with reviewers identified by the assignment matrix.

Example 4

This example demonstrates the **override_if_exist** behavior when the AMX handler is used alongside the EPM-adhoc-signoffs handler. The handler sequence is important: EPM-adhoc-signoffs executes first, followed by the AMX handler. The AMX handler looks for an assignment matrix related to the **-target_type** object **ProblemReportRevision**. It then reads all assignees from that assignment matrix (for the **General** application context and the **Reviewer** assignment type).

Because the **-assignment_option** argument is explicitly set to **override_if_exist**, the AMX handler removes the ad hoc assignees identified by the EPM-adhoc-signoffs handler and replaces them with assignees from the assignment matrix.

In this scenario, the EPM-adhoc-signoffs handler initially identifies User_1 and User_2. However, when the AMX handler executes with **override_if_exist**, it removes User_1 and User_2 and assigns only User_3 and User_4 (identified from the assignment matrix) to the task.

Argument	Values
-target_type	ProblemReportRevision
-application_context	General
-assignment_type	Reviewer
-relation	IMAN_reference
-partial_match_option	All
-assignment_option	override_if_exist

Note

When **-assignment_option** is set to **override_if_exist**, the AMX handler removes all ad hoc reviewers identified by previous handlers (such as EPM-adhoc-signoffs) and replaces them with reviewers identified by the assignment matrix.

Example 5

This example finds an assignment matrix attached with **items_tag** property to the target type object **ChangeRequestRevision** and finds **All** assignees, then assigns tasks in the workflow process to the specified user defined on entries of the related assignment matrix, based on the **General** application context and **Reviewer** assignment type:

Argument	Values
-target_type	ChangeRequestRevision
-application_context	General
-assignment_type	Reviewer

Argument	Values
-property	items_tag
-partial_match_option	All

Example 6

This example finds an assignment matrix attached with **IMAN_reference** relation to the target type object **ProblemReportRevision** and finds **All** assignees and assigns tasks in the workflow process to the specified user defined on entries of related assignment matrix, based on **General** application context and **Reviewer** assignment type. The **-assignment_option** argument is explicitly set to **override_if_exist** to override all existing assignments (except **profile sign-off**) with the new assignees from the assignment matrix.

Argument	Values
-target_type	ProblemReportRevision
-application_context	General
-assignment_type	Reviewer
-relation	IMAN_reference
-partial_match_option	All
-assignment_option	override_if_exist

Example 7

This example finds an assignment matrix attached with **IMAN_reference** relation to the target type object **ProblemReportRevision**, then finds **All** assignees and assigns tasks in the workflow process to the specified user defined on entries of the related assignment matrix, based on **General** application context and **Reviewer** assignment type. The **-assignment_option** is explicitly set to **merge_if_exist**, the default behavior, to merge all existing assignments with new assignees from the assignment matrix.

Argument	Values
-target_type	ProblemReportRevision
-application_context	General
-assignment_type	Reviewer
-relation	IMAN_reference
-partial_match_option	All
-assignment_option	merge_if_exist

Example 8

This example finds the assignment matrix by searching for the **DocumentRevision** type of the secondary object associated with the target object using the **IMAN_specification** relation.

Argument	Values
-target_type	ProblemReportRevision
-application_context	General
-assignment_type	Reviewer
-relation	IMAN_specification
-secondary_type	DocumentRevision

Example 9

This example finds an assignment matrix attached to the workflow target type object **ChangeRequestRevision** and assigns tasks to the specified user defined on entries of the related assignment matrix, based on the **General** application context and **Reviewer** assignment type. The **-target_task** argument specifies that assignees should be added to a Signoff team selection task within a specific task in the workflow.

Argument	Values
-target_type	ChangeRequestRevision
-application_context	General
-assignment_type	Reviewer
-partial_match_option	All
-target_task	Design Review:EngineeringReview

Example 10

This example demonstrates how the assignment matrix uses properties from a **ChangeRequestRevision** object to determine assignees. The handler retrieves the **ChangeRequestRevision** object from the workflow and reads its property values (such as **Priority**, **Change Type**, and **Project**). The assignment matrix configured with the **Engineering** application context contains rows with decisive properties that define conditions for each row. The handler evaluates which assignment matrix rows match based on the target object's property values, and assignees from all matching rows are added to the task where this handler is attached.

Argument	Values
-target_type	ChangeRequestRevision
-application_context	Engineering
-assignment_type	Reviewer

Argument	Values
-partial_match_option	All
-assignment_option	merge_if_exist

AR-mark-archive

Description

Note

This handler is deprecated and will be obsolete in a future release. Do not add this handler to new workflow processes.

Generates archive requests for datasets of item revisions with the specified status. This handler should be used only when the targets of a workflow process are item revisions. This handler is very useful in archiving the experimental, prototype data and keeping only the real data.

Syntax

```
AR-mark-archive [-exclude_related=<relation::type
[, relation::type..]> ],-status_to_keep=<status::number-of-item-revs-to-keep
[, status::number-of-item-revs-to-keep..]>
```

Arguments

-exclude_related

Excludes the specified relation or type or type in relation from having an archive request being generated. This argument is optional. If this argument is used, either a relation or type should be specified. If only a relation is specified, :: need not be appended (for example:

-exclude_related=IMAN_specification). If only a type is used, prepend the type with :: (for example:

-exclude_related=::UGPART).

-status_to_keep

Release status **names::number** of item revisions to keep.

This means not to mark for archive the datasets of a specified number of item revisions with the specified release status.

Siemens Digital Industries Software recommends that the number of revisions to keep should be 1 or more. This way, at least one item revisions per release status is not archived. This assures that there are no product structure configuration problems.

Placement

Requires no specific placement. Typically placed on the **Complete** action of the root task so that the objects are marked for archive at the end of completion of the workflow process.

Restrictions

Target objects must be item revisions.

Examples

In this example, consider the scenario:

An item has 20 item revisions out of which item revisions 1-4 have no release status, item revisions 5-9 have release status **Released**, item revisions 10-14 have release status **R**, and item revisions 15-19 have release status **X** set.

The **AR-mark-archive** handler with the following arguments is added to the **Complete** action of the root task.

Argument	Values
-exclude_related	IMAN_manifestation::UGPART
-status_to_keep	R::3, X::2

The previously created item revision workflow process template is initiated on the 20th item revision. After the workflow process is completed, the following results are expected.

All datasets except those:

- With manifestation relation
- Of type **UGPART**

of the item revisions, 10-11 and 15-17, are marked for archive.

ASBUILT-attach-physical-components

Description

Traverses the as-built structure and attaches as-built physical parts as targets to the workflow.

Syntax

```
ASBUILT-attach-physical-components [-depth=<level> | all]
[-owned_by_initiator] [-owned_by_initiator_group]
[initiator_has_write_prev]
{[-exclude_released] [-traverse_released_component]}
[-exclude_types=types]
[-add_excluded_as_ref][-include_missing]
```

Arguments

-depth

Defines the depth to which the traversal should take place.

- For example, specify **1** to traverse one level deep or **all** to traverse all levels.
- If not specified, the handler traverses all levels.

-owned_by_initiator

Adds the components owned by the initiator as targets to the workflow process.

-owned_by_initiator_group

Adds all components owned by the initiator's group as targets to the workflow process.

-initiator_has_write_prev

Adds all component item revisions where the initiator has write access as targets to the workflow process.

-exclude_released

Excludes released components from being added as targets.

If the released component is a subassembly, the handler does not traverse the components of the released component unless **-traverse_released_component** is also specified.

-traverse_released_component

Note

This argument can only be used in conjunction with the **-exclude_released** argument.

Traverses the structure of the released component and adds the components as targets to the workflow process.

- If the **-depth** argument is set to **1**, **-traverse_released_component** only traverses one level deep.

- If the **-depth** argument is set to **all**, **-traverse_released_component** traverses all levels of the subassembly.

-exclude_types

Defines the types to be excluded from being added as targets.

-add_excluded_as_ref

Adds components that are not included as targets to the workflow process as references.

-include_missing

Includes missing components as targets.

If this is not specified, an error is displayed for structures that contain missing components.

Placement

Requires no specific placement, but preferably after review/approval completion, if any.

Restrictions

None.

ASBUILT-release-asbuilt-structure

Description

Releases or freezes the as-built physical structures. Given a top or root physical part revision, this handler navigates the as-built structure relationships and releases each of the physical part revision objects in the structure by attaching a release status object. Target objects are officially released after this handler runs.

Syntax

```
ASBUILT-release-asbuilt-structure -release status [-depth=<level>| all] [-owned_by_initiator] [-owned_by_initiator_group] [-initiator_has_write_prev] [{[-exclude_released] [-traverse_released_component]}] [-exclude_types=<types>] [-add_excluded_as_ref] [-include_missing]
```

Arguments

-release status

Applies the specified release status to each of the physical parts.

-depth

Defines the depth to which the traversal should take place.

For example, specify **1** to traverse one level deep or **all** to traverse all levels.

If not specified, the handler traverses all levels.

-owned_by_initiator

Adds the components owned by the initiator as targets to the workflow process.

-owned_by_initiator_group

Adds all components owned by the initiator's group as targets to the workflow process.

-initiator_has_write_prev

Adds all component item revisions where the initiator has write access as targets to the workflow process.

-exclude_released

Excludes released components from being added as targets.

If the released component is a subassembly, the handler does not traverse the components of the released component unless **-traverse_released_component** is also specified.

-traverse_released_component

Traverses the structure of the released component and adds the components as targets to the workflow process.

This argument can only be used in conjunction with the **-exclude_released** argument.

If the **-depth** argument is set to **1**, **-traverse_released_component** only traverses one level deep. If the **-depth** argument is set to **all**, **-traverse_released_component** traverses all levels of the subassembly.

-exclude_types

Defines the types to be excluded from being added as targets.

-add_excluded_as_ref

Adds components that are not included as targets to the workflow process as references.

-include_missing

Includes missing components as targets.

If this is not specified, an error is displayed for structures that contain missing components.

Placement

Requires no specific placement, but preferably after review/approval completion, if any.

Restrictions

None.

ASMAINTAINED-attach-physical-components

Description

Traverses the as-maintained structure and attaches as-built physical parts as targets to the workflow.

Syntax

```
ASMAINTAINED-attach-physical-components [-depth=<level> | all]
[-owned_by_initiator] [-owned_by_initiator_group]
[initiator_has_write_prev]
{[-exclude_released] [-traverse_released_component]}
[-exclude_types=types]
[-add_excluded_as_ref][-include_missing]
```

Arguments

-depth

Defines the depth to which the traversal should take place.

- For example, specify **1** to traverse one level deep or **all** to traverse all levels.
- If not specified, the handler traverses all levels.

-owned_by_initiator

Adds the components owned by the initiator as targets to the workflow process.

-owned_by_initiator_group

Adds all components owned by the initiator's group as targets to the workflow process.

-initiator_has_write_prev

Adds all component item revisions where the initiator has write access as targets to the workflow process.

-exclude_released

Excludes released components from being added as targets.

If the released component is a subassembly, the handler does not traverse the components of the released component unless **-traverse_released_component** is also specified.

-traverse_released_component

Note

This argument can only be used in conjunction with the **-exclude_released** argument.

Traverses the structure of the released component and adds the components as targets to the workflow process.

- If the **-depth** argument is set to **1**, **-traverse_released_component** only traverses one level deep.

- If the **-depth** argument is set to **all**, **-traverse_released_component** traverses all levels of the subassembly.

-exclude_types

Defines the types to be excluded from being added as targets.

-add_excluded_as_ref

Adds components that are not included as targets to the workflow process as references.

-include_missing

Includes missing components as targets.

If this is not specified, an error is displayed for structures that contain missing components.

Placement

Requires no specific placement, but preferably after review/approval completion, if any.

Restrictions

None.

ASMAINTAINED-release-asmaintained-structure

Description

Releases or freezes the as-maintained physical structures. Given a top or root physical part revision, this handler navigates the as-maintained structure relationships and releases each of the physical part revision objects in the structure by attaching a release status object. Target objects are officially released after this handler runs.

Syntax

```
ASMAINTAINED-release-asmaintained-structure -release status [-depth=<level >| all] [-owned_by_initiator] [-owned_by_initiator_group] [-initiator_has_write_prev] {[exclude_released] [-traverse_released_component]} [-exclude_types=<types >] [-add_excluded_as_ref] [-include_missing]
```

Arguments

-release status

Applies the specified release status to each of the physical parts.

-depth

Defines the depth to which the traversal should take place.

For example, specify **1** to traverse one level deep or **all** to traverse all levels.

If not specified, the handler traverses all levels.

-owned_by_initiator

Adds the components owned by the initiator as targets to the workflow process.

-owned_by_initiator_group

Adds all components owned by the initiator's group as targets to the workflow process.

-initiator_has_write_prev

Adds all component item revisions where the initiator has write access as targets to the workflow process.

-exclude_released

Excludes released components from being added as targets.

If the released component is a subassembly, the handler does not traverse the components of the released component unless **-traverse_released_component** is also specified.

-traverse_released_component

Traverses the structure of the released component and adds the components as targets to the workflow process.

This argument can only be used in conjunction with the **-exclude_released** argument.

If the **-depth** argument is set to **1**, **-traverse_released_component** only traverses one level deep. If the **-depth** argument is set to **all**, **-traverse_released_component** traverses all levels of the subassembly.

-exclude_types

Defines the types to be excluded from being added as targets.

-add_excluded_as_ref

Adds components that are not included as targets to the workflow process as references.

-include_missing

Includes missing components as targets.

If this is not specified, an error is displayed for structures that contain missing components.

Placement

Requires no specific placement, but preferably after review/approval completion, if any.

Restrictions

None.

BC-perform-export

Description

Performs a Briefcase/PDX export using a workflow process.

Syntax

```
BC-perform-export -sitename=<site-name> [-optionset=<transfer-option-set> ] [-usegs=True |
False] [-revisionrule=<revision-rule-name>] [-bomlevel=<depth>] [-vendors=<vendor-names>] [-
reason=<export-reason-string>] [-immediate=True | False] [-notify=True|False] [-emailaddrs=<email-
ids>]
```

Arguments

-sitename

Specifies the destination site name where the Briefcase or PDX package is to be exported.

-optionset

Specifies the transfer option set to be used during export. If none is specified, the system uses either **TIEPDXOptionSetDefault** (for a PDX export) or **TIEUnconfiguredExportDefault** (for a Briefcase export) based on availability of the set.

-usegs

Specifies whether the transaction should go through Global Services or not. Valid values are **True** and **False**. The default value is **False**, which is a non-Global Services-based transaction.

-revisionrule

Specifies the revision rule to be used to perform the BOM configuration.

-bomlevel

Specifies the depth to which the BOM must be traversed for export.

-vendors

Specifies the list of vendor names whose manufacturer parts are to be exported. Only parts from these vendors get exported.

-reason

Specifies the reason for the export (up to 240 characters).

-immediate

Specifies whether the transaction should be performed immediately or not. This argument is applicable only when **-usegs=True**. Valid values are **True** and **False**. The default value is **False**.

-notify

Specifies whether the users listed in the **-emailaddrs** argument are notified when the transaction is completed. This argument is applicable only when **-usegs=True**. Valid values are **True** and **False**. The default value is **False**.

-emailaddrs

Lists the email IDs of users to be notified when the transaction is completed. This argument is applicable only when **-usegs=True** and when the **-notify=True**.

Separate the email IDs with commas or the character specified by the **EPM_ARG_target_user_group_list_separator** preference.

Placement

Requires no specific placement.

Restrictions

None.

Examples

This example exports a package to **Supplier-site-1** using a custom option set without using Global Services.

Argument	Values
-sitename	Supplier-site-1
-optionset	CustomOptionSet1
-usegs	False

CAE-attach-related-cae-folder-objects

Description

At some sites, not all simulation tools are integrated with Teamcenter. In such cases, simulation analysts can run the simulation tools on their local desktop and periodically upload or download the data to or from Teamcenter. The analysts can create a CAE folder structure within an item revision to manage the different types of files from different simulation tools.

Simulation administrators can configure a workflow process using the **CAE-attach-related-cae-folder-objects** action handler to allow simulation analysts to release the item revision containing the CAE folder structure and its contents.

This action handler attaches the specified related **CAE Folder (CAE0FileCollection)** objects of the target objects as target or reference attachments to the workflow process. It searches all the target objects, finds the **CAE Folder** objects recursively, and then adds them as a target or as reference attachments. If a **CAE Folder** object is already part of the target list, it is ignored.

Syntax

CAE-attach-related-cae-folder-objects -attachment=<target | reference>

Arguments

-attachment <target | reference>

The attachment type with which the objects are attached to the workflow process.

The **-tool** argument is mandatory and requires the simulation tool ID value. The rest of the arguments are optional and can be specified without any values.

Placement

It is typically placed on the **Start** action of the root task so that the list of target attachments is updated during the workflow process initiation.

Restrictions

Requires one or more target objects to find the related **CAE Folder** objects. The placement should allow at least one target object before the execution of this handler takes place.

Examples

This example attaches all the **CAE Folder** objects as target objects to the workflow process when a workflow process is initiated on a CAE item revision.

Argument	Values
-attachment	target

CAE-decode-token-and-update-model-attributes

Description

This action handler is for managing Simcenter Client for Git default workflows. The user must provide the Simcenter Client for Git token in the **Comments** box of the workflow process.

This action handler must be configured on the complete action of a **Do** task. It decodes the token and updates the **Model Identifier**, **Model Version**, and **Model Name** properties on the **CAE 1D Model** revision.

Syntax

CAE-decode-token-and-update-model-attributes

Arguments

None

Placement

It is typically placed on the complete action of the **Do** task only. It requires inputs and users should provide them in the **Comments** box of the workflow process.

Restrictions

Configure only for **CAE 1D Model** revisions.

CAE-mark-up-to-date

Description

In a complex product development environment, different analysts perform different tasks of the overall analysis. For example, abstractions are delivered by one group, models built by another group, and load cases defined by another group. In such scenarios, it becomes critical to know when the analysis data, possibly with multiple dependencies, is out-of-date. The analyst can then act on it and ensure that the analysis is built with the correct set of data to deliver accurate results.

When analysts complete their work, they have to mark the revisions they worked on as up-to-date. Instead of the analyst manually doing this, the simulation administrator can configure a workflow process using the **CAE-mark-up-to-date** action handler. This allows the system to automatically mark revisions as up-to-date when they are released through a workflow process.

Syntax

CAE-mark-up-to-date

Arguments

None

Placement

Typically, before the release action.

Restrictions

Configure only for CAE items.

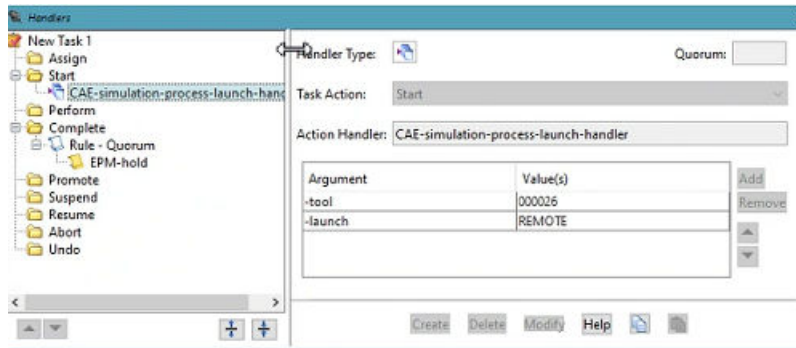
CAE-simulation-process-launch-handler

Description

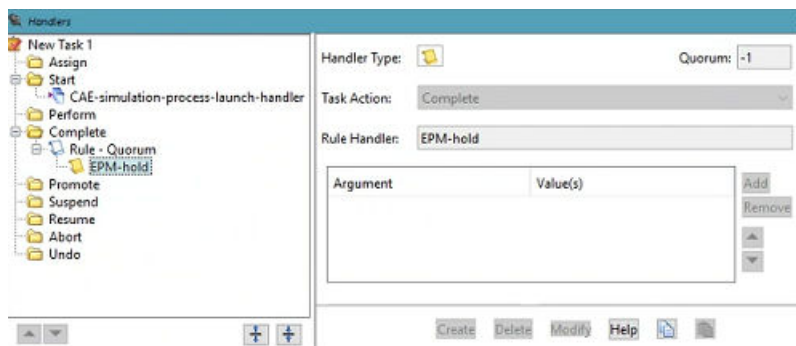
Launches the specified simulation tool.

Placement

- Place the **CAE-simulation-process-launch-handler** action handler on the **Start** action.



- Place the **EPM-hold** rule handler on the **Complete** action. This stops the task from automatically completing when started.



Syntax

CAE-simulation-process-launch-handler -tool=<tool_ID> -launch=<LOCAL_OR_SERVER_OR_REMOTE> -nosync -continue -noref -param::

Note

When you use **local** as the input for the **-launch** argument, the workflow initiates a server launch. The input means local to the server.

Arguments

-tool

The ID of the simulation tool to launch.

Note

The simulation tool ID you specify here must match the simulation tool ID defined in the **Simulation Tool Configuration** dialog box in CAE Manager.

The **-tool** argument is mandatory and requires the simulation tool ID value. The rest of the arguments are optional and can be specified without any values.

Tool names and revisions are no longer supported. The tool is now launched with the latest released revision. If you have an existing action handler with a tool name and revision values, you must modify them and use only the tool ID value.

-launch

This argument is mandatory if you select the **Remote Launch** option in the **Simulation Tool Configuration** dialog box in CAE Manager.

Note

If this value is not specified, the handler assumes the launch type to be local, this is, the machine on which Teamcenter server is running.

-nosync

If specified, a synchronous process running in the background does not inform the task about its completion. As a result, the control from the current task goes to the next task (if any) as soon as the current task starts.

If not specified, the system displays the following warning:

<A simulation batch run is in progress. The task will complete offline after the process completes.>

Note

This argument is valid for local launch only. Remote launch is always run in non-synchronous mode.

This parameter is deprecated. To run the process in synchronous mode, use the **EPM-hold** rule handler on the **Complete action**. Do not use the rule handler if want to run the process in non-synchronous mode.

-continue

If specified, the current task moves to the next task after completion even if the current task fails.

If not specified, the task stops on failure.

Note

This argument is valid for local launch only. Remote launch is always run in nonsynchronous mode.

This argument is not valid if you specify the **-nosync** argument.

-noref

If specified, the handler does not add output objects as reference attachments.

If not specified, the handler adds output objects as reference attachments in the **Reference** folder.

Note

This argument is valid for local launch only. Remote launch is always run in nonsynchronous mode and output objects are never added as reference attachments.

This argument is not valid if you specify the **-nosync** argument.

-param::<paramName>

Used to assign run-time parameter values for any parameters already defined as part of the tool configuration in the **Simulation Tool Configuration** dialog box in CAE Manager.

Launches the tool with the <paramValue> value for the <paramName> parameter as defined in the tool configuration. The specified parameters are processed according to the defined configuration.

Note

The <paramName> value must be defined as a run-time parameter for the tool configuration in the **Simulation Tool Configuration** dialog box. Any run-time parameters defined in the tool configuration that are not indicated as action handler arguments get the default values defined in the tool configuration. The <paramValue> value can be an empty string, in which case the default value of the corresponding <paramName> is overridden with an empty value.

Restrictions

None.

CAE-structuremap-execution-handler

Description

Performs the structure map execution on the target objects.

Note

Users can configure multiple CAE-structuremap-execution-handler with different action handler configurations.

Syntax

CAE-structuremap-execution-handler **-sm**=<structure-map-ID> **-revrule**=<Revision-Rule> **-svr**=<Saved-Variant-Rule> **-target_owning_user**=<owning_user> **-target_owning_group**=<owning_group>

Arguments

-sm <target | reference>

(Mandatory) Structure Map Item ID.

-svr

(Optional) Saved variant rules to be applied on the target structure. For multiple SVRs, use comma (,) as a separator.

Note

Users can configure the SVRs present on other item revisions.

If users provide only the SVR name, the system considers that SVR present on the input item revision, which is a target object.

If users provide the SVR name in the format <item_id>::svr, then the system considers the SVR present on the latest working revision of the item to which the **item_id** corresponds.

If users provide the SVR name in the format <item_id>::<rev_id>::svr, then the system considers the SVR present on the revision of the item to which the **item_id** and **rev_id** corresponds.

If multiple SVRs are provided the system applies those SVRs which are found. Not found SVRs are skipped.

-revrule

(Optional) Revision rule to be applied on the target structure.

-target_owning_user

(Optional) Owing user to be set for the resulting CAE structure. If not specified, then the current workflow process owner is the owner of the CAE BOM.

-target_owning_group

(Optional) Group name of the owning user to set. If not specified, then the current workflow process owner's group is the owning group of the CAE BOM.

Placement

Not applicable.

Restrictions

None.

Examples

Not applicable.

CFG0-attach-allocations

Description

Attaches allocation objects that reference features, families, or groups. Such objects may be located in the target attachment or reference attachment folder. The **-configuration** argument specifies whether to attach the allocation's **Latest Working** or **Latest Released** revision.

For more information, see *Create workflows to release configurator data in Product Configurator — Deployment and Administration*.

Syntax

```
CFG0-attach-allocations
[-attachment = {target | reference}]
[-configuration = {Latest Working | Latest Released}]
[-attachedConfiguratorContext = {false | true}]
[-debug = {false | true}]
```

Arguments

-attachment

Attachment type with which the objects are attached to the workflow process. If any subsequent workflow handler depends on the allocation objects to be attached, either as a reference or as a target attachment, use this argument to configure two instances of this handler in the same workflow process. In such cases, the first handler is configured with **-attachment=<target>-configuration=<Latest Working>** in order to attach the working revisions (if any). It is followed by the same handler configured with **-attachment=<reference>-configuration=<Latest Released>** to attach the related released objects (if any). Possible values are:

- **target**

Allocation revisions are attached as target objects. This is the default value.

- **reference**

Allocation revisions are attached as reference objects.

Note

If another revision of the same configurator object thread is already attached to this workflow (either as target or reference), the handler silently skips the object. That is, the handler does not attach a second revision of the same thread.

-configuration

Specifies whether to attach the **Latest Working** or **Latest Released** revision. If any subsequent workflow handler depends on the allocation objects to be attached, either as a reference or as a target attachment, use this argument to configure two instances of this handler in the same workflow

process. In such cases, the first handler is configured with **-attachment=<target>-configuration=<Latest Working>** in order to attach the working revisions (if any). It is followed by the same handler configured with **-attachment=<reference> -configuration=<Latest Released>** to attach the related released objects (if any). Possible values are:

- **Latest Working**

The most recently created revision with no release status is attached. This is the default value.

- **Latest Released**

The most recently released revision is attached.

-debug

Whether or not to log status information to the syslog file. Possible values are:

- **false**

No status information is written to the syslog file. This is the default value.

- **true**

Status information is written to the syslog file for debugging purposes.

Placement

A typical placement is below the **EPM-create-status** action handler that creates and adds the release status to the workflow process. In many cases, it is useful to add the **CFG0-attach-allocations** handler below a **CFG0-attach-familygroups** handler.

Restrictions

None

Examples

This example illustrates the use of the handler that attaches **Latest Working** revisions of feature, family, and group allocations for features, families, and groups in this workflow process as target attachments so that they are processed along with the variability that is already attached to the workflow. The list of allocations to add is filtered by the **Configurator Context** items attached to this workflow.

Argument	Values
-attachment	target
-configuration	Latest Working
-attachedConfiguratorContext	true

CFG0-attach-constraint-rules

Description

Attaches configurator constraint rules that reference a feature or variant option family. Such objects may be located in the target attachment or referenced attachment folder. The **-configuration** argument specifies whether to attach the **Latest Working** or **Latest Released** revision of the constraint rules.

Note

A configurator constraint rule references the option family if the family has free-form values. Otherwise, it references the option value directly.

For more information, see *Create workflows to release configurator data* in *Product Configurator — Deployment and Administration*.

Syntax

CFG0-attach-constraint-rules

[-attachment = {target | reference}]

[-configuration = {Latest Working | Latest Released}]

[-attachedConfiguratorContext = {false | true}]

[-debug = {false | true}]

Arguments

-attachment

Attachment type with which the objects are attached to the workflow process. If any subsequent workflow handler depends on constraint rules to be attached, for example, **CFG0-attach-rule-variability**, either as a reference or as a target attachment, use this argument to configure two instances of this handler in the same workflow process. In such cases, the first handler is configured with **-attachment=<target>** **-configuration=<Latest Working>** in order to attach the working revisions (if any). It is followed by the same handler configured with **-attachment=<reference>** **-configuration=<Latest Released>** to attach the related released objects (if any). Possible values are:

- **target**

Constraint rules are attached as target objects. This is the default value.

- **reference**

Constraint rules are attached as reference objects.

Note

If another revision of the same configurator object thread is already attached to this workflow (either as target or reference), the handler silently skips the object. That is, the handler does not attach a second revision of the same thread.

-configuration

Specifies whether to attach the **Latest Working** or **Latest Released** revisions. If any subsequent workflow handler depends on constraint rules to be attached, for example, **CFG0-attach-rule-variability**, either as a reference or as a target attachment, use this argument to configure two instances of this handler in the same workflow process. In such cases, the first handler is configured with **-attachment=<target>** **-configuration=<Latest Working>** in order to attach the working revisions (if any). It is followed by the same handler configured with **-attachment=<reference>** **-configuration=<Latest Released>** to attach the related released objects (if any). Possible values are:

- **Latest Working**

The most recently created revision that does not have any release status is attached. This is the default value.

- **Latest Released**

The most recently released revision is attached. Use this setting with care as there could be a large number of released constraint rules to attach.

-attachedConfiguratorContext

Specifies whether **Configurator Context** items that are attached to the workflow process should be used to filter constraint rules. This argument can be used as a filter to attach only constraint rules that are targeting product contexts, which are attached to the workflow process. This is useful when releasing variant features or families that are also used in constraint rules for other contexts: Filtering by configurator context prevents from accidentally attaching (and hence releasing) additional constraint rules for the configurator contexts that are not intended. Possible values are:

- **false**

The configured revision of all constraint rules are attached, irrespective of their **Configurator Context** item scope. This is the default.

- **true**

The configured revision of constraint rules are attached that reference a **Configurator Context** item that is attached to this workflow, for example, as a reference attachment. If no **Configurator Context** items are found to be attached to the workflow process, no additional constraint rules are added to the workflow process.

-debug

Whether or not to log status information to the syslog file. Possible values are:

- **false**

No status information is written to the syslog file. This is the default value.

- **true**

Status information is written to the syslog file for debugging purposes.

Placement

A typical placement is below the **EPM-create-status** action handler that creates and adds the release status to the workflow process. In many cases it is useful to add the **CFG0-attach-constraint-rules** action handler followed by a **CFG0-attach-rule-variability** action handler.

Restrictions

None

Examples

This example illustrates the use of the handler that attaches **Latest Working** revisions of constraint rules as target attachments so that they are processed along with the values and families that are already attached to the workflow. The list of constraint rules to attach is not filtered by **Configurator Context**.

Argument	Values
-attachment	target
-configuration	Latest Working
-attachedConfiguratorContext	false

CFG0-attach-families

Description

Attaches to the workflow process variant option families that are referenced by features in the target attachment or reference attachment folder. The **-configuration** argument specifies whether to attach the **Latest Working** or **Latest Released** revisions of the variant option families.

For more information, see *Create workflows to release configurator data in Product Configurator — Deployment and Administration*.

Syntax

CFG0-attach-families

[-attachment = {target | reference}]

[-configuration = { Latest Working | Latest Released}]

[-debug = {false | true}]

Arguments

-attachment

Attachment type with which the objects are attached to the workflow process. Possible values are:

- **target**

Variant option families are attached as target objects. This is the default value.

- **reference**

Variant option families are attached as reference objects.

Note

If another revision of the same configurator object thread is already attached to this workflow (either as target or reference), the handler silently skips the object. That is, the handler does not attach a second revision of the same thread.

-configuration

Specifies whether to attach the Latest Working or Latest Released revisions. Possible values are:

- **Latest Working**

The most recently created revision that doesn't have any release status is attached. This is the default value.

- **Latest Released**

The most recently released revision is attached.

-debug

Whether to log status information to the syslog file. Possible values are:

- **false**

No status information is written to the syslog file. This is the default value.

- **true**

Status information is written to the syslog file for debugging purposes.

Placement

A typical placement is below the **EPM-create-status** action handler that creates and adds the release status to the workflow process. In many cases, it is useful to add the **CFG0-attach-families** action handler between a **CFG0-attach-rule-variability** handler and a **CFG0-attach-familygroups** handler.

Restrictions

None

Examples

This example illustrates the use of the handler that attaches **Latest Released** revisions of variant option families for the features in this workflow process as reference attachments so that they are processed along with the features that are already attached to the workflow.

Argument	Values
-attachment	reference
-configuration	Latest Released

CFG0-attach-familygroups

Description

Attaches to the workflow process variant option groups that reference variant option families in the target attachment or reference attachment folder. The **-configuration** argument specifies whether to attach the **Latest Working** or **Latest Released** revisions of the variant option families.

Note

Group objects are not subject to revision rule configuration. You cannot revise groups.

For more information, see *Create workflows to release configurator data in Product Configurator — Deployment and Administration*.

Syntax

```
CFG0-attach-familygroups  
[-attachment = {target | reference}]  
[-configuration = {Latest Working | Latest Released}]  
[-debug = {false | true}]
```

Arguments

-attachment

Attachment type with which the objects are attached to the workflow process. Possible values are:

- **target**

Variant option groups are attached as target objects. This is the default value.

- **reference**

Variant option groups are attached as reference objects.

Note

If another revision of the same configurator object thread is already attached to this workflow (either as target or reference), the handler silently skips the object. That is, the handler does not attach a second revision of the same thread.

-configuration

Specifies whether to attach the **Latest Working** or **Latest Released** revisions. Possible values are:

- **Latest Working**

The most recently created revision that doesn't have any release status is attached. This is the default value.

- **Latest Released**

The most recently released revision is attached.

- **attachedConfiguratorContext**

Specifies whether relevant **Configurator Context** items for which allocation objects are to be added are attached to this workflow process. This argument can be used as a filter to attach only the allocation objects that are targeting product contexts, which are attached to the workflow process. This is useful when releasing variant features, families, or groups that are allocated to multiple contexts: Filtering by configurator context prevents from accidentally attaching (and hence releasing) additional allocations to other configurator contexts that are not intended. Possible values are:

- **false**

Configured revisions for allocations to all **Configurator Context** items will be attached. This is the default value.

- **true**

The configured allocation revisions to attach are filtered by the **Configurator Context** items attached to this workflow. If no **Configurator Context** items are found to be attached to the workflow process, no additional allocations are added to the workflow process.

- **debug**

Whether or not to log status information to the syslog file. Possible values are:

- **false**

No status information is written to the syslog file. This is the default value.

- **true**

Status information is written to the syslog file for debugging purposes.

Placement

A typical placement is below the **EPM-create-status** action handler that creates and adds the release status to the workflow process. In many cases, it is useful to add the **CFG0-attach-familygroups** action handler between a **CFG0-attach-families** handler and a **CFG0-attach-allocations** handler.

Restrictions

None

Examples

This example illustrates the use of the handler that attaches **Latest Released** revisions of variant option groups for the variant option families in this workflow process as reference attachments so that they are processed along with the variant option families that are already attached to the workflow.

Argument	Values
-attachment	reference
-configuration	Latest Released

CFG0-attach-rule-variability

Description

Attaches features and families that are referenced by a constraint rule. Such constraint rules may be located in the target attachment or reference attachment folder. The **-configuration** argument specifies whether to attach the **Latest Working** or **Latest Released** revisions of the values, families, and groups.

Note

A configurator constraint rule references the option family if the family has free-form values. Otherwise, it references the option value directly.

For more information, see *Create workflows to release configurator data in Product Configurator — Deployment and Administration*.

Syntax

```
CFG0-attach-rule-variability
[attachment = {target | reference }]
[-configuration = {Latest Working | Latest Released }]
[-attachConfiguratorContext = {false | true }]
[-debug = { false | true }]
```

Arguments

-attachment

Attachment type with which the objects are attached to the workflow process. Possible values are:

- **target**

Variant option families and values are attached as target objects. This is the default value.

- **reference**

Variant option families and values are attached as reference objects.

Note

If another revision of the same configurator object thread is already attached to this workflow (either as target or reference), the handler silently skips the object. That is, the handler does not attach a second revision of the same thread.

-configuration

Specifies whether to attach the **Latest Working** or **Latest Released** revisions. Possible values are:

- **Latest Working**

The most recently created revision that has no release status is attached. This is the default value.

- **Latest Released**

The most recently released revision is attached.

-attachConfiguratorContext

Specifies whether **Configurator Context** items that are referenced by the constraint rules in this workflow process should be attached as **reference** attachments.

Note

The **Configurator Context** items are always added as **reference** attachments. This behavior is not affected by the **-attachment** parameter value.

Use this argument as a filter to attach only features and families that are allocated to intended product contexts.

This argument is evaluated after attaching the product context in response to the **-attachConfiguratorContext** argument (if any). The handler first attaches the product contexts of each rule as a **reference** attachment and then filters the set of features and families in each constraint rule if both **-attachConfiguratorContext** and **attachedConfiguratorContext** arguments are provided.

This is useful when releasing multi-context constraint rules. Not all variant features and families in a multi-context constraint rule are allocated to all product contexts. Use this argument when you want to release multi-context constraint rules for a specific subset of the product context scope that this constraint rule is targeting. Filtering by configurator context prevents accidentally attaching (and hence releasing) variant features and families for a configurator context that is not intended.

Options are:

- **true**

Configurator Context items that are referenced by the constraint rules in this workflow process are attached as **reference** attachments. This is the default value.

- **false**

No additional **Configurator Context** items are attached.

-debug

Whether to log status information to the syslog file. Possible values are:

- **false**

No status information is written to the syslog file. This is the default value.

- **true**

Status information is written to the syslog file for debugging purposes.

Placement

A typical placement is below the **EPM-create-status** action handler that creates and adds the release status to the workflow process. In many cases, it is useful to add the **CFG0-attach-rule-variability** action handler between a **CFG0-attach-constraint-rules** handler and a **CFG0-attach-families** handler.

Restrictions

None

Examples

This example illustrates the use of the handler that attaches **Latest Working** revisions of features and families that are used in the constraint rules in this workflow process as target attachments so that they are processed along with the constraint rules that are already attached to the workflow. The list of **Configurator Context** items to which the constraint rules apply are added as reference attachments to this workflow.

Argument	Values
-attachment	target
-configuration	Latest Working
-attachConfiguratorContext	true

CFG0-find-constraint-conflict

Description

Creates the report of constraint conflicts for a given variant rule and its subtypes. The generated report is attached to the workflow process as a reference to execute this handler.

The solve profile to find the constraint conflicts are taken from the input variant rule. If no session info (solve profile) is saved on the variant rule, the system displays an error.

Similarly, the other session information such as revision rule and rule date are considered from the session information saved on the input variant rule.

If the argument values mentioned below are provided, those values override the values from session information.

The results of workflow handler are in the form of a **.json** file report with the specific schema as below:

For more information, see *Create workflows to release configurator data* in *Product Configurator — Deployment and Administration*.

- `<TC_DATA>\json\configurator\schema\CFG0_configurator_definitions.json`
- `<TC_DATA>\json\configurator\schema\CFG0_report_constraint_conflicts.json`

After you generate a report using the workflow handler, you can refer these schemas to get more information about the report such as constraints, severity, conflicts, session info, and variant rule name.

Syntax

CFG0-find-constraint-conflicts

-revisionRuleName=<revision-rule>
-ruleDate=
 <rule date>

Arguments

-revisionRuleName
 Specifies the revision rule for generating the report.

If the value is empty, then the revision rule from the input variant rule is considered.

-ruleDate
 Specifies the rule date for generating the report.

The date should be in the **ISO 8601** format.

Examples

Variability data:

Family	Values
Engine	Diesel
	Petrol
	Hybrid
Powertrain	Manual
	Automatic

Configurator rules:

DefaultRule D1 = (Powertrain=Manual → Engine=Diesel)

DefaultRule D2 = (Powertrain=Manual → Engine=Petrol)

Variant rule:

Variant rule VR1 = Powertrain = Manual

Note

All data is configured for **Latest Working Revision Rule**.

When we start the workflow on variant rule **VR1**.

The workflow handler output report contains the conflicts between **D1** and **D2**.

CM-Approve-ECO-Markup-Handler

Description

Applies the markups created under the target object (ECN) to a new revision of the corresponding impacted item revisions, provided the following conditions are met.

- The impacted item revision is not currently checked out.
- Another revision may be created on which to apply the markup.

Note

To verify if you can create another revision, the handler checks **MaxAllowedWorkRevsForItemCreate**, **MaxAllowedWorkRevsForItemCopyRev**, and **MaxAllowedWorkRevsItemCpRevExist**.

- The **BOMViewRevision** of the impacted item revision is of the default view type.
- The **BOMViewRevision** of the impacted item revision has a release status attached.

Note

The item revision and the **BOMViewRevision** require a release status.

If there is an error during **Revise** or **Apply Markup**, the impacted item revision and its markup will revert to their original state.

The **BOMViewRevision** of the new item revision will have the same release status attached.

Note

Designed for use in the **Review and Apply BOM Markups** process template.

Syntax

CM-Approve-ECO-Markup

Arguments

None.

Placement

Requires no specific placement. Typically placed on the **Complete** action.

Restrictions

The item revision and the **BOMViewRevision** require a release status.

CM-baseline-solution-item-revisions-on-change-notice

Description

Performs a smart baseline on the assemblies of any item revisions listed as **Solution Items** on target **ChangeNoticeRevisions**.

Syntax

CM-baseline-solution-item-revisions-on-change-notice

[-baseline_rev_rule=<revision rule name>]

[-baseline_process=<workflow process name>]

Arguments

-baseline_rev_rule

Defines the name of the revision rule used to configure the item revision's assembly structure for baselining.

Will use the default **Structure Manager** revision rule if omitted.

-baseline_process

Defines the name of the workflow process used to release the baseline revisions.

Will use the default baseline process **TC Default Baseline Process** if omitted.

Placement

Place on any action. Typically attached to the **Complete** action.

Restrictions

None.

CM-cancel-change-notice-handler

Description

Provides the ability to cancel a change notice (ECN) prior to its approval at any stage of the workflow. However, once an ECN is canceled, the solution objects within is also get canceled and cannot be reused.

Syntax

CM-cancel-change-notice-handler -include_type

Arguments

-include_type – List of solution item types that need to be canceled.

Placement

Place this workflow handler on the **Complete** action:

Argument	Values
-include_type	WorkspaceObject (set by default) You can include different business objects in a comma-separated list. For example, you can set the value as Part Revision, Design Revision .

Restrictions

None

CM-Cancel-ECO-Markup-Handler

Description

Sets the active markups for the impacted assemblies of the target object (ECN) to inactive.

If an error occurs, the markup will revert to its original state.

Note

Designed for use in the **Review and Apply BOM Markups** process template.

Syntax

CM-Cancel-ECO-Markup

Arguments

None.

Placement

Requires no specific placement. Typically placed on the **Complete** action.

Restrictions

None.

CM-concurrent-notification

Calculates concurrent changes and identifies associated participants. Depending on its placement and the corresponding JSON file arguments, it sends alerts when a reviewer approves a concurrent change or when the change is released.

Syntax

CM-concurrent-notification

Arguments

-type

(Required)

Specifies the nature of the task this handler is placed on.

- **Review**

Use this option if the handler is placed on the Perform Signoff task.

- **Release**

Use this option if the handler is placed on the task that releases the Change object.

Placement

Place this handler on the **Complete** action of the Perform Signoff task, as well as the task that releases the change notice.

Restrictions

None.

Examples

- This example sends notifications once the Perform Signoff task is completed.

Argument	Values
-type	Review

- This example sends notifications once the task that releases the change is completed.

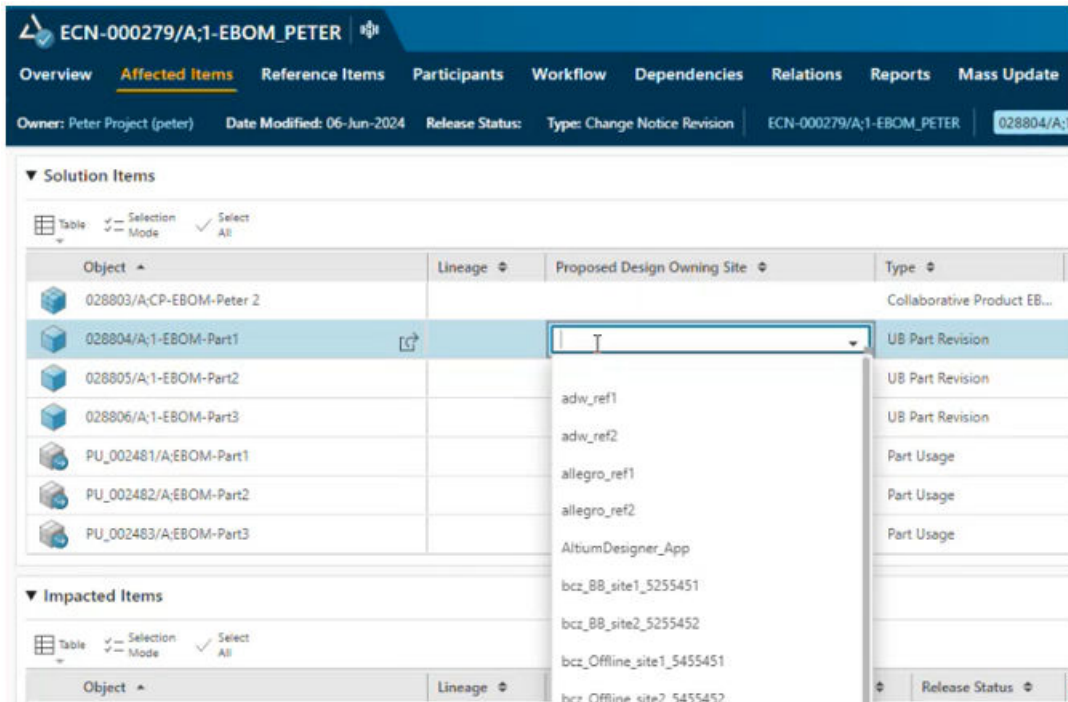
Argument	Values
-type	Release

CM-create-multisite-change-notice-objects

Description

This handler is used for transferring change notice objects from an EBOM to DBOM in a multisite environment to facilitate part design, authoring, and alignment. This handler does the following:

1. Creates new DBOM change notice objects for each DBOM site. Configure the DBOM site in the **Proposed Design Owning Site** column of the **Solutions Items** table in the master EBOM ECN.



2. Attaches the respective EBOM solutions to the newly created DBOM change notice objects.
3. Transfers each newly created DBOM change notice object to the corresponding remote DBOM site.
4. After transfer to the DBOM site, the workflow process automatically starts on the DBOM change notice object.

Syntax

CM-create-multisite-change-notice-objects

Arguments

Mandatory arguments include:

dbom_workflow_template_name

The value for this argument is the name of the workflow template used to create the process on the DBOM site.

callback_workflow_task_to_complete

The value of this argument is the name of the Workflow task from the EBOM process template that should be automatically triggered to complete when the DBOM workflow finishes and the DBOM change is released.

An **optional** argument includes:

change_type

New DBOM change notice objects are created based on the type specified by this argument's value. For example, if the argument value is **CM0SimpleChange**, then new DBOM change notice objects of the **Simple Change** type are created. If this argument is not provided, the system defaults to creating change objects of the **Change Notice** type.

Placement

No specific placement is required.

Examples**Example 1**

This example creates new DBOM ECN(s) of type **Change Notice** and initiates new remote process(es) using the template **DBOM_Multisite_Change_Process**. A callback task named **Wait for DBOM ECN Release** is present on the EBOM master workflow and is triggered for completion once the DBOM process is completed.

Argument	Values
- dbom_workflow_template_name	DBOM_Multisite_Change_Process
- callback_workflow_task_to_complete	Wait for DBOM ECN Release

Example 2

This example creates new DBOM ECNs of type **Simple Change** and initiates new remote processes using the **DBOM_Multisite_Master_Template**. The callback task named **Wait for DBOM ECN Release** on the EBOM master workflow is triggered for completion once the DBOM process is finished.

Argument	Values
- dbom_workflow_template_name	DBOM_Multisite_Change_Process
- callback_workflow_task_to_complete	Wait for DBOM ECN Release

CM-derive-change

Description

A user would be able to update COTS workflow templates using the **CM-derive-change** handler to automate deriving a change request from a problem report, or a change notice from a change request. Setting **auto_derive** to *<true>* will automatically derive the proper result when submitted using the default workflow template.

Syntax

CM-derive-change

[-from_type=<<valid change object type>>]

[-to_type=<<valid change object type>>]

[-synopsis=<<valid string>>]

[-description=<<valid string or empty>>]

[-template=<<process-template-name or default>>]

Arguments

-from_type

Target object type to derive from. **Derive** will derive from all workflow targets of this type, or subtypes of this type.

-to_type

The change object type to be created by the **Derive** operation. Must be a valid change type that can implement the designated **-from_type**.

-synopsis

Synopsis to be assigned to create change object.

-description

Description to be assigned to created change object.

-template

If the value component of the **-template** argument specifies the name of a workflow template, the newly created change object is submitted to a workflow using that defined workflow template.

If the value component of the **-template** argument is empty, the newly created change object is submitted to a workflow using the default workflow template of the created change type.

Note

If the **-template** argument is removed, then the newly created change object will not be submitted to a workflow.

Placement

Place on any action. Typically attached to the **Complete** action.

Restrictions

Only change objects can be included as a target of the workflow. If the workflow includes a non-Change target object, it will fail.

CM-inactivate-edit-context

Description

Deactivates the change space associated with the change notice revision of the target.

Syntax

CM-inactivate-edit-context

Arguments

None.

Placement

Requires no specific placement.

Restrictions

None.

CM-merge-changes

Description

Automatically merges the redlined solution items from the target/released change object of the workflow into the concurrent work-in-progress change.

CM-merge-changes utilizes two important preferences that define the course of action for change notice merge behavior. The **CM_automerge_conflicting_changes** preference determines if conflicting changes merge automatically or manually, with **true** as the default for automatic merging.

If **CM_automerge_overwrite_conflicting_changes** is set to **target**, the target assembly remains unchanged, the merge status is marked complete, and an auto-merge notification is sent while enabling merge candidates. See Managing change notice conflicts during an auto-merge and Conflicts resulting from concurrent change notices for more information.

Syntax

CM-merge-changes

```
[-recipient=
{OS:<user-name>
| user:<user>
| person:<person>| addresslist:<value>

| resourcepool:<group::role>

| allmembers:<group::role>
| $USER
| $REVIEWERS | $PROPOSED_REVIEWERS
| $RESPONSIBLE_PARTY
| $PROPOSED_RESPONSIBLE_PARTY
| $PROJECT_ADMINISTRATOR |
| $PROJECT_TEAM_ADMINISTRATOR
$PROJECT_AUTHOR | $PROJECT_MEMBER<[group::role]>
| $TARGET_OWNER | $PROCESS_OWNER
| $RESOURCE_POOL_ALL | $RESOURCE_POOL_NONE
| $RESOURCE_POOL_SUBSCRIBED
| $REQUESTOR
| $ANALYST
| $CHANGE_SPECIALIST1
| $CHANGE_SPECIALIST2
| $CHANGE_SPECIALIST3
| $CHANGE_REVIEW_BOARD
| $CHANGE_IMPLEMENTATION_BOARD]
```

```
[-notify_to = {target | source | both}]
```

[-notify_on= {success| failure | both}]

Arguments

The value can be any of the following:

recipient

(Required) Specifies the task reviewers receiving notification. Any surrogates for the specified users are also notified. Accepts one of the following values:

- **OS**

Sends a notification to the OS user name specified.

<user-name> is a single valid user name.

- **user**

Sends notification to the user specified.

user is a single valid Teamcenter user ID.

- **person**

Sends a notification to user whose name is specified.

person is a single valid Teamcenter person.

If the person's name includes a comma, you must include an escape character (\) to add the correct person. For example, to use **wayne, joan**:

-recipient=person:wayne\, joan

- **addresslist**

Sends a notification to all members of the address list.

value is a valid Teamcenter address list.

- **resourcepool**

Sends notification to members of a group/role combination. Notification is sent to all members, subscribed members, or none based on the **EPM_resource_pool_recipients** preference.

The preference value can be overridden with:

\$RESOURCE_POOL_ALL

\$RESOURCE_POOL_SUBSCRIBED

\$RESOURCE_POOL_NONE

You can define role in groups in the form of **group::**, **group::role** or **role**.

value is a valid Teamcenter resource pool and these keywords:

\$GROUP: Current user's current group.

\$ROLE: Current user's current role.

\$TARGET_GROUP[*type*]: Owing group of the first target object of the specified type. The type value is optional. If not specified, the first target is used.

\$PROCESS_GROUP: Owing group of the workflow process.

- **allmembers**

Sends notification to all members of a group/role combination.

value is all members of a Teamcenter group and role. You can define a role in groups in the form of **group::, group::role** or **role**.

Accepts valid Teamcenter resource pool names and these keywords:

\$GROUP, \$ROLE, \$TARGET_GROUP and **\$PROCESS_GROUP**.

Note that the **\$ROLE_IN_GROUP** keyword (formerly **\$ROLEINGROUP**) cannot be used. Use **allmembers:\$GROUP::\$ROLE** instead.

- **\$USER**

Sends email to the current user.

- **\$REVIEWERS**

Builds a list of all users who are reviewers in the same task level as the current reviewer and sends email to all of them.

- **\$PROPOSED_REVIEWERS**

Sends email to all members assigned as the proposed reviewers of the first target object in the workflow process.

- **\$RESPONSIBLE_PARTY**

Sends email to the designated responsible party for the task. If you use **\$RESPONSIBLE_PARTY**, add the handler to the **Start** action of the task, not the **Assign** action.

- **\$PROPOSED_RESPONSIBLE_PARTY**

Sends email to the member assigned as the proposed responsible party of the first target object in the workflow process.

- **\$PROCESS_OWNER**

Sends email to the workflow process owner.

- **\$TARGET_OWNER** [*type*]

Sends email to the target owner of the first target of the specified type. The *type* value is optional. If it is not specified, the first target is used.

- **\$PROJECT_ADMINISTRATOR**

\$PROJECT_TEAM_ADMINISTRATOR

\$PROJECT_AUTHOR

\$PROJECT_MEMBER[*group::role*]

These values dynamically evaluate project team members belonging to the role specified in the argument value and send a notification to them. The project team is determined by the project team associated with the first target object.

If the **\$PROJECT_MEMBER[group::role]** argument is specified, only the project members of the qualifying projects which belong to the specified group and role are selected as recipients. If the group and role are not specified, all the project members from qualifying projects are selected.

- **\$REQUESTOR, \$ANALYST, \$CHANGE_SPECIALIST1, \$CHANGE_SPECIALIST2, \$CHANGE_SPECIALIST3**

\$CHANGE_REVIEW_BOARD, \$CHANGE_IMPLEMENTATION_BOARD

Dynamically resolves to the user or resource pool associated with the first Change target object in the process. The particular user or resource pool is determined by the role specified in the argument value.

If custom participants are defined by the customer, those participants can be used as recipients.

Change-related keywords apply only to change objects. If the process does not contain a change object as a target, the argument resolves to null.

Change Manager does not need to be enabled before these keywords take effect, but during installation, **Change Management** must be selected in Deployment Center.

- **RESOURCE_POOL_ALL**

Identifies all members of the resource pool.

This argument has an effect only when it is used along with **\$RESPONSIBLE_PARTY** or any dynamic participant, such as **\$CHANGE_IMPLEMENTATION_BOARD**, which point to a resource pool.

When this argument is used along with **\$RESPONSIBLE_PARTY**, and if a resource pool is assigned as responsible party, the email is sent to all members of that resource pool.

- **\$RESOURCE_POOL_NONE**

Identifies all members of the resource pool.

This argument has an effect only when it is used along with **\$REVIEWERS, \$UNDECIDED, or \$RESPONSIBLE_PARTY**.

When this argument is used along with **\$REVIEWERS** or **\$UNDECIDED**, and if a resource pool is assigned as a reviewer, email is not sent to members or subscribers of the resource pool.

When this argument is used along with **\$RESPONSIBLE_PARTY**, and if a resource pool is assigned as responsible party, the email is not sent to members or subscribers of resource pool.

- **\$RESOURCE_POOL_SUBSCRIBED**

Identify the users who have subscribed to resource pool.

This argument has an effect only when it is used along with **\$RESPONSIBLE_PARTY**.

When this argument is used along with **\$RESPONSIBLE_PARTY**, and if a resource pool is assigned as a responsible party, the email is sent to users who have subscribed to the resource pool.

If the **\$RESOURCE_POOL_XXXXX** argument is not defined and the **\$RESPONSIBLE_PARTY** arguments are used for a case where assignments are made to resource pools, the email is sent using the **EPM_resource_pool_recipients** preference.

EPM_resource_pool_recipients can take one of the following values:

all

Sends mail to all members of resource pool.

none

Does not send a mail to members or subscribers of resource pool.

subscribed

Sends mail to Teamcenter users who have subscribed to resource pool.

If the **\$RESOURCE_POOL_XXXXX** argument is defined, the argument takes precedence over preference value.

If this argument is not defined and the **EPM_resource_pool_recipients** preference is not set, **subscribed** is considered the default value.

The **-recipient** and **-notify** arguments must be used together. If you use one argument, you must use the other.

-notify_to

(Optional) This argument takes effect when the **recipient** argument is provided. It determines whether notifications are sent to the recipients of the source change notice, the target change notice, or both, after completing an auto-merge.

Use **notify_to** when you have a dynamic participant as a value in the same **recipient** argument.

-notify_on

(Optional) This argument takes effect when the **recipient** argument is provided.

This argument specifies the conditions or events that will trigger notifications when using the auto-merge feature. This could include:

- The successful completion of the auto-merge.
- Any conflicts or errors encountered during the auto-merge.

By setting this argument, users are informed about the success or failure of the auto-merge process, which lets them take appropriate actions if necessary.

Examples

- This example will send notifications to the recipients of both the source and target change notices after completing an auto-merge.

Argument	Values
-recipient	user@example.com
notify_to	source and target

- The recipient argument specifies the email address of the user to notify, and the **notify_on** argument is set to **both**, meaning notifications will be sent to the recipients of both the source and target change notices after completing an auto-merge.

Argument	Values
-recipient	user@example.com
-notify_on	both

CM-promote-change-notice

Description

Performs the following operations within a transaction and rolls back all changes if there is a failure:

1. Applies release status to **ChangeNoticeRevision** workflow target objects.
2. Receives any change space data objects from the **ChangeNoticeRevision** POM Space and promotes or shares them to public data objects.

Note

If specific object types contained in the POM space require pre- or post-promote validation, this can be accomplished by overriding the following methods on the respective types:

- **fnd0ValidateBOTypePrePromote**
- **fnd0ValidateBOTypePostPromote**

3. Applies release status to all of the solution items of the **ChangeNoticeRevision** target objects and any other targets not addressed in step 1.

Note

If specific object types require pre-release validation, this can be accomplished by overriding the following method on the respective type:

- **fnd0ValidateBOTypeForRelease**

Note

The arguments and their effect on the behavior are all related to how the release status is applied to the target objects and **ChangeNoticeRevision** solution objects. The handler arguments are a copy of the arguments and processing behavior of the **EPM-set-status** handler.

Syntax

```
CM-promote-change-notice -action=append | replace | rename | delete [-status=<old_name>],[
[-new_status=<new_name> ] [-retain_release_date] [-set_effectivity] [-status_not_shared] [-
promote=share]
```

Arguments

-action

append

Attaches the status objects from the root task to the target objects, not impacting any previous status objects applied to the same targets.

replace

Deletes all existing status objects attached to target objects and attaches the status objects from the root task to the target objects.

rename

Renames an existing status object attached to the target objects from **old_name** to **new_name**.

- If a status object with the **old_name** status is not found, it renames the last status object attached to the target objects.

If the target object has an existing status, the status object is renamed from **old_name** to **new_name**.

delete

Deletes the status **status_name** specified by the **-status** argument from the target object.

- If the **delete** argument is not used in combination with the **-status** argument, all status objects are removed from the target objects.
- If the status objects being removed from the target objects were created in the same workflow, they are attached to the root task upon creation and are not removed from the root task by this handler.

-status

Used with the **-action** argument to define the status.

- If the action is **append** or **replace** and the status by the name given is not present on the root task, it will create a new status with this name and attach it to the root task.
- If the action is **delete**, it deletes the status objects from the target object but does not delete it from the root task.
- If the action is **rename**, it renames the status objects to the new value specified in **-new_status**.

The value provided should be the name of a status type already defined in the Business Modeler IDE, not the display name.

-new_status

Specifies the new name for the status object.

- The name provided should be the name of a status type already defined in the Business Modeler IDE, not the display name.
- This argument is only used in case of **rename** option for **-action** argument.
- If the status type is not already defined, a status object is not based on a status type, which means that effectivity and configuration may not work against it.

-retain_release_date

Retains the original release date on the target object if it had previously been released.

Note

This option is not valid when **-action=replace** is used.

-set_effectivity

When used, the system creates the open-ended date effectivity with release date as start date.

-status_not_shared

The default behavior is to share a single release status object reference for all target objects. When this argument is present, it changes that behavior and an individual copy of the release status object is added to each target object.

-promote
share

Specifies that the change space data objects from the **ChangeNoticeRevision** POM Space will be shared to public.

Any value other than **share** promotes the change space data objects from the **ChangeNoticeRevision** POM Space to public.

Placement

Place on any action. Typically attached to the **Complete** action.

Restrictions

If no argument is supplied or if an argument other than the one specified is supplied to the handler, the default behavior is to treat it as an action **append** argument.

If **replace** is used and there is more than one status object attached to the root task, the status on the target objects is replaced by the latest status on the root task.

Examples

- This example adds the status object of the root task to the target object.

Argument	Values
-action	append

- This example creates a new status with this name and attaches to the root task if status by the name given is not present on the root task already.

Argument	Values
-action	append
-status	released

- This example adds the status object of the root task to the target object and retains the original released date of the target object.

Argument	Values
-action	append
-retain_release_date	

- This example replaces all existing status objects with the status object of the root task.

Argument	Values
----------	--------

-action	replace
---------	---------

- This example replaces existing status objects with the status object of the root task. It also sets an open-ended effectivity with release date as the start date on the new status object.

Argument	Values
----------	--------

-action	replace
---------	---------

-set_effectivity	
------------------	--

- This example renames all the status objects named **pre-released** to the name of the new status object, **released**.

Argument	Values
----------	--------

-action	rename
---------	--------

-status	pre-released
---------	--------------

-new_status	released
-------------	----------

- This example deletes all status objects from the target object but does not delete it from the root task.

Argument	Values
----------	--------

-action	delete
---------	--------

- This example deletes a **released** status from the target object but does not delete it from the root task.

Argument	Values
----------	--------

-action	delete
---------	--------

-status	released
---------	----------

- This example takes the release status attached to root task and creates an individual copy of the release status object for each target object.

Argument	Values
----------	--------

-action	append
---------	--------

-status_not_shared	
--------------------	--

- This example creates a new status with name **released** and attaches it to the root task if status by the name given is not present on the root task already. Also it creates an individual copy of the release status object for each target object.

Argument	Values
-action	append
-status_not_shared	
-status	released

- This example shares the change space contents to public and attaches **shared** status to the root task. Each time the **shared** operation is performed, the **shared** status is replaced and a copy of the release status object for each target object is created.

Argument	Values
-action	replace
-status_not_shared	
-status	Cm0TC Shared
-promote	share

CM-propagate-change-notice-effectivity

Description

Propagates the release effectivity of an active change notice as the element effectivity of the part usages that are added or updated in a collaborative product engineering BOM.

The effectivity propagation is enabled by using the **Cm0UsageEffectivityPropagationMode** business object constant.

Syntax

CM-propagate-change-notice-effectivity -target

Arguments

-target – A change notice revision object.

Placement

Place the following workflow handlers on the **Start** action:

- EPM-attach-related-objects

Argument	Values
-relation	IMAN_specification
-attachment	target

- EPM-attach-related-objects

Argument	Values
-relation	PSBOMViewRevision
-attachment	target

- EPM-assign-team-selector

Argument	Values
-assignee	\$PROCESS_OWNER

- EPM-auto-assign-rest

Restrictions

None

CM-show-concurrent-warning-on-task

Description

Calculates simultaneous changes and alerts reviewers with a warning. The warning appears on the Task Perform panel.

When the Start action of a Signoff task is initiated, a message indicates that there are pending changes for certain affected items. Approving this change could result in the conflicting redlines of open changes not being displayed.

When the Start action of a Condition task is initiated, a message indicates that there are pending changes for certain affected items. Releasing this change might hide the conflicting redlines of open changes.

Syntax

CM_show_concurrent_warning_on_task

Arguments

None.

Placement

Place on the Start action of Perform Signoff task or the Start action of a Condition task.

Restrictions

For workflows acting on an ECN target.

CM-transfer-multisite-parent-structure

Description

Replicates the assemblies of solution item(s) associated with the target (for example, a change notice) to the proposed design owning site linked with the solution items.

Syntax

CM-transfer-multisite-change-notice-objects

Arguments

None.

Placement

Requires no special placement.

CM-validate-ECO-Markup-Handler

Description

Verifies that the **Review and Apply BOM Markups** process template contains at least one target object.

Note

If the target object is missing, it triggers a **PS_markup_target_invalid** error.

Syntax

CM-validate-ECO-Markup

Arguments

None.

Placement

Requires no specific placement. Typically placed on the **Start** action of the root task.

Restrictions

None.

CONFMGMT-cut-back-effectivity

Description

Reduces the effectivity range of problem item objects attached to a change object so it does not overlap with the combined effectivity range of the solution items. This facilitates the release of solution items to replace problem items for a given effectivity range.

Note

This handler should be used only for 4th Generation Design (4GD) and Product Configurator objects.

For example, a cast component C is a solution item for a forged component F, a problem item with a unit effectivity of 1 through 10 in 4G Designer on 4GD data. C is assigned the same effectivity (unit 1 through 10) because it has the same purpose. To replace C with F with unit effectivity 3 through 10, a change notice is created that tracks F as a problem item and C as a solution item. The change notice is assigned a unit effectivity of 3 and up. The handler applies the change notice effectivity to the solution item and then reduces the effectivity range of the problem item. As a result, C has an effectivity range of 3 through 10 and F's effectivity is reduced to 1 through 2. For every unit in the range of 1 through 10, either C or F is effective. The effective ranges of C and F neither overlap nor do they have a gap.

The effectivity range of the change is determined either by the release status attachment of the workflow process or by the effectivity range on the change object using **EffectivityConfigurable** behavior.

If the process does not have a release status attachment, the release statuses of the change object are used. An error occurs if multiple release statuses with effectivity data are found and handler arguments are used that require the definition of the effectivity range of the change object. By default, the system uses the effectivity range of the release statuses, unless user provides the **useECNEffectivity** argument.

If the **useECNEffectivity** argument is used, the effectivity range of the change object is determined as the effectivity of the change object using **EffectivityConfigurable** behavior. An error is returned if the change object does not have **EffectivityConfigurable** behavior

EffectivityConfigurable objects with no effectivity data behave as if they had an effectivity condition **Unit=1 OR Unit!=1** (in other words, **TRUE** unless explicitly stated otherwise). For more information, see the **defaultSolveTypePreferenceName** argument.

The effectivity range to be subtracted from a problem item attachment is the combined effectivity range of all **EffectivityConfigurable** objects in the corresponding solution item set. You can use the **designatorProperty** argument to define corresponding sets of solution and problem items. Solution item sets that do not correspond to a problem item set do not affect problem item effectivity ranges. Problem item sets that do not correspond to a solution item will be effected out permanently. Solution items without **EffectivityConfigurable** behavior (for example, datasets) are skipped in the computation of the effectivity range to be subtracted.

The handler only modifies problem item objects exposing **EffectivityConfigurable** behavior, such as **Cpd0DesignElement**. These modifications are not subject to access control rules.

Syntax

CONFMGMT-cut-back-effectivity

```
[ -engineeringChangeTypeName = { ChangeNoticeRevision | <object-type-name> } ]
[ -problemItemRelationshipName = { CMHasProblemItem | <relationship-type-name> } ]
[ -solutionItemRelationshipName = { CMHasSolutionItem | <relationship-type-name> } ]
[ -verifyEffectivity = { NoAction | Compare | Validate } ]
[ -solutionItemEffectivity = { NoAction | ApplyCMEffectivity | MergeCMEffectivity |
ResetToCMEffectivity } ]
[ -designatorProperty = { "" | <property-name> } ]
[ -defaultSolveTypePreferenceName = { "" | <preference-name> } ]
[ -dropEndItemQualification ]
[ -useECNEffectivity ]
```

Arguments

-engineeringChangeTypeName

Sets the type of the target object managing the change. Any object type name is valid as long as there is only one such target attachment and the object type supports the relationship types specified below. The default value is **ChangeNoticeRevision**.

-problemItemRelationshipName

Sets the type name of the relationship that associates objects to be replaced by the objects specified by the **-solutionItemRelationshipName** argument with the change object. The type name must be compatible with the above change object type. The default value is **CMHasProblemItem**, but Siemens Digital Industries Software recommends you use **CMHasImpactedItem** as the relationship name.

-solutionItemRelationshipName

Sets the type name of the relationship that associates objects, which replace the objects specified by the **-problemItemRelationshipName** argument, with the change object. The type name must be compatible with the change object type. The default value is **CMHasSolutionItem**.

-verifyEffectivity

Specifies the action to take with respect to the effectivity range of the change object and its solution item attachments. The action skips solution items for which no **EffectivityConfigurable** effectivity is saved or which do not expose **EffectivityConfigurable** behavior. Possible values are:

- **NoAction**

Takes no action. This is the default.

- **Compare**

Displays a separate warning for every solution item whose effectivity range does not equal the effectivity range of the change object. An error is returned if no effectivity has been saved for the change object.

- **Validate**

Returns an error if any solution item's **EffectivityConfigurable** effectivity range does not equal the effectivity range of the change object. An error is returned if no effectivity has been saved for the change object.

-solutionItemEffectivity

Specifies the action to take for solution item effectivity. Possible values are:

- **NoAction**

Takes no action. This is the default.

- **ApplyCMEffectivity**

Reduces the **EffectivityConfigurable** effectivity range of each solution item to be within the range of the change object (in other words, combines both with a logical **AND**). An error is returned if no release status effectivity is saved for the change object. The result is identical to action **ResetToCMEffectivity** for solution items, for which no **EffectivityConfigurable** effectivity has been saved, or which do not expose **EffectivityConfigurable** behavior.

- **MergeCMEffectivity**

Sets the **EffectivityConfigurable** effectivity range of each solution item to equal the range of the change object for the common effectivity intent; the effectivity range of the solution item having other intents are kept unchanged.

- If effectivity ranges of the solution item and the change object do not have a common effectivity intent, then the solution item effectivity range is extended with the effectivity range of the change object.
- An error is returned if no effectivity range has been saved for the change object or the effectivity range on the solution item or the change object has multiple effectivity intents or intent families.

Note

This mode is supported only with the **-useECNEffectivity** parameter.

- **ResetToCMEffectivity**

Sets the **EffectivityConfigurable** effectivity range of each solution item to equal the range of the release status effectivity of the change object. An error is returned if no release status effectivity has been saved for the change object. The result is identical to action **NoAction** for solution items, which do not expose **EffectivityConfigurable** behavior.

-designatorProperty

Specifies the property to use to group change object attachments into sets for the purpose of replacing problems items with corresponding solution items. These sets are formed by virtue of having a common value for the same property (for example, a logical designator as stored on a partition membership in the preferred partition scheme). If a property is specified, the solution item attachments of the change object are grouped into sets formed by the value for this property. If the property name is an empty string (the default) there is one set for all solution items that corresponds to one set for all problem items.

-defaultSolveTypePreferenceName

By default, **EffectivityConfigurable** objects without effectivity condition behave as if they had an effectivity condition **Unit=1 OR Unit!=1**, that is, equivalent to the Boolean constant **TRUE**. If the value for this argument is different from the empty string (default) it is expected to specify a preference having the same semantics as defined for **TC_Default_Solve_Type** in the **confmgmt** module, which can be used to define whether **EffectivityConfigurable** objects without effectivity condition pass effectivity filter criteria. If the given preference is not found in the scope specified by the **defaultSolveTypePreferenceScope** argument a default solve type of **529** is assumed, that is

solveMismatch|solveFalse|solveInvert except where explicitly otherwise stated. The effectivity range that is assumed for **EffectivityConfigurable** objects without effectivity condition can be configured to be the following:

- **Unit=1 OR Unit!=1**

Equivalent to the Boolean constant **TRUE**, if the solve type specifies that **EffectivityConfigurable** objects without effectivity condition pass effectivity filters.

- **Unit=1 AND Unit!=1**

Equivalent to the Boolean constant **FALSE**, if the solve type specifies that **EffectivityConfigurable** objects without effectivity condition do not pass effectivity filters.

-dropEndItemQualification

(Optional) If provided and if an end item qualification is present, it is dropped and changed to an effectivity condition when it is copied from

- the ReleaseStatus attachment of the workflow process.
- the ReleaseStatus of the attached change notice if the workflow process does *not* have a ReleaseStatus attachment.

-useECNEffectivity

(Optional) If provided, the effectivity range of the change is determined by the effectivity range on the change notice object. The change notice object should carry the effectivity range using **EffectivityConfigurable** behavior. An error is returned if this argument is provided and the change notice object does not have **EffectivityConfigurable** behavior.

The handler argument should be absent, if you manage effectivity for the change notice as described in *About setting change effectivities*. In that case, the system fetches the effectivity (if any) from the release status.

Otherwise, the system fetches effectivity from the ECN itself.

Placement

A typical placement is to precede the **add-status** action handler that attaches the release status to the change object, so that the release status is not attached to the change object if this handler errors out.

Restrictions

None.

Examples

- This example illustrates the use of the handler with a change object type that is available in the Teamcenter foundation template. It configures the handler to reduce the effectivity of the solution item attachments to not be effective beyond the effective range of the change.

Argument	Values
-engineeringChangeTypeName	ItemRevision
-problemItemRelationshipName	IMAN_reference
-solutionItemRelationshipName	IMAN_manifestation
-verifyEffectivity	NoAction
-solutionItemEffectivity	ApplyCMEffectivity
-designatorProperty	object_desc
-defaultSolveTypePreferenceName	TC_Default_Solve_Type
-dropEndItemQualification	None
-useECNEffectivity	None

CONTMGMTS1000D-increment

Description

Sets properties depending on whether the **Civ0DM4Revision** object in a workflow is rejected or released.

- If the **Civ0DM4Revision** object is *rejected*, the **inWork** number is incremented.
- If the **Civ0DM4Revision** object is *released*, the following properties are set:
 - The **issueNum** property is incremented.
 - The **inWork** number is reset to **00**.
 - The **issue_day**, **issue_month** and **issue_year** properties are set to the current date.

Syntax

CONTMGMTS1000D-increment {-inclnWork | -inclIssueNum}

Arguments

-inclnWork

Increments only the **inWork** number. Use this argument for this handler on tasks after reviewers rejections.

-inclIssueNum

Increments **issueNum**, resets **inWork** to **00**, and sets **issue_day**, **issue_month** and **issue_year** to the current date. Use this argument for this handler on a task after the document gets final approval.

Placement

Place on the **Start** or **Perform** action of a normal task.

Restrictions

This handler can be used only with **Civ0DM4Revision** objects.

CONTMGMTS1000D-setQAStatus

Description

Sets the **Quality Assurance Status** property of the data module and updates the XML of the data module to reflect the QA status.

Syntax

CONTMGMTS1000D-setQAStatus -verification=<status>-vertype=<type>

Arguments

-verification

Sets the QA verification status for the data module. You can use one of the following three values:

- **unverified**
- **firstVerification**
- **secondVerification**

-vertype

Sets the verification type of the QA status on the data module. You can use one of the following three values:

- **tabtop**

The content was verified without the physical presence of the equipment or system, such as with design documentation.

- **onobject**

The content was verified by practical demonstration of the procedure on the product.

- **ttandoo**

Both table top and on object verifications have been performed.

This argument is ignored if the **-verification** argument is set to **unverified**:

Placement

Place on the **Start** or **Perform** action of a **Do** task.

Restrictions

This handler can be used only with **Civ0DM4Revision** objects.

CPD-collect-related-items

Description

Collects objects related to design elements from a designated source pseudofolder in a change object. For example, this handler collects the source item revision, parent design elements (such as reuse) and their corresponding source item revisions, and adds them to designated target pseudofolder of the change object.

Syntax

CPD-collect-related-items

```
-source_folder_relation_type=<relation-name>
-processing_type=parent|assembly|default
-destination_folder_relation_type=<relation-name >
```

Arguments

-source_folder_relation_type

Processes the design elements from the pseudofolder of the change object specified by the relation type. The value can be one of the following:

- **CMHasProblemItem**
- **CMHasImpactedItem**
- **CMHasSolutionItem**

-processing_type

Defines how the design elements from the source folder are navigated to collect the related objects. The following modes are supported:

- **parent**

The parent design element corresponding to the input design element and its source object are retrieved and copied to the target pseudofolder of the change object.

- **assembly**

Reuse design element for the input design element and the corresponding source object that are retrieved and copied to the target pseudofolder of the change object.

- **default**

Reuse design element and parent design element for the input design element and their corresponding source objects that are retrieved and copied to the target pseudofolder of the change object.

-destination_folder_relation_type

The related objects collected for the objects in the source folder based on the processing type are copied to the pseudofolder of the change object. Processes the design elements from the pseudofolder of the change object specified by the relation type. The value can be one of the following:

- CMHasProblemItem
- CMHasImpactedItem
- CMHasSolutionItem

Placement

Requires no specific placement.

Restrictions

This handler is specific to design elements as the source objects.

Examples

- This example collects the reuse design element for the input design element in the **Problems** folder of an ECN, which would be a subordinate design element, and the source item revision for them. It then copies them to the **Impacted** folder of the ECN.

Argument	Values
-source_folder_relation_type	CMHasProblemItem
-processing_type	assembly
-destination_folder_relation_type	CMHasImpactedItem

- This example collects the immediate parent for the input design element in the **Problems** folder of an ECN and the source item revision. It then copies them to the same **Problems** folder of the ECN.

Argument	Values
-source_folder_relation_type	CMHasProblemItem
-processing_type	parent
-destination_folder_relation_type	CMHasProblemItem

CPD-update-item-realization

Description

Updates the realization of all reuse design elements attached as references, using the source assembly item revision or installation assembly item revision provided by the target.

If the realization update fails, this handler reports the failed subordinates and corresponding error codes in the log file.

Syntax

CPD-update-item-realization

Arguments

None.

Placement

Place on the **Complete** action of any task.

Restrictions

None.

CPD-where-used-item-revision

Description

Finds all realized reuse design elements in the database for a specific revision of the source item assembly or installation assembly provided by the target in the process. If specified, the search scope is restricted to certain collaborative designs that are attached as references to the process.

All found reuse design elements are added to the references.

Syntax

CPD-where-used-item-revision

Arguments

None.

Placement

Place on the **Complete** action of any task.

Restrictions

None.

CPD0WORKSET_collect_impacted_worksets_handler

Description

Collects disclosed worksets where impacted 4GD design components of the change item are a part of its subset content.

Syntax

CPD0WORKSET_collect_impacted_worksets_handler-disclosureType=<disclosure-workset-type>-sourceRelationType=<source-relation-type>-reuseOnly-targetRelationType=<target-relation-type>

Arguments

-disclosureType

Defines the type of the disclosure workset revision. The default value is **Cpd0WorksetRevision**.

-sourceRelationType

Defines the type of relation to extract the design components attached to the change item. The default value is **CMHasImpactedItem**.

-reuseOnly

Fetches the disclosed workset for reuse design components only, if specified. Else, fetches the disclosed workset for all included design components from **sourceRelationType**.

-targetRelationType

Defines the type of relation to attach the disclose workset revision to the change item. The default value is **CMHasImpactedItem**.

Examples

The following example shows how a workflow set with the **CPD0WORKSET_collect_impacted_worksets_handler** handler finds a disclosed workset and attaches it to a change item.

```

Workset1
  Subset1
    ReuseDE1
  Workset2
    Subset2
      SubordinateDE1
  ChangeNotice1
    Problems
      RootItem1
      ChildItem1
    Impacted
      ReuseDE1

```

SubordinateDE1

Scenario 1

ChangeNotice1 is submitted to the workflow set with the default arguments. In this case, the workflow first gets **ReuseDE1** and **SubordinateDE1** design elements from **ChangeNotice1**. The workflow then locates the worksets, **Workset1** and **Workset2**, that contain these design elements and attaches the worksets to the **Impacted** folder.

ChangeNotice1

Problems

RootItem1

ChildItem1

Impacted

ReuseDE1

SubordinateDE1

Workset1

Workset2

Scenario 2

ChangeNotice1 is submitted to the workflow with **-reuseOnly** and **targetRelationType=CMHasSolutionItem** arguments. In this case, the workflow only finds **Workset1** and attached it to the **Solution** folder.

ChangeNotice1

Problems

RootItem1

ChildItem1

Impacted

ReuseDE1

SubordinateDE1

Solution

Workset1

CSI-propagate-folder-contents

Description

Copies change objects in the change folders to the corresponding schedule task change folders.

Syntax

```
CSI-propagate-folder-contents -relation=<relation-name> [-no_condition_check= true|
false][[-exclude_type=<types-to-be-excluded>] | [-include_type=<types-to-be-included>]][[-
allowed_status=<status-to-be-propagated>] | [-disallowed_status=<status-to-not-be-propagated>]]
```

Arguments

-relation

Propagates the change objects with the specified relation. The value can be one of the following:

- CMHasProblemItem
- CMHasImpactedItem
- CMHasSolutionItem
- CMReferences

To propagate objects that have different relations, add another instance of the handler to the task. For example, to propagate objects with the **CMHasProblemItem** and the **CMHasImpactedItem** relation, add the **CSI-propagate-folder-contents** handler with the **-relation=CMHasProblemItem** argument and value along with another **CSI-propagate-folder-contents** handler with the **-relation=CMHasImpactedItem** argument and value.

-bypass_condition_check

(Optional) Specifies whether to bypass condition checking. Valid values are **true** and **false**. If this argument is not specified, condition checking is used.

-exclude_type=<object-type>

(Optional) Does not propagate objects of the specified type.

The **-exclude_type** and **-include_type** arguments are mutually exclusive. Only one of these can be specified as arguments to the handler. If both arguments are specified, an error is displayed when running a workflow process using this handler.

-include_type=<object-type>

(Optional) Propagates objects of the specified type.

The **-exclude_type** and **-include_type** arguments are mutually exclusive. Only one of these can be specified as arguments to the handler. If both arguments are specified, an error is displayed when running a workflow process using this handler.

-allowed_status

(Optional) Propagates objects with the specified status.

The **-allowed_status** and **-disallowed_status** arguments are mutually exclusive. Only one of these can be specified as arguments to the handler. If both arguments are specified, an error is displayed when running a workflow process using this handler.

-disallowed_status

(Optional) Does not propagate objects with the specified status.

The **-allowed_status** and **-disallowed_status** arguments are mutually exclusive. Only one of these can be specified as arguments to the handler. If both arguments are specified, an error is displayed when running a workflow process using this handler.

Placement

Place on the **Start** task of the workflow process.

Restrictions

None.

Examples

- This example propagates change objects with the **CMHasProblemItem** relation.

Argument	Values
-relation	CMHasProblemItem

- This example propagates change objects with the **CMHasProblemItem** relation, but does not check conditions.

Argument	Values
-relation	CMHasProblemItem
-bypass_condition_check	true

- This example propagates change item revisions with the **CMHasProblemItem** relation and **Completed** status, but does not check conditions.

Argument	Values
-relation	CMHasProblemItem
-bypass_condition_check	true
-include_type	ItemRevision
-allowed_status	Completed

DOCMGTAPP-apply-pdf-control

Description

Applies a system stamp, watermark, logo (if attached), distribution statement text (if attached), workflow signoff table (if the target object is in a review task), and Teamcenter attributes when the logical object is related to the attached PDF dataset. A target object can be an item, an item revision or its subtype, or the PDF dataset itself.

The system stamp is an imprint comprising data such as a watermark and optional boilerplate text. In Business Modeler IDE, the data model administrator creates a system stamp configuration, associating the configuration with the XML command file that defines the watermark and text.

For this handler to apply the stamp and watermark, the following conditions are required:

- The PDF dataset must be related to the item revision or its subtype.
- For adding workflow signoff table, the workflow target must be an item revision object.
- The system stamp configuration must be enabled for the item revision or its subtype. The **Applies To** attribute of the system stamp configuration must be set to **PDF_Control**.
- The **PDF Control** access privilege must be granted.

Syntax

DOCMGTAPP-apply-pdf-control -user_stamp=<text string>

Arguments

-user_stamp

(Optional) Specifies any string for the text portion of the stamp.

-ignore_error

(Optional) Specifies that the current task can be continued on to the next task even if the current task fails. If this optional argument is not specified, the current task might stop on failures (including password protected file or dataset is checked out, or object cannot be saved), and might not continue to the next task.

-logical_object_names

(Optional) This argument accepts either a single Logical Object (such as, `DMSampleLODocAttach`) or a comma-separated list of Logical Objects (such as, `DMSampleLODocAttach, LOname2` and so on). These specified Logical Objects will be attached and related to the target PDF Datasets. Once the stamping process is complete, the Logical Objects will be automatically removed.

Placement

Place on the **Start** action or the **Complete** action.

Restrictions

None

DOCMGTAPP-insert-pdf-cover-page

Description

Inserts a cover page to a PDF dataset attached to the target being sent in the workflow. The target can be an item, an item revision or its subtype, or the PDF dataset itself. The cover page is a PDF dataset that is related to the item revision by using the **Document Page Type** relation. Its **Page Type** relation property is set to **Cover Page**.

For this handler to insert a PDF cover page, the following conditions are required:

- The PDF dataset must be related to the item revision or its subtype. If it is related using the **Document Page Type** related, its **Page Type** relation property must be set to **Base Document**.
- The PDF cover page must be related to the item revision or its subtype.

Syntax

DOCMGTAPP-insert-pdf-cover-page [-create_new_dataset= <<true|false>> [-new_dataset_suffix= <<text>>]]

Arguments

-create_new_dataset

(Optional) If **true**, creates a new PDF dataset with the cover page inserted. If **false**, the original PDF file is modified.

-new_dataset_suffix

If **-create_new_dataset** argument is specified as true, you can enter any text string for the dataset suffix name.

-ignore_error

(Optional) Specifies that the current task can be continued on to the next task even if the current task fails. If this optional argument is not specified, the current task might stop on failures (including password protected file or dataset is checked out, or object cannot be saved), and might not continue to the next task.

Placement

Place on the **Start** action or the **Complete** action.

Restrictions

None

DOCMGT-render-document-revision

Description

Translates Source Datasets associated with target Item revisions to Derived Datasets, for example, MSWordX Datasets to PDF Datasets. Settings from the **Item Revision Definition Configuration (IRDC)** and **Dispatcher Service Configuration** determine the file formats of the input Source Dataset and output Derived Datasets.

Note

- This handler requires Teamcenter Dispatcher RenderMgtTranslator for the translation, previewservice and Teamcenter Visualization Convert and Print.
- Target item revisions must be valid and checked in.

The translation is asynchronous; the workflow can continue while translation begins and runs to completion. The translated files are stored as Derived Datasets in Teamcenter and may be related to the input Source Datasets and Item revisions.

Tip

You can use a **Do** task to wait for the RenderMgtTranslator dispatcher translation process to set the **Complete** action before the workflow continues. The RenderMgtTranslator dispatcher process sets the task state to **Completed** when the translation is successful.

Syntax

DOCMGT-render-document-revision -existing_file=[replace | preserve]

Arguments

-existing_file

- **replace**
 - The new translated file replaces the existing Derived Dataset file (*the Source Dataset must be related to its Derived Dataset*) or added to the newly created output Derived Dataset.
 - For Released Items or Source Datasets, set preferences FndODM_AllowRenderAllReleasedItemsOrSrcDatasets to "true" in order to replace existing Derived Dataset file.
- **preserve**

This is the default value.

 - The existing Derived Dataset must be related to its Source Dataset

- If the Source Dataset last modified date is later than its related Derived Dataset last modified date, then the new translated file will replace the existing Derived Dataset file. Otherwise the new translated file will not replace its existing Derived Dataset file.

-ignore_errors

(Optional) Specifies that the current task can be continued on to the next task even if the current task fails. If this optional argument is not specified, the current task might stop on failures (including password protected file or dataset is checked out, or object cannot be saved), and might not continue to the next task.

Placement

Place on the **Start** action of a **Do** task.

Note

Whenever this handler is used, upon successful completion, an Active Workspace user gets notified: either the process initiator, the task responsible party, or the *<Dispatcher-client-proxy-user>* user.

When the workflow administrator sets up the workflow:

- If there is only one **Do** task in the workflow to render documents, the handler is placed on the **Start** action of the **Do** task and the workflow initiator gets a notification.
- If there are several tasks in the workflow, including a **Do** task for rendering documents, and the handler is placed on the **Start** action of the **Do** task, the notification goes to the user who completed the predecessor task.
- If a successor task invokes this same handler or the **DOCMGT-update-document-property** handler, an administrator can add a predecessor **Do** task to ensure that the user who completes the predecessor task receives the notification. Otherwise, the Dispatcher client user receives the notification.

You can use a **Do** task to wait for the RenderMgtTranslator dispatcher translation process to set the **Complete** action before the workflow continues.

Caution

Do not place this handler on the **perform** action of the **perform-signoffs** task. Otherwise, this handler runs multiple times.

Restrictions

- Requires Dispatcher to translate the dataset's file.
- Item revisions with attached datasets such as Microsoft Word and Microsoft Excel must be included as targets of the workflow.
- Do not use this handler with a workflow that is running in the background.

DOCMGT-update-docprop-logicalobject

Description

Updates the datasets (for example, MSWordX with a **.docx** extension or MSEXcelX with a **.xlsx** extension) associated with the target item revisions with the latest attribute exchange data. Attribute exchange data can include Teamcenter properties, logos, distribution statements, and workflow sign off tables, if the target object is in a review task. Attributes are exchanged between Teamcenter and the files.

Note

- The generic (logical object) attribute exchange currently supports Microsoft Word, Excel, and PowerPoint datasets only.
 - The Microsoft Word, Excel, and PowerPoint datasets must be related to the logical objects for the generic attribute exchange to occur.
 - Target item revisions must be valid and checked in.
- The attribute exchange process from this workflow action handler bypasses the **Fnd0TriggerLOAttrExch** business object constant configuration.
- Logos and distribution statements must be enabled based on their document configuration setting.
 - System stamp must be enabled for a business object revision and logical objects must be defined for its datasets.

The update is synchronous.

Syntax

```
DOCMGT-update-docprop-logicalobject[ -add_wf_history_table=legacy_workflow_signoff_table |
workflow_history_table |reset_workflow_history_table ]
```

Arguments

-ignore_errors

(Optional) Specifies that the current task can be continued on to the next task even if the current task fails. If this optional argument is not specified, the current task might stop on failures (including password protected file or dataset is checked out, or object cannot be saved), and might not continue to the next task.

```
-add_wf_history_table= legacy_workflow_signoff_table | workflow_history_table |
reset_workflow_history_table
```

(Optional) Argument value **workflow_history_table** adds workflow signoff history table using **WorkflowHistoryTable Rich Text** control. Argument value **legacy_workflow_signoff_table** adds workflow signoff history table using **WorkflowSignoffTable Rich Text** control (requires Logical Object).

If argument value is **reset_workflow_history_table**, the **WorkflowHistoryTable** will be reset to default state (only support for the **workflow_history_table** option). If this optional argument is not specified, the default is **workflow_history_table**.

The operation is synchronous.

Placement

Place on the **Start** action of a **Task**.

Place on the **Perform** or **Complete** action of a **ReviewTask**.

Caution

Do not place this handler on the **perform** action of the **perform-signoffs** task. Otherwise, this handler runs multiple times.

Do not place this handler with argument **-add_wf_history_table=reset_workflow_history_table** on the **ReviewTask**.

Restrictions

Item revisions with attached datasets such as Microsoft Word, Excel, or PowerPoint must be included as targets of the workflow process.

DOCMGT-update-document-property

Description

Update the MSWordX Datasets with .docx file extension associated with the target Item **Revisions** with the latest attribute exchange data, if there are any from Teamcenter to file (.docx file).

Note

- This handler works only if the property name in the MSOffice document contains only ASCII characters. Language-specific characters should be avoided.
- This handler requires Teamcenter Dispatcher for the update.
- The **RenderMgtTranslator** service must be enabled.
- Use the Business Modeler IDE to set up and deploy IRDC and dispatcher service configuration objects to the Teamcenter database.
- Target item revisions must be valid and checked in.

The update is asynchronous. The workflow continues while the update begins and runs to completion.

Tip

You can use a **Do** task to wait for the update process to initiate the **Complete** action before the workflow continues. The update process sets the task state to **Completed** when the update is successful.

Syntax

DOCMGT-update-document-property

Arguments

-ignore_errors

(Optional) Specifies that the current task can be continued on to the next task even if the current task fails. If this optional argument is not specified, the current task might stop on failures (including password protected file or dataset is checked out, or object cannot be saved), and might not continue to the next task.

Placement

Place on the **Start** action of a **Do** task.

Note

Whenever this handler is used, upon successful completion, a user gets notified: either the process initiator, the task responsible party, or the *<Dispatcher-client-proxy-user>* user.

When the workflow administrator sets up the workflow:

- If there is only one **Do** task in the workflow to update document properties, the handler is placed on the **Start** action of the **Do** task and the workflow initiator gets a notification.
- If there are several tasks in the workflow, including a **Do** task for updating document properties, and the handler is placed on the **Start** action of the **Do** task, the notification goes to the user who completed the predecessor task.
- If a successor task invokes this same handler or the **DOCMGT-render-document-revision** handler, an administrator can add a predecessor **Do** task to ensure that the user who completes the predecessor task receives the notification. Otherwise, the Dispatcher client user receives the notification.

Caution

Do not place this handler on the **perform** action of the **perform-signoffs** task. Otherwise, this handler runs multiple times.

Restrictions

- Requires Dispatcher to update the dataset's files.
- Item revision with attached datasets containing Microsoft WordX .docx extension must be included as targets of the workflow process.
- Do not use this handler with a workflow that is running in the background.

EPM-adhoc-signoffs

Description

Determines the behavior of the **Ad-hoc done** check box in the **select-signoff-team** task's interface, allowing the initializing user, address list members, and resource pool members to add users to the signoff team in an ad hoc manner. If the task template contains predefined signoff profiles, the ad hoc selections add one-time-only additions to the required signoff team. Alternatively, if the task template contains no predefined signoff profiles, the ad hoc additions comprise the whole of the signoff team.

Note

Active Workspace does not display the **Ad Hoc done** check box.

When this handler is attached to the **select-signoff-team** task, the checkbox is not selected by default. You can modify this behavior using the **-auto_complete** argument.

Note

When this handler is *not* attached to the **select-signoff-team** task, the checkbox displays by default as checked, in expectation that ad hoc additions are not required. Users can still clear the checkbox, add additional signoff team members to the signoff team, and then select the checkbox again.

Remember that the checkbox only indicates that the user has completed any ad hoc additions to the signoff team; it does not signify that the required profiles have been added to the signoff team. Even if the user fits into any of the signoff profiles, it is added only as an ad hoc user and not as the signoff profile member.

Using the **-auto_complete** argument with this handler allows the **select-signoff-team** task to complete automatically. If the system's **ad hoc done** query is returned as **true** and any predefined signoff profiles have been selected, the task automatically completes without user interaction. Therefore, the **select-signoff-team** task template can be configured to automatically choose a signoff team and decide whether or not to allow users to modify this predefined signoff team at execution of the task.

This handler's arguments are listed in order of precedence, meaning that the system attempts to find a match for the argument as a user before it tries to find a match as an address list, and so on. When a **select-signoff-team** task is created based on a task template that uses this handler, it parses these arguments and adds those signoffs to the task.

If the **-required** argument is specified, the signoffs will be added as required signoffs which cannot be removed or marked as optional by users. After that point, the ad hoc signoff functionality allows subsequent modifications to the signoff list. Therefore, what is specified in this handler is only used to initialize this task; during execution of the workflow process, the ad hoc signoff functionality accepts further changes.

By default, this handler is run at workflow process initiation, rather than at the task where it is assigned. It initializes the signoff lists at workflow process initiation, allowing the workflow process initiator to view

signoff assignments early in the workflow process and set the assignments as desired. However, this also means that assignments are based on target/assignment data as it exists at the time of initiation. For instance, if you use the **\$TARGET_GROUP** keyword argument with this handler, and the handler is run at workflow process initiation, it looks at the group that owns the targets when the workflow process is initiated, not when the task using this handler is run. When you use this method, keyword arguments always resolve to the workflow process initiator.

Alternatively, if the **-ce** argument is used, the handler is not run when the workflow process is initiated. The handler is run instead when the **select-signoff-team** task starts.

If the **-condition_name** argument is specified, the handler will add the reviewers or set auto complete only if the condition is met. However, it will not reset the auto-complete flag if it is already set on the select-signoff-team task.

Syntax

EPM-adhoc-signoffs

[-auto_complete]

**[-assignee= {user:<user> | person:<person>
| addresslist:<list>**

**| resourcepool:<group>::<role>
| allmembers:<group>::<role>**

| user:PROP::<property_name>

**| resourcepool:PROP::<property_name>
| allmembers:PROP::<property_name>
| \$PROPOSED_RESPONSIBLE_PARTY | \$PROPOSED_REVIEWERS | \$USER
| \$PROCESS_OWNER | \$TARGET_OWNER <[type]>**

**| \$PROJECT_ADMINISTRATOR
| \$PROJECT_TEAM_ADMINISTRATOR
| \$PROJECT_AUTHOR | \$PROJECT_MEMBER<[group]::<role>>
| \$REQUESTOR | \$ANALYST
| \$CHANGE_SPECIALIST1 | \$CHANGE_SPECIALIST2 | \$CHANGE_SPECIALIST3**

| \$CHANGE_REVIEW_BOARD | \$CHANGE_IMPLEMENTATION_BOARD}}

[-from_include_type=<object-type1[,object-type2,...]>|

[-from_exclude_type=<object-type1[,object-type2,...]>]

[-from_attach=target | reference | schedule_task]

[-from_relation=<relation-type>]

[-from_include_related_type=<object-type1[,object-type2,...]> |

[-from_exclude_related_type=<object-type1[,object-type2,...]>]

[-quorum=<quorum-value>]

[-ce]
 [-clear_signoffs]
 [-clear_comments]
 [-target_task=<multilevel-task-path>]

[-required]

[-project_scope=all | owning_project]

[-check_first_object_only=<true | false>]

[-condition_name=<condition1>]

[-condition_scope=all | any | none]

Arguments

-auto_complete

(Optional) Allows the task to complete without user interaction. Automatically selects the **Ad-hoc done** check box in the **select-signoff-team** task interface. The task is assumed to be populated; no **select-signoff-team** task needs to be performed through the interface (providing at least one of the signoff profiles has been fulfilled).

When this argument is not used, the system does not automatically select the **Ad-hoc done** check box, preventing the **select-signoff-team** task from completing until the user manually checks it, typically after ad hoc signoffs have been added. Absence of the **EPM-adhoc-signoffs** handler implies the presence of this argument, and the **Ad-hoc done** check box is selected and behaves accordingly.

-assignee

(Optional) Assigns signoff members to **select-signoff-team** or **Notify** task under a **Route** task.

Separate multiple assignees with commas or the character specified by the **EPM_ARG_target_user_group_list_separator** preference.

The following value formats are allowed:

- **user:<user>**

Adds the user specified to the signoff member list for the task to which it is attached. Accepts a valid Teamcenter user ID.

- **user:PROP::<property_name>**

Adds the user specified by the property name to the signoff member list for the task to which it is attached.

If the property is a multi-value property, only the first value is used when only a single user is assigned in the workflow. When more than one user is assigned, all property values are used.

- **resourcepool:PROP::<property_name>**

Adds the resource pool specified by the property name to the signoff member list for the task to which it is attached.

If the property is a multi-value property, only the first value is used when only a single user is assigned in the workflow. When more than one user is assigned, all property values are used.

- **allmembers:PROP::<property_name>**

Adds all members of a group/role combination that is specified by the property name to the signoff member list.

If the property is a multi-value property, only the first value is used when a single user is assigned in the workflow. When more than one user is assigned, all property values are used.

- **person:<person>**

Adds the user whose name is specified to the signoff member list for the task to which it is attached. Accepts a valid Teamcenter person name.

Note

If the person's name includes a comma, you must include an escape character (\) to add the correct person. For example, to use **wayne, joan**:

-assignee=person:waynel, joan

- **addresslist:<list>**

Adds all members of the address list specified to the signoff member list.

- **resourcepool:<group>::<role>**

Results in a single assignment which can be performed by any single member of this group/role.

You can define resource pools in the form of <group>::, <group>::<role>, or <role>.

Note

When a resource pool task is performed by a user, it is automatically claimed by that user. If that task is a **Review** task and it is started again, the task is assigned to the user who performed it in the previous iteration, rather than the resource pool, unless the following arguments are used.

- **-auto_complete**
- **-clear_signoffs**
- **-ce**

Accepts valid Teamcenter resource pool names and these keywords:

- **\$GROUP**

Current user's current group.

- **\$ROLE**

Current user's current role.

- **\$TARGET_GROUP[<type>]**

Owning group of the first target object of the specified type. The <type> value is optional. If not specified, the first target is used.

- **\$PROCESS_GROUP**

Owning group of the workflow process.

- **allmembers:<group>::<role>**

Adds all members of a group/role combination to the signoff member list. You can define role in groups in the form of <group>::, <group>::<role>, or <role>. Accepts valid Teamcenter resource pool names and these keywords:

- **\$GROUP**

Current user's current group.

- **\$ROLE**

Current user's current role.

- **\$TARGET_GROUP[<type>]**

Owning group of the first target object of the specified type. The <type> value is optional. If not specified, the first target is used.

- **\$PROCESS_GROUP**

Owning group of the workflow process.

- **\$PROPOSED_RESPONSIBLE_PARTY**

Affects assignments based on the user assigned as the responsible party for the first target object.

- **\$PROPOSED_REVIEWERS**

Affects assignments based on members assigned as reviewers for the first target object.

- **\$USER**

Adds the current user to the signoff member list.

- **\$PROCESS_OWNER**

Adds the workflow process owner to the signoff member list.

- **\$TARGET_OWNER [<type>]**

Adds the owner of the first target of specified type to the signoff member list. The <type> value is optional. If not specified, the first target is used.

- **\$PROJECT_ADMINISTRATOR, \$PROJECT_TEAM_ADMINISTRATOR, \$PROJECT_AUTHOR, \$PROJECT_MEMBER[<group>::<role>]**

Dynamically adds the project team members belonging to the role specified in the argument value. The project team is determined by the project team associated with the first target object.

Both configurations (**\$PROJECT_MEMBER[group::role]** and **\$PROJECT_MEMBER**) are valid. If the **\$PROJECT_MEMBER[<group>::<role>]** argument is specified, only the project members of the qualifying projects which belong to the specified group and role are selected for assignment. If the group and role are not specified, all the project members from qualifying projects are selected.

You can specify a sub-group with the syntax **< sub-group.group>** or **<sub-group.group++>**.

Format subgroups by starting with the lowest subgroup, followed by a parent group, and the top-level group (each level separated by a period).

Use **++** when adding each additional subgroup. The syntax **<group++>** indicates that everything below the specified subgroup is included. Ensure that **++** are the last two characters in the string. If you use **++**, do so only once at the end.

For example, if the organizational hierarchy of groups looks like this:

```
Group1
  Group2
    Group3
```

Sample configurations include:

- **Syntax:**
`< $PROJECT_MEMBER[Group1.Group::Designer]>`
Specified role:
 Designer role from Group1
- **Syntax:**
`< $PROJECT_MEMBER[Group2.Group1.Group::Designer]>`
Specified role:
 Designer role from Group2
- **Syntax:**
`< $PROJECT_MEMBER[Group1.Group++::Designer]>`
Specified role:
 Designer role from Group1 and any of its sub-groups
- **Syntax:**
`< $PROJECT_MEMBER[Group++::Designer]>`
Specified role:
 Designer role from Group and any of its sub-groups
- **Syntax:**
`<$PROJECT_MEMBER[Group1.Group::]>`

Specified role:

Any role from Group1

- **Syntax:**

< \$PROJECT_MEMBER[Group1.Group++::]>

Specified role:

Any role from Group1 and any of its sub-groups

- **\$REQUESTOR, \$ANALYST, \$CHANGE_SPECIALIST1, \$CHANGE_SPECIALIST2, \$CHANGE_SPECIALIST3, \$CHANGE_REVIEW_BOARD, \$CHANGE_IMPLEMENTATION_BOARD**

Dynamically resolves to the user or resource pool associated with the first Change target object in the workflow process. The particular user or resource pool is determined by the role specified in the argument value.

If custom participants are defined by the customer, those participants can be used as recipients.

Note

Change-related keywords apply only to change objects. If the workflow process does not contain a change object as a target, the argument resolves to null.

Change Manager does not need to be enabled before these keywords take effect, but during installation, make sure you install the **Change Management** application using your preferred installation tool (Deployment Center or Teamcenter Environment Manager) to ensure change-related keywords are enabled.

-from_include_type=<object-type1[,object-type2,...]>

(Optional) Specifies the object types to be used to get the property value from when a property is specified in the **-assignee** argument (for example, **-assignee=user:PROP::<property_name>**). They must be valid object types.

You can use this argument only when you get the assignee from a property on an object (**user:PROP::** or **resourcepool:PROP::**).

-from_exclude_type=<object-type1[,object-type2,...]>

(Optional) Specifies the object types to be excluded when getting the property value when it is specified in the **-assignee** argument (for example, **-assignee=user:PROP::<property_name>**). They must be valid object types.

You can use this argument only when you get the assignee from a property on an object (**user:PROP::** or **resourcepool:PROP::**).

-from_attach= target | reference | schedule_task

(Optional) Specifies which type of attachment (**target**, **reference**, or **schedule_task**) to get the property value from when a property is specified in the **-assignee** argument (for example, **-assignee=user:PROP::<property_name>**). If this argument is not specified, the default is **target**.

You can use this argument only when you get the assignee from a property on an object (**user:PROP::** or **resourcepool:PROP::**).

-from_relation

(Optional) Specifies the relation of the objects to get the property value from when a property is specified in the **-assignee** argument (for example, **-assignee=user:PROP::<property_name>**). It must be a valid relation.

- For manifestations, use **IMAN_manifestation**.
- For specifications, use **IMAN_specification**.
- For requirements, use **IMAN_requirement**.
- For references, use **IMAN_reference**.
- For BOM views, use **PSBOMViewRevision**.

You can use this argument only when you get the assignee from a property on an object (**user:PROP::** or **resourcepool:PROP::**).

-from_include_related_type=<object-type1[,object-type2]>

(Optional) Specifies the related object types to be used to get the property value from when a property is specified in the **-assignee** argument (for example, **-assignee=user:PROP::<property_name>**). They must be valid object types.

You can use this argument only when you get the assignee from a property on an object (**user:PROP::** or **resourcepool:PROP::**) and you use the **-from_relation** argument.

This argument should not be used with the **-from_exclude_related_type** argument.

-from_exclude_related_type=<object-type1[,object-type2]>

(Optional) Specifies related object types to be excluded when getting the property value when it is specified in the **-assignee** argument (for example, **-assignee=user:PROP::<property_name>**). They must be valid object types.

You can use this argument only when you get the assignee from a property on an object (**user:PROP::** or **resourcepool:PROP::**) and you use the **-from_relation** argument.

This argument should not be used with the **-from_include_related_type** argument.

-quorum

(Optional.) Determines the approval quorum for the **perform-signoffs** task. The value can either be a percentage or a number. For example, if it is set to **51%** then of all the signoff members, 51% of members need to approve for the task to move ahead. If it is set to **5**, then 5 members need to approve for the task to move ahead. The value specified here will override the quorum specified on the **select-signoff-team** task template. If no value is specified, the quorum specified on the **select-signoff-team** task template is used. This argument is ignored if the handler is placed on a **Notify** task.

-ce

(Optional) Disables the default behavior of running this handler when the workflow process is initiated. Instead, the handler is run when the **select-signoff-team** task is initiated in the workflow.

If **-ce** is specified, the **select-signoff-team** task does not auto-complete even if a process assignment list is assigned during process initiation. For the **select-signoff-team** task to auto-complete, you must also use the **-auto_complete** handler argument.

-clear_signoffs

(Optional) If specified, all existing signoffs are removed from the **select-signoff-team** subtask before the new signoffs are added. If you specify this argument, you must also use the **-ce** argument before it.

-clear_comments

(Optional) If specified, removes the comments when Review/Acknowledge tasks are restarted. If you specify this argument, you must also use the **-ce** argument before it. This action cannot be overridden by the user once the workflow is initiated. If your Review/Acknowledge task template is set to retain approvals for a task during rework, then even though the **-clear_comments** argument is configured, those signoff comments are not cleared.

Note

The **-clear_comments** and **-clear_signoffs** arguments are mutually exclusive, meaning you cannot use them together. Since **-clear_comments** focuses on removing comments and **-clear_signoff** focuses on removing signoff statuses, using both simultaneously could cause conflicts. Therefore, the system requires you to choose one or the other, but not both at the same time.

-target_task

(Optional) Specifies the multilevel task path to which the reviewers are added. The path is from the root task to the **select-signoff-team** subtask, with the path levels separated with colons (:). For example:

Change Request Review:QA Review:select-signoff-team

-required

(Optional) If specified, all signoffs added through this handler instance are marked as mandatory.

-project_scope

(Optional) Specifies which projects are used to resolve project-based assignments. The **all** value specifies all projects in the list of projects. The **owning_project** value specifies the owning project only.

If this argument is not specified, the default value is the first project in the project list.

-check_first_object_only

(Optional) The **true** value specifies that only the first object is checked. If the value is **false**, all objects are checked. If this argument is not specified, or if it is specified without a value, only the first object is checked.

If the **-include_type**, **-exclude_type**, **-include_related_type**, or **-exclude_related_type** arguments are specified, they determine the types of objects that are checked.

-condition_name

(Optional) The name of the condition to evaluate against the objects identified for assigning reviewers from. The condition signature should accept a **WorkspaceObject & UserSession**. The handler assigns the reviewers only if the condition results are successful, based on the **-condition_scope** argument.

-condition_scope

(Optional) The criteria for evaluating condition results against workflow objects.

all All objects should meet the condition. This is the default behavior if this argument is not supplied with the **-condition_name** argument.

any Any object should meet the condition.

none No object should meet the condition.

Placement

Place on the **Start** action of a **select-signoff-team** subtask.

This handler runs at workflow process initiation if the **-ce** argument is not specified. If **-ce** is specified, the handler runs in a conventional manner at the point of handler placement on the task action.

If the workflow involves a rejection path and it is required that the Signoff Team is to be selected again, then add the **-ce** argument to the handler present on the **Start** action of the **select-signoff-team** subtask.

Place on the **Undo** action of a **select-signoff-team** subtask and specify the **-ce** argument to clear the **Ad-hoc done** check box when the subtask is demoted. In this situation, the next time the subtask reaches the **Start** action of the **select-signoff-team** subtask, the user is again prompted to select a signoff team.

Restrictions

Ignores any invalid arguments without reporting an error.

The keywords always refer to the initiating user because all instances of this handler in a workflow process are run when the workflow process is initiated, not when tasks are approved.

If the **-ce** argument is not specified, all instances of this handler are run when the workflow process is initiated, and in this case, the keywords refer to the initiating user.

Examples

- This example places the handler on the **Undo** action of the **select-signoff-team** subtask. If the **select-signoff-team** subtask is demoted, the **-ce** argument clears the **Ad-hoc done** check box. When the workflow process returns to the **select-signoff-team** subtask, the responsible party is again prompted to select the signoff team because the **Ad-hoc done** check box is clear, indicating the task is not yet complete.

Argument	Values
-ce	

- This example has a valid user, resource pool, address list, and handler-specific keywords as argument values. So **Smith**, the current logged on users group/role resource pool, members of the **List1** address list, and the members assigned as reviewers are added as signoff attachments to the **select-signoff-team** task on which this handler is added. The handler is run at the time of workflow process initiation.

Argument	Values
-assignee	user:Smith, resourcepool:\$GROUP::\$ROLE, addresslist:List1, \$PROPOSED_REVIEWERS
-quorum	80%

If the handler with the above arguments is specified on the **Notify** task under the **Route** task, the signoff attachments are added to the **Notify** task and used for sending notifications. The quorum is set to **80%** which means that of all the signoff members, 80% need to approve for the task to move ahead.

- This example has a valid user, resource pool, address list, and handler-specific keywords as argument values. So **Smith**, the current logged on users group/role resource pool, members of **List1** address list, and the members assigned as reviewers are added as signoff attachments to the **select-signoff-team** task on which this handler is added. Because of the **-ce** option, the handler is run when the task action on which it is attached is run. The quorum is set to **80%** which means that of all the signoff members, 80% need to approve for the task to move ahead.

Argument	Values
-assignee	user:Smith, resourcepool:\$GROUP::\$ROLE, addresslist:List1, \$PROPOSED_REVIEWERS
-quorum	80%
-ce	

If the handler with the above arguments is specified on the **Notify** task under the **Route** task, the signoff attachments are added to the **Notify** task and used for sending notifications.

- This example assigns the user whose ID is Smith to the signoff team

Argument	Values
-assignee	user:Smith

- This example assigns the owning user ID of the first **UGMASTER** target found by the system to the signoff team.

Argument	Values
-assignee	user:\$TARGET_OWNER[UGMASTER]

- This example assigns the project team administrator of the project team associated with the first target found by the system to the signoff team.

Argument	Values
-assignee	user:\$PROJECT_TEAM_ADMINISTRATOR

- This example assigns all members of the **jhList** address list to the signoff team.

Argument	Values
----------	--------

-assignee **addresslist:jhList**

- This example assigns the **manufacturing** resource pool (any role within the manufacturing group) to the signoff team.

Argument	Values
----------	--------

-assignee **resourcepool:manufacturing::**

- This example assigns the **\$PROCESS_GROUP** resource pool (any role within the **xyz** group, where **xyz** is the owning group of the workflow process) to the signoff team.

Argument	Values
----------	--------

-assignee **resourcepool:\$PROCESS_GROUP::**

- This example assigns the **\$TARGET_GROUP** resource pool (any roles within the **abc** group, where **abc** is the group of the first item revision target) to the signoff team.

Argument	Values
----------	--------

-assignee **resourcepool:\$TARGET_GROUP::**

- This example assigns the engineer role within the manufacturing group resource pool to the signoff team.

Argument	Values
----------	--------

-assignee **resourcepool:manufacturing::engineer**

- This example assigns the current logged on role within the current logged on group resource pool to the signoff team.

Argument	Values
----------	--------

-assignee **resourcepool:\$GROUP::\$ROLE**

- This example assigns the engineer role within any group resource pool to the signoff team.

Argument	Values
----------	--------

-assignee **resourcepool:::engineer**

- This example adds user **smith** and all reviewers of the first target item revision object to the signoff team. The quorum is set to **51%**, which means that at least more than half of the signoff members need to approve for the **perform-signoffs** task to move ahead. Because of the **-ce** option, the handler is run when the task action on which it is attached is run.

Argument	Values
-assignee	user:smith, \$PROPOSED_REVIEWERS
-quorum	51%
-ce	

- This example adds all members of the **Engineering** group and **Engineer** role to the signoff team. The members are dynamically evaluated when the **select-signoff-team** task completes. The quorum is set to **80%**, which means that of all the signoff members, 80% need to approve for the task to move ahead. Because of the **-ce** option, the handler is run when the task action on which it is attached is run.

Argument	Values
-assignee	allmembers:Engineering::Engineer
-quorum	80%
-ce	

- This example adds all members of the **list1** address list and the **Engineering:Engineer** resource pool to the signoff team. The quorum is set to **5**, which means that of all the signoff members, 5 need to approve for the task to move ahead. Because of the **-ce** option, the handler is run when the task action on which it is attached is run.

Argument	Values
-assignee	resourcepool:Engineering::Engineer, addresslist:list1
-quorum	5
-ce	

- This example has a valid user, resource pool, address list, and handler specific keywords as argument values. So **smith**, the current logged on users group/role resource pool, members of the **list1** address list, and the members assigned as reviewers are assigned to the signoff team. Because of the **-ce** option, the handler is run when the task action on which it is attached is run.

Argument	Values
-assignee	user:smith,resourcepool:\$GROUP::\$ROLE, addressList:list1,\$PROPOSED_REVIEWERS
-ce	

If the handler with these arguments is specified on the **Notify** task under the **Route** task, the signoff attachments are added to the **Notify** task and used for sending notifications.

- This example has a valid user, resource pool, and handler-specific keywords as values. So **smith**, the current logged on users group/role resource pool, members of the project associated with the first target object, and members assigned as reviewers are added to the signoff team. Because of the **-ce** option, the handler is run when the task action on which it is attached is run.

Argument	Values
-assignee	user:smith,resourcepool:\$GROUP::\$ROLE, \$PROJECT_MEMBER,\$PROPOSED_REVIEWERS
-ce	

If the handler with these arguments is specified on the **Notify** task under the **Route** task, the signoff attachments are added to the **Notify** task and used for sending notifications.

- This example has a valid user, resource pool, and handler-specific keywords as values. So **smith**, the current logged-on user group/role resource pool, and **CHANGE_REVIEW_BOARD** and **ANALYST** associated with the first change target object are added to the signoff team. Because of the **-ce** option, the handler is run when the task action on which it is attached is run.

Argument	Values
-assignee	user:smith,resourcepool:\$GROUP::\$ROLE, \$CHANGE_REVIEW_BOARD,\$ANALYST
-ce	

If the handler with these arguments is specified on the **Notify** task under the **Route** task, the signoff attachments are added to the **Notify** task and used for sending notifications.

- This example removes all existing members of the signoff team and adds **PROPOSED_RESPONSIBLE_PARTY**. Because of the **-ce** option, the handler is run when the task action on which it is attached is run. The **-auto_complete** option allows the task to complete without user interaction by automatically selecting the **Ad-hoc done** check box in the **select-signoff-team** subtask interface, and the task does not need to be performed through the interface.

Argument	Values
-ce	
-clear_signoffs	
-assignee	\$PROPOSED_RESPONSIBLE_PARTY
-auto_complete	

If the handler with these arguments is specified on the **Notify** task under the **Route** task, the signoff attachments are added to the **Notify** task and used for sending notifications.

- This example assigns all members of the **Engineering** group and **Designer** role of the first project team associated with the first target found by the system to the signoff team as optional signoffs.

Argument	Values
-assignee	\$PROJECT_MEMBER[Engineering::Designer]

- This example assigns all members of the **Engineering** group and **Designer** role of the owning project team associated with the first target found by the system to the signoff team as required signoffs.

Argument	Values
-assignee	\$PROJECT_MEMBER[Engineering::Designer]
-project_scope	owning_project
-check_first_object_only	
-required	

- This example assigns all members of the **Engineering** group and **Designer** role of all project teams associated with the first target found by the system to the signoff team as optional signoffs.

Argument	Values
-assignee	\$PROJECT_MEMBER[Engineering::Designer]
-project_scope	all
-check_first_object_only	true

- This example assigns all members of the **Engineering** group and **Designer** role of the first project team associated with each target found by the system to the signoff team as optional signoffs.

Argument	Values
-assignee	\$PROJECT_MEMBER[Engineering::Designer]
-check_first_object_only	false

- This example places the handler on the **Start** action of the **select-signoff-team** subtask. The **-ce** argument ensures that the **\$PROPOSED_REVIEWERS** variable is not set until the **select-signoff-team** subtask is initiated. Without the **-ce** argument, the **\$PROPOSED_REVIEWERS** variable is assigned the values of **\$PROPOSED_REVIEWERS** that existed at process initiation.

Note

These dynamic variables can change values throughout a process, so care needs to be taken to ensure the desired functionality.

Argument	Values
-ce	
-assignee	\$PROPOSED_REVIEWERS

- This example prevents duplicate task assignments when a participant is replaced and the workflow returns to a previously approved Review task. When a participant assigned by a keyword (such as **\$ANALYST**) is replaced and the workflow loops back to the original signoff task, the **-clear_signoffs** argument ensures that only the new participant receives the task assignment. This prevents duplicate assignments to both the original and replacement participants.

The **-ce** argument ensures the handler runs when the task is initiated rather than at the workflow's initiation, allowing it to evaluate the current participant values. The **-assignee** argument uses the

\$ANALYST keyword to dynamically resolve to the user or resource pool associated with the first Change target object in the workflow.

Argument	Values
-ce	
-clear_signoffs	
-assignee	\$ANALYST

EPM-apply-digital-signature

Description

Applies the digital signature of the logged-on user to the target objects and, optionally, the schedule task.

Syntax

EPM-apply-digital-signature [-include_schedule_task]

Arguments

-include_schedule_task

(Optional) Applies the digital signature to the schedule task and all target objects of the workflow. If this argument is not provided, the digital signature is applied only on the target objects of the workflow.

Placement

Place either on the **Perform** action of the **perform-signoffs** task or the **Complete** action of the following tasks:

- **Do** task
- **Condition** task
- **select-signoff-team** task

On a **Route** task, place on the **Complete** action of the **select-signoff-team** subtask of the **Review** task.

Restrictions

Do not place a workflow handler that modifies digital signature key property values before this handler on the same action on the same workflow task. Modifying digital signature key properties after applying a digital signature voids the signature.

EPM-assign-responsible-party-dynamic-participant

Description

Assigns the specified user or resource pool as the dynamic participant to the target attachment.

Note

Participants can be assigned to **Item Revision** subtypes only. **Non-Revision Items** are removed from processing and, if no **Targets** are left, may result in this warning: **No attachment of the specified type can be found.**

If the BMIDE constant **Fnd0ParticipantEligibility** is defined for the dynamic participant, the handler gets the corresponding group member which matches the group and role criteria defined in the BMIDE constant. If the user identified through the **-assignee** argument does not have the correct group and role membership, the handler reports an error and does not assign the user to the dynamic participant.

If the value is specified as **\$PROJECT_AUTHOR** or **\$PROJECT_MEMBER[<group>::<role>]**, the relevant first project member is selected.

Note

Use the **WRKFLW_display_participants** preference to specify which dynamic-participant types are displayed when assigning dynamic participants for an object.

Syntax

EPM-assign-responsible-party-dynamic-participant

```
-name= {PROPOSED_RESPONSIBLE_PARTY
| ANALYST
| CHANGE_SPECIALIST1
| CHANGE_SPECIALIST2
| CHANGE_SPECIALIST3}
[-assignee= [user:<user> | person:<person>
| resourcepool:<group>::<role>
| user:PROP::<property_name>
| resourcepool:PROP::<property_name>
| $PROPOSED_RESPONSIBLE_PARTY | $USER
| $PROCESS_OWNER | $TARGET_OWNER [<type>]
| $PROJECT_ADMINISTRATOR
| $PROJECT_TEAM_ADMINISTRATOR
| $PROJECT_AUTHOR | $PROJECT_MEMBER[<group>::<role>]
| $REQUESTOR | $ANALYST
| $CHANGE_SPECIALIST1
| $CHANGE_SPECIALIST2
| $CHANGE_SPECIALIST3]]
[-from_include_type=<object-type1[,object-type2,...]>|
```

```

-from_exclude_type=<object-type1[,object-type2,...]>]
[-to_include_type=<object-type1[,object-type2,...]>]
-to_exclude_type=<object-type1[,object-type2,...]>]
[-from_attach= target | reference | schedule_task]
[-from_relation=<relation-type>]
[-from_include_related_type=<object-type1[,object-type2,...]> |
-from_exclude_related_type=<object-type1[,object-type2,...]>]
[-first_object_only]
[-bypass_condition_check]
[-project_scope=all | owning_project]
[-check_first_object_only=<true | false>]
[-condition_name=<condition1>]
[-condition_scope=all | any | none]

```

Arguments

-name

(Required) Specifies the keyword of the dynamic participant that you want to assign participants. Accepts one of the following:

- PROPOSED_RESPONSIBLE_PARTY
- ANALYST
- CHANGE_SPECIALIST1
- CHANGE_SPECIALIST2
- CHANGE_SPECIALIST3

Note

Change-related keywords apply only to change objects. If the workflow process does not contain a change object as a target, the argument resolves to null.

Change Manager does not need to be enabled before these keywords take effect, but during installation, make sure you install the **Change Management** application using your preferred installation tool (Deployment Center or Teamcenter Environment Manager) to ensure change-related keywords are enabled.

-assignee

Makes the user or resource pool the specified keyword evaluates to the responsible party for the task to which this handler is added. Accepts one of the following in the format specified below:

- **user:**<user>

Adds the user specified to the signoff member list for the task to which it is attached. Accepts a valid Teamcenter user ID.

- **person:**<person>

Adds the user whose name is specified to the signoff member list for the task to which it is attached. Accepts a valid Teamcenter person name.

Note

If the person's name includes a comma, you must include an escape character (\) to add the correct person. For example, to use **wayne, joan**:

-assignee=person:wayne\, joan

- **resourcepool:<group>::<role>**

Results in a single assignment which can be performed by any single member of this group/role.

You can define resource pools in the form of *<group>::*, *<group>::<role>*, or *<role>*. Accepts valid Teamcenter resource pool names and these keywords:

- **\$GROUP**

Current user's current group.

- **\$ROLE**

Current user's current role.

- **\$TARGET_GROUP [<type>]**

Owning group of the first target object of the specified type. The *<type>* value is optional. If not specified, the first target is used.

- **\$PROCESS_GROUP**

Owning group of the workflow process.

- **user:PROP::<property_name>**

Adds the user specified by the property name to the signoff member list for the task to which it is attached.

If the property is a multi-value property, only the first value is used when only a single user is assigned in the workflow. When more than one user is assigned, all property values are used.

- **resourcepool:PROP::<property_name>**

Adds the resource pool specified by the property name to the signoff member list for the task to which it is attached.

If the property is a multi-value property, only the first value is used when only a single user is assigned in the workflow. When more than one user is assigned, all property values are used.

- **\$PROPOSED_RESPONSIBLE_PARTY**

Affects assignments based on the user assigned as the responsible party for the first target object.

- **\$USER**

Adds the current user to the signoff member list.

- **\$PROCESS_OWNER**

Adds the workflow process owner to the signoff member list.

- **\$TARGET_OWNER** [<type>]

Adds the owner of the first target of specified type to the signoff member list. The <type> value is optional. If not specified, the first target is used.

- **\$PROJECT_ADMINISTRATOR, \$PROJECT_TEAM_ADMINISTRATOR, \$PROJECT_AUTHOR, \$PROJECT_MEMBER**[<group>::<role>]

Dynamically adds the project team members belonging to the role specified in the argument value. The project team is determined by the project team associated with the first target object.

If the **\$PROJECT_MEMBER**[<group>::<role>] argument is specified, only the project members of the qualifying projects which belong to the specified group and role are selected for assignment. If the group and role are not specified, all the project members from qualifying projects are selected.

You can specify a sub-group with the syntax <group>++<sub-group>::<role>.

- **\$REQUESTOR, \$ANALYST, \$CHANGE_SPECIALIST1, \$CHANGE_SPECIALIST2, \$CHANGE_SPECIALIST3**

Dynamically resolves to the user or resource pool associated with the first change target object in the workflow process. The particular user or resource pool is determined by the role specified in the argument value.

Note

Change-related keywords apply only to change objects. If the workflow process does not contain a change object as a target, the argument resolves to null.

Change Manager does not need to be enabled before these keywords take effect, but during installation, make sure you install the **Change Management** application using your preferred installation tool (Deployment Center or Teamcenter Environment Manager) to ensure change-related keywords are enabled.

Change Management must be selected under **Extensions**→**Enterprise Knowledge Foundation** in Teamcenter Environment Manager.

-from_include_type=<object-type1[,object-type2,...]>

(Optional) Specifies the object types to be used to get the property value from when a property is specified in the **-assignee** argument (for example, **-assignee=user:PROP::<property_name>**). They must be valid object types.

You can use this argument only when you get the assignee from a property on an object (**user:PROP::** or **resourcepool:PROP::**).

-from_exclude_type=<object-type1[,object-type2,...]>

(Optional) Specifies the object types to be excluded when getting the property value when it is specified in the **-assignee** argument (for example, **-assignee=user:PROP::<property_name>**). They must be valid object types.

You can use this argument only when you get the assignee from a property on an object (**user:PROP::** or **resourcepool:PROP::**).

-to_include_type=<object-type1[,object-type2,...]>

(Optional) Specifies the object types to be used while assigning participants on the target attachment. They must be valid object types.

The **-to_include_type** and **-to_exclude_type** arguments are mutually exclusive. If you use one, you cannot use the other.

-to_exclude_type=<object-type1[,object-type2,...]>

(Optional) Specifies the object types to be excluded while assigning participants on the target attachment. They must be valid object types.

The **-to_include_type** and **-to_exclude_type** arguments are mutually exclusive. If you use one, you cannot use the other.

-from_attach= target | reference | schedule_task

(Optional) Specifies which type of attachment (**target**, **reference**, or **schedule_task**) to get the property value from when a property is specified in the **-assignee** argument (for example, **-assignee=user:PROP::<property_name>**). If this argument is not specified, the default is **target**.

You can use this argument only when you get the assignee from a property on an object (**user:PROP::** or **resourcepool:PROP::**).

-from_relation

(Optional) Specifies the relation of the objects to get the property value from when a property is specified in the **-assignee** argument (for example, **-assignee=user:PROP::<property_name>**). It must be a valid relation.

- For manifestations, use **IMAN_manifestation**.
- For specifications, use **IMAN_specification**.
- For requirements, use **IMAN_requirement**.
- For references, use **IMAN_reference**.
- For BOM views, use **PSBOMViewRevision**.

You can use this argument only when you get the assignee from a property on an object (**user:PROP::** or **resourcepool:PROP::**).

-from_include_related_type=<object-type1[,object-type2]>

(Optional) Specifies the related object types to be used to get the property value from when a property is specified in the **-assignee** argument (for example, **-assignee=user:PROP::<property_name>**). They must be valid object types.

You can use this argument only when you get the assignee from a property on an object (**user:PROP::** or **resourcepool:PROP::**) and you use the **-from_relation** argument.

-from_exclude_related_type=<object-type1[,object-type2]>

(Optional) Specifies related object types to be excluded when getting the property value when it is specified in the **-assignee** argument (for example, **-assignee=user:PROP::<property_name>**). They must be valid object types.

You can use this argument only when you get the assignee from a property on an object (**user:PROP::** or **resourcepool:PROP::**) and you use the **-from_relation** argument.

-first_object_only

(Optional) Sets the participants on the first target attachment matching the **-to_include_type** and **-to_exclude_type** arguments. If this argument is not specified, the participants are set on all target attachments matching the **-to_include_type** and **-to_exclude_type** arguments.

-bypass_condition_check

(Optional) Bypasses the Business Modeler IDE condition check before assigning participants. If this argument is not specified, the Business Modeler IDE conditions are checked before assigning participants.

-project_scope

(Optional) Specifies which projects are used to resolve project-based assignments. The **all** value specifies all projects in the list of projects. The **owning_project** value specifies the owning project only.

If this argument is not specified, the default value is the first project in the project list.

-check_first_object_only

(Optional) The **true** value specifies that only the first object is checked. If the value is **false**, all objects are checked. If this argument is not specified, or if it is specified without a value, only the first object is checked.

If the **-include_type**, **-exclude_type**, **-include_related_type**, or **-exclude_related_type** arguments are specified, they determine the types of objects that are checked.

-condition_name

(Optional) The name of the condition to evaluate against the identified objects from which to assign participants. The condition signature should accept a **WorkspaceObject & UserSession**. The handler assigns the reviewers only if the condition results are successful, based on the **-condition_scope** argument.

-condition_scope

(Optional) The criteria for evaluating condition results against workflow objects. Values are the following:

- all** All objects should meet the condition. This is the default behavior if this argument is not supplied with the **-condition_name** argument.
- any** Any object should meet the condition.
- none** No object should meet the condition.

Placement

Place on the **Start** action.

Restrictions

Can only be used to assign dynamic participants that resolve to a single user. For example:

PROPOSED_RESPONSIBLE_PARTY or **ANALYST**

Examples

- Assigns the user **Smith** as the **PROPOSED_RESPONSIBLE_PARTY** participant for all target objects in the workflow process.

Argument	Values
-name	PROPOSED_RESPONSIBLE_PARTY
-assignee	user:Smith

- Reads the **owning_user** property from the target and assigns the user as the **PROPOSED_RESPONSIBLE_PARTY** participant for the first target object only.

Argument	Values
-name	PROPOSED_RESPONSIBLE_PARTY
-assignee	user:PROP::owning_user
-first_object_only	

- Reads the **owning_user** property from the **Document Revision** type target and assigns the user as the **PROPOSED_RESPONSIBLE_PARTY** participant.

Argument	Values
-name	PROPOSED_RESPONSIBLE_PARTY
-assignee	user:PROP::owning_user
-from_include_type	DocumentRevision

- Traverses the **References** relation from the **Part Revision** types of the targets to get the **Document Revision** objects. It then reads the **owning_user** property from the **Document Revision** and assigns the user as the **PROPOSED_RESPONSIBLE_PARTY** participant for all target objects.

Argument	Values
-name	PROPOSED_RESPONSIBLE_PARTY
-assignee	user:PROP::owning_user
-from_include_type	Part Revision
-from_relation	IMAN_reference
-from_include_related_type	DocumentRevision

- This example assigns the first member of the **Engineering** group and **Designer** role of the first project team associated with the first target found by the system to the dynamic participant.

Argument	Values
-name	PROPOSED_RESPONSIBLE_PARTY
-assignee	\$PROJECT_MEMBER[Engineering::Designer]

EPM-assign-signoff-dynamic-participant

Description

Assigns the specified users or resource pools as the dynamic participant to the target attachment.

If the BMIDE constant **Fnd0ParticipantEligibility** is defined for the dynamic participant, the handler gets the corresponding group member which matches the group and role criteria defined in the BMIDE constant. If the user identified through the **-assignee** argument does not have the correct group and role membership, the handler reports an error and does not assign the user to the dynamic participant.

Syntax

```
EPM-assign-signoff-dynamic-participant
-name= {PROPOSED_REVIEWERS
| CHANGE_REVIEW_BOARD
| CHANGE_IMPLEMENTATION_BOARD}
[-assignee= [user:<user> | person:<person> | resourcepool:<group>::<role>
| user:PROP::<property_name>
| resourcepool:PROP::<property_name>
| $PROPOSED_RESPONSIBLE_PARTY | $USER
| $PROCESS_OWNER | $TARGET_OWNER [<type>]
| $PROJECT_ADMINISTRATOR
| $PROJECT_TEAM_ADMINISTRATOR
| $PROJECT_AUTHOR | $PROJECT_MEMBER<[group>::<role>]
| $REQUESTOR | $ANALYST
| $CHANGE_SPECIALIST1
| $CHANGE_SPECIALIST2
| $CHANGE_SPECIALIST3]]
[-from_include_type=<object-type1[,object-type2,...]>|
[-from_exclude_type=<object-type1[,object-type2,...]>]
[-to_include_type=<object-type1[,object-type2,...]>|
[-to_exclude_type=<object-type1[,object-type2,...]>]
[-from_attach= target | reference | schedule_task]
[-from_relation=<relation-type>]
[-from_include_related_type=<object-type1[,object-type2,...]> |
[-from_exclude_related_type=<object-type1[,object-type2,...]>]
[-clear] [-first_object_only]
[-bypass_condition_check]
[-project_scope=all | owning_project]
[-check_first_object_only=<true | false>]
[-condition_name=<condition1>]
[-condition_scope=all | any | none]
```

Arguments

-name

(Required) Specifies the keyword of the dynamic participant that you want to assign participants to. Accepts one of the following:

- **PROPOSED_REVIEWERS**
- **CHANGE_REVIEW_BOARD**
- **CHANGE_IMPLEMENTATION_BOARD**

Note

Change-related keywords apply only to change objects. If the workflow process does not contain a change object as a target, the argument resolves to null.

Change Manager does not need to be enabled before these keywords take effect, but during installation, make sure you install the **Change Management** application using your preferred installation tool (Deployment Center or Teamcenter Environment Manager) to ensure change-related keywords are enabled.

-assignee

(Required) Makes the user or resource pool the specified keyword evaluates to the responsible party for the task to which this handler is added. Accepts one of the following in the format specified below:

- **user:<user>**

Adds the user specified to the signoff member list for the task to which it is attached. Accepts a valid Teamcenter user ID.

- **person:<person>**

Adds the user whose name is specified to the signoff member list for the task to which it is attached. Accepts a valid Teamcenter person name.

Note

If the person's name includes a comma, you must include an escape character (\) to add the correct person. For example, to use **wayne, joan**:

-assignee=person:wayne\, joan

- **resourcepool:<group>::<role>**

Results in a single assignment which can be performed by any single member of this group/role.

You can define resource pools in the form of **<group>::, <group>::<role>**, or **<role>**. Accepts valid Teamcenter resource pool names and these keywords:

- **\$GROUP**

Current user's current group.

- **\$ROLE**

Current user's current role.

- **\$TARGET_GROUP** [<type>]
Owning group of the first target object of the specified type. The <type> value is optional. If not specified, the first target is used.
- **\$PROCESS_GROUP**
Owning group of the workflow process.
- **user:PROP::<property_name>**
Adds the user specified by the property name to the signoff member list for the task to which it is attached.

If the property is a multi-value property, only the first value is used when only a single user is assigned in the workflow. When more than one user is assigned, all property values are used.
- **resourcepool:PROP::<property_name>**
Adds the resource pool specified by the property name to the signoff member list for the task to which it is attached.

If the property is a multi-value property, only the first value is used when only a single user is assigned in the workflow. When more than one user is assigned, all property values are used.
- **\$PROPOSED_RESPONSIBLE_PARTY**
Affects assignments based on the user assigned as the responsible party for the first target object.
- **\$USER**
Adds the current user to the signoff member list.
- **\$PROCESS_OWNER**
Adds the workflow process owner to the signoff member list.
- **\$TARGET_OWNER** [<type>]
Adds the owner of the first target of specified type to the signoff member list. The <type> value is optional. If not specified, the first target is used.
- **\$PROJECT_ADMINISTRATOR, \$PROJECT_TEAM_ADMINISTRATOR, \$PROJECT_AUTHOR, \$PROJECT_MEMBER**[<group>::<role>]
Dynamically adds the project team members belonging to the role specified in the argument value. The project team is determined by the project team associated with the first target object.

If the **\$PROJECT_MEMBER**[<group>::<role>] argument is specified, only the project members of the qualifying projects which belong to the specified group and role are selected for assignment. If the group and role are not specified, all the project members from qualifying projects are selected.

You can specify a sub-group with the syntax <group>++<sub-group>::<role>.
- **\$REQUESTOR, \$ANALYST, \$CHANGE_SPECIALIST1, \$CHANGE_SPECIALIST2, \$CHANGE_SPECIALIST3**

Dynamically resolves to the user or resource pool associated with the first change target object in the workflow process. The particular user or resource pool is determined by the role specified in the argument value.

Note

Change-related keywords apply only to change objects. If the workflow process does not contain a change object as a target, the argument resolves to null.

Change Manager does not need to be enabled before these keywords take effect, but during installation, **Change Management** must be selected.

-from_include_type=<object-type1[,object-type2,...]>

(Optional) Specifies the object types to be used to get the property value from when a property is specified in the **-assignee** argument (for example, **-assignee=user:PROP::<property_name>**). They must be valid object types.

You can use this argument only when you get the assignee from a property on an object (**user:PROP::** or **resourcepool:PROP::**).

-from_exclude_type=<object-type1[,object-type2,...]>

(Optional) Specifies the object types to be excluded when getting the property value when it is specified in the **-assignee** argument (for example, **-assignee=user:PROP::<property_name>**). They must be valid object types.

You can use this argument only when you get the assignee from a property on an object (**user:PROP::** or **resourcepool:PROP::**).

-to_include_type=<object-type1[,object-type2,...]>

(Optional) Specifies the object types to be used while assigning participants on the target attachment. They must be valid object types.

The **-to_include_type** and **-to_exclude_type** arguments are mutually exclusive. If you use one, you cannot use the other.

-to_exclude_type=<object-type1[,object-type2,...]>

(Optional) Specifies the object types to be excluded while assigning participants on the target attachment. They must be valid object types.

The **-to_include_type** and **-to_exclude_type** arguments are mutually exclusive. If you use one, you cannot use the other.

-from_attach= target | reference | schedule_task

(Optional) Specifies which type of attachment (**target**, **reference**, or **schedule_task**) to get the property value from when a property is specified in the **-assignee** argument (for example, **-assignee=user:PROP::<property_name>**). If this argument is not specified, the default is **target**.

You can use this argument only when you get the assignee from a property on an object (**user:PROP::** or **resourcepool:PROP::**).

-from_relation

(Optional) Specifies the relation of the objects to get the property value from when a property is specified in the **-assignee** argument (for example, **-assignee=user:PROP::<property_name>**). It must be a valid relation.

- For manifestations, use **IMAN_manifestation**.
- For specifications, use **IMAN_specification**.
- For requirements, use **IMAN_requirement**.
- For references, use **IMAN_reference**.
- For BOM views, use **PSBOMViewRevision**.

You can use this argument only when you get the assignee from a property on an object (**user:PROP::** or **resourcepool:PROP::**).

-from_include_related_type=<object-type1[,object-type2]>

(Optional) Specifies the related object types to be used to get the property value from when a property is specified in the **-assignee** argument (for example, **-assignee=user:PROP::<property_name>**). They must be valid object types.

Use this argument when a property is designated

You can use this argument only when you get the assignee from a property on an object (**user:PROP::** or **resourcepool:PROP::**) and you use the **-from_relation** argument.

-from_exclude_related_type=<object-type1[,object-type2]>

(Optional) Specifies related object types to be excluded when getting the property value when it is specified in the **-assignee** argument (for example, **-assignee=user:PROP::<property_name>**). They must be valid object types.

You can use this argument only when you get the assignee from a property on an object (**user:PROP::** or **resourcepool:PROP::**) and you use the **-from_relation** argument.

-clear

(Optional) Removes all previously assigned participants before assigning new participants. If this argument is not specified, new participants are appended to existing participants list.

-first_object_only

(Optional) Sets the participants on the first target attachment matching the **-to_include_type** and **-to_exclude_type** arguments. If this argument is not specified, the participants are set on all target attachments matching the **-to_include_type** and **-to_exclude_type** arguments.

-bypass_condition_check

(Optional) Bypasses the Business Modeler IDE condition check before assigning participants. If this argument is not specified, the Business Modeler IDE conditions are enforced before assigning participants.

-project_scope

(Optional) Specifies which projects are used to resolve project-based assignments. The **all** value specifies all projects in the list of projects. The **owning_project** value specifies the owning project only.

If this argument is not specified, the default value is the first project in the project list.

-check_first_object_only

(Optional) The **true** value specifies that only the first object is checked. If the value is **false**, all objects are checked. If this argument is not specified, or if it is specified without a value, only the first object is checked.

If the **-include_type**, **-exclude_type**, **-include_related_type**, or **-exclude_related_type** arguments are specified, they determine the types of objects that are checked.

-condition_name

(Optional) The name of the condition to evaluate against the identified objects from which to assign participants. The condition signature should accept a **WorkspaceObject & UserSession**. The handler assigns the reviewers only if the condition results are successful, based on the **-condition_scope** argument.

-condition_scope

(Optional) The criteria for evaluating condition results against workflow objects. Values are the following:

- all** All objects should meet the condition. This is the default behavior if this argument is not supplied with the **-condition_name** argument.
- any** Any object should meet the condition.
- none** No object should meet the condition.

Placement

Place on the **Start** action.

Restrictions

Can only be used to assign dynamic participants that resolve to multiple users. For example:

PROPOSED_REVIEWERS or **CHANGE_REVIEW_BOARD**

Examples

- Assigns the users **Smith** and **David** as the **PROPOSED_REVIEWERS** participant for all target objects in the workflow process.

Argument	Values
-name	PROPOSED_REVIEWERS
-assignee	user:Smith,David

- Reads the **owning_user** and **last_mod_user** properties from the target and assigns the user as the **PROPOSED_REVIEWERS** participant for the first target object only.

Argument	Values
-name	PROPOSED_REVIEWERS
-assignee	user:PROP::owning_user,user:PROP::last_mod_user
-first_object_only	

- Reads the **owning_user** and **last_mod_user** properties from the **Document Revision** type target and assigns the user as the **PROPOSED_REVIEWERS** participant.

Argument	Values
-name	PROPOSED_REVIEWERS
-assignee	user:PROP::owning_user,user:PROP::last_mod_user
-from_include_type	DocumentRevision

- Traverses the **References** relation from the **Part Revision** types of the targets to get the **Document Revision** objects. It then reads the **owning_user** and **last_mod_user** properties from the **Document Revision** and assigns the user as the **PROPOSED_REVIEWERS** participant for all target objects.

Argument	Values
-name	PROPOSED_REVIEWERS
-assignee	user:PROP::owning_user,user:PROP::last_mod_user
-from_include_type	Part Revision
-from_relation	IMAN_reference
-from_include_related_type	DocumentRevision

- This example assigns all members of the **Engineering** group and **Designer** role of the first project team associated with the first target found by the system to the dynamic participant.

Argument	Values
-name	PROPOSED_RESPONSIBLE_PARTY
-assignee	\$PROJECT_MEMBER[Engineering::Designer]

- This example assigns all members of the **Engineering** group and **Designer** role of the owning project team associated with the first target found by the system to the dynamic participant.

Argument	Values
-name	PROPOSED_REVIEWERS
-assignee	\$PROJECT_MEMBER[Engineering::Designer]

Argument	Values
-project_scope	owning_project
-check_first_object_only	

- This example assigns all members of the **Engineering** group and **Designer** role of all project teams associated with the first target found by the system to the dynamic participant.

Argument	Values
-name	PROPOSED_REVIEWERS
-assignee	\$PROJECT_MEMBER[Engineering::Designer]
-project_scope	all
-check_first_object_only	true

- This example assigns all members of the **Engineering** group and **Designer** role of the first project team associated with each target found by the system to the dynamic participant.

Argument	Values
-name	PROPOSED_REVIEWERS
-assignee	\$PROJECT_MEMBER[Engineering::Designer]
-check_first_object_only	false

EPM-assign-team-selector

Description

Assigns all **select-signoff-team** tasks in the entire workflow process to the specified user, person, initiator (owner), or resource pool of the workflow process. Only one argument can be defined; all arguments are mutually exclusive of each other.

Syntax

EPM-assign-team-selector

```
-assignee= [user:<user> | person:<person> | resourcepool:<group>::<role>
| user:PROP::<property_name>
| resourcepool:PROP::<property_name>
| $PROPOSED_RESPONSIBLE_PARTY | $USER
| $PROCESS_OWNER | $TARGET_OWNER [<type>]
| $PROJECT_ADMINISTRATOR
| $PROJECT_TEAM_ADMINISTRATOR
| $PROJECT_AUTHOR | $PROJECT_MEMBER<[group]::<role>]
| $REQUESTOR | $ANALYST
| $CHANGE_SPECIALIST1
| $CHANGE_SPECIALIST2
| $CHANGE_SPECIALIST3]
[-from_include_type=<object-type1[,object-type2,...]>|
[-from_exclude_type=<object-type1[,object-type2,...]>]
[-from_attach= target | reference | schedule_task]
[-from_relation=<relation-type>]
[-from_include_related_type=<object-type1[,object-type2,...]> |
- from_exclude_related_type=<object-type1[,object-type2,...]>]
[-target_task=<multilevel-task-path>]
[-project_scope=all | owning_project]
[-check_first_object_only=<true | false>]
[-condition_name=<condition1>]
[-condition_scope=all | any | none]
```

Arguments

-assignee

(Required) Makes the user or resource pool the specified keyword evaluates to the responsible party for the task to which this handler is added. Accepts one of the following in the format specified below:

- **user:**<user>

Adds the user specified to the signoff member list for the task to which it is attached. Accepts a valid Teamcenter user ID.

- **person:**<person>

Adds the user whose name is specified to the signoff member list for the task to which it is attached. Accepts a valid Teamcenter person name.

Note

If the person's name includes a comma, you must include an escape character (\) to add the correct person. For example, to use **wayne, joan**:

-assignee=person:wayne\, joan

- **resourcepool:<group>::<role>**

Results in a single assignment which can be performed by any single member of this group/role.

You can define resource pools in the form of <group>::, <group>::<role>, or <role>.

Note

When a resource pool task is performed by a user it is automatically claimed by that user. If that task is a **Review** task, and it is started again, the task is assigned to the user who performed it in the previous iteration, rather than the resource pool.

Accepts valid Teamcenter resource pool names and these keywords:

- **\$GROUP**

Current user's current group.

- **\$ROLE**

Current user's current role.

- **\$TARGET_GROUP [<type>]**

Owning group of the first target object of the specified type. The <type> value is optional. If not specified, the first target is used.

- **\$PROCESS_GROUP**

Owning group of the workflow process.

- **user:PROP::<property_name>**

Adds the user specified by the property name to the signoff member list for the task to which it is attached.

If the property is a multi-value property, only the first value is used when only a single user is assigned in the workflow. When more than one user is assigned, all property values are used.

- **resourcepool:PROP::<property_name>**

Adds the resource pool specified by the property name to the signoff member list for the task to which it is attached.

If the property is a multi-value property, only the first value is used when only a single user is assigned in the workflow. When more than one user is assigned, all property values are used.

- **\$PROPOSED_RESPONSIBLE_PARTY**

Affects assignments based on the user assigned as the responsible party for the first target object.

- **\$USER**

Adds the current user to the signoff member list.

- **\$PROCESS_OWNER**

Adds the workflow process owner to the signoff member list.

- **\$TARGET_OWNER [<type>]**

Adds the owner of the first target of specified type to the signoff member list. The <type> value is optional. If not specified, the first target is used.

- **\$PROJECT_ADMINISTRATOR, \$PROJECT_TEAM_ADMINISTRATOR, \$PROJECT_AUTHOR, \$PROJECT_MEMBER[<group>::**

Dynamically makes the first project team member belonging to the role specified in the argument value as the responsible party. The project team is determined by the project team associated with the first target object.

- If the **\$PROJECT_MEMBER[<group>:: argument is specified, only the project members of the qualifying projects which belong to the specified group and role are selected for assignment. The first member from that selected list will assign as a responsible party.**

If the group and role are not specified, all the project members from qualifying projects are considered. The first member from that selected list will assign as a responsible party.

- If the value is specified as **\$PROJECT_AUTHOR** or **\$PROJECT_MEMBER[<group>::, the relevant first project member is selected.**
- You can specify a sub-group with the syntax **<group>++<sub-group>::.**

- **\$REQUESTOR, \$ANALYST, \$CHANGE_SPECIALIST1, \$CHANGE_SPECIALIST2, \$CHANGE_SPECIALIST3**

Dynamically resolves to the user or resource pool associated with the first change target object in the workflow process. The particular user or resource pool is determined by the role specified in the argument value.

Note

Change-related keywords apply only to change objects. If the workflow process does not contain a change object as a target, the argument resolves to null.

Change Manager does not need to be enabled before these keywords take effect, but during installation, make sure you install the **Change Management** application using your preferred installation tool (Deployment Center or Teamcenter Environment Manager) to ensure change-related keywords are enabled.

-from_include_type=<object-type1[,object-type2,...]>

(Optional) Specifies the object types to be used to get the property value from when a property is specified in the **-assignee** argument (for example, **-assignee=user:PROP::). They must be valid object types.**

You can use this argument only when you get the assignee from a property on an object (**user:PROP::** or **resourcepool:PROP::**).

-from_exclude_type=<object-type1[,object-type2,...]>

(Optional) Specifies the object types to be excluded when getting the property value when it is specified in the **-assignee** argument (for example, **-assignee=user:PROP::<property_name>**). They must be valid object types.

You can use this argument only when you get the assignee from a property on an object (**user:PROP::** or **resourcepool:PROP::**).

-from_attach= target | reference | schedule_task

(Optional) Specifies which type of attachment (**target**, **reference**, or **schedule_task**) to get the property value from when a property is specified in the **-assignee** argument (for example, **-assignee=user:PROP::<property_name>**). If this argument is not specified, the default is **target**.

You can use this argument only when you get the assignee from a property on an object (**user:PROP::** or **resourcepool:PROP::**).

-from_relation

(Optional) Specifies the relation of the objects to get the property value from when a property is specified in the **-assignee** argument (for example, **-assignee=user:PROP::<property_name>**). It must be a valid relation.

- For manifestations, use **IMAN_manifestation**.
- For specifications, use **IMAN_specification**.
- For requirements, use **IMAN_requirement**.
- For references, use **IMAN_reference**.
- For BOM views, use **PSBOMViewRevision**.

This argument must be used with the **-from_attach** argument. A derived object is identified by starting with objects of the specified attachment type indicated by the **-from_attach** argument and then locating the first secondary object with the specified relation indicated by the **-relation** argument.

You can use this argument only when you get the assignee from a property on an object (**user:PROP::** or **resourcepool:PROP::**).

-from_include_related_type=<object-type1[,object-type2]>

(Optional) Specifies the related object types to be used to get the property value from when a property is specified in the **-assignee** argument (for example, **-assignee=user:PROP::<property_name>**). They must be valid object types.

You can use this argument only when you get the assignee from a property on an object (**user:PROP::** or **resourcepool:PROP::**) and you use the **-from_relation** argument.

This argument should not be used with the **-from_exclude_related_type** argument.

-from_exclude_related_type=<object-type1[,object-type2]>

(Optional) Specifies related object types to be excluded when getting the property value when it is specified in the **-assignee** argument (for example, **-assignee=user:PROP::<property_name>**). They must be valid object types.

You can use this argument only when you get the assignee from a property on an object (**user:PROP::** or **resourcepool:PROP::**) and you use the **-from_relation** argument.

This argument should not be used with the **-from_include_related_type** argument.

-target_task

(Optional) Specifies the multilevel task path to which the reviewers are added. The path is from the root task to the **select-signoff-team** subtask with the path levels separated with colons (:). For example:

Change Request Review:QA Review:select-signoff-team

-project_scope

(Optional) Specifies which projects are used to resolve project-based assignments. The **all** value specifies all projects in the list of projects. The **owning_project** value specifies the owning project only.

If this argument is not specified, the default value is the first project in the project list.

-check_first_object_only

(Optional) The **true** value specifies that only the first object is checked. If the value is **false**, all objects are checked. If this argument is not specified, or if it is specified without a value, only the first object is checked.

If the **-include_type**, **-exclude_type**, **-include_related_type**, or **-exclude_related_type** arguments are specified, they determine the types of objects that are checked.

-condition_name

(Optional) The name of the condition to evaluate against the identified objects from which to assign tasks. The condition signature should accept a **WorkspaceObject & UserSession**. The handler assigns the reviewers only if the condition results are successful, based on the **-condition_scope** argument.

-condition_scope

(Optional) The criteria for evaluating condition results against workflow objects. Values are the following:

- all** All objects should meet the condition. This is the default behavior if this argument is not supplied with the **-condition_name** argument.
- any** Any object should meet the condition.
- none** No object should meet the condition.

Placement

Place on the **Start** action of the root task.

Restrictions

None.

Examples

- This example assigns the user **jim** all **select-signoff-team** tasks in that workflow process.

Argument	Values
-assignee	user:jim

- This example assigns the person **Jim Smith** all **select-signoff-team** tasks in that workflow process.

Argument	Values
-assignee	person:Jim Smith

- This example assigns the owner of the workflow process all **select-signoff-team** tasks in that workflow process.

Argument	Values
-assignee	\$PROCESS_OWNER

- This example assigns the user or resource pool assigned as the responsible party for all **select-signoff-team** tasks in that workflow process.

Argument	Values
-assignee	\$PROPOSED_RESPONSIBLE_PARTY

- This example makes the project administrator of the project associated with the first target the responsible party for all **select-signoff-team** tasks in that workflow process.

Argument	Values
-assignee	\$PROJECT_ADMINISTRATOR

- This example makes the user or resource pool associated as **REQUESTOR** with the first change target the responsible party for all **select-signoff-team** tasks in the workflow process.

Argument	Values
-assignee	\$REQUESTOR

- This example assigns the first member of the **Engineering** group and **Designer** role of the first project team associated with the first target found by the system to the **select-signoff-team** task.

Argument	Values
-assignee	\$PROJECT_MEMBER[Engineering::Designer]

EPM-attach-related-objects

Description

Attaches the specified related objects of the target objects as target or reference attachments to the workflow process. This handler searches all target objects, finds the secondary objects with the specified relation or in the specified reference property and type (if specified), then adds them as target or reference attachments.

If the handler is configured to attach a secondary object as a target, but that secondary object already exists as a target, then the secondary target object is ignored and is not attached (since it already exists). Likewise, if you want to attach a reference as a secondary object, but the secondary object already exists as a reference, then that secondary object is ignored and is not attached. Note, however, that this handler manages targets and references as two different lists; an object can be attached to one list even if it is already attached to the other list.

Note

If the **WRKFLW_allow_replica_targets** preference is set to **true** and if any replica object qualifies to be attached as a workflow target, that object is attached as a **Replica Proposed Target** to the workflow process. If the intended attachment type is not a target, the replica object is attached as the attachment type defined in **-attachment** argument.

If the preference is set to **false** or is undefined, the handler reports an error and attaches replica objects as targets.

Further, if the **-from_attach** argument is set to **schedule_task** and if the attached schedule task is a proxy link, the handler ignores the schedule task proxy link for any processing.

Note

If the handler attempts to attach related objects that are checked out, the workflow process fails. You can use a **Validate** task to branch to a workflow path to have the objects checked in.

Note

To replace the obsolete **EPM-attach-item-revision-targets** handler, use the following two instances of the **EPM-attach-related-objects**:

- **EPM-attach-related-objects**

Arguments	Values
-relation	IMAN_specification
-attachment	target

- **EPM-attach-related-objects**

Argument	Values
-relation	PSBOMViewRevision
-attachment	target

Note

Enable debugging functionality for this handler with the **TC_HANDLERS_DEBUG** environment variable.

Syntax**EPM-attach-related-objects**

```
{-relation=<relation-name> | -property=<property-name>}
[include_related_type=<object-type1[,object-type2,...]> |
|-exclude_related_type=<object-type1[,object-type2,...]>] ] |
[-lov=<lov-name>]
-attachment= target | reference
[-from_attach= target | reference | schedule_task]
[-allowed_status=<status1>
[,null,<status2,...>] | * | all | any | null | none | IN_PROCESS]
[-disallowed_status=<status1>
[,null,<status2,...>] | * | all | any | null | none | IN_PROCESS]
-user_access
```

Arguments

-relation=<relation-name> | -property=<property-name>

(Optional) Specifies the relation or the property that is used to locate and attach secondary objects. You can use only one of these two arguments.

-relation=<relation-name>

Specifies the relation of the secondary objects to be attached to the target. The relation name must be valid for the relation type.

Relation type	Valid relation name
Manifestation	IMAN_manifestation
Specification	IMAN_specification
Requirement	IMAN_requirement
Reference	IMAN_reference
BOM view	PSBOMViewRevision

Note

You cannot use this argument with the **-property** argument.

-property=<property-name>

Specifies the target object property whose value lists the secondary objects to be attached to the target.

Note

You cannot use this argument with the **-relation** argument.

-include_related_type=<object-type1[,object-type2]>

(Optional) Specifies object types to be attached.

They must be valid object types. This argument is optional.

This argument should not be used with the **-exclude_related_type** argument.

-exclude_related_type=<object-type1[,object-type2]>

(Optional) Specifies object types to be excluded.

They must be valid object types. This argument is optional.

This argument should not be used with the **-include_related_type** argument.

-lov=<lov-name>

(Optional) Specifies a list of values (LOV) to use to define which objects to attach.

Use only with the **-attachment**, **-allowed_status** and **-disallowed_status** arguments. This argument is mutually exclusive of the **-relation**, **-include_related_type**, and **-exclude_related_type** arguments.

For an overview of using LOVs in handlers, see [Using lists of values \(LOVs\) in handler arguments](#).

-attachment= target | reference

(Required) Attachment type with which the objects are attached to the workflow process.

-from_attach= target | reference | schedule_task

(Optional) Finds the related objects with the specified relation or property argument from the specified types of attachments (**target**, **reference**, or **schedule_task**).

-allowed_status=<status1>[,null,<status2>,...] | * | all | any | null | none | IN_PROCESS

(Optional) Defines allowed statuses. Only objects with a release status matching a status defined in the list are attached.

null | NULL | none | NONE matches no status (or WIP).

*** | all | ALL | any | ANY** matches any status set, excluding null.

IN_PROCESS checks whether the object is currently in a workflow process.

Note

The **-allowed_status** and **-disallowed_status** arguments are mutually exclusive. If you use one of them, you cannot use the other in the same handler instance.

-disallowed_status=<status1>[,null,<status2>,...] | * | all | any | null | none | IN_PROCESS

(Optional) Defines statuses that are not allowed. Only objects with a release status not matching a status defined in the list are attached.

null | NULL | none | NONE matches no status (or WIP).

*** | all | ALL | any | ANY** matches any status set, excluding null.

IN_PROCESS checks whether the object is currently in a workflow process.

Note

The **-allowed_status** and **-disallowed_status** arguments are mutually exclusive. If you use one of them, you cannot use the other in the same handler instance.

-user_access

(Optional) By default, this handler automatically turns on bypass, which means if it encounters an object that the current user does not have access to, the handler will still be allowed to attach the object without an error.

However, you can use **-user_access** to prevent the handler from using bypass. If the handler is configured with **-user_access** and the handler encounters an object that the current user does not have access to, then the object will not attach, and an error will occur.

This argument does not take a value.

Note

This functionality may be affected by preference **WRKFLW_access_level_for_handlers_execution**. The default behavior for the **-user_access** argument is used when **WRKFLW_access_level_for_handlers_execution** is set to its default value of **regular**. However, if **WRKFLW_access_level_for_handlers_execution** is set to **system**, then bypass will always be used regardless of how the **EPM-attach-related-objects** handler is configured. So in this case, **-user_access** will have no effect.

LOV

For an overview of using LOVs in handlers, see [Using lists of values \(LOVs\) in handler arguments](#).

The LOV can contain multiple optional lines containing filter options followed by multiple lines containing multilevel object paths.

Note

For an overview and examples of multilevel object paths in handlers, see [Defining multilevel object paths](#).

Each multilevel object path line can optionally have a filter option added as a second field after a tilde (~).

OPTION=<value>

OPTION=<value>

{\$TARGET|\$REFERENCE}.<multi.level.object.path>[~ OPTION=<value>]

{\$TARGET|\$REFERENCE}.<multi.level.object.path>[~ OPTION=<value>]

OPTION=<value>

Defines a configurable option to filter object selection.

If you supply an option on an LOV line on its own, it applies to all subsequent lines containing multilevel object paths. The option does not affect any multilevel object paths listed before the option.

If you supply an option on the same line as a multiple level object path, as a second field after a tilde (~) character, it only applies to that line.

Valid values are:

- **REV RULE={LATEST|<Rule>}**

Specifies the revision rule used to select the revision attached to the workflow process if initiated on an item. Use the **LATEST** keyword to select only the latest revision.

- **INCLUDE PARENTS=YES**

Specifies that all objects found by traversing a multilevel path are attached to the workflow process, not just the last set of objects in a path. For example, when a multilevel path is used to first find items in a workflow process, then find revisions in the item, then find datasets in the revisions, it is only the datasets that are attached by default. Setting this argument to **YES** causes both the revisions and the datasets to be attached.

This argument reduces the number of lines required in the LOV and improves performance.

\$TARGET|\$REFERENCE

Defines the starting point from which to look for objects. Valid values are:

- **\$TARGET**

Defines the starting point as the workflow process target attachments.

- **\$REFERENCE**

Defines the starting point as the workflow process reference attachments.

<multi.level.object.path>

Defines a multilevel object path to traverse to find the required objects to attach to the workflow process. For an overview of using multilevel object paths in handlers, see [Defining multilevel object paths](#).

For example, **(ItemRevision).IMAN_specification.(Dataset)**.

Attaches any datasets attached to the specification relation to any revisions found.

For more examples, see the **Examples** section.

Placement

Typically placed on the **Start** action of the root task so that the list of target attachments is updated at workflow process initiation.

To allow targets to be added to a workflow process containing a task on which this handler has been placed (other than the root task), verify that the **EPM-disallow-adding-targets** handler does not exist on the root task of the respective workflow process template and ensure that the affected users have change access to the workflow process object. You may use the **EPM-set-rule-based-protection** handler to ensure that the required change access is asserted. See [Executing workflow handlers](#) for more information.

Restrictions

- Requires one or more target objects to find the related objects. Placement should allow at least one target object before the execution of this handler takes place.
- To attach both targets and references using LOVs requires two occurrences of the handler: one to attach the targets by setting the **-attachment** argument to **target**, and one to attach the references using the **-attachment** argument to **reference**.
- The LOV argument cannot be used to attach objects based on properties.

Examples

- This example attaches all objects with a specification relation as target objects to the workflow process, when a workflow process is initiated on an item revision:

Arguments	Values
-relation	IMAN_specification
-attachment	target

Note

If an object is already attached as target, it is not added.

- In this example, all objects listed in the **altid_list** property value are attached to the workflow process as target objects, when a workflow process is initiated on an item revision:

Arguments	Values
-property	altid_list
-attachment	target

Note

- The property named in the argument value must exist on the target.
- If an object is already attached as target, it is not added.

- To attach all objects with a reference relation as reference objects, add this handler one more time with the syntax:

Argument	Values
-relation	IMAN_reference
-attachment	reference

- This example attaches the BOM view revision type **View** as a target:

Argument	Values
-relation	PSBOMViewRevision
-include_related_type	view
-attachment	target

Alternatively, you can use these LOV settings:

Argument	Values
-lov	SYS_EPM_attach_view_bvr

where the **SYS_EPM_attach_view_bvr** LOV contains the value:

\$TARGET.(ItemRevision).PSBOMViewRevision.BOMView Revision

- This example attaches the **UGMASTER** and the **UGPART** datasets (associated by the **IMAN_specification** relation to the item revision) to the item revision as target objects.

Argument	Values
-relation	IMAN_specification
-include_related_type	UGMASTER, UGPART
-attachment	target

Alternatively, you can use these LOV settings:

Argument	Values
-lov	SYS_EPM_attach_UGMASTER_UGPART

where the **SYS_EPM_attach_UGMASTER_UGPART** LOV contains the data:

\$TARGET.(ItemRevision).IMAN_specification.UGMASTER,UGPART

- This example uses the **-exclude_related_type** argument to specify object types that are not to be attached as targets to the workflow process. It attaches all objects attached to the **Specification** relation in any target revisions as targets to the workflow process, except for the dataset types **UGMASTER** and **Text**.

Argument	Values
-relation	IMAN_specification
-exclude_related_type	UGMASTER, Text
-attachment	target

Alternatively, you can use these LOV settings:

Argument	Values
-lov	SYS_EPM_exclude_UGMASTER

where the **SYS_EPM_exclude_UGMASTER** LOV contains the data:

\$TARGET.(ItemRevision).IMAN_specification.(*)!UGMASTER!Text

Note

Use an * for any class, then exclude **UGMASTER** and **Text**:

- This example attaches all specification objects, all BOM view revisions, all forms attached to datasets through a **Form** reference (except **UGPartAttr** forms), and all forms attached through a **manifestation** relation. Only attach objects that not released.

Argument	Values
-lov	SYS_EPM_attach_main_objects
-attachment	target
-allowed_status	null

Where the **SYS_EPM_attach_main_objects** LOV contains the data:

Value	Description
<code>\$TARGET.(ItemRevision).Specification.*</code>	Attach all objects in target revision Specification relation
<code>\$TARGET.(ItemRevision).IMAN_specification.UGMASTER.UGPART-ATTR.UGPartAttr</code>	Attach all forms attached to datasets in target revision # Specification relation as a Form reference, but excluding the # form type UGPartAttr.
<code>\$TARGET.(ItemRevision).PSBOMViewRevision.*</code>	Attach all BOM View Revisions in target revision
<code>\$TARGET.(ItemRevision).Manifestation.(Form)</code>	Attach all forms in target revision Manifestation relation

- This example attaches all required revision attachments, such as specification objects and BOM view revisions, regardless of whether the workflow process is initiated on revisions, items or folders containing the items or revisions. If the method of initiating workflow processes on items or folders is convenient, use the **EPM-remove-objects** handler to remove the items and/or folders from the targets after this handler.

When the targets are item revisions, attach all specification objects, all BOM view revisions and any objects attached to specification datasets as relations and references (only attaches workspace objects).

When the targets are items, attach all the latest revisions and all objects mentioned above for each revision.

When the targets are folders, attach any items in the folders and the item revisions and the revision attachments. For any revisions in the folder, attach the revisions' attachments.

Only attach objects not already released or with a status of **Pending**.

Argument	Values
<code>-lov</code>	<code>SYS_EPM_attach_main_objects</code>
<code>-attachment</code>	<code>target</code>
<code>-allowed_status</code>	<code>null, Pending</code>

Where the `SYS_EPM_attach_main_objects` LOV contains the data:

Value	Description
<code>INCLUDE PARENTS = YES</code>	Set option for all lines to include all objects found
<code>REV RULE = LATEST</code>	Set the revision rule for any items
<code>\$TARGET.(ItemRevision).IMAN_specification, PSBOMViewRevision.*.* ~</code>	Attach required objects from REVISION targets

Value	Description
<code>\$TARGET.(Item).Revisions.*.IMAN_specification, PSBOMViewRevision.*.*</code>	Attach required objects from latest revisions in ITEM targets
<code>\$TARGETS.(Folder).*(Item).Revisions.*.IMAN_specification, PSBOMViewRevision.*.*</code>	Attach required objects from FOLDER targets
<code>\$TARGETS.(Folder).*(ItemRevision).IMAN_specification, PSBOMViewRevision.*.*</code>	Look for items and revisions in the folders

Additional Information

With the addition of this handler, these handlers are deprecated:

EPM-attach-item-revision-target

As the **EPM-attach-item-revision-target** handler attaches BOM view revisions and objects with **IMAN_specification** relation, this handler can be replaced using one of the following options:

- Adding the **EPM-attach-related-objects** handler two times (one for specification relation and one for BOM view revisions) with the syntax:

EPM-attach-related-objects

Argument	Values
<code>-relation</code>	IMAN_specification
<code>-attachment</code>	target

EPM-attach-related-objects

Argument	Values
<code>-relation</code>	PSBOMViewRevision
<code>-attachment</code>	target

- Adding the **EPM-attach-related-objects** handler once using an LOV:

EPM-attach-related-objects

Argument	Values
<code>-lov</code>	SYS_EPM_attach_default_objects
<code>-attachment</code>	target

Where the **SYS_EPM_attach_main_objects** LOV contains the data:

`$TARGET . (ItemRevision) . Specification, PSBOMViewRevision . *`

EPM-auto-assign

Description

Makes the specified user or resource pool the responsible party for the task to which the handler is added. Optionally, you can make the same specified user or resource pool the responsible party for all subtasks of the parent task.

Note

If you use keyword arguments to dynamically generate this assignment, and the system resolve the argument to a user or resource pool, then the argument is ignored.

Syntax

```
EPM-auto-assign [-subtasks]
[-assignee= {user:<user> | person:<person> | resourcepool:<group>::<role>
| user:PROP::<property_name>
| resourcepool:PROP::<property_name>
| $PROPOSED_RESPONSIBLE_PARTY | $USER
| $PROCESS_OWNER | $TARGET_OWNER [<type>]
| $PROJECT_ADMINISTRATOR
| $PROJECT_TEAM_ADMINISTRATOR
| $PROJECT_AUTHOR | $PROJECT_MEMBER<[group>::<role>]
| $REQUESTOR | $ANALYST
| $CHANGE_SPECIALIST1
| $CHANGE_SPECIALIST2
| $CHANGE_SPECIALIST3}]
[-from_include_type=<object-type1[,object-type2,...]>|
[-from_exclude_type=<object-type1[,object-type2,...]>]
[-from_attach= target | reference | schedule_task]
[-from_relation=<relation-type>]
[-from_include_related_type=<object-type1[,object-type2,...]> |
- from_exclude_related_type=<object-type1[,object-type2,...]>]
[-target_task=<multilevel-task-path>]
[-project_scope=all | owning_project]
[-check_first_object_only=<true | false>]
[-condition_name=<condition1>]
[-condition_scope=all | any | none]
```

Arguments

-subtasks

(Optional) Propagates task assignments to subtasks of the current task (nonrecursively).

-assignee

(Required) Assigns as the responsible party for the task to which this handler is added either the specified person, user, resource pool, or the user or resource pool the specified keyword evaluates to.

Accepts one of the following in the format specified below:

- **user:***<user>*

Adds the specified user to the signoff member list and as the responsible party for the task to which the handler is attached. Accepts a valid Teamcenter user ID.

- **person:***<person>*

Adds the person whose name is specified to the signoff member list and as the responsible party for the task to which the handler is attached. Accepts a valid Teamcenter person name.

Note

If the person's name includes a comma, you must include an escape character (\) to add the correct person. For example, to use **wayne, joan**:

-assignee=person:wayne\, joan

- **resourcepool:***<group>::<role>*

Results in a single assignment which can be performed by any single member of this group/role.

You can define resource pools in the form of *<group>::*, *<group>::<role>*, or *<role>*.

Note

When a resource pool task is performed by a user it is automatically claimed by that user. If that task is a **Review** task and it is started again, the task is assigned to the user who performed it in the previous iteration, rather than the resource pool.

Accepts valid Teamcenter resource pool names and these keywords:

- **\$GROUP**

Current user's current group.

- **\$ROLE**

Current user's current role.

- **\$TARGET_GROUP**[*<type>*]

Owning group of the first target object of the specified type. The *<type>* value is optional. If not specified, the first target is used.

- **\$PROCESS_GROUP**

Owning group of the workflow process.

Note

The **\$ROLE_IN_GROUP** keyword (formerly **\$ROLEINGROUP**) cannot be used. Use **resourcepool:\$GROUP::\$ROLE** instead.

- **user:PROP:***<property_name>*

Adds the user specified by the property name to the signoff member list for the task to which it is attached.

If the property is a multi-value property, only the first value is used when only a single user is assigned in the workflow. When more than one user is assigned, all property values are used.

- **resourcepool:PROP::<property_name>**

Adds the resource pool specified by the property name to the signoff member list for the task to which it is attached.

If the property is a multi-value property, only the first value is used when only a single user is assigned in the workflow. When more than one user is assigned, all property values are used.

- **\$PROPOSED_RESPONSIBLE_PARTY**

Affects assignments based on the user assigned as the responsible party for the first target object.

- **\$USER**

Adds the current user to the signoff member list and as the responsible party.

- **\$PROCESS_OWNER**

Adds the workflow process owner to the signoff member list and as the responsible party.

- **\$TARGET_OWNER [<type>]**

Adds the owner of the first target of the specified type to the signoff member list and as the responsible party. The <type> value is optional. If not specified, the first target is used.

- **\$PROJECT_ADMINISTRATOR, \$PROJECT_TEAM_ADMINISTRATOR, \$PROJECT_AUTHOR, \$PROJECT_MEMBER[<group>::<role>]**

Dynamically makes the first project team member belonging to the role specified in the argument value as the responsible party. The project team is determined by the project team associated with the first target object.

- If the **\$PROJECT_MEMBER[<group>::<role>]** argument is specified, only the project members of the qualifying projects which belong to the specified group and role are selected for assignment. If the group and role are not specified, all the project members from qualifying projects are selected.
- If the value is specified as **\$PROJECT_AUTHOR** or **\$PROJECT_MEMBER[<group>::<role>]**, the relevant first project member is selected.
- You can specify a sub-group with the syntax **<group>+<sub-group>::<role>**.

- **\$REQUESTOR, \$ANALYST, \$CHANGE_SPECIALIST1, \$CHANGE_SPECIALIST2, \$CHANGE_SPECIALIST3**

Dynamically resolves to the user or resource pool associated with the first change target object in the workflow process. The particular user or resource pool is determined by the role specified in the argument value.

Note

Change-related keywords apply only to change objects. If the workflow process does not contain a change object as a target, the argument resolves to null.

Change Manager does not need to be enabled before these keywords take effect, but during installation, make sure you install the **Change Management** application using your preferred installation tool (Deployment Center or Teamcenter Environment Manager) to ensure change-related keywords are enabled.

-from_include_type=<object-type1[,object-type2,...]>

(Optional) Specifies the object types to be used to get the property value from when a property is specified in the **-assignee** argument (for example, **-assignee=user:PROP::<property_name>**). They must be valid object types.

-from_exclude_type=<object-type1[,object-type2,...]>

(Optional) Specifies the object types to be excluded when getting the property value when it is specified in the **-assignee** argument (for example, **-assignee=user:PROP::<property_name>**). They must be valid object types.

-from_attach= target | reference | schedule_task

(Optional) Specifies which type of attachment (**target**, **reference**, or **schedule_task**) to get the property value from when a property is specified in the **-assignee** argument (for example, **-assignee=user:PROP::<property_name>**). If this argument is not specified, the default is **target**.

-from_relation

(Optional) Specifies the relation of the objects to get the property value from when a property is specified in the **-assignee** argument (for example, **-assignee=user:PROP::<property_name>**). It must be a valid relation.

- For manifestations, use **IMAN_manifestation**.
- For specifications, use **IMAN_specification**.
- For requirements, use **IMAN_requirement**.
- For references, use **IMAN_reference**.
- For BOM views, use **PSBOMViewRevision**.

This argument must be used with the **-from_attach** argument. A derived object is identified by starting with objects of the specified attachment type indicated by the **-from_attach** argument and then locating the first secondary object with the specified relation indicated by the **-relation** argument.

-from_include_related_type=<object-type1[,object-type2]>

(Optional) Specifies the related object types to be used to get the property value from when a property is specified in the **-assignee** argument (for example, **-assignee=user:PROP::<property_name>**). They must be valid object types.

Use this argument when a property is designated and you use the **-from_relation** argument.

This argument should not be used with the **-from_exclude_related_type** argument.

-from_exclude_related_type=<object-type1[,object-type2]>

(Optional) Specifies related object types to be excluded when getting the property value when it is specified in the **-assignee** argument (for example, **-assignee=user:PROP::<property_name>**). They must be valid object types.

Use this argument when a property is designated and you use the **-from_relation** argument.

This argument should not be used with the **-from_include_related_type** argument.

-target_task

(Optional) Specifies the multilevel task path to which the reviewers are added. The path is from the root task to the subtask with the path levels separated with colons (:). For example: **Change Request Review:QA Review:perform-signoff**

-project_scope

(Optional) Specifies which projects are used to resolve project-based assignments. The **all** value specifies all projects in the list of projects. The **owning_project** value specifies the owning project only.

If this argument is not specified, the default value is the first project in the project list.

-check_first_object_only

(Optional) The **true** value specifies that only the first object is checked. If the value is **false**, all objects are checked. If this argument is not specified, or if it is specified without a value, only the first object is checked.

If the **-include_type**, **-exclude_type**, **-include_related_type**, or **-exclude_related_type** arguments are specified, they determine the types of objects that are checked.

-condition_name

(Optional) The name of the condition to evaluate against the identified objects from which to assign tasks. The condition signature should accept a **WorkspaceObject & UserSession**. The handler assigns the reviewers only if the condition results are successful, based on the **-condition_scope** argument.

-condition_scope

(Optional) The criteria for evaluating condition results against workflow objects. Values are the following:

- all** All objects should meet the condition. This is the default behavior if this argument is not supplied with the **-condition_name** argument.
- any** Any object should meet the condition.
- none** No object should meet the condition.

Placement

Place on the **Start** action.

Restrictions

None.

Examples

- This example makes **Smith** the responsible party for the task to which this handler is assigned and all of the task's subtasks.

Argument	Values
-subtasks	
-assignee	user:Smith

- This example makes the workflow process owner the responsible party for the task to which this handler is assigned.

Argument	Values
-assignee	\$PROCESS_OWNER

- This example makes the engineer role within manufacturing group resource pool the responsible party for the task to which this handler is assigned.

Argument	Values
-assignee	resourcepool:manufacturing::engineer

- This example makes the responsible party group the responsible party for the task to which this handler is assigned.

Argument	Values
-assignee	\$PROPOSED_RESPONSIBLE_PARTY

- This example makes the project administrator of the project associated with the first target the responsible party for the task to which this handler is assigned.

Argument	Values
-assignee	\$PROJECT_ADMINISTRATOR

- This example makes the user or resource pool associated as **ANALYST** with the first change target the responsible party for the task to which this handler is assigned.

Argument	Values
-assignee	\$ANALYST

- This example assigns the first member of the **Engineering** group and **Designer** role of the first project team associated with the first target found by the system to the task as responsible party.

Argument	Values
-assignee	\$PROJECT_MEMBER[Engineering::Designer]

EPM-auto-assign-rest

Description

Automatically makes the specified assignee the responsible party for any unassigned subtasks of the parent task to which this handler is added.

- If this handler is attached to the root task with no argument specified, the workflow process initiator is made the responsible party for all tasks in the workflow process.
- If this handler is attached to the root task and one or more entries are contained in the list, the first valid user or resource pool is made the responsible party for all tasks in the workflow process.

Syntax

EPM-auto-assign-rest

```
-assignee= [user:<user> | person:<person> | resourcepool:<group>::<role>
| user:PROP::<property_name>
| resourcepool:PROP::<property_name>
| $PROPOSED_RESPONSIBLE_PARTY | $USER
| $PROCESS_OWNER | $TARGET_OWNER [<type>]
| $PROJECT_ADMINISTRATOR
| $PROJECT_TEAM_ADMINISTRATOR]
| $PROJECT_AUTHOR | $PROJECT_MEMBER<[group]::<role>>
| $REQUESTOR | $ANALYST
| $CHANGE_SPECIALIST1
| $CHANGE_SPECIALIST2
| $CHANGE_SPECIALIST3
[-from_include_type=<object-type1[,object-type2,...]>]
[-from_exclude_type=<object-type1[,object-type2,...]>]
[-from_attach= target | reference | schedule_task]
[-from_relation=<relation-type>]
[-from_include_related_type=<object-type1[,object-type2,...]> |
-from_exclude_related_type=<object-type1[,object-type2,...]>]
[-project_scope=all | owning_project]
[-check_first_object_only=<true | false>]
[-condition_name=<condition1>]
[-condition_scope=all | any | none]
```

Arguments

-assignee

(Optional) Makes the user or resource pool the specified keyword evaluates to the responsible party for the task to which this handler is added. Assignee is an *optional* argument.

Accepts one of the following in the format specified below:

- **user:**<user>

Adds the user specified to the signoff member list and as the responsible party for the task to which the handler is attached. Accepts a valid Teamcenter user ID.

- **person:***<person>*

Adds the person whose name is specified to the signoff member list and as the responsible party for the task to which the handler is attached. Accepts a valid Teamcenter person name.

Note

If the person's name includes a comma, you must include an escape character (\) to add the correct person. For example, to use **wayne, joan**:

-assignee=person:wayne\, joan

- **resourcepool:***<group>::<role>*

Results in a single assignment which can be performed by any single member of this group/role.

You can define resource pools in the form of *<group>::*, *<group>::<role>*, or *<role>*.

Note

When a resource pool task is performed by a user it is automatically claimed by that user. If that task is a **Review** task and it is started again, the task is assigned to the user who performed it in the previous iteration, rather than the resource pool.

Accepts valid Teamcenter resource pool names and these keywords:

- **\$GROUP**

Current user's current group.

- **\$ROLE**

Current user's current role.

- **\$TARGET_GROUP**[*<type>*]

Owning group of the first target object of the specified type. The *<type>* value is optional. If not specified, the first target is used.

- **\$PROCESS_GROUP**

Owning group of the workflow process.

Note

The **\$ROLE_IN_GROUP** keyword (formerly **\$ROLEINGROUP**) cannot be used. Use **resourcepool:\$GROUP::\$ROLE** instead.

- **user:PROP:***<property_name>*

Adds the user specified by the property name to the signoff member list for the task to which it is attached.

If the property is a multi-value property, only the first value is used when only a single user is assigned in the workflow. When more than one user is assigned, all property values are used.

- **resourcepool:PROP::<property_name>**

Adds the resource pool specified by the property name to the signoff member list for the task to which it is attached.

If the property is a multi-value property, only the first value is used when only a single user is assigned in the workflow. When more than one user is assigned, all property values are used.

- **\$PROPOSED_RESPONSIBLE_PARTY**

Affects assignments based on the user assigned as the responsible party for the first target object.

- **\$USER**

Adds the current user to the signoff member list and as the responsible party.

- **\$PROCESS_OWNER**

Adds the workflow process owner to the signoff member list and as the responsible party.

- **\$TARGET_OWNER [<type>]**

Adds the owner of the first target of the specified type to the signoff member list and as the responsible party. The <type> value is optional. If not specified, the first target is used.

- **\$PROJECT_ADMINISTRATOR, \$PROJECT_TEAM_ADMINISTRATOR, \$PROJECT_AUTHOR, \$PROJECT_MEMBER[<group>::<role>]**

Dynamically makes the first project team member belonging to the role specified in the argument value as the responsible party. The project team is determined by the project team associated with the first target object.

- If the **\$PROJECT_MEMBER[<group>::<role>]** argument is specified, only the project members of the qualifying projects which belong to the specified group and role are selected for assignment. If the group and role are not specified, all the project members from qualifying projects are selected.
- If the value is specified as **\$PROJECT_AUTHOR** or **\$PROJECT_MEMBER[<group>::<role>]**, the relevant first project member is selected.
- You can specify a sub-group with the syntax **<group>+<sub-group>::<role>**.

- **\$REQUESTOR, \$ANALYST, \$CHANGE_SPECIALIST1, \$CHANGE_SPECIALIST2, \$CHANGE_SPECIALIST3**

Dynamically resolves to the user or resource pool associated with the first change target object in the workflow process. The particular user or resource pool is determined by the role specified in the argument value.

Note

Change-related keywords apply only to change objects. If the workflow process does not contain a change object as a target, the argument resolves to null.

Change Manager does not need to be enabled before these keywords take effect, but during installation, make sure you install the **Change Management** application using your preferred

installation tool (Deployment Center or Teamcenter Environment Manager) to ensure change-related keywords are enabled.

-from_include_type=<object-type1[,object-type2,...]>

(Optional) Specifies the object types to be used to get the property value from when a property is specified in the **-assignee** argument (for example, **-assignee=user:PROP::<property_name>**). They must be valid object types.

-from_exclude_type=<object-type1[,object-type2,...]>

(Optional) Specifies the object types to be excluded when getting the property value when it is specified in the **-assignee** argument (for example, **-assignee=user:PROP::<property_name>**). They must be valid object types.

-from_attach= target | reference | schedule_task

(Optional) Specifies which type of attachment (**target**, **reference**, or **schedule_task**) to get the property value from when a property is specified in the **-assignee** argument (for example, **-assignee=user:PROP::<property_name>**). If this argument is not specified, the default is **target**.

-from_relation

(Optional) Specifies the relation of the objects to get the property value from when a property is specified in the **-assignee** argument (for example, **-assignee=user:PROP::<property_name>**). It must be a valid relation.

- For manifestations, use **IMAN_manifestation**.
- For specifications, use **IMAN_specification**.
- For requirements, use **IMAN_requirement**.
- For references, use **IMAN_reference**.
- For BOM views, use **PSBOMViewRevision**.

This argument must be used with the **-from_attach** argument. A derived object is identified by starting with objects of the specified attachment type indicated by the **-from_attach** argument and then locating the first secondary object with the specified relation indicated by the **-relation** argument.

-from_include_related_type=<object-type1[,object-type2]>

(Optional) Specifies the related object types to be used to get the property value from when a property is specified in the **-assignee** argument (for example, **-assignee=user:PROP::<property_name>**). They must be valid object types.

Use this argument when a property is designated and you use the **-from_relation** argument.

This argument should not be used with the **-from_exclude_related_type** argument.

-from_exclude_related_type=<object-type1[,object-type2]>

(Optional) Specifies related object types to be excluded when getting the property value when it is specified in the **-assignee** argument (for example, **-assignee=user:PROP::<property_name>**). They must be valid object types.

Use this argument when a property is designated and you use the **-from_relation** argument.

This argument should not be used with the **-from_include_related_type** argument.

-project_scope

(Optional) Specifies which projects are used to resolve project-based assignments. The **all** value specifies all projects in the list of projects. The **owning_project** value specifies the owning project only.

If this argument is not specified, the default value is the first project in the project list.

-check_first_object_only

(Optional) The **true** value specifies that only the first object is checked. If the value is **false**, all objects are checked. If this argument is not specified, or if it is specified without a value, only the first object is checked.

If the **-include_type**, **-exclude_type**, **-include_related_type**, or **-exclude_related_type** arguments are specified, they determine the types of objects that are checked.

-condition_name

(Optional) The name of the condition to evaluate against the identified objects from which to assign tasks. The condition signature should accept a **WorkspaceObject & UserSession**. The handler assigns the responsible party for any unassigned subtasks only if the condition results are successful, based on the **-condition_scope** argument.

-condition_scope

(Optional) The criteria for evaluating condition results against workflow objects. Values are the following:

- all** All objects should meet the condition. This is the default behavior if this argument is not supplied with the **-condition_name** argument.
- any** Any object should meet the condition.
- none** No object should meet the condition.

Placement

Place on the **Start** action. Typically placed on the root task after the **EPM-assign-team-selector** handler.

Restrictions

None.

Examples

- In this example, a five-task workflow process containing the task templates below is initiated by user **Jones**. The **EPM-auto-assign-rest** handler is placed on the root task, and the **EPM-auto-assign** handler is placed on the fourth task, set with the **-assignee=\$PROCESS_OWNER** argument.

The workflow consists of a **Do** task, **Review** task, **Review** task, and **Do** task.

Because the **EPM-auto-assign-rest** handler is placed on the root task and **Smith** is specified with the **-assignee** argument, **Smith** is the responsible party for the first three tasks (and their subtasks). Because the **EPM-auto-assign -assignee=\$PROCESS_OWNER** handler is placed on the fourth task, **Jones** is the responsible party for the fourth task and its subtasks. **Smith** is the owner of the fifth task.

Argument	Values
-assignee	user:Smith

- This example assigns the user or resource pool assigned as the responsible party for the subtasks of the task to which this handler is assigned.

Argument	Values
-assignee	\$PROPOSED_RESPONSIBLE_PARTY

- This example assigns the user or resource pool associated as **ANALYST** with the first change target object the responsible party for the subtasks of the task to which this handler is assigned.

Argument	Values
-assignee	\$ANALYST

- This example assigns the first member of the **Engineering** group and **Designer** role of the first project team associated with the first target found by the system to the remaining tasks as responsible party.

Argument	Values
-assignee	\$PROJECT_MEMBER[Engineering::Designer]

EPM-auto-check-in-out

Description

Automatically checks in/out the target objects of a workflow process to the assigned reviewer or the responsible party. This prevents other users who have write access to the target objects from being able to modify them. Optionally, when a dataset is checked in/out, it checks in/out the BOM view of the type specified.

Syntax

```
EPM-auto-check-in-out
-assignee=$REVIEWERS | $RESPONSIBLE_PARTY
-action=check-in | check-out
[-include_related_type=<dataset-type::bom-view-type>]
[-include_replica]
```

Arguments

-assignee
(Required)

Note

The **-assignee** argument is *optional* and not required for **-action=check-in**.

Use **\$REVIEWERS** for **Review** tasks. Use **\$RESPONSIBLE_PARTY** otherwise.

Note

The object is checked out to the first reviewer.

-action
(Required) Action to check in (**check-in**) or check out (**check-out**) the objects.

-include_related_type
(Optional) Also check in/out the type specified in the form of **<dataset-type>::<bom-view-type>**. This value works for BOM views only. A BOM view of the specified type is checked in/out if a dataset of a specified type is checked in/out.

-include_replica
(Optional) Remote checks-in or remote checks-out the **Replica Proposed Targets** objects of the workflow along with the target objects. For remote check-outs, the objects are checked out to the current site executing the workflow.

Placement

- For **Review** and **Route** tasks where **-assignee=\$REVIEWERS**:

- If **-action=check-out**, place the handler on the **Complete** action of the **select-signoff-team** subtask, or **Start** action of the **perform-signoffs** subtask.
- If **-action=check-in**, place the handler on the **Complete** action of the **perform-signoffs** subtask.
- For all other tasks or where **-assignee=\$RESPONSIBLE_PARTY**:
Requires no specific placement.

Restrictions

Placement of the **EPM-auto-check-in-out** handler with the **-action=check-out** defined should be determined considering the placement of **EPM-assert-targets-checked-in** rule handler, which displays an error if target objects are not checked in. If this handler is used in a **Review** task, this should be used only when the number of reviewers equals one.

Examples

This example, placed on a **Review** task, checks out the objects to the reviewer once the task is assigned to the reviewer and checks in the objects once the reviewer signs off. You can place this action handler in the **Complete** action of the **select-signoff-team** subtask using the **Check out** action, and in the **Complete** action of the **perform-signoffs** subtask using the **Check in** action.

Argument	Values
-assignee	\$REVIEWERS
-action	check-out
-include_related_type	UGMASTER::view

This setting checks out all the target objects; if a **UGMASTER** is checked out, the BOM view of type **view** is also checked out. If **UGMASTER** is referenced in multiple item revisions, the BOM view of the first item revision is checked out.

This example, placed on a **Review** task, checks in the objects once the task is completed and all reviewers sign off. You can place this action handler in the **Complete** action of the **Review** task using the **Check in** action, or in the **Complete** action of the **perform-signoffs** subtask using the **Check in** action.

Argument	Values
-action	check-in

EPM-change-all-started-to-pending

Description

Ensures that all tasks that are started, but not are not completed, are cleaned up at the conclusion of the workflow process.

Syntax

EPM-change-all-started-to-pending

Arguments

None.

Placement

Place on the **Complete** action of the root task.

Restrictions

None.

EPM-change-group-owner

Description

Changes the owning group for the item master of any item type whose revision is attached as target.

Syntax

EPM-change-group-owner -group=<group-id>

Arguments

-group
(Required) A valid Teamcenter **group_id**.

Placement

Place on the **Complete** action.

Restrictions

None.

Examples

- This example is used with a workflow initiated with an item revision and document revision attached as targets. It sets the owning group of the respective master item and master document to **engineering**.

Argument	Values
-group	engineering

EPM-change-ownership

Description

Changes the ownership of all target objects to the group and user ID of the reviewer or the responsible party.

The advantage of changing ownership is to allow a revision rule to configure WIP (work in process) data based on owner and group settings.

If this handler is used in **Review** tasks, the number of reviewers should be one.

To save processing time and/or improve robustness, the handler can be configured to be active only in one or more actions (**-active=<action>**). If the handler is called as part of trigger to another action, the handler silently returns immediately.

Syntax

```
EPM-change-ownership -assignee=$REVIEWERS | $RESPONSIBLE_PARTY
[-active= <action> [-active=<other-action>]][-depth=<level>]
```

Arguments

-assignee

(Required) User to whom the ownership is given.

Use **\$REVIEWERS** if this handler is used in a **Review** task. Use **\$RESPONSIBLE_PARTY** otherwise.

[-active=<action> [-active=<other-action>]]

(Optional) Name of the action for which this handler is valid.

If this argument is used, and the handler is called as part of a trigger to an unlisted action, the handler silently returns immediately. You can use the following valid action names as values.

EPM_add_attachment_action

EPM_remove_attachment_action

EPM_approve_action

EPM_reject_action

EPM_promote_action

EPM_demote_action

EPM_refuse_action

EPM_assign_approver_action

EPM_notify_action

This argument can be useful when the handler is placed on the **Perform** action. These actions automatically run the following **Perform** action handlers, raising the potential for unnecessary processing.

This argument is optional.

-depth

(Optional) Recursion depth. This argument is optional and the default is set to **1**.

Placement

Requires no specific placement.

Restrictions

Set the number of reviewers to **1** when this handler is placed on a **Review** task.

Examples

This example, when placed on the **Complete** action of the **select-signoff-team** subtask of a **Review** task, changes the ownership of all the target objects to reviewers and their groups.

Argument	Values
-assignee	\$REVIEWERS

EPM-change-target-group

Description

Changes the group ownership of the target objects to the current **group_id** of the user. If the target is an item revision object, the group of its item master is set to the current group ID of the user as well.

Syntax

EPM-change-target-group

Arguments

None.

Placement

Place on the **Complete** action.

Restrictions

None.

EPM-change-target-group-owner

Description

Changes the owner and/or the owning group for the target objects.

Note

The handler does not validate if the owning user belongs to the owning group. It makes the change even if the user does not belong to the group.

Syntax

EPM-change-target-group-owner [-owner=<user-id>][-group=<group-id>]

Arguments

-owner

(Optional) Valid Teamcenter **user_id**.

-group

(Optional) Valid Teamcenter **group_id**.

Placement

Place on the **Complete** action.

Restrictions

None.

Examples

- This example changes the group and owner of the targets to **engineering** and **jim**, respectively.

Argument	Values
-owner	jim
-group	engineering

- This example changes the only group of the targets to **production**.

Argument	Values
-group	production

- This example changes only the owner of the targets to **smith**.

Argument	Values
-owner	smith

EPM-check-signoff-comments

Description

Requires users to type a comment when making a signoff decision. You can specify whether the comment is required for the approve decision or the reject decision. If neither decision is specified, comments are required to complete either signoff decision.

Syntax

EPM-check-signoff-comments [-decision= approve | reject]

Arguments

-decision

(Optional) Specifies which signoff decision requires comments to be entered when making a signoff decision for either a **Review** task or an **Acknowledge** task.

Use **approve** to require comments to be added before selecting **Approve** for a **Review** task, or **Acknowledge** for an **Acknowledge** task.

Use **reject** to require comments to be added before rejecting a signoff for a **Review** task.

If this argument is not used, comments are required for either decision before completing a signoff.

Placement

Place on the **Perform** action of the **perform-signoffs** task.

Restrictions

Place on the **perform-signoffs** task.

Note

The **EPM-check-signoff-comments** handler is only necessary in specialized cases where it is combined with other rule handlers containing a rule quorum.

Otherwise, signoff quorum handling is internalized in the workflow engine and does not need this handler.

This usage is rare, and unless this specialized need is required, it should be avoided.

Examples

- This example requires that the user type comments before rejecting a signoff:

Argument	Values
-decision	reject

- This example requires the user to type comments before approving a signoff:

Argument	Values
-decision	approve

EPM-create-form

Description

Creates an instance of a specified form and attaches that form to the specified task. For more information, refer to the **EPM-display-form** handler.

Configuring a task to display forms using EPM-create-form, EPM-display-form, and EPM-hold

To configure a task to display a form when a user performs a specified action, use the **EPM-hold** handler. This handler pauses the task, requiring the user to perform an action on the task before the task can complete. Without the use of this handler, a task completes automatically once started.

To create an instance of a specified form and attach the form to the specified task, use the **EPM-create-form** handler.

Therefore, the **EPM-create-form** handler creates the form when the **Start** action is initiated, the **EPM-display-form** handler displays the form when the **Perform** action is initiated, and the **EPM-hold** handler prevents the task from automatically completing, allowing the form to be completed by the user.

Variations on the above example may be required for a more sophisticated interaction when it is required that the task not complete until required fields are entered in the form. This type of configuration requires the creation of customized rule handlers.

Syntax

```
EPM-create-form -type=<formtype> [-name=<string>] [-description=<string>]
[ [-property=<field-name>] [-value=<value>]] [-target_task=<task-name.attachment-type>]
```

Arguments

-type

(Required) Valid **FormType** object.

-name

(Optional) User-defined form name. Default is the workflow process name.

-description

(Optional) User-defined description of the form. Default value is **null**.

-property

(Optional) Specifies the particular field of the form that has a default value. Users can choose to set the default value to more than one field by adding the field names separated by commas or the character specified by the **EPM_ARG_target_user_group_list_separator** preference. The default value for each field is set by the **-value** argument. Do not use this argument for field names of **Typed_Reference** and **Untyped_Reference** types. This argument is optional.

Note

Use this argument with the **-value** argument to populate the initial values in forms created by a workflow. If you do not use this argument and instead set the initial value in the business object definition, the workflow process defines the value as empty until you perform an edit and save it.

-value

(Optional) Specifies the default value for a particular field of the form specified by the **-property** argument. Users can choose to set the default values for more than one field by adding the values separated by commas or the character specified by the **EPM_ARG_target_user_group_list_separator** preference in the same order as listed in the **-property** argument values. Do not use this argument for field names of **Typed_Reference** and **Untyped_Reference** types. This argument is optional.

Note

Use this argument with the **-property** argument to populate the initial values in forms created by a workflow. If you do not use this argument and instead set the initial value in the business object definition, the workflow process defines the value as empty until you perform an edit and save it.

-target_task

(Optional) Task name and attachment type receiving the new form as an attachment. The default value is the current task.

Accepts one of four <keywords> for <attachment-type>:

- **\$REFERENCE**
Reference attachments
- **\$TARGET**
Target object attachments

The default value is **\$REFERENCE**.

Placement

Requires no specific placement.

Restrictions

None.

Examples

- This example shows how to create form type **ECN Form**, form name **ECN**, form description **Engineering Change Management Form**, and attachment type **EPM_reference** attachment. The form is attached to the root task of the workflow process.

Argument	Values
-type	ECN Form
-name	ECN
-description	Engineering Change Management Form
-target_task	\$ROOT.\$REFERENCE

- This example attaches the form as a target attachment to the current task:

Argument	Values
-target_task	\$ROOT.\$TARGET

To attach the form as a reference attachment to the current task, do not set the **-target_task** argument, because this is the default location this handler uses when this argument is not defined.

EPM-create-relation

Description

Creates a specified relation between the target/reference objects of the workflow process. The relation to be created must be a valid relation. The handler goes through all the primary objects of the specified type and creates a specified relation with all the secondary objects of the specified type.

Syntax

```
EPM-create-relation -relation=<relation-name> -primary_attachment= target | reference
-primary_type=<type-of-primary-object> -secondary_attachment=target | reference
-secondary_type=<type-of-secondary-object>
```

Arguments

-relation

(Required) The relation type to be created.

-primary_attachment

(Required) The objects that have to be considered as primary objects (target or reference).

-primary_type

(Required) Type of object to be considered as primary object.

Considers all the target or reference attachments of this type as primary objects. Target or reference is specified in **-primary** argument.

This argument checks for the exact type name and does not consider the subtypes.

-secondary_attachment

(Required) The objects that have to be considered as secondary objects (target or reference).

-secondary_type

(Required) Type of object to be considered as secondary object.

Considers all the target or reference attachments of this type as secondary objects. Target or reference is specified in **-secondary** argument.

This argument checks for the exact type name and does not consider the subtypes.

Placement

Place on the **Complete** action of the task.

Restrictions

None.

Examples

In this example, the workflow process has two item revisions as target objects and one **UGPART** object as a reference object. There is no relation between the two item revisions and the **UGPART**. To create a requirements relationship between the two, with the item revisions as primary and the **UGPART** as secondary:

Argument	Values
-relation	IMAN_requirement
-primary_attachment	target
-primary_type	ItemRevision
-secondary_attachment	reference
-secondary_type	UGPART

EPM-create-status

Description

Attaches the specified status type to the root task.

Syntax

EPM-create-status -status=<status-type>

Arguments

-status

(Optional) Adds the specified status type to the root task. If this argument is not supplied, the task name where the handler is attached is used. The name provided should be the name of a status type already defined in the Business Modeler IDE, not the display name.

If the status type is not already defined, a status object is created that is not based on a status type.

Placement

Requires no specific placement.

Restrictions

None.

Examples

- This example attaches the **Released** status to the root task.

Argument	Values
-status	Released

EPM-create-sub-process

Description

This handler starts subprocesses from a workflow process. The new subprocess can take on attachments of the parent process, and those attachments can be grouped by property.

This action handler creates subprocesses and attaches the specified target/reference objects of the parent process as target/reference attachments to the new subprocesses. This handler goes through all the target/reference objects of the parent process, finds the corresponding object type, and adds them as target/reference attachments of the new subprocess. This handler allows you to launch one or multiple workflow processes from within a parent process. You can use this handler to set a dependency between the parent process and subprocess in a way that causes the parent process to wait for the subprocess's (task) completion. The action handler can be added multiple times to a task action to provide abilities such as using different workflow process templates per target object type or other combinations.

If you want the progress of the parent process to be dependent on the subprocess completing, use the **-dependency** argument with this handler and place the handler on the **Start** action of the parent task to start the subprocess correctly. However, the parent task checks if the dependent subprocess is complete only when the parent task reaches the **Complete** action.

For example, if you place this handler with the **-dependency** argument on a **Review** task, it starts the subprocess, allows users to select a signoff team and perform signoffs, then checks the subprocess for its completion status. If the dependency subprocess is not complete when the signoffs are completed, the parent process task is removed from the **Tasks To Perform** folder (or **My Tasks** in Active Workspace) and placed in the **Tasks To Track** folder (or **Tracking** in Active Workspace).

If the parent process is dependent on the subprocess, then when the subprocess completes it will automatically trigger its parent process task to complete.

The **-include_replica** argument adds the parent's **Replica Proposed Targets** to the newly created subprocesses.

Note

When this handler creates a subprocess, the process owner and responsible parties for the new subprocess are defined as the current session's user. It may not match the responsible party of the workflow task having this handler, particularly when the task is automated and its actions are triggered after completing a previous task.

If the process owner and responsible parties should be different from the currently logged-in user, use **EPM-auto-assign** or **EPM-assign-team-selector** in the subprocess template.

The **WRKFLW_parent_behavior_on_sub_process_abort** preference ensures the parent process's task proceeds after the dependent subprocess is aborted:

- 1 - Triggers the dependent parent task to complete (default value)

2 - No action occurs on the dependent parent task. The user decides what action to take.

Syntax

EPM-create-sub-process

-template=<process-template-name>
[-from_attach=Target | Reference | ALL]
[-to_attach=Target | Reference | ALL]
[-include_type=<object-type>]
[-exclude_type=<object-type>]
[-process_name=<name-for-process>]
[-description=<string>]
[-multiple_processes]
[-dependency= <multilevel-parent-process-task-path::multilevel-sub-process-task-path>]
[-transfer]
[-process_assembly]
-depth=<depth-of-traversal>
-rev_rule=<revision-rule-to-apply>
-relation=<relation-type-to-look>
[-include_related_type=<type-of-related-components-to-be-included>]
[-exclude_related_type=<type-of-related-components-to-be-excluded>]
[-include_replica]
[-query]
[-group_by_property=<property-to-be-used-for-grouping>]

Arguments

-template=<process-template-name>
 (Required) The workflow process template name that is used to start a new workflow process.
-from_attach=Target | Reference | ALL

(Optional) The following are the objects attachments to be inherited from the parent process target and/or reference folder:

- **Target**

Takes the attachments from the target folder of the parent process.

- **Reference**

Takes the attachments from the reference folder of the parent process

- **ALL**

Takes targets and reference attachments.

The **-from_attach** and **-to_attach** arguments must be used together. If you use one argument, you must use the other.

The preference to enable for multiple workflow processes for the same objects needs to be set if **-from_attach** is used with either the **Target** or **ALL** option. The **EPM_multiple_processes_targets** preference attaches components that are currently in process as targets if it is set to **ON**.

-to_attach=Target | Reference | ALL

(Optional) The following are the objects to attach with the new workflow process:

- **Target**

Attaches to target folder of new workflow process.

- **Reference**

Attaches to reference folder of new workflow process

- **ALL**

Attached from target folder of the parent process to the target folder of a new workflow process and reference folder of the parent process to the reference folder of a new process.

The **-from_attach** and **-to_attach** arguments must be used together. If you use one argument, you must use the other.

This argument is optional.

-include_type=<object-type>

(Optional) Defines the types to be included as targets and/or references.

- Must be valid workspace object types. For example: **ItemRevision** and **ITEM**.
- If this argument is specified as **Dataset**, any type of dataset (**UGMASTER**, **UGPART**, **Text**, and so on) is considered.
- If this argument is specified as **ItemRevision**, any type of item revision (**DocumentRevision** and any custom item revision types) is considered.

If this argument is passed to the handler, **-from_attach** and **-to_attach** should also be passed to the handler.

-exclude_type=<object-type>

(Optional) Defines the types to be excluded from being adding as targets/reference.

- Must be valid workspace object types. For example: **ItemRevision** and **ITEM**
- If this argument is specified as **Dataset**, any type of dataset (**UGMASTER**, **UGPART**, **Text**, and so on) is considered.
- If this argument is specified as **ItemRevision**, any type of item revision (**DocumentRevision**, and so on, and any custom item revision types) is considered.

If this argument is passed to the handler, **-from_attach** and **-to_attach** should also be passed to the handler.

-process_name=<name-of-process>

(Optional) The name used identifies the new workflow process. You can use the **\$TARGET** keyword, which is replaced by the target display name *<targetname-item-name>*.

When a workflow process name is given as **subprocess** and no **-multiple_processes** arguments are used, the workflow process name alone is used as there is only one, so the subprocess would be called **subprocess**. In this case, to include a number in the name, put it in the argument name and only one is created. If the workflow process name is not given and the **-multiple_process** argument is not used, the parent process name is **parentprocess**; in this case, it is **parentprocess:1**. The same is true for cases where there are no targets on the parent process.

If the workflow process name is not given, and the **-multiple_processes** argument is used, the name assigned is in the format of *<subprocesstargetdisplayname>-<item-name>:<count>*. In this case, that would be **item1/A-wheel:1**, **item2/B-axle:2**, **item3/A-bearing:3**. In the case where the parent had no targets, the name is **parentprocess:1**.

If the workflow process name is given with the **\$TARGET** keyword, such as **subprocess1_\$TARGET**, and the **-multiple_processes** argument is used, the name assigned is in the format *<subprocess1_subprocesstargetname>-<item-name>:<count>* format. In this case, that is **subprocess1_item1/A-wheel:1**, **subprocess1_item2/B-axle:2**, **subprocess1_item3/A-bearing:3**. In a case where the parent had no targets, the name is **subprocess1_:1**.

-description=<string>

(Optional) Workflow process description.

If the description is not specified, it is set to blank.

This argument is optional.

-multiple_processes

(Optional) Each target object to be considered becomes a target in its own individual subprocess. If not specified, all targets are in a single subprocess.

To learn how to use this argument, see the example section.

-dependency=<multilevel-parent-process-task-path::multilevel-sub-process-task-path>

(Optional) Creates a dependency between a parent process task and a specified subprocess task; the parent process's task proceeds after the subprocess's task completes.

You must use a multilevel path to specify the task templates. Separate path levels with colons (:). Separate the multilevel path of the parent task from the multilevel path of the subprocess task with a double colon (: :). For example:

Change Approval:QA Review:perform-signoffs::Design Change:
Part Review:perform-signoffs

If you use a double colon (::) only without specifying either a source or target task, a subprocess task is created, and a dependency is established between the parent process task and the newly created subprocess task.

If a parent process task is not specified, the task containing this handler is designated as the parent process task. If a subprocess task is not specified, or not found, the dependency is not set.

This argument is optional.

Note

- If you try to complete a task that has a dependency on an uncompleted subprocess task, you receive a warning indicating that the interprocess task dependencies are not met for the dependent task.
- By default, if you do not use this argument, the signoff details for the subprocess are not included in the parent process signoff report for standard tasks. To include the details for an independent subprocess, change the value of the **WRKFLW_signoff_report_show_sub_process** preference.

-transfer

(Optional) Transfers attachments of the parent process to the subprocess. The parent process has no attachments as target/reference that exists in the subprocess.

-process_assembly

(Optional) Signals the handler to traverse the assembly and start a subprocess on its components. Multiple workflow processes can be started if the **-multiple_processes** argument is specified. This argument works in conjunction with **-depth**, **-rev_rule**, **-include_related_type**, and **-exclude_related_type** arguments. This argument can be used together with the **-relation** argument. Both arguments can be specified on the same instance of the handler.

-depth=<depth of traversal>

(Optional) Specifies the depth of traversal for an assembly. Specify **all** to traverse all levels. If not specified, the default value is 1.

-rev_rule=<revision-rule-to-apply>

(Optional) Defines the name of the revision rule to be applied for BOM traversal. If not supplied, the default revision rule would be used

-relation=<relation-type-to-look>

(Optional) Finds the objects attached to the target objects with the given relation. The value must be a valid relation.

Specifies whether a relation is used to locate secondary objects. The relation of the objects to be attached to the target object. Must be a valid relation.

To specify manifestation, use **IMAN_manifestation**.

For specification use **IMAN_specification**.

For requirement use **IMAN_requirement**.

For reference use **IMAN_reference**.

For BOM views use **PSBOMViewRevision**.

This argument works in conjunction with **-include_related_type**, and **-exclude_related_type** arguments. This argument can be used together with the **-process_assembly** argument. Both arguments can be specified on the same instance of the handler.

-include_related_type=<type-of-related-components-to-be-included>

(Optional) Defines the types of related component objects to be included as targets and/or references.

- Must be valid workspace object types. For example: **ItemRevision** and **ITEM**.
- If this argument is specified as **Dataset**, any type of dataset (**UGMASTER**, **UGPART**, **Text**, and so on) is considered.
- If this argument is specified as **ItemRevision**, any type of item revision (**DocumentRevision** and any custom item revision types) is considered.

This argument works in conjunction with **-process_assembly** and **-relation** arguments.

This argument is optional.

-exclude_related_type=<type-of-related-components-to-be-excluded>

(Optional) Defines the types of related component objects to be excluded from being adding as targets and/or reference.

- Must be valid workspace object types. For example: **ItemRevision** and **ITEM**
- If this argument is specified as **Dataset**, any type of dataset (**UGMASTER**, **UGPART**, **Text**, and so on) is considered.
- If this argument is specified as **ItemRevision**, any type of item revision (**DocumentRevision**, and so on, and any custom item revision types) is considered.

This argument works in conjunction with **-process_assembly** and **-relation** arguments.

Note

The **-include_related_type** and **-exclude_related_type** arguments can be used in conjunction with each other. If used in conjunction, the **-include_related_type** argument takes precedence; first the objects are processed against **-include_related_type**, and then **-exclude_related_type**.

-include_replica

(Optional) Adds the parent's **Replica Proposed Targets** to the newly created subprocesses under these conditions:

- If the **-from_attach** argument specifies either **Target** or **ALL**, the **Replica Proposed Targets** are also attached to subprocess with the targets.
- If the **-to_attach** argument specifies **Target** and any of the qualified objects are replicas, they are attached as **Replica Proposed Targets** instead of targets.
- If the **-include_replica** argument is not used, the handler does not add the **Replica Proposed Targets** attachments to the subprocess.

-group_by_property

(Optional)

- Input attachments are grouped according to the property assigned such as **object_type** and **object_owner**. One subprocess is spawned for each group. Each subprocess has objects (attachments) in that group.
- When used with the **-multiple_processes** arguments, one subprocess is spawned for each target object.

This argument is optional, but must be used with **-from attach**.

-query

(Optional)

You can set **-query** to the name of a query. In Active Workspace, this query will appear in the query list for easy selection. However, in rich client, you must manually type the query name. When the **EPM-create-sub-process** handler runs, it will execute the query on the objects linked to the parent process, and the results will be utilized in the subprocess.

Placement

Place in the **Start** or **Complete** action of a task template.

Note

If you use the **-dependency** argument and the current task is dependent on the subprocess, you must place the handler on the **Start** action. If you place it on the **Complete** action, the **-dependency** argument causes an error.

The handler can be added multiple times to a task action to provide abilities such as using different workflow process templates per target object type or other combinations.

Restrictions

- When using **-relation** or **-process assembly**, the targets/reference attachments for the subprocess are processed based on the secondary related/assembly components of the parent target/reference attachments.
- If a user demotes a task that already created subprocesses, when the task gets activated again, it creates another subprocess. Depending on the user's choice, they should either delete the original subprocess or the new subprocess. Currently this is a manual step for the user.
- The **-depth** and **-rev_rule** arguments are used only when the **-process_assembly** argument is used. The **-exclude_related_type** and **-include_related_type** arguments are used only when **-process_assembly** or **-relation** is used.
- For the **group_by_property** argument, these p are not supported: **PROP_operationinput**, **PROP_unknown**, or properties containing multiple values, for instance, **array/list**.
- For the **group_by_property** argument, these property value types are not supported: **PROP_external_reference**, **PROP_untyped**.

Examples

The following examples illustrate how to configure the handler arguments. These examples illustrate creating a parent process template containing a **Do** task and adding the handler to the task to create a subprocess.

- The examples where the current task is dependent on the subprocess and that use the **-dependency** argument must be placed on the **Start** action.
- The examples without the **-dependency** argument can be placed on either the **Start** or **Complete** action of a task.

Note

You can add this handler to any action from which you want to create the subprocess. Use the following examples to understand how to configure the handler arguments.

- This example launches a new process using the **Change Approval** template and sets the dependency between the parent process initiating task that starts a new subprocess and **SubProcess_001**. The task that initiates the new subprocess cannot be completed until **SubProcess_001** is completed. Place this handler on the **Start** action.

Argument	Values
-template	Change Approval
-dependency	::
-process_name	SubProcess_001

- The example creates a new workflow process using the **Change Approval** template with no attachments. The **-process_name** and **-process_desc** are optional.

Argument	Values
-template	Change Approval
-process_name	0006/A_Change Approval
-description	This is a demo description text

- This example creates a new workflow process on the **Change Approval** template by inheriting all the targets/reference attachments of the parent process as target/reference attachments, respectively, of the newly created workflow process. If the workflow process name is not defined, it generates a workflow process name for the child process in the `<Parentprocess>:<count>` format. The workflow process description is left blank.

Argument	Values
-template	Change Approval
-from_attach	ALL
-to_attach	ALL

- This example creates a new workflow process on the **Change Approval** template by inheriting all the target attachments of the parent process as target attachments for the subprocess.

Argument	Values
-template	Change Approval
-from_attach	TARGET
-to_attach	TARGET

- This example creates a new workflow process on the **Change Approval** template by inheriting all the attachments (target and reference) of the parent process as target attachments for the subprocess.

Argument	Values
-template	Change Approval
-from_attach	ALL
-to_attach	TARGET

- This example launches a new workflow process on the **Change Approval** template. All target and reference attachments of the **ItemRevision** and **UGMASTER** types of the parent process are attached as targets for the new process.

Argument	Values
-template	Change Approval
-from_attach	ALL
-to_attach	TARGET
-include_type	ItemRevision, UGMASTER

- This example launches a new workflow process on the **Change Approval** template. All objects (both target and reference attachments) of the **ItemRevision** and **UGMASTER** type of the parent process are attached as target and reference attachments respectively for the new workflow process.

Argument	Values
-template	Change Approval
-include_type	ItemRevision, UGMASTER
-from_attach	TARGET
-to_attach	ALL

- This example launches a new workflow process on the **Change Approval** template. All objects of the **ItemRevision** type of the parent process are excluded as targets for the new workflow process.

Argument	Values
-template	Change Approval
-from_attach	ALL
-to_attach	TARGET
-exclude_type	ItemRevision

- This example launches a new workflow process on the **Change Approval** template by specifying the **-include_type** and **-exclude_type** arguments. It specifies the list of attachment types to be included in **-include_type** and the list of types to be excluded in **-exclude_type**. This argument launches a subprocess with only **ItemRevision**.

Argument	Values
-template	Change Approval
-from_attach	ALL
-to_attach	ALL
-include_type	ItemRevision
-exclude_type	UGMASTER

- This example launches a new workflow process on the **Change Approval** template and sets the dependency between the **DoChecklist** task in the **DesignReview** parent process and the **perform-signoffs** subtask of the **QA Review** task of the **Change Approval_001** subprocess. The **DoChecklist** task of the parent process cannot complete until the **perform-signoffs** task in the subprocess completes. Place this handler on the **Start** action.

Argument	Values
-template	Change Approval
-dependency	DesignReview:DoChecklist::Change Approval_001:QA Review:perform-signoffs

- This example launches a new workflow process using the **Change Approval** template. Because no path is specified for the parent process, the task containing this handler is used as the parent process task. A dependency is created between the task containing this handler and the **perform-signoffs** subtask of the **QA Review** task of the **Change Approval_001** subprocess. The task containing this handler cannot complete until the **perform-signoffs** task in the subprocess completes. Place this handler on the **Start** action.

Argument	Values
-template	Change Approval
-dependency	::Change Approval_001:QA Review:perform-signoffs

- This example launches new workflow processes on the **Change Approval** template. Each object instance of the **ItemRevision** type on target attachments of the parent process launches a new workflow process with that instance as target. For example, if the parent process has three **ItemRevision** objects as the target, three different workflow processes are launched.

Argument	Values
-template	Change Approval
-from_attach	ALL
-to_attach	TARGET
-include_type	ItemRevision
-multiple_processes	

- The following handler configuration looks for an assembly in the targets, configures it as per the **Latest Working** revision rule and starts multiple workflow processes on all its components.

Argument	Values
-template	Change Approval
-from_attach	TARGET
-to_attach	TARGET
-multiple_processes	

Argument	Values
-process_assembly	
-depth	All
-rev_rule	Latest Working

- The following handler configuration starts a subprocess on the **UGMaster** dataset attached to the target objects with **Iman_specification** relation.

Argument	Values
-template	Change Approval
-from_attach	TARGET
-to_attach	TARGET
-multiple_processes	
-relation	Iman_specification
-include_related_type	UGMaster

- The following handler configuration looks for an assembly in the targets, configures it as per the **Latest Working** revision rule and starts multiple workflow processes on all its components. It also starts a subprocess on the objects that are attached to the target objects with the **Iman_specification** relation.

Argument	Values
-template	Change Approval
-from_attach	TARGET
-to_attach	TARGET
-multiple_processes	
-process_assembly	
-depth	All
-rev_rule	Latest Working
-relation	Iman_specification

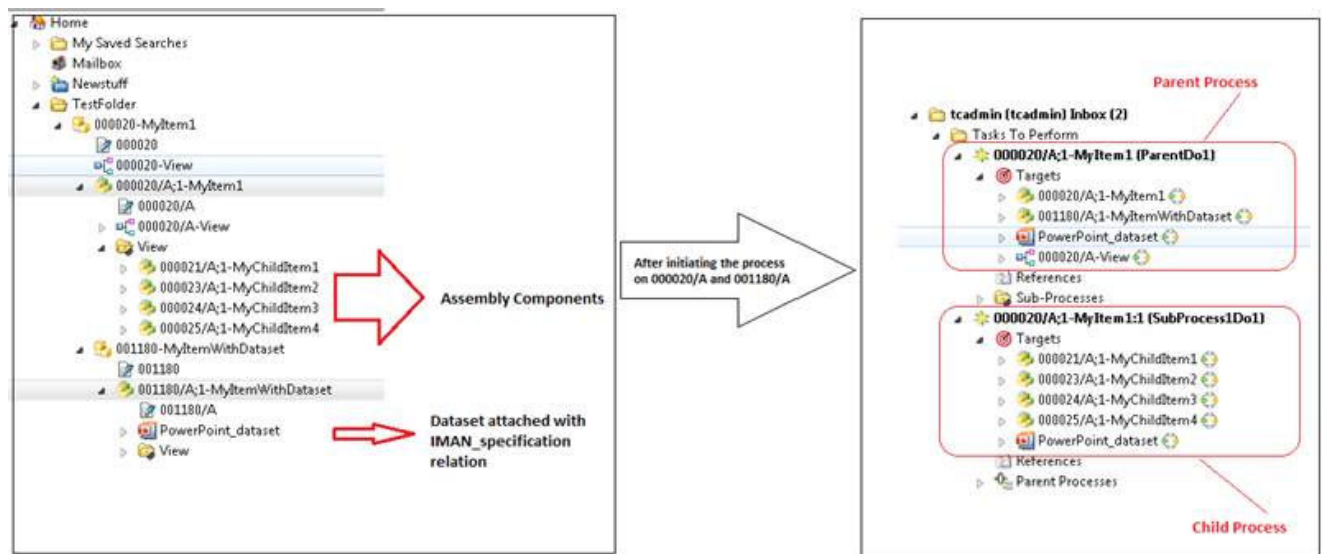
- The following handler configuration starts a subprocess using the **Change Approval** template. All target objects of the **Dataset** type except for **MSWord** type objects are attached as targets to the subprocess.

Argument	Values
-template	Change Approval
-from_attach	TARGET
-to_attach	TARGET

Argument	Values
-include_type	Dataset
-exclude_type	MSWord

- The following configuration initiates the parent process on 000020/A (with assembly components) and 001180/A (with a dataset).

Argument	Values
-template	SubProcess1
-from_attach	ALL
-to_attach	ALL
-relation	IMAN_specification
-process_assembly	



- The following handler configuration starts a subprocess using the **Change Approval** template. It spawns a Change Approval subprocess for each group formed.

Argument	Values
-template	Change Approval
-from_attach	ALL
-to_attach	ALL
-group_by_property	Object_type

- The following handler configuration starts a subprocess using the **Change Approval** template. It spawns one **Change Approval** subprocess for each target object in each group. The subprocesses spawned are named per the value in the **-process_name** argument.

Argument	Values
-template	Change Approval
-from_attach	ALL
-group_by_property	Object_type
-to_attach	ALL
-multiple_processes	
-process_name	newSubprocess

- In this example, a parent process includes two target attachments and one reference attachment (all are ItemRevisions).

Each of the three ItemRevisions have their description set as follows:

ItemRevision Name	ItemRevision Description	Target or reference in parent process
ItemRev1	Review	Target
ItemRev2	Review	Reference
ItemRev3	Final	Target

There is a Teamcenter query named **QRY_Review** which is configured to search for ItemRevisions that have a description of **Review**. The EPM-create-sub-process handler is configured as follows:

The screenshot shows the configuration for the 'EPM-create-sub-process' handler. The 'Handlers' tab is selected, and the 'EPM-create-sub-process' handler is highlighted in the list. The configuration details on the right show the following settings:

- Name: EPM-create-sub-process
- Argument: -query, Value(s): QRY_Review
- include_type: ItemRevision
- to_attach: target
- from_attach: all
- template: sub-assembly-review

The **-query** argument triggers the execution of the **QRY_Review** query, which identifies attached objects that meet the specified criteria (that is, **Description = Review**). In this example, it will find **ItemRev1** and **ItemRev2** since they match the query criteria (their **Description** is **Review**). However **ItemRev3** will be filtered out since it does not meet the query criteria (its description is not **Review**).

The end result is the attachments that the query found (**ItemRev1** and **ItemRev2**) will be used as targets in the subprocess. **ItemRev3** will not be used in the subprocess.

RESTRICTIONS on arguments

These examples show how *not* to use this handler.

- Do not create a workflow process without specifying the **-template** name.

Argument	Values
-process_name	0006/A_Change Approval
-from_attach	TARGET
-to_attach	TARGET

- Do not create a workflow process with the **-multiple_processes** argument but not providing the **-from_attach** and **-to_attach** arguments.

Argument	Values
-template	Change Approval
-multiple_processes	

- Do not create a workflow process by only specifying either one of the arguments: **-from_attach** or **-to_attach**.

Argument	Values
-template	Change Approval
-from_attach	TARGET

EPM-debug

Description

Prints information about the last action triggered. For example, it will print the current task state, the proposed task state, the task action, and optionally it will print the comment. Typically used for debugging.

Note

Teamcenter logging must be activated for printing.

Syntax

EPM-debug -comment=<string>

Arguments

-comment

(Optional) Additional descriptive string appended to the action name.

Placement

Requires no specific placement.

Restrictions

None.

Examples

This example notifies the user when the **Complete** action runs by printing **Complete, action is executing** to the standard output device.

Argument	Values
-comment	action is executing

Note

This example assumes you have attached this handler to a **Complete** action.

EPM-demote

Description

Clears all signoff decisions from the current and previous **Review** tasks. An optional argument allows the user to specify the task name that the workflow process is demoted to.

Syntax

EPM-demote [-target_task=<task-name>]

Arguments

-target_task
(Optional) Specifies to which previous task the workflow process is demoted. Must specify a valid task in the current workflow process.

If this argument is not specified, the workflow process is demoted to the previous task.

Placement

None.

Restrictions

None.

Examples

This example shows how to demote the workflow process to the task named **design**.

Argument	Values
-target_task	design

EPM-demote-on-reject

Description

Demotes the current task to the previous task, or to the task specified on the **-target_task** argument of the **EPM-demote** handler placed on the **Undo** action of the current task.

By default, the handler checks the approval quorum requirements at each rejection and demotes the task when the quorum limit cannot be met. Consider a **perform-signoffs** task assigned to seven reviewers with an approval quorum of three. The first four rejections do not demote the task. The fifth rejection, which would prevent the approval quorum of three from being met, demotes the task.

You can override the default behavior and specify the number of rejections required to demote the workflow process using the **-num_rejections** argument. Using the above example, override the quorum requirement by setting this argument to **2**. The task demotes on the second rejection, instead of the fifth.

To set the number of rejections needed to the number where the quorum cannot be met, set **-num_rejections** to **-1**. Using the above example of seven reviewers with a quorum of three, the **-1** value sets the required number of rejections to five. When five rejections are recorded, the task is demoted.

Note

This handler takes precedence if success and failure paths exist.

Syntax

EPM-demote-on-reject [-num_rejections=<number-of-rejections>]

Arguments

-num_rejections

(Optional) Number of rejections required to demote the task.

Specifying **-1** reads the approval quorum value and demotes the task when the number of rejections recorded makes it no longer possible to meet the quorum.

Placement

Place on the **Perform** action of the **perform-signoffs** subtask of a **Review** task.

Restrictions

This handler assumes that all target objects, reference objects, and status types are attached to the root task.

Examples

- This example demotes a process when the number of rejections exceed the quorum limit:

EPM-demote-on-reject

- This example demotes a process when the second rejection is received:

Argument	Values
-num_rejections	2

- This example demotes a process when the number of rejections recorded prevents the quorum from being met. For example:
 - If there are two reviewers and a quorum of one, both reviewers would have to reject the signoff.
 - If there are three reviewers and a quorum of two, two reviewers would have to reject the signoff.
 - If there are four reviewers and a quorum of two, three reviewers would have to reject the signoff.

Argument	Values
-num_rejections	-1

EPM-display-form

Description

Displays specified forms attached to a specified *custom* task , which is an instance of the **EPMTaskTemplate**. By default, all attachments of the **FormType** object attached to the current task are displayed.

The custom task template is used to define custom forms and other site-specific tasks for the user to complete and is designed to accept customization. This template contains no innate customized interface behavior.

Note

Do not use this handler on other task templates, such as **Do**, **Review**, and **Route**. Other task templates have their own user interface that overrides any attached forms. The task templates either are not meant to display a customized interface (such as the **Add Status** task template) or already have customized interface behavior assigned (such as the **Review** and **Route** task templates).

For example, the **Do** task template already has customized interface behavior assigned. While form handlers can be added to the **Do** task template, the template's original interface behavior is displayed, not the forms. If the default display required is a customized form, use an instance of the custom task template.

The default **Perform** action for any template can be overridden using the **.properties** file. It is more effective, however, to use the task template when the required default **Perform** action is the display of forms.

Configuring a task to display forms using EPM-display-form, EPM-hold, and EPM-create-form

To configure a task to display a form when a user performs a specified action, use the **EPM-hold** handler. This handler pauses the task, requiring the user to perform an action on the task before the task can complete. If this handler is not used, a task completes automatically once started.

To create an instance of a specified form and attach the form to the specified task, use the **EPM-create-form** handler.

The **EPM-create-form** handler creates the form when the **Start** action is initiated, the **EPM-display-form** handler displays the form when the **Perform** action is initiated, and the **EPM-hold** handler prevents the task from automatically completing, allowing the form to be completed by the user.

Variations on the above example may be required for a more sophisticated interaction when it is required that the task not complete until required fields are entered in the form. This type of configuration requires the creation of customized rule handlers.

Syntax

EPM-display-form -type=<form-type> [-source_task=<task-name.attachment-type>]

Arguments

-type

(Required) Valid **FormType** object.

-source_task

(Optional) Form to be displayed. The default values for this optional argument are reference attachments of the **FormType** attached to the current **task_name**.

<attachment-type>

Accepts one of the following reserved keywords:

- **\$REFERENCE**
Reference attachments
- **\$TARGET**
Target object attachments
- **\$SIGNOFF**
Signoff attachments
- **\$RELEASE_STATUS**
Release status attachments

Placement

Requires no specific placement. Typically placed on the **Perform** action of a task. If this task has no other perform user interface, the form is used as its **Perform** action user interface.

Restrictions

None.

Examples

This example lists handler definitions to be entered on a task template to display customized forms:

- On the **Start** action: **EPM-create-form**

Argument	Values
-type	ItemRevision Master
-name	MyForm
-target_task	\$ROOT.\$REFERENCE

- On the **Perform** action: **EPM-display-form**

Argument	Values
-type	ItemRevision Master
-source_task	\$ROOT.\$REFERENCE

- On the **Complete** action: **EPM-hold**

Argument	Values
true	

EPM-end-bypass

Description

Disables bypass after **EPM-start-bypass** had enabled bypass earlier during action handler processing.

Note

The **WRKFLW_access_level_for_handlers_execution** preference has precedence over bypass subset handlers when it is set to **system** access. The system setting allows all handlers to execute with system access. Therefore, the **EPM-start-bypass** and **EPM-end-bypass** handlers have no effect.

Syntax

EPM-end-bypass

Arguments

EPM-end-bypass does not accept any arguments.

Placement

There are no restrictions on placement.

Restrictions

EPM-end-bypass must be paired with **EPM-start-bypass** on an action. This identifies when system authority should start (identified by **EPM-start-bypass**) and when system authority ends (identified by **EPM-end-bypass**).

The system authority granted by the **EPM-start-bypass/EPM-end-bypass** handler pair will not span multiple actions, so you must configure both handlers as a pair on a single action. However, you can configure the handler pair on multiple actions.

Examples

- This example will grant bypass for the **EPM-set-property** handler.

Start Action:
EPM-start-bypass

EPM-set-property

EPM-end-bypass
- This example will grant bypass for the **EPM-set-property** handler, but

Complete Action:

not for the **EPM-set-parent-result**
handler.

EPM-start-bypass

EPM-set-property

EPM-end-bypass

EPM-set-parent-result

EPM-execute-follow-up

Description

Runs a specified ITK program. During the ITK execution the parameter internally passed to the executable is **-zo=<object>**, where **<object>** is the tag of the workflow process in string format.

You can use the process tag in the ITK program by retrieving the **-zo** argument as shown in the sample program below. You can then use the POM tag to obtain process attachments, references, signoffs, and so on, using ITK functions.

Note

The ITK executable must be placed in the **<TC_ROOT>/bin** folder of the Teamcenter installation.

By default, this handler is placed on the **Complete** action of the **Review** task. If left unset, no action is taken.

Note

The user is already authenticated in the instance of the same Teamcenter server. For this reason, the code does not perform the login process again and auto login flags are not checked.

Syntax

EPM-execute-follow-up -command=<argument>

Arguments

-command

(Optional) A valid ITK program name.

Placement

Requires no specific placement.

Restrictions

The ITK executable must be placed in the **<TC_ROOT>/bin** folder of the Teamcenter installation.

Examples

This sample code converts the argument output **-zo=process_tag** from a string to a POM tag. Use the POM tag to obtain process attachments, references, signoffs, and so on, using ITK functions.

```
/* Sample code; file: test_itk_main.c */
#include <tc.h>
#include <pom.h>
int ITK_user_main(
    int argc,          /* <I> number of command line arguments */
    char* argv[]       /* <I> list of command line arguments */
)
/*
 * Description: This program is a follow-up action.
 */
{
    int    ifail = ITK_ok;
    tag_t  job_tag = NULLTAG;
    char*  job_tag_string = 0;
    ITK_initialize_text_services (ITK_BATCH_TEXT_MODE);
    if ( (ifail = ITK_auto_login ()) != ITK_ok)
    {
        printf ("ERROR: login failed - error code = %d\n",ifail);
        return ( ifail );
    }
    printf("Get process tag string ...\n"); fflush(stdout);
    job_tag_string = ITK_ask_cli_argument("-zo=");
    if (!job_tag_string)
    {
        printf ("ERROR: no process tag string passed\n");
        ITK_exit_module(TRUE);
        return 1;
    }
    printf("process tag string = %s\n", job_tag_string);
    fflush(stdout);
    printf("Convert process tag string to process tag ...\n");
    fflush(stdout);
    if ( (ifail = POM_string_to_tag(job_tag_string, &job_tag))
        != ITK_ok)
    {
        printf ("ERROR: POM_string_to_tag failed - error code
                = %d\n",ifail);
        return ( ifail );
    }
    /* start required code here */
    /* Use the process tag to get attachments, references,
       signoffs etc */
    /* ... */
    /* end required code here */
}
```

EPM-fill-in-reviewers

Description

Automatically assigns signoff reviewers that meet specified user, group, or role criteria for the specified **Review** task. This criteria populates the signoff profiles.

This handler compares the assigned user with the profile definition in the corresponding **select-signoff-team** task. If the assigned user does not match the profile definition, automatic assignment does not occur and the **select-signoff-team** task must be performed manually.

If the **-required** argument is specified; the signoffs will be added as required signoffs which cannot be altered by users.

If the **-condition_name** argument is specified; the handler will add the reviewers only if the condition is met.

Note

A user is added to **select-signoff-team** task as a reviewer only once. If the same user participates in multiple signoff profiles, use the value **resourcepool:<group>::<role>** with the **-assignee** argument.

Syntax

EPM-fill-in-reviewers

```
-assignee= [user:<user> | person:<person> | addresslist:<list>
| resourcepool:<group>::<role> | allmembers:<group>::<role>
| user:PROP::<property_name>
| resourcepool:PROP::<property_name>
| allmembers:PROP::<property_name>
| $PROPOSED_RESPONSIBLE_PARTY | $PROPOSED_REVIEWERS | $USER
| $PROCESS_OWNER | $TARGET_OWNER [<type>]
| $PROJECT_ADMINISTRATOR
| $PROJECT_TEAM_ADMINISTRATOR
| $PROJECT_AUTHOR | $PROJECT_MEMBER[<group>::<role>]
| $REQUESTOR | $ANALYST
| $CHANGE_SPECIALIST1 | $CHANGE_SPECIALIST2 | $CHANGE_SPECIALIST3
| $CHANGE_REVIEW_BOARD | $CHANGE_IMPLEMENTATION_BOARD]
[-from_include_type=<object-type1[,object-type2,...]>]
[-from_exclude_type=<object-type1[,object-type2,...]>]
[-from_attach= target | reference | schedule_task]
[-from_relation=<relation-type>]
[-from_include_related_type=<object-type1[,object-type2,...]> |
-from_exclude_related_type=<object-type1[,object-type2,...]>]
[-add_excess_as_adhoc]
[-target_task=<review-task-name | multilevel-task-path>]
[-required]
```

```
[-project_scope=all | owning_project]
[-check_first_object_only=<true | false>]
[-condition_name=<condition1>]
[-condition_scope=all | any | none]
```

Arguments

-assignee

(Required) Assigns the specified users, role members, group members, and/or resource pool members to the signoff team.

- **user:***<user>*

Adds the user specified to the signoff member list for the task to which it is attached. Accepts a valid Teamcenter user ID.

- **person:***<person>*

Adds the user whose name is specified to the signoff member list for the task to which it is attached. Accepts a valid Teamcenter person name.

Note

If the person's name includes a comma, you must include an escape character (\) to add the correct person. For example, to use **wayne, joan**:

```
-assignee=person:wayne\, joan
```

- **addresslist:***<list>*

Adds all members of the address list specified to the signoff member list.

- **resourcepool:***<group>::<role>*

Results in a single assignment which can be performed by any single member of this group/role.

You can define resource pools in the form of *<group>::*, *<group>::<role>*, or *<role>*.

Note

When a resource pool task is performed by a user it is automatically claimed by that user. If that task is a **Review** task and it is started again, the task is assigned to the user who performed it in the previous iteration, rather than the resource pool.

Accepts valid Teamcenter resource pool names and these keywords:

- **\$GROUP**

Current user's current group.

- **\$ROLE**

Current user's current role.

- **\$TARGET_GROUP[<type>]**

Owning group of the first target object of the specified type. The <type> value is optional. If not specified, the first target is used.

- **\$PROCESS_GROUP**

Owning group of the workflow process.

Note

The **\$ROLE_IN_GROUP** keyword (formerly **\$ROLEINGROUP**) cannot be used. Use **resourcepool:\$GROUP::\$ROLE** instead.

- **allmembers:<group>::<role>**

Adds all members of a group/role combination to the signoff member list. You can define role in groups in the form of <group>::, <group>::<role>, or <role>. Accepts valid Teamcenter resource pool names and these keywords:

- **\$GROUP**

Current user's current group.

- **\$ROLE**

Current user's current role.

- **\$TARGET_GROUP[<type>]**

Owning group of the first target object of the specified type. The <type> value is optional. If not specified, the first target is used.

- **\$PROCESS_GROUP**

Owning group of the workflow process.

- **user:PROP::<property_name>**

Adds the user specified by the property name to the signoff member list for the task to which it is attached.

If the property is a multi-value property, only the first value is used when only a single user is assigned in the workflow. When more than one user is assigned, all property values are used.

- **resourcepool:PROP::<property_name>**

Adds the resource pool specified by the property name to the signoff member list for the task to which it is attached.

If the property is a multi-value property, only the first value is used when only a single user is assigned in the workflow. When more than one user is assigned, all property values are used.

- **allmembers:PROP::<property_name>**

Adds all members of a group/role combination that is specified by the property name to the signoff member list.

If the property is a multi-value property, only the first value is used when only a single user is assigned in the workflow. When more than one user is assigned, all property values are used.

- **\$PROPOSED_RESPONSIBLE_PARTY**

Affects assignments based on the user assigned as the responsible party for the first target object.

- **\$PROPOSED_REVIEWERS**

Affects assignments based on members assigned as reviewers for the first target object.

- **\$USER**

Adds the current user to the signoff member list.

If **\$USER** is used, and the current user belongs to several groups and roles, the behavior of the **\$USER** keyword depends on the value of the **SIGNOFF_fill_in_reviewers** preference, as follows:

- **1**

Attempts to match the current user's group/role values with the profile first, default values second, then any other groups/roles of which the current user is a member. This is the default setting.

- **2**

Attempts to match the current user's group/role values first, default values of which the current user is a member second.

- **3**

Attempts to match the current user's group/role values.

- **\$PROCESS_OWNER**

Adds the workflow process owner to the signoff member list.

- **\$TARGET_OWNER [<type>]**

Adds the owner of the first target of specified type to the signoff member list. The <type> value is optional. If not specified, the first target is used.

- **\$PROJECT_ADMINISTRATOR, \$PROJECT_TEAM_ADMINISTRATOR, \$PROJECT_AUTHOR, \$PROJECT_MEMBER[<group>::<role>]**

Dynamically adds the project team members belonging to the role specified in the argument value. The project team is determined by the project team associated with the first target object.

If the **\$PROJECT_MEMBER[<group>::<role>]** argument is specified, only the project members of the qualifying projects which belong to the specified group and role are selected for assignment. If the group and role are not specified, all the project members from qualifying projects are selected.

You can specify a sub-group with the syntax **<group>++<sub-group>::<role>**.

- **\$REQUESTOR, \$ANALYST, \$CHANGE_SPECIALIST1, \$CHANGE_SPECIALIST2, \$CHANGE_SPECIALIST3, \$CHANGE_REVIEW_BOARD, \$CHANGE_IMPLEMENTATION_BOARD**

Dynamically resolves to the user or resource pool associated with the first Change target object in the process. The particular user or resource pool is determined by the role specified in the argument value.

Note

Change-related keywords apply only to change objects. If the process does not contain a change object as a target, the argument resolves to null.

Change Manager does not need to be enabled before these keywords take effect, but during installation, make sure you install the **Change Management** application using your preferred installation tool (Deployment Center or Teamcenter Environment Manager) to ensure change-related keywords are enabled.

-from_include_type=<object-type1[,object-type2,...]>

(Optional) Specifies the object types to be used to get the property value from when a property is specified in the **-assignee** argument (for example, **-assignee=user:PROP::<property_name>**). They must be valid object types.

You can use this argument only when you get the assignee from a property on an object (**user:PROP::** or **resourcepool:PROP::**).

-from_exclude_type=<object-type1[,object-type2,...]>

(Optional) Specifies the object types to be excluded when getting the property value when it is specified in the **-assignee** argument (for example, **-assignee=user:PROP::<property_name>**). They must be valid object types.

You can use this argument only when you get the assignee from a property on an object (**user:PROP::** or **resourcepool:PROP::**).

-from_attach= target | reference | schedule_task

(Optional) Specifies which type of attachment (**target**, **reference**, or **schedule_task**) to get the property value from when a property is specified in the **-assignee** argument (for example, **-assignee=user:PROP::<property_name>**). If this argument is not specified, the default is **target**.

You can use this argument only when you get the assignee from a property on an object (**user:PROP::** or **resourcepool:PROP::**).

-from_relation

(Optional) Specifies the relation of the objects to get the property value from when a property is specified in the **-assignee** argument (for example, **-assignee=user:PROP::<property_name>**). It must be a valid relation.

- For manifestations, use **IMAN_manifestation**.
- For specifications, use **IMAN_specification**.
- For requirements, use **IMAN_requirement**.
- For references, use **IMAN_reference**.
- For BOM views, use **PSBOMViewRevision**.

This argument must be used with the **-from_attach** argument. A derived object is identified by starting with objects of the specified attachment type indicated by the **-from_attach** argument and then locating the first secondary object with the specified relation indicated by the **-relation** argument.

You can use this argument only when you get the assignee from a property on an object (**user:PROP::** or **resourcepool:PROP::**).

-from_include_related_type=<object-type1[,object-type2]>

(Optional) Specifies the related object types to be used to get the property value from when a property is specified in the **-assignee** argument (for example, **-assignee=user:PROP::<property_name>**). They must be valid object types.

You can use this argument only when you get the assignee from a property on an object (**user:PROP::** or **resourcepool:PROP::**) and you use the **-from_relation** argument.

This argument should not be used with the **-from_exclude_related_type** argument.

-from_exclude_related_type=<object-type1[,object-type2]>

(Optional) Specifies related object types to be excluded when getting the property value when it is specified in the **-assignee** argument (for example, **-assignee=user:PROP::<property_name>**). They must be valid object types.

You can use this argument only when you get the assignee from a property on an object (**user:PROP::** or **resourcepool:PROP::**) and you use the **-from_relation** argument.

This argument should not be used with the **-from_include_related_type** argument.

-add_excess_as_adhoc

(Optional) Adds the rest of the assignees as ad hoc users if the profile is satisfied.

-target_task

(Optional) Specifies either the single **Review** task name or multilevel task path to which the reviewers are added. The path is from the root task to the **select-signoff-team** subtask with the path levels separated with colons (:). For example: **Change Request Review:QA Review:select-signoff-team**

-required

(Optional) If specified, all signoffs added through this handler instance are marked as mandatory.

-project_scope

(Optional) Specifies which projects are used to resolve project-based assignments. The **all** value specifies all projects in the list of projects. The **owning_project** value specifies the owning project only.

If this argument is not specified, the default value is the first project in the project list.

-check_first_object_only

(Optional) The **true** value specifies that only the first object is checked. If the value is **false**, all objects are checked. If this argument is not specified, or if it is specified without a value, only the first object is checked.

If the **-include_type**, **-exclude_type**, **-include_related_type**, or **-exclude_related_type** arguments are specified, they determine the types of objects that are checked.

-condition_name

(Optional) The name of the condition to evaluate against the objects identified for assigning reviewers from. The condition signature should accept a **WorkspaceObject & UserSession**. The handler assigns the reviewers only if the condition results are successful, based on the **-condition_scope** argument.

-condition_scope

(Optional) The criteria for evaluating condition results against workflow objects. Values are the following:

- all** All objects should meet the condition. This is the default behavior if this argument is not supplied with the **-condition_name** argument.
- any** Any object should meet the condition.
- none** No object should meet the condition.

Placement

Place either on the **Start** action of the relevant **select-signoff-team** task or on the root task with the **-review_task_name** argument.

Restrictions

Use only with the **select-signoff-team** task or on the root task.

Examples

- This example designates the user **tom** and all members of the **engineering** group as reviewers for the **Review** task called **Review Task 1**.

Argument	Values
-assignee	user:tom, allmembers:engineering::
-target_task	\$ROOT.Review Task 1

- This example shows the current user added as a reviewer.

Argument	Values
-assignee	user:\$USER
-target_task	Review Task 1

- This example designates members assigned as reviewers for the first target object as reviewers for the **Review** task called **Review Task 1**.

Argument	Values
-assignee	\$PROPOSED_REVIEWERS
-target_task	Review Task 1

- This example designates user **tom**, all the members of the **Engineering** group, and the **REQUESTOR** associated with the first change target object as reviewers for the **Review** task named **Review Task 1**.

Argument	Values
-assignee	user:tom, allmembers:engineering::\$REQUESTOR
-target_task	Review Task 1

If the handler with these arguments is specified on the **Notify** task under the **Route** task, the signoff attachments are added to the **Notify** task and used for sending notifications.

- This example assigns all members of the **Engineering** group and **Designer** role of the first project team associated with the first target found by the system to the signoff team as optional signoffs.

Argument	Values
-assignee	\$PROJECT_MEMBER[Engineering::Designer]

- This example assigns all members of the **Engineering** group and **Designer** role of the owning project team associated with the first target found by the system to the signoff team as optional signoffs.

Argument	Values
-assignee	\$PROJECT_MEMBER[Engineering::Designer]
-project_scope	owning_project
-check_first_object_only	

- This example assigns all members of the **Engineering** group and **Designer** role of all project teams associated with the first target found by the system to the signoff team as required signoffs.

Argument	Values
-assignee	\$PROJECT_MEMBER[Engineering::Designer]
-project_scope	all
-check_first_object_only	true
-required	

- This example assigns all members of the **Engineering** group and **Designer** role of the first project team associated with each target found by the system to the signoff team as optional signoffs.

Argument	Values
-assignee	\$PROJECT_MEMBER[Engineering::Designer]
-check_first_object_only	false

EPM-inherit

Description

Inherits specified attachment types from a specified task.

Syntax

EPM-inherit -task=\$PREVIOUS | \$CALLER | \$ROOT
-attachment=target | reference | signoffs

Arguments

-task

(Optional) Task that contains the attachments to be inherited. Choices are the **\$PREVIOUS** task, the parent task (**\$CALLER**) or the **\$ROOT** task. You can use multiple values by separating them with commas or the character specified by the **EPM_ARG_target_user_group_list_separator** preference.

-attachment

(Optional) Attachment types that are inherited from the tasks specified in the **-task** argument. Choices are **target**, **reference**, or **signoffs**. You can use multiple values by separating them with commas or the character specified by the **EPM_ARG_target_user_group_list_separator** preference.

Placement

Requires no specific placement.

Restrictions

None.

Examples

- This example copies the reference attachments from the parent task to the current task.

Argument	Values
-task	\$CALLER
-attachment	reference

- This example copies the signoffs from the previous task and the targets from the root task to the current task. The handler is placed on the **perform-signoffs** subtask of the second **Review** task.

Argument	Values
-task	\$PREVIOUS, \$ROOT
-attachment	signoffs, target

EPM-invoke-system-action

Description

Runs an external command (specified with the **-command** argument) such as Perl scripts, shell scripts, or external ITK programs, then continues or halts the workflow process based on the return code of the external command.

Use this handler for increased control of the workflow process. For example, to synchronize NX attributes and structure with Teamcenter, or to generate JT tessellation from CAD files.

This handler writes workflow process-related information to an XML file. The file is passed to the external script/program as **-f <XML-file-name>**. APIs are provided (in the form of Perl modules) to read the XML file and perform functions on its data objects. The APIs are located in the **Workflow.pm** file in the **<TC_ROOT>/bin/tc** directory.

Write Perl scripts (for example, **<TC_ROOT>/bin/tc_check_renderings.pl** for background tessellation of CAD data) using the provided APIs to read the XML file and perform required functions on its data objects. Then use the Perl script as the value of the **-command** argument (for example, **-command=<perl-script-name>**) in the workflow process template.

Note

Siemens Digital Industries Software recommends you place the Perl scripts in the **<TC_ROOT>/bin** folder.

Alternatively, you can place the script in an alternate location and provide an absolute path to the location (for example, **c:\temp\test.bat**). However, using an absolute path requires that you update the template if there are any changes. In the previous example, **c:\temp\test.bat** is a path on a Windows platform. If you were to change to a Linux platform, the template would need to be updated. This second method is not recommended.

The handler returns a code that is mapped to:

- **ITK_ok** when the external script returns **0** and no other errors are returned
- **CR_error_in_handler** in all other cases

Syntax

```
EPM-invoke-system-action -command=<name-of-the-external-program>
[-trigger_on_go= [<TASK>:]<ACTION>] [-trigger_on_nogo= [<TASK>:]<ACTION>]
[-trigger_on_undecided= [<TASK>:]<ACTION>] [-skip_unreadable_objs]
[-change_status_on_go=<[[old-status-name]:[new-status-name]]>]
[-change_status_on_nogo=<[[old-status-name]:[new-status-name]]>]
[-change_status_on_undecided=<[[old-status-name]:[new-status-name]]>]
[-add_occurrence_notes] [-comment=<comment>]
[-responsible_party= User:<responsible-party>; Task:<task-name>]]
```



```
[-reviewer=User:<user-id>] [; Group:<group>] [; Role:<role>] [; Level:<level>]]
[-send_mail=<user-ids>] [-initiate_process]
[-where_used=<itemrevtype>] [-expand=<itemrevtype>]
[-list_sibling_processes=<wildcarded-procname>]
[-depth=<maximum-recursion-depth>] [-debug]
```

Arguments

-command=<name-of-the-external-program>

(Required) Name of the external executable. This executable can be an external Perl script that reads and modifies the XML file written by this handler, or an ITK program to perform specific functionality.

-trigger_on_go= [<TASK>:]<ACTION>

(Optional) Triggers an action in the same workflow process when **EPM_go** is returned.

Trigger an action in another task by specifying an action and task name. If another task name is unspecified, the specified action in the current task is triggered.

The system supports the following actions:

ASSIGN, START, PERFORM, COMPLETE, SUSPEND, RESUME, SKIP, ABORT, REFUSE, UNDO, REJECT, APPROVE, PROMOTE, DEMOTE.

Action names are not case-sensitive.

Task names cannot contain a colon or a period. If the task name is ambiguous (for example, **select-signoff-team**), hierarchical notation is required.

-trigger_on_nogo= [<TASK>:]<ACTION>

(Optional) Triggers an action in the same workflow process when **EPM_nogo** is returned.

Trigger an action in another task by specifying an action and task name. If another task name is unspecified, the specified action in the current task is triggered.

The system supports the following actions:

ASSIGN, START, PERFORM, COMPLETE, SUSPEND, RESUME, SKIP, ABORT, REFUSE, UNDO, REJECT, APPROVE, PROMOTE, DEMOTE.

Action names are not case-sensitive.

Task names cannot contain a color or period. If the task name is ambiguous (for example, **select-signoff-team**), hierarchical notation is required.

-trigger_on_undecided= [<TASK>:]<ACTION>

(Optional) Triggers an action in the same workflow process when **EPM_undecided** is returned.

Trigger an action in another task by specifying an action and task name. If another task name is unspecified, the specified action in the current task is triggered.

The system supports the following actions:

ASSIGN, START, PERFORM, COMPLETE, SUSPEND, RESUME, SKIP, ABORT, REFUSE, UNDO, REJECT, APPROVE, PROMOTE, DEMOTE.

Action names are not case-sensitive.

Task names cannot contain a color or period. If the task name is ambiguous (for example, **select-signoff-team**), hierarchical notation is required.

-skip_unreadable_objs

(Optional) Unreadable objects are not processed. The handler does not attempt to write information about unreadable objects into the XML file; the objects are skipped.

If this argument is not specified, the handler displays an error when a failure occurs when there is no read access.

-change_status_on_go=<[[old-status-name]:[new-status-name]]>

(Optional) Adds, removes or changes the status of attachments when **EPM_go** is returned.

Both the old and new status names are optional.

- If both status names are specified, the new status name replaces the old status name.
- If only the new status name is specified, the corresponding status is added.
- If only the old status name is specified, the corresponding status name is removed.
- If neither status name is specified, no action is taken.

If a value is not provided for this argument, the value set by the external Perl script is read.

-change_status_on_nogo=<[[old-status-name]:[new-status-name]]>

(Optional) Adds, removes, or changes the status of attachments when **EPM_nogo** is returned.

Both the old and new status names are optional.

- If both status names are specified, the new status name replaces the old status name.
- If only the new status name is specified, the corresponding status is added.
- If only the old status name is specified, the corresponding status name is removed.
- If neither status name is specified, no action is taken.

If a value is not provided for this argument, the value set by the external Perl script is read.

-change_status_on_undecided=<[[old-status-name]:[new-status-name]]>

(Optional) Adds, removes or changes the status of attachments when **EPM_undecided** is returned.

Both the old and new status names are optional.

- If both status names are specified, the new status name replaces the old status name.

- If only the new status name is specified, the corresponding status is added.
- If only the old status name is specified, the corresponding status name is removed.
- If neither status name is specified, no action is taken.

If a value is not provided for this argument, the value set by the external Perl script is read.

-add_occurrence_notes

(Optional) Sets occurrence notes of target assemblies. Can be used in combination with the **-expand** argument to set **OccurrenceNotes** for components of assembly structures.

-comment=<comment>

(Optional) The signoff decision is set depending on the return code of the external program:

- 0=Approve
- 1=Reject
- 2=No Decision

If a value is not provided for this argument, the value set by the external Perl script is read.

-responsible_party= User:<responsible-party>[; Task:<task-name>]

(Optional) Assigns a responsible party. If no user ID is specified for this argument, the value set by the external Perl script is read.

-reviewer=[User:<user-id>] [; Group:<group>] [; Role:<role>] [; Level:<level>]

(Optional) Assigns a reviewer for a release level. If no reviewer is specified for this argument, the value set by the external Perl script is read.

-send_mail=<user-id[,user-id,...]>

(Optional) Sends target, reference, or sibling objects through the program mail. If one or more user IDs are defined for this argument, the workflow process is sent to the specified users through the program mail.

Separate multiple user IDs with a space, a comma, or the character specified by the **EPM_ARG_target_user_group_list_separator** preference.

If no user IDs are defined for this argument, the recipients and the contents of the envelope set by the external Perl script are read.

-initiate_process

(Optional) Initiates a workflow process for another object. Target objects are defined by the values set by the external Perl script.

-where_used=<itemrevtype>

(Optional) Reports the where-used of item and item revision target attachments by writing the hierarchy of all parent and grandparent assemblies of item and item revision target attachments into the XML file to allow the external Perl script to perform required functions.

If an item revision type is specified, the type argument is compared to the corresponding item revision type. For example, **ItemRevision** matches objects of the type **Item**.

If an item revision type is specified, the parent assemblies of only those target attachments that match this type are listed.

-expand=<itemrevtype>

(Optional) Reports the assembly of item and item revision target attachments by writing the hierarchy of all child and grandchild components of item and item revision target attachments into the XML file to allow the external Perl script to perform required functions.

If an item revision type is specified, the type argument is compared to the corresponding item revision type. For example, **ItemRevision** matches objects of the type **Item**. The assembly structure is expanded for all item revisions of all matching item target attachments.

If an item revision is specified, the child components of only those target attachments are listed that match this type.

-list_sibling_processes=<wildcarded-procname>

(Optional) Writes information regarding processes that belong to the same change item into the XML file to allow the external Perl script to perform required functions. The information concerns processes sharing the same change item as reference attachment.

If a process template name is specified in the procedure definition, only the processes that match the procedure name are included.

-depth=<maximum-recursion-depth>

(Optional) Increases the maximum incursion depth. The **-trigger_on_go** or **-initiate_process** arguments could cause the triggered action to use the same handler in a deeper level of recursion. If this is what you intend, you must set the maximum level of recursion to the desired number. If necessary, it can be disabled by setting it to **0**. The default is set to **1**, to avoid infinite loops.

-debug

(Optional) Enables debugging. Each occurrence of this argument increases the debug level by one. Debug messages are written to the Teamcenter error stack for display in the rich client user interface, as well as written to the **syslog** file.

Placement

Place on the **Start** or **Complete** action of any task. If this handler is configured to set the signoff decisions on a **perform-signoffs** task (for example, if the **-comment** argument is specified), then place on the **Complete** action of the **perform-signoffs** task.

Restrictions

- Do not add to a workflow process containing *any* handler using resource pools.
- You cannot use the **-trigger_on_go** argument to start a task if any of the tasks previous to it in the workflow process are not complete.

Examples

- This example shows how to run the **tc_check_renderings_pl** script using the **-command** argument. Do not list the file extension in the value. This value runs either the **tc_check_renderings_pl.bat** (Windows) or **tc_check_renderings_pl** (Linux) script, depending on which platform the server is running.

Note

The script should be placed in the <TC_ROOT>/bin directory.

Argument	Values
-command	tc_check_renderings_pl

- This example shows how to run the **test_action.bat** script in a Windows system. The script is the following:

```
set rc=2
echo %rc% >> c:\temp\test.log
exit 0
```

It is used in the following workflow process:



Create one signoff profile for the **Review** task and place the **EPM-invoke-system-action** handler on the **Complete** action of the **Review** task with the following arguments:

Argument	Values
-command	c:\temp\test_action.bat
-expand	
-debug	

- This example shows how to run the **test_action** script in a Linux system. The script is the following:

```
#!/bin/sh
rc=2
export rc
echo $rc > /tmp/test.log
exit $rc
```

It is used in the following workflow process:



Create one signoff profile for the **Review** task and place the **EPM-invoke-system-action** handler on the **Complete** action of the **Review** task with the following arguments:

Argument	Values
-command	/tmp/test_action
-expand	
-debug	

- This example, placed on the **Complete** action of the **perform-signoffs** task, runs the **tc_check_install_assembly_pl** script using the **-command** argument. The script looks at a vehicle structure and checks to ensure each component has:
 - A valid release status for the structure development stage and not **In Process**.
 - All occurrences are precise and have an occurrence note indicating its usage at this stage.
 - Every target attachment is a component of only one multilevel product item.

If the target of the original workflow process is a component of only one multilevel product item, the **-initiate_process** argument starts the **Initiate VPPS** workflow process specified in the Perl script and attaches the vehicle as a target and its work orders as references.

Note

The script is in the **sample\task_handlers** directory and should be placed in the **<TC_ROOT>/bin** directory.

Argument	Values
-command	tc_check_install_assembly_pl
-initiate_process	

EPM-late-notification

Description

Serves as an initializer to store the specified members of the default recipient's list. Notification of a late task is triggered when the **Task Manager** daemon identifies the late task in a worklist. An email is then sent to the task's specified recipients, notifying the recipients that the task is late. The **Task Manager** daemon be installed using your preferred installation tool (Deployment Center or Teamcenter Environment Manager).

Note

If a Review task is rejected and is demoted back to an earlier task, its due date does not get reset. Once a due date is set on a Review task, then even if the task is demoted for rework, that date remains the same to maintain the schedule.

However, if the desired behavior is to reset the due date when a task is rejected or demoted, then add a reject path. A reject will cause the task to change to the Failed state, and therefore the task will not be reported as being late.

Syntax

EPM-late-notification

```
-recipient= | <user> | <group>
| $OWNER
| $REVIEWERS | $PROPOSED_REVIEWERS
| $RESPONSIBLE_PARTY | $PROPOSED_RESPONSIBLE_PARTY
| $UNDECIDED
| $PROJECT_ADMINISTRATOR
| $PROJECT_TEAM_ADMINISTRATOR
| $PROJECT_AUTHOR | $PROJECT_MEMBER
| $TARGET_OWNER | $PROCESS_OWNER
| $RESOURCE_POOL_ALL | $RESOURCE_POOL_NONE
| $RESOURCE_POOL_SUBSCRIBED
| $REQUESTOR
| $ANALYST
| $CHANGE_SPECIALIST1
| $CHANGE_SPECIALIST2
| $CHANGE_SPECIALIST3
| $CHANGE_REVIEW_BOARD
| $CHANGE_IMPLEMENTATION_BOARD | <distribution-list>
```

Arguments

-recipient
(Required)

- `<user>`
Specifies a specific user. It must be the name of a valid Teamcenter user.
- `<group>`
Specifies a specific group. It must be the name of a valid Teamcenter group.
- **\$OWNER**
Specifies the task owner.
- **\$REVIEWERS**
Specifies all users who are reviewers in the same task level as the current reviewer.
- **\$PROPOSED_REVIEWERS**
Sends email to all members assigned as the proposed reviewers of the first target object in the workflow process.
- **\$RESPONSIBLE_PARTY**
Specifies the responsible party of the task.
- **\$PROPOSED_RESPONSIBLE_PARTY**
Sends email to the member assigned as the proposed responsible party of the first target object in the workflow process.
- **\$UNDECIDED**
Specifies the users who have not set the decision for the task.
- **\$PROJECT_ADMINISTRATOR**
\$PROJECT_TEAM_ADMINISTRATOR
\$PROJECT_AUTHOR
\$PROJECT_MEMBER

These values dynamically evaluate project team members belonging to the role specified in the argument value and send notifications to those members. The project team is determined by the project team associated with the first target object.
- **\$TARGET_OWNER**
Sends email to the target owner of the first target of the specified type.
The type value is optional. If it is not specified, the first target is used.
- **\$PROCESS_OWNER**
Sends email to the workflow process owner.
- **\$RESOURCE_POOL_ALL**
Specifies all the members of the resource pool.

This argument has an effect only when it is used along with **\$REVIEWERS**, **\$UNDECIDED**, or **\$RESPONSIBLE_PARTY**.

When this argument is used along with **\$REVIEWERS**, and if a resource pool is assigned as a reviewer, then email is sent to all the members of that resource pool.

When this argument is used along with **\$UNDECIDED**, and if a resource pool is assigned as a reviewer, and no signoff decision has been made for this resource pool assignment, then all members of that resource pool are notified.

When this argument is used along with **\$RESPONSIBLE_PARTY**, and if a resource pool is assigned as responsible party, then the email is sent to all members of that resource pool.

- **\$RESOURCE_POOL_NONE**

This argument has an effect only when it is used along with **\$REVIEWERS**, **\$UNDECIDED**, or **\$RESPONSIBLE_PARTY**.

When this argument is used along with **\$REVIEWERS** or **\$UNDECIDED**, and if a resource pool is assigned as a reviewer, then the email is not sent to members or subscribers of the resource pool.

When this argument is used along with **\$RESPONSIBLE_PARTY**, and if a resource pool is assigned as a responsible party, then the email is not sent to members or subscribers of resource pool.

- **\$RESOURCE_POOL_SUBSCRIBED**

Specifies the users who have subscribed to resource pool.

This argument has an effect only when it is used along with **\$REVIEWERS**, **\$UNDECIDED**, or **\$RESPONSIBLE_PARTY**.

When this argument is used along with **\$REVIEWERS**, and if a resource pool is assigned as a reviewer, then the email is sent to users are subscribed to the resource pool.

When this argument is used along with **\$UNDECIDED**, and if a resource pool is assigned as a reviewer and no signoff decision has been made for this resource pool assignment, then email is sent to users who are subscribed to the resource pool.

When this argument is used along with **\$RESPONSIBLE_PARTY**, and if a resource pool is assigned as a responsible party, then, the email is sent to users who are subscribed to the resource pool.

- **\$REQUESTOR, \$ANALYST, \$CHANGE_SPECIALIST1, \$CHANGE_SPECIALIST2, \$CHANGE_SPECIALIST3, \$CHANGE_REVIEW_BOARD, \$CHANGE_IMPLEMENTATION_BOARD**

Dynamically resolves to the user or resource pool associated with the first Change target object in the process. The particular user or resource pool is determined by the role specified in the argument value.

Note

Change-related keywords apply only to change objects. If the process does not contain a change object as a target, the argument resolves to null.

Change Manager does not need to be enabled before these keywords take effect, but during installation, **Change Management** must be selected.

- **<distribution-list>**

Specifies all members of the specified distribution list. This entry can either be the name of a valid address list, or any one of several keywords that represent a distribution list.

Placement

Place on the **Start** action.

When **\$REVIEWERS** or **\$UNDECIDED** is used as the key word, place on the **Start** action of the **perform-signoffs** task.

To add the **EPM-late-notification** handler to the task, select the task and the **Display the Task Attributes Panel**. Insert the duration and recipients.

Restrictions

None.

Examples

- This example builds a list of all users assigned as reviewers for the **perform-signoffs** task, along with the owner of the task, and sends email to them.

Argument	Values
-recipient	\$REVIEWERS, \$OWNER

- This example sends email to reviewers of the task who have not performed the signoff.

Argument	Values
-recipient	\$UNDECIDED

- This example sends email to user **Smith**, a distribution list (**VendorList**), and members of the **Purchase** group.

Argument	Values
-recipient	Smith, VendorList, Purchase

Note

The **Task Attributes** shortcut menu in Workflow Designer populates the arguments to handler. However, you can insert the keywords argument using the **Task Handlers Panel**.

- This example represents a late notification email.

Subject:
Late Notification for <PROCESS_NAME/TASK_NAME>

Contents:
You have a late task in Teamcenter that requires attention. Details shared below:

Overview:

Process Name: <PROCESS_NAME>

Current Task: <TASK_NAME>

Due Date: <Due date for the task>

Instructions: <TASK_INSTRUCTIONS>

Select the preferred client to view the task:

<Link to Rich Client>

<Link to Active Workspace>

This email was sent from Teamcenter.

EPM-move-attached-objects

Description

Changes or copies workflow attachments from one attachment type to another. If the handler requires attaching replica objects as workflow targets, the handler attaches them as **Replica Proposed Targets**.

Syntax

EPM-move-attached-objects **-from_attach**=<attachment-type> **-to_attach**=<attachment-type>

[**-include_type**=<comma-separated-type-list> | **-exclude_type**=<comma-separated-type-list>]

[**-copy**]

Arguments

-from_attach

(Required) Specify one of the following attachment types from which the attached objects should be selected. This is a mandatory argument.

- **target**
- **reference**
- **problem_item**
- **solution_item**
- **impacted_item**

-to_attach

(Required) Specifies one of the following new attachment types for the attached objects. This is a mandatory argument.

- **target**
- **reference**
- **problem_item**
- **solution_item**
- **impacted_item**

-include_type

(Optional) Specifies the object types whose attachment type is to be changed. The handler changes the attachment type defined in the **-from_attach** argument of objects that are the types or their subtypes specified in this argument. Do not use this argument with the **-exclude_type** argument.

Separate multiple types with commas or the character specified by the **EPM_ARG_target_user_group_list_separator** preference.

-exclude_type

(Optional) Ignores the object types specified by this argument. Attachments to these objects are not changed by this handler. Do not use this argument with the **-include_type** argument.

Separate multiple types with commas or the character specified by the **EPM_ARG_target_user_group_list_separator** preference.

-copy

(Optional) Adds the attachments with the new relation defined by the **-to_attach** argument and leaves the attachments with the original relation. If this argument is not specified, the objects are removed from the attachment type specified by the **-from_attach** argument.

Placement

Place on the **Start** or **Complete** action of any task. Do not place on the **Perform** action.

Restrictions

None.

EPM-notify

Description

Informs users of a task's status through an email notification.

The **EPM-notify** handler can send notifications to users through Teamcenter mail only if the **Mail_internal_mail_activated** preference is set to **True**. **OS** mail is not controlled by this preference.

The **-report** argument on the **EPM-notify-report** handler differentiates the **EPM-notify-report** handler from the **EPM-notify** handler. In the email notification, the **-report** argument appends a report describing the signoff data associated with the **perform-signoffs** task. Therefore, you should use the **EPM-notify-report** handler on the **perform-signoffs** task, whereas the **EPM-notify** handler is more generic and can be used on any type of task.

If you place the **EPM-notify** handler on the **Perform** action (**EPM_perform_action**), an email notification is sent each time a **Perform** sub-action is triggered. These multiple notifications can cause unnecessary processing.

For example, a handler on the **Perform** action is executed three times by the **Add Attachment** sub-action (**EPM_add_attachment_action**). If the handler is **EPM-notify**, reviewers receive the same notification at three different intervals.

In addition to **Add Attachment**, the **Perform** action can include the following sub-actions:

- **Remove Attachment** (**EPM_remove_attachment_action**)
- **Approve** (**EPM_approve_action**)
- **Reject** (**EPM_reject_action**)
- **Promote** (**EPM_promote_action**)
- **Demote** (**EPM_demote_action**)
- **Assign Approver** (**EPM_assign_approver_action**)

Use the **Mail_OS_from_address** preference to specify the **From** address displayed in the notification email. The preference value must be a valid email address.

When placed on the **Start** action of **perform-signoffs** task, the **EPM-notify** or **EPM-notify-report** handlers are automatically re-executed when a signoff is delegated.

Note

Use caution when entering special characters into argument fields of notification handlers. Depending on your configuration and email client, using special characters and character entities in argument values may not display correctly in email notifications. These characters can interfere with the mail notification utility **tc_mail_smtp**, and should be tested before deployment.

Syntax

EPM-notify

```

-recipient=
{OS:<user-name>
| user:<user>
| person:<person>| addresslist:<value>

| resourcepool:<group::role>

| allmembers:<group::role>
| $USER
| $REVIEWERS | $PROPOSED_REVIEWERS
| $RESPONSIBLE_PARTY
| $PROPOSED_RESPONSIBLE_PARTY
| $UNDECIDED
| $PROJECT_ADMINISTRATOR |
| $PROJECT_TEAM_ADMINISTRATOR
| $PROJECT_AUTHOR | $PROJECT_MEMBER<[group>::<role]>
| $TARGET_OWNER | $PROCESS_OWNER
| $RESOURCE_POOL_ALL | $RESOURCE_POOL_NONE
| $RESOURCE_POOL_SUBSCRIBED
| $REQUESTOR
| $ANALYST
| $CHANGE_SPECIALIST1
| $CHANGE_SPECIALIST2
| $CHANGE_SPECIALIST3
| $CHANGE_REVIEW_BOARD
| $CHANGE_IMPLEMENTATION_BOARD

[-subject=
<string | $TARGET | string $TARGET string | >
<string | $STATE | string $STATE string | >
<$PROCESS | string $PROCESS string | >
<$TASK | string $TASK string |>]
[-comment=<string>]
[-url = {rich | activeworkspace | none}]

[-attachment={target | process | reference}]

```

Arguments

Adds an attachment to Teamcenter mail and adds attachment information for operating system email. The value can be any of the following:

-recipient

(Required) Specifies the task reviewers receiving notification. Any surrogates for the specified users are also notified. Accepts one of the following values:

- **OS**

Sends a notification to the OS user name specified.

`<user-name>` is a single valid user name.

- **user**

Sends notification to the user specified.

`<user>` is a single valid Teamcenter user ID.

- **person**

Sends a notification to user whose name is specified.

`<person>` is a single valid Teamcenter person.

Note

If the person's name includes a comma, you must include an escape character (\) to add the correct person. For example, to use **wayne, joan**:

-recipient=person:wayne\, joan

- **addresslist**

Sends a notification to all members of the address list.

`<value>` is a valid Teamcenter address list.

- **resourcepool**

Sends notification to members of a group/role combination. Notification is sent to all members, subscribed members, or none based on the **EPM_resource_pool_recipients** preference.

The preference value can be overridden with:

\$RESOURCE_POOL_ALL

\$RESOURCE_POOL_SUBSCRIBED

\$RESOURCE_POOL_NONE

You can define role in groups in the form of **group::**, **group::role** or **role**.

`<value>` is a valid Teamcenter resource pool and these keywords:

\$GROUP

Current user's current group.

\$ROLE

Current user's current role.

\$TARGET_GROUP[<type>]

Owning group of the first target object of the specified type. The type value is optional. If not specified, the first target is used.

\$PROCESS_GROUP

Owning group of the workflow process.

- **allmembers**

Sends notification to all members of a group/role combination.

<value> is all members of a Teamcenter group and role.

You can define role in groups in the form of **group::, group::role** or **role**.

Accepts valid Teamcenter resource pool names and these keywords: **\$GROUP**, **\$ROLE**, **\$TARGET_GROUP** and **\$PROCESS_GROUP**.

Note

The **\$ROLE_IN_GROUP** keyword (formerly **\$ROLEINGROUP**) cannot be used. Use **allmembers:\$GROUP::\$ROLE** instead.

- **\$USER**

Sends email to the current user.

- **\$REVIEWERS**

Builds a list of all users who are reviewers in the same task level as the current reviewer and sends email to all of them.

- **\$PROPOSED_REVIEWERS**

Sends email to all members assigned as the proposed reviewers of the first target object in the workflow process.

- **\$RESPONSIBLE_PARTY**

Sends email to the designated responsible party for the task.

If you use **\$RESPONSIBLE_PARTY**, add the handler to the **Start** action of the task, not the **Assign** action.

- **\$PROPOSED_RESPONSIBLE_PARTY**

Sends email to the member assigned as the proposed responsible party of the first target object in the workflow process.

- **\$PROCESS_OWNER**

Sends email to the workflow process owner.

- **\$TARGET_OWNER** [<type>]

Sends email to the target owner of the first target of the specified type. The <type> value is optional. If it is not specified, the first target is used.

- **\$UNDECIDED**

Sends email to the users who have not set the decision for the task.

- **\$PROJECT_ADMINISTRATOR**
\$PROJECT_TEAM_ADMINISTRATOR
\$PROJECT_AUTHOR
\$PROJECT_MEMBER[<group>::<role>]

These values dynamically evaluate project team members belonging to the role specified in the argument value and send a notification to them. The project team is determined by the project team associated with the first target object.

If the **\$PROJECT_MEMBER[<group>::<role>]** argument is specified, only the project members of the qualifying projects which belong to the specified group and role are selected as recipients. If the group and role are not specified, all the project members from qualifying projects are selected.

- **\$REQUESTOR, \$ANALYST, \$CHANGE_SPECIALIST1, \$CHANGE_SPECIALIST2,**
\$CHANGE_SPECIALIST3
\$CHANGE_REVIEW_BOARD, \$CHANGE_IMPLEMENTATION_BOARD

Dynamically resolves to the user or resource pool associated with the first Change target object in the process. The particular user or resource pool is determined by the role specified in the argument value.

If custom participants are defined by the customer, those participants can be used as recipients.

Note

Change-related keywords apply only to change objects. If the process does not contain a change object as a target, the argument resolves to null.

Change Manager does not need to be enabled before these keywords take effect, but during installation, make sure you install the **Change Management** application using your preferred installation tool (Deployment Center or Teamcenter Environment Manager) to ensure change-related keywords are enabled.

- **\$RESOURCE_POOL_ALL**

Identifies all members of the resource pool.

This argument has an effect only when it is used along with **\$REVIEWERS, \$UNDECIDED, \$RESPONSIBLE_PARTY** or any Dynamic participant, such as **\$CHANGE_IMPLEMENTATION_BOARD**, which point to a resource pool.

When this argument is used along with **\$REVIEWERS**, and if a resource pool is assigned as a reviewer, email is sent to all the members of that resource pool.

When this argument is used along with **\$UNDECIDED**, and if a resource pool is assigned as a reviewer, and no signoff decision has been made for this resource pool assignment, all members of that resource pool are notified.

When this argument is used along with **\$RESPONSIBLE_PARTY**, and if a resource pool is assigned as responsible party, the email is sent to all members of that resource pool.

- **\$RESOURCE_POOL_NONE**

Identifies all members of the resource pool.

This argument has an effect only when it is used along with **\$REVIEWERS**, **\$UNDECIDED**, or **\$RESPONSIBLE_PARTY**.

When this argument is used along with **\$REVIEWERS** or **\$UNDECIDED**, and if a resource pool is assigned as a reviewer, email is not sent to members or subscribers of the resource pool.

When this argument is used along with **\$RESPONSIBLE_PARTY**, and if a resource pool is assigned as responsible party, the email is not sent to members or subscribers of resource pool.

- **\$RESOURCE_POOL_SUBSCRIBED**

Identifies the users who have subscribed to resource pool.

This argument has an effect only when it is used along with **\$REVIEWERS**, **\$UNDECIDED**, or **\$RESPONSIBLE_PARTY**.

When this argument is used along with **\$REVIEWERS**, and if a resource pool is assigned as a reviewer, the email is sent to users who have subscribed to the resource pool.

When this argument is used along with **\$UNDECIDED**, and if a resource pool is assigned as a reviewer and no signoff decision has been made for this resource pool assignment, email is sent to users who have subscribed to the resource pool.

When this argument is used along with **\$RESPONSIBLE_PARTY**, and if a resource pool is assigned as a responsible party, the email is sent to users who have subscribed to the resource pool.

Note

If the **\$RESOURCE_POOL_<XXXXX>** argument is not defined and the **\$REVIEWERS**, **\$UNDECIDED**, or **\$RESPONSIBLE_PARTY** arguments are used for a case where assignments are made to resource pools, the email is sent using the **EPM_resource_pool_recipients** preference.

EPM_resource_pool_recipients can take one of the following values:

- **all**

Sends mail to all members of resource pool.

- **none**

Does not send a mail to members or subscribers of resource pool.

- **subscribed**

Sends mail to Teamcenter users who have subscribed to resource pool.

If the **\$RESOURCE_POOL_<XXXXX>** argument is defined, the argument takes precedence over preference value.

If this argument is not defined and the **EPM_resource_pool_recipients** preference is not set, **subscribed** is considered the default value.

-subject

(Optional) Displays the string identified by this argument in the subject line of the OS email. The **-subject** argument supplies value options, such as "**-subject=\$TARGET**." Variants of the **-subject** argument values allow for a prefix or suffix string to the target name.

Note

If the “**-subject\$TARGET**” produces zero targets then the default subject line is used.

When no subject argument is provided, the default subject line for OS email is **"Process_name (Task_name)" is being <upcoming state>**.

- **\$STATE**

Appends action to the notification.

- **\$PROCESS**

Appends process name to the notification.

- **\$TASK**

Appends task name to the notification.

-comment

(Optional) Embeds user-defined comments in the body of the email.

-url

(Optional) Insert links to the workflow process into the notification email, based on values for **-url**. If no value is specified for **-url**, the links are added into the notification email.

If the **-url** argument is not defined, the notification email contains links depending on the values set in the **EPM_notify_url_format** preference.

If the **-url** argument is not defined and the **EPM_notify_url_format** preference is not set in the preference file, the links are added to the notification email by default.

The **-url** argument accepts multiple comma-separated values separated. For example, enter **rich, activeworkspace**.

This argument and the **EPM_notify_url_format** preference can take the following values:

- **rich**

Inserts a rich client link to the workflow process into the notification email.

Note

Rich client URL functionality must be enabled for links to rich client workflow processes to launch the rich client.

- **activeworkspace**

Inserts a link to the workflow process into the notification email.

Note

One of the two following preferences must be defined:

- **ActiveWorkspaceHosting.URL**
- **ActiveWorkspaceHosting.WorkflowEmail.URL**

- **none**

No links are inserted into the notification email.

-attachment

(Optional) Adds an attachment to Teamcenter mail and adds table(s) containing information on the specified attachments to the **OS** mail. Accept a comma separated or single value from the following options.

Warning

Hide target names from users without read access rights by using the **-url** argument.

- **target**

The workflow target attachments are included in the email.

- **process**

The workflow process is included in the email.

- **reference**

The task attachments reference list is included in the email.

Placement

There are no specific restrictions on placement for this handler except the following:

- When **\$REVIEWERS** or **\$UNDECIDED** is used as the keyword, place on the **Start** or **Complete** action of the **perform-signoffs** task.
- When **\$RESPONSIBLE_PARTY** is used as the keyword, place on the **Start** action of the task, not the **Assign** action.

Restrictions

None.

Examples

- This example sends an email with the subject **Lower Right Subassembly Review** to all users on the **design** and **qualityControl** address lists. The comment described in the example appears in the body of the email text. In addition to the email, the recipients also receive a Teamcenter mail that contains both the workflow target attachments and the current workflow process.

Argument	Values
-subject	Lower Right Subassembly Review
-recipient	addresslist:design, addresslist:qualityControl
-comment	Please review new subassembly and report any concerns directly to the Product Manager
-attachment	target, process

- This example sends an email and Teamcenter mail to the designated responsible party for the task. If the responsible party is a resource pool, no email is sent.

Argument	Values
-recipient	\$RESPONSIBLE_PARTY, \$RESOURCE_POOL_NONE

- This example designates OS users **peters** and **john**, user **Smith**, members of the group **manufacturing**, and members of the address list **purchasing** as recipients of an email with the subject **Manufacturing Release Procedure Completed**.

Argument	Values
-subject	Manufacturing Release Procedure Completed
-recipient	OS:peters, OS:john, User:smith, Group:manufacturing, Role:manager, addresslist:purchasing

- This example designates OS users **peters** and **john**, user **Smith**, all members of the group **manufacturing**, and members of the **CHANGE_REVIEW_BOARD** of the first change target object as recipients of an email with the subject **Manufacturing Release Procedure Completed**.

Argument	Values
-subject	Manufacturing Release Procedure Completed
-recipient	OS:peters, OS:john, User:smith, allmembers:manufacturing::, \$CHANGE_REVIEW_BOARD

- This example designates the recipient **PROCESS_OWNER** of an email with the subject **"Process Notification for Design_item"** when **Design_item** is the first target object of the workflow process.

Argument	Values
-subject	Process Notification for \$TARGET
-recipient	\$PROCESS_OWNER

- This example appends arguments to the subject line to include the **\$STATE**, **\$PROCESS**, and **\$TASK** arguments. This example provides a notification with a subject line showing the process with the task and the current state of the task.

Argument	Values
-subject	Performing signoffs notification for \$PROCESS: (\$TASK), \$STATE
-recipient	\$PROCESS_OWNER

EPM-notify-report

Description

Sends a report through the operating system (OS) email to all task reviewers. **EPM-notify-report** does not notify users through Teamcenter email. If you want to send the report using Teamcenter email, use the **EPM-notify** handler.

The **-report** argument differentiates **EPM-notify-report** handler from the **EPM-notify** handler. In notification email, the **-report** argument appends a report describing the signoff data associated with the **perform-signoffs** task. **EPM-notify-report** is designated for use on the **perform-signoffs** task. The **EPM-notify** handler is used on any type of task.

Note

- Use the **Mail_OS_from_address** preference to specify the **From** address displayed in the notification email. The preference value must be a valid email address.
- When placed on the **Start** action of **perform-signoffs** task, the **EPM-notify** or **EPM-notify-report** handlers are automatically re-executed when a signoff is delegated.

Syntax

EPM-notify-report

```
-report={review|rejection|progress|level}
[-recipient=
{OS:<user-name>| user:<user>| person:<person>| addresslist:<value>

| resourcepool:<group::role>

| allmembers:<group::role>

| $USER | $REVIEWERS | $PROPOSED_REVIEWERS

| $RESPONSIBLE_PARTY| $PROPOSED_RESPONSIBLE_PARTY

| $PROCESS_OWNER | $TARGET_OWNER [<type>]

| $UNDECIDED | $RESOURCE_POOL_ALL

| $RESOURCE_POOL_NONE | $RESOURCE_POOL_SUBSCRIBED

| $PROJECT_ADMINISTRATOR | $PROJECT_MEMBER<[group>::<role]>

| $PROJECT_TEAM_ADMINISTRATOR

| $PROJECT_AUTHOR}
| $REQUESTOR | $ANALYST
```


| \$CHANGE_SPECIALIST1 | \$CHANGE_SPECIALIST2 | \$CHANGE_SPECIALIST3

| \$CHANGE_REVIEW_BOARD | \$CHANGE_IMPLEMENTATION_BOARD}]

[-subject=

<string | \$TARGET | string \$TARGET string | >

<string | \$STATE | string \$STATE string | >

<\$PROCESS | string \$PROCESS string | >

<\$TASK | string \$TASK string | >]

[-comment=<string>]

[-url={rich | activeworkspace|none}]

[-attachment= {target | process | reference }]

Arguments

-report

(Required) Indicates the report type sent to recipients. Accepts one of these values:

- **review**

Notifies all recipients when they must review target objects. The report lists target and reference object IDs and types.

- **rejection**

Notifies recipients that the **Review** task has been rejected. The report lists target and reference object IDs, as well as the **Review** task reviewers, decisions, dates, and comments for each **Review** task. Do not use this value unless you want the workflow process to always send a rejection notice.

- **progress**

Notifies recipients of the current state of the workflow process. The report lists the target and reference object names, object IDs (if applicable for the object), as well as the **Review** task reviewers, decisions, dates, and comments for each **Review** task.

- **level**

Notifies recipients when the **Review** task completes. The report lists the target and reference object IDs, as well as the current **Review** task reviewers, decisions, dates, and comments.

-recipient

(Optional) Specifies the task reviewers to receive notification. Any surrogates for the specified users are also notified. Accepts one of these values:

- **OS:<user-name>**

Sends a notification to the OS user name specified.

<user-name> is a single valid OS user name.

- **user:<user>**

Sends notification to the user specified.

`<user>` is a single valid Teamcenter user ID.

- **person:**`<person>`

Sends a notification to user whose name is specified.

`<person>` is a single valid Teamcenter person.

Note

If the person's name includes a comma, you must include an escape character (\) to add the correct person. For example, to use **wayne, joan**:

-recipient=person:wayne\, joan

- **addresslist:**`<list>`

Adds all members of the address list specified to the signoff member list. Sends notification to all members of a group/role combination.

`<list>` is a valid Teamcenter address list.

- **resourcepool:**`<group::role>`

Sends notification to members of a group/role combination. Notification is sent to all members, subscribed members, or none based on the **EPM_resource_pool_recipients** preference.

The preference value can be overridden with:

- **\$RESOURCE_POOL_ALL**
- **\$RESOURCE_POOL_SUBSCRIBED**
- **\$RESOURCE_POOL_NONE**

You can define role in groups in the form of `<group>::`, `<group::role>`, or `<role>`.

Accepts valid Teamcenter resource pool names and these keywords:

- **\$GROUP**

The current user's current group.

- **\$ROLE**

The current user's current role.

- **\$TARGET_GROUP** [`<type>`]

The owning group of the first target object of the specified type. The `<type>` value is optional. If not specified, the first target is used.

- **\$PROCESS_GROUP**

The owning group of the workflow process.

- **allmembers:**`<group::role>`

Sends notification to all members of a group/role combination.

You can define role in groups in the form of `<group>::`, `<group::role>`, or `<role>`.

Accepts valid Teamcenter group and role names and these keywords:

- **\$GROUP**

The current user's current group.

- **\$ROLE**

The current user's current role.

- **\$TARGET_GROUP** [`<type>`]

The owning group of the first target object of the specified type. The `<type>` value is optional. If not specified, the first target is used.

- **\$PROCESS_GROUP**

The owning group of the workflow process.

Note

The **\$ROLE_IN_GROUP** keyword (formerly **\$ROLEINGROUP**) cannot be used. Use **allmembers:\$GROUP::\$ROLE** instead.

- **\$USER**

Send notification to the current user.

- **\$REVIEWERS**

Builds a list of all users who are reviewers in the same task level as the current reviewer, and sends email to them all.

- **\$PROPOSED_REVIEWERS**

Builds a list of all users who are reviewers in the same task level as the current reviewer, and sends notification to all of them.

- **\$RESPONSIBLE_PARTY**

Sends the notification to the designated responsible party for the task.

- **\$PROPOSED_RESPONSIBLE_PARTY**

Sends the notification to the designated responsible party for the task.

- **\$PROCESS_OWNER**

Sends notification to the workflow process owner.

- **\$TARGET_OWNER** [`<type>`]

Adds the owner of the first target of specified type to the signoff member list. The `<type>` value is optional. If not specified, the first target is used.

- **\$UNDECIDED**

Sends notification to the users who have not set the decision for the task.

- **\$RESOURCE_POOL_ALL**

Identifies all members of the resource pool.

This argument has an affect only when it is used along with **resourcepool**, **\$REVIEWERS**, **\$PROPOSED_REVIEWERS**, **\$UNDECIDED**, **\$RESPONSIBLE_PARTY**, **\$PROPOSED_RESPONSIBLE_PARTY**, or any Dynamic participant, such as **\$CHANGE_IMPLEMENTATION_BOARD**, which point to a resource pool.

If custom participants are defined by the customer, those participants can be used as recipients.

When this argument is used along with **resourcepool**>, email is sent to all the members of the resource pool.

When this argument is used along with **\$REVIEWERS** or **\$PROPOSED_REVIEWERS**, and if a resource pool is assigned as a reviewer, email is sent to all the members of that resource pool.

When this argument is used with **\$UNDECIDED**, and if a resource pool is assigned as a reviewer, and no signoff decision has been made for this resource pool assignment, all members of that resource pool are notified.

When this argument is used along with **\$RESPONSIBLE_PARTY** or **\$PROPOSED_RESPONSIBLE_PARTY**, and if a resource pool is assigned as responsible party, email is sent to all members of that resource pool.

- **\$RESOURCE_POOL_NONE**

This argument has an effect only when it is used along with **resourcepool**, **\$REVIEWERS**, **\$PROPOSED_REVIEWERS**, **\$UNDECIDED**, **\$RESPONSIBLE_PARTY**, or **\$PROPOSED_RESPONSIBLE_PARTY**.

When this is used along with **resourcepool**, email is not sent to members of the resource pool. (This combination is allowed, but of no value.)

When this argument is used along with **\$REVIEWERS**, **\$PROPOSED_REVIEWERS**, or **\$UNDECIDED**, and if a resource pool is assigned as a reviewer, email is not sent to members or subscribers of the resource pool.

When this argument is used along with **\$RESPONSIBLE_PARTY** or **\$PROPOSED_RESPONSIBLE_PARTY**, and if a resource pool is assigned as a responsible party, email is not sent to members or subscribers of resource pool.

- **\$RESOURCE_POOL_SUBSCRIBED**

Identifies the users who have subscribed to resource pool.

This argument has an effect only when it is used along with **resourcepool**, **\$REVIEWERS**, **\$PROPOSED_REVIEWERS**, **\$UNDECIDED**, **\$RESPONSIBLE_PARTY**, or **\$PROPOSED_RESPONSIBLE_PARTY**.

When this is used along with **resourcepool**, email is sent to users who are subscribed to the resource pool.

When this argument is used with **\$REVIEWERS** or **\$PROPOSED_REVIEWERS**, and if a resource pool is assigned as a reviewer, email is sent to users who are subscribed to the resource pool.

When this argument is used with **\$UNDECIDED**, and if a resource pool is assigned as a reviewer and no signoff decision has been made for this resource pool assignment, email is sent to users who subscribed to the resource pool.

When this argument is used with **\$RESPONSIBLE_PARTY** or **\$PROPOSED_RESPONSIBLE_PARTY**, and if a resource pool is assigned as a responsible party, email is sent to users who subscribed to the resource pool.

- **\$PROJECT_ADMINISTRATOR**
\$PROJECT_MEMBER
\$PROJECT_TEAM_ADMINISTRATOR
\$PROJECT_AUTHOR

Dynamically evaluates project team members belonging to the role specified in the argument value and sends notification to them. The project team is determined by the project team associated with the target object.

If the **\$PROJECT_MEMBER[<group>::<role>]** argument is specified, only the project members of the qualifying projects which belong to the specified group and role are selected as recipients. If the group and role are not specified, all the project members from qualifying projects are selected.

- **\$REQUESTOR**
\$ANALYST
\$CHANGE_SPECIALIST1
\$CHANGE_SPECIALIST2
\$CHANGE_SPECIALIST3
\$CHANGE_REVIEW_BOARD
\$CHANGE_IMPLEMENTATION_BOARD

Dynamically resolves to the user or resource pool associated with the first change target object in the process. The particular user or resource pool is determined by the role specified in the argument value.

Note

Change-related keywords apply only to change objects. If the process does not contain a change object as a target, the argument resolves to null.

Change Manager does not need to be enabled before these keywords take effect, but during installation, **Change Management** must be selected.

Note

If the **\$RESOURCE_POOL_<XXXXX>** argument is not defined and the **\$REVIEWERS**, **\$UNDECIDED**, or **\$RESPONSIBLE_PARTY** arguments are used for a case where assignments are made to resource pools, the email is sent using the **EPM_resource_pool_recipients** preference.

The **EPM_resource_pool_recipients** preference can have one of the following values:

- **all**
Sends email to all members of resource pool.
- **none**

Does not send an email to members or subscribers of resource pool.

- **subscribed**

Sends email to Teamcenter users who have subscribed to resource pool.

If the **\$RESOURCE_POOL_<XXXXX>** argument is defined, the argument takes precedence over the preference value. If this argument is not defined and the **EPM_resource_pool_recipients** preference is not set, then **subscribed** is the default preference.

The **-recipient** argument can have multiple values by using a delimiter specified by the **EPM_ARG_target_user_group_list_separator** preference. The default value for this preference is a comma.

-subject

(Optional) Displays the string identified by this argument in the subject line of the OS email. The **-subject** argument supplies value options, such as **"-subject=\$TARGET."** Variants of the **-subject** argument values allow for a prefix or suffix string to the target name.

Note

If the **"-subject\$TARGET"** produces zero targets then the default subject line is used.

Reports are formatted by type and e-mailed with a default subject line.

- The progress report (report=progress) default subject line is: **Review of "Process_name (Task_name)" is in progress.**
- The level report (report=level) default subject line is: **"Process_name (Task_name)" is being <upcoming state>.**
- The rejection report (report=rejection) default subject line is: **"Process_name (Task_name)" is in rejected.**
- **\$STATE**
Appends action to the notification.
- **\$PROCESS**
Appends process name to the notification.
- **\$TASK**
Appends task name to the notification.

-comment

(Optional) Inserts the specified string in the body of the email.

-url

(Optional) Inserts a link to the workflow process into the notification email, based on the value for **-url**. If no value is specified for **-url**, the links are added into the notification email.

If the **-url** argument is not defined, the notification email contains links depending on the values set in the **EPM_notify_url_format** preference.

If the **-url** argument is not defined and the **EPM_notify_url_format** preference is not set in the preference file, the links are added to the notification email by default.

The **-url** argument accepts multiple comma-separated values separated. For example, enter **rich, activeworkspace**.

This argument and the **EPM_notify_url_format** preference can take the following values:

- **rich**

Inserts a rich client link to the workflow process into the notification email.

Note

Rich client URL functionality must be enabled for links to rich client workflow processes to launch the rich client.

- **activeworkspace**

Inserts a link to the workflow process into the notification email.

Note

One of the two following preferences must be defined:

- **ActiveWorkspaceHosting.URL**
- **ActiveWorkspaceHosting.WorkflowEmail.URL**

- **none**

No links are inserted into the notification email.

-attachment

(Optional) Adds an attachment and adds table(s) containing information on the specified attachments to the **OS** email.

Accept a comma separated or single value from the following options.

Warning

Hide target names from users without read access rights by using the **-url** argument.

- **target**

The workflow target attachments are included in the email.

- **process**

The workflow process is included in the email.

- **reference**

The task attachments reference list is included in the email.

Placement

review

Place on the **Start** action of the **perform-signoffs** task when using this argument.

rejection

Place on the **Perform** or **Undo** actions of the **perform-signoffs** task when using this argument.

When placed on a **Perform** action, an email is sent on a **Reject** action.

Only place on an **Undo** action when the next task is a **Review** task, and the design of the workflow process requires that the task is demoted on a **Reject** action. This is achieved by placing the **EPM-demote-on-reject** handler on the **Perform** action of the **perform-signoffs** task. A **Reject** action causes a demotion to the previous task, which invokes the **Undo** action, and the **EPM-notify-report** handler sends out the required notification.

progress

The recommended placement when using this argument is attached to the **Start**, **Perform**, or **Complete** actions of a **perform-signoffs** task.

level

The recommended placement when using this argument is attached to the **Complete** action of a **perform-signoffs** task.

Restrictions

Use only on the **perform-signoffs** task.

Examples

- This example designates the user **smith**, members of the **manufacturing** group, the OS users **peters** and **john**, users with the **manager** role, members of the **VendorList** address list, and project members as recipients of a progress report with the subject **Manufacturing Release Process Completed**.

The **EPM-notify-report** handler should be placed on **Complete** action of **perform-signoffs** task.

Argument	Values
-report	progress
-subject	Manufacturing Release Process Completed
-recipient	user:smith, os:peters, os:john, allmembers:manufacturing, allmembers:::manager, addresslist:VendorList, \$PROJECT_MEMBER

- This example designates the workflow process owner as the recipient of a progress report with the subject **Manufacturing Release Process Completed**.

The **EPM-notify-report** handler should be placed on **Complete** action of **perform-signoffs** task.

Argument	Values
-report	progress
-subject	Manufacturing Release Process Completed
-recipient	\$PROCESS_OWNER

- This example builds a list of all users assigned as reviewers for the **perform-signoffs** task.

The **EPM-notify-report** handler should be placed on **Start** action of **perform-signoffs** task.

Argument	Values
-report	progress
-recipient	\$PROPOSED_REVIEWERS

- This example designates the task owner and task reviewers as recipients of a review report with the subject **TASK REVIEW NOTIFICATION**.

If any resource pool is assigned as a reviewer, then all users who have subscribed to that resource pool receive notification email.

Place the **EPM-notify-report** handler on the **Start** action of the **perform-signoffs** task.

Argument	Values
-report	review
-subject	TASK REVIEW NOTIFICATION
-comment	Please review the task
-recipient	\$PROCESS_OWNER, \$PROPOSED_REVIEWERS, \$RESOURCE_POOL_SUBSCRIBED

- This example illustrates creating a workflow process template with a **Review** task. Add the **EPM-notify-report** handler in the **Undo** action of the **perform-signoffs** task. Place an **EPM-demote-on-reject** handler on the **Perform** action of the **perform-signoffs** task.

The notification is sent to task owner, responsible party, and reviewers. If any resource pool is assigned as a responsible party and/or as a reviewer, then notification is sent to all group members of that resource pool.

Argument	Values
-report	rejection
-subject	TASK REJECTION & DEMOTE NOTIFICATION
-recipient	\$RESOURCE_POOL_ALL, \$PROCESS_OWNER, \$PROPOSED_RESPONSIBLE_PARTY, \$PROPOSED_REVIEWERS

- This example designates the **REQUESTOR** of the first change target object the recipient of a progress report with the subject **Manufacturing Release Process Completed**.

Place the **EPM-notify-report** handler on the **Complete** action of the **perform-signoffs** task.

Argument	Values
-report	Progress
-subject	Manufacturing Release Process Completed
-recipient	\$REQUESTOR

- This example builds a list of all users in the current task level where the handler has been placed and sends email to all of them.

Argument	Values
-report	Progress

- This example appends arguments to the subject line to include the **\$STATE**, **\$PROCESS**, and **\$TASK** arguments. This example provides a notification with a subject line showing the process with the task and the current state of the task.

Argument	Values
-subject	Performing signoffs notification for \$PROCESS: (\$TASK), \$STATE
-recipient	\$PROCESS_OWNER

EPM-notify-signoffs

Description

Informs users of a **Route** task's status through Teamcenter email and **OS** email. Any surrogates for the specified users are also notified. If the **-attachment** argument is included in the definition of the **EPM-notify-signoffs** handler, the recipients also receive program email. The recipients list is filled dynamically when running the **Review** task with the **Route** task. Links to the workflow process are added based on the value of the **EPM_notify_url_format** preference.

Note

Use the **Mail_OS_from_address** preference to specify the **From** address displayed in the notification email. The preference value must be a valid email address.

Syntax

EPM-notify-signoffs

```
[-subject=
<string | $TARGET | string $TARGET string | >

<string | $STATE | string $STATE string | >
<$PROCESS | string $PROCESS string |>< $TASK | string $TASK string |>]
[-comment=<string>]
[-url={rich | activeworkspace|none}]

[-attachment= {target | process | reference }]
[-log]
```

Arguments

-subject

(Optional) Displays the string identified by this argument in the subject line of the Teamcenter email and **OS** email. The **-subject** argument supplies value options, such as "**-subject=\$TARGET**." Variants of the **-subject** argument values allow for a prefix or suffix string to the target name.

Note

If the "**-subject\$TARGET**" produces zero targets then the default subject line is used.

When no subject argument is provided, the default subject line for **OS** email is **Review of "<Process_name (Parent Task_name)>" is in progress.**

- **\$STATE**

Appends action to the notification.

- **\$PROCESS**

Appends process name to the notification.

- **\$TASK**

Appends task name to the notification.

-comment

(Optional) User-defined comment that is embedded in the body of the email.

-url

(Optional) Inserts URLs into the notification email that links to the workflow process in either the rich client (**rich**), Active Workspace (**activeworkspace**), or all (no value). Rich client URL functionality must be enabled for links to rich client workflow processes to launch the rich client.

The **-url** argument accepts multiple comma-separated values separated. For example, enter **rich, activeworkspace**.

- If the argument is specified with no value, rich client and Active Workspace links are added to the notification email.
- If the argument is not specified, the notification email contains links depending on the value of the **EPM_notify_url_format** preference, which can be one or more of the following:
 - **rich**
 - **activeworkspace**

Note

One of the two following preferences must be defined:

- ◊ **ActiveWorkspaceHosting.URL**
- ◊ **ActiveWorkspaceHosting.WorkflowEmail.URL**

- **none**

No links are inserted into the notification email.

- If the argument is not specified and the **EPM_notify_url_format** preference is not set, rich client and Active Workspace links are added.

-attachment

(Optional) Adds an attachment to Teamcenter mail and adds table(s) containing information on the specified attachments to the **OS** email. Accept a comma separated or single value from following:

- **target**

Attaches the target to the program email.

- **process**

Attaches the workflow process to the program email.

- **reference**

The task attachments reference list is included in the email.

-log

(Optional) Records notification activity in the workflow audit file.

Placement

Place on the **Complete** action of the **Notify** task.

Restrictions

None.

EPM-remove-objects

Description

Removes the specified target or reference objects from the workflow process. This handler can use either a set of arguments to define which objects to remove or keep, or a list of values (LOV) to define a list of object types to remove.

The **-include_replica** argument keeps or removes the **Replica Proposed Targets** along with the targets specified by the **-keep_targets** or **-remove_targets** argument.

This handler can be used effectively with the **EPM-attach-related-objects** handler. For example, consider a task where users can manually add objects to any target revisions, such as new datasets through a specification relation. Users can also attach objects directly as targets to the workflow process. To ensure only allowable objects are attached as targets on approval, remove all objects except for the revisions using the **EPM-remove-objects** handler with the **-keep_targets=(ItemRevision)** argument. Then re-add the revision's attachments using the **EPM-attach-related-objects** handler.

Note

Enable debugging functionality for this handler with the **TC_HANDLERS_DEBUG** environment variable.

For more information about implementing this environment variable, see the *Teamcenter Environment Variables*.

Several arguments in this handler documentation show the following format for their values:

```
[[!(Class)![Type1][,(Class2)[,Type1[,...]]]] Type1[,Type2][,...]
```

The following explanation helps describe how to interpret this format. The explanation uses the **-remove_targets** argument in its examples, but the same value configurations shown in the examples can be applied to other arguments as well. Additional examples are shown in the **EXAMPLES** section.

The argument values in this handler indicate what objects are affected by configuring either an object class or an object type or a combination of both.

One approach is to specify a class. Surrounding a name in parentheses indicates that it is a class. For example, if the **-remove_targets** argument is configured with the Dataset class, then all Datasets regardless of their type would be removed from the targets list. The configuration for this example would look like this:

```
-remove_targets=(Dataset)
```

A type can also be specified. The absence of parentheses indicates that it is a type. For example, to remove only Datasets of type Text and leave all other Dataset types on the targets list, the **-remove_targets** argument would be configured like this:

-remove_targets=Text

It is also possible to indicate the type(s) that should not be affected. This is done by making use of the NOT (!) operator. When using the NOT operator, the syntax is to combine the class name and the type name separated by the NOT operator. For example, to remove all Dataset targets except those of type Text, the **-remove_targets** argument would be configured like this:

-remove_targets=(Dataset)!Text

Additional information can be found in [Differentiating between classes and types](#).

Syntax

```
EPM-remove-objects [{-remove_targets=<types> | -keep_targets=<types>}]
[{-remove_refs=<types> | -keep_refs=<types>}] | -lov=<lov-name>
[-include_replica]
```

Arguments

-remove_targets

(Optional) Defines the classes and/or types of target objects to remove from the workflow process.

Accepts a comma-separated list of classes and/or types in the format:

```
[(<Class>)[!<Type1>][,<Class2>][,<Type1>[,...]]] [<Type1>[,<Type2>][,...]]
```

For example, to specify datasets and forms:

(Dataset),(Form)

For an overview and examples of multilevel object paths in handlers, see [Defining multilevel object paths](#).

Note

The **-keep_targets** and **-remove_targets** arguments are mutually exclusive.

-keep_targets

(Optional) Defines the classes and/or types of target objects to be kept. All other target objects are removed from the workflow process.

Accepts a comma-separated list of classes and/or types in the format:

```
[(<Class>)[!<Type1>][,<Class2>][,<Type1>[,...]]] [<Type1>[,<Type2>][,...]]
```

For example, to specify datasets and forms:

(Dataset),(Form)

For an overview of using multilevel object paths in handlers, see [Defining multilevel object paths](#).

Note

The **-keep_targets** and **-remove_targets** arguments are mutually exclusive.

The **keep_targets** argument removes all targets of types that do not match the types specified by the **keep_targets** argument.

-remove_refs

(Optional) Defines the classes and/or types of reference objects to remove from the workflow process.

Accepts a comma-separated list of classes and/or types in the format:

[(<Class>)[!<Type1>][,<Class2>][,<Type1>[,...]]]] <Type1>[,<Type2>][,...]

For example, to specify datasets and forms:

(Dataset),(Form)

For an overview of using multilevel object paths in handlers, see [Defining multilevel object paths](#).

Note

The **-keep_refs** and **-remove_refs** arguments are mutually exclusive.

-keep_refs

(Optional) Defines the classes and/or types of reference objects to be kept in the workflow process.

Accepts a comma-separated list of classes and/or types in the format:

[(<Class>)[!<Type1>][,<Class2>][,<Type1>[,...]]]] <Type1>[,<Type2>][,...]

For example, to specify datasets and forms:

(Dataset),(Form)

For an overview of using multilevel object paths in handlers, see [Defining multilevel object paths](#).

Note

The **-keep_refs** and **-remove_refs** arguments are mutually exclusive.

The **keep_refs** argument removes all reference objects of types that do not match the types specified by the **keep_refs** argument.

-lov

Specifies a LOV to use to define which objects to remove. This argument is mutually exclusive of all other arguments.

For an overview of using LOVs in handlers, see [Using lists of values \(LOVs\) in handler arguments](#). See the LOV row, next, for required LOV format.

-include_replica

(Optional) Keeps or removes the **Replica Proposed Targets** as well as the target objects specified by the **-keep_targets** or **-remove_targets** argument.

LOV

{\$TARGET|\$REFERENCE}.<types>

{\$TARGET|\$REFERENCE}.<types>

...

{\$TARGET|\$REFERENCE}

(Optional) Specifies whether to remove targets, or to remove references.

Accepts a comma-separated list of classes and/or types in the format:

[(<Class>)[!<Type1>][,<Class2>][,<Type1>[,...]]][<Type1>[,<Type2>][,...]]

For example, to specify datasets and forms:

(Dataset),(Form)

For an overview of using multilevel object paths in handlers, see [Defining multilevel object paths](#).

Placement

Place on the **Start** or **Complete** action of any task.

To allow the removal of targets, ensure that the **EPM-disallow-removing-targets** handler is not placed on the root task of the respective workflow process template and the affected users have change access to the workflow target objects. You may use the **EPM-set-rule-based-protection** handler to ensure that the required change access to target objects is asserted. See the topic Executing workflow handlers for more information.

Restrictions

When using a LOV, you can only define objects to be removed. You cannot define objects to be kept.

Examples

- This example removes any folders or items attached as targets:

Argument	Values
-remove_targets	(Folder), (Item)

Alternatively, you can use these LOV settings:

Argument	Values
-lov	SYS_EPM_remove_folders_items

where the **SYS_EPM_remove_folders_items** LOV contains the data:

\$TARGET.(Folder),(Item)

- This example retains only item revisions, removing all other targets:

Argument	Values
-keep_targets	(ItemRevision)

- This example retains item revisions, document revisions, and PDF datasets, and removes all other targets:

Argument	Values
-keep_targets	(ItemRevision)(DocumentRevision),PDF

- This example removes MS ExcelX datasets and MS WordX datasets, and retains all other targets:

Argument	Values
-remove_targets	MSEcelX,MSWordX

- This example removes BOMView revisions and datasets other than UGPARTs, and retains all other targets:

Argument	Values
-remove_targets	BOMView Revision,(Dataset)!UGPART

EPM-request-PKI-authentication

Description

Displays a PKI authentication box in the **Perform** dialog box or panel of the task within which it has been placed. Users must type their PKI PIN in the box before the task can be completed.

Note

This handler requires an environment configured with PKI enabled Teamcenter client communication system (TCCS) security services to use the PKI serial number as **userid**, with the value of the **WRKFLW_PKI_user_validation_fieldname** preference set to **SERIALNUMBER**.

Syntax

EPM-request-PKI-authentication

Arguments

None.

Placement

Place either on the **Perform** action of the **perform-signoffs** task or the **Complete** action of the following tasks:

- **Do task**
- **Condition task**
- **select-signoff-team** task

On a **Route** task, place on the **Complete** action of the **select-signoff-team** subtask of the **Review** task.

Restrictions

None.

EPM-require-authentication

Description

Displays a password box in the **Perform** dialog box or panel of the task within which it has been placed. Users must type their logon password in the password box. The **password** and **username** are authenticated before the task can be completed.

You can also use this handler to require reauthentication for SSO.

Syntax

EPM-require-authentication

Arguments

None.

Placement

Place on the **Perform** action of the following tasks:

- **Do** task
- **perform-signoffs** task
- **Condition** task

When working with a **Route** task, place on the **Perform** action of the **perform-signoffs** subtask of either the **Review** or **Acknowledge** tasks.

Restrictions

- Place on the **Perform** action of these tasks.
- Do not use this handler when the user logs on with PKI authentication. Use the [EPM-request-PKI-authentication](#) handler to prompt for the PKI PIN.

EPM-run-external-command

Description

Runs external system commands. The external command can be sent a variety of information that includes configurable arguments, a configuration file, a list of data and a list of target and attachment details. If dataset details are required there is also an optional export feature to export specified files from the specified datasets to a specified export directory. All options are configured using a list of values (LOV), hence there is only one argument. Nearly all options can be specified in the LOV using specially formatted lines to extract object properties.

Note

Enable **debugging** functionality for this handler with the **TC_HANDLERS_DEBUG** environment variable.

Syntax

EPM-run-external-command

[-lov=<lov-name>]

[-auto_login]

Arguments

-lov

(Required) Specifies the List of Values (LOV) used to configure all options.

-auto_login

This argument is optional.

LOV

For an overview of using LOVs in handlers, see [Using lists of values \(LOVs\) in handler arguments](#).

<lov-name> can contain several lines in the following format:

```
<KEYWORD>~<OPTION>=<Value>
<KEYWORD>~<OPTION>=<%formatted string%>
<KEYWORD>~<%formatted string%>
```

- **KEYWORD**

Specifies a keyword to indicate the type of information to extract and send to the external command. Keywords are described below:

- **INPUT**

Specifies options to configure the handler.

INPUT~<OPTION>=<Value>

<OPTION> can contain any of the following values:

♦ **Target**

Indicates the main workflow process objects to extract data. The following example sets all item revision targets of the workflow process as the main objects:

```
INPUT~Target=$TARGET.(ItemRevision)
```

The following example uses references of the workflow process. These objects are the main objects that <%property%> fields relate to in <%formatted strings%>.

```
INPUT~Target=$REF.(ItemRevision)
```

♦ **Application**

Indicates the system application to run.

```
INPUT~Application=${TC_ROOT}\local\tools\run_ext_app
```

♦ **CallPerTarget**

Controls the application execution, once or per target found from **INPUT~Target**.

INPUT~CallPerTarget=YES | NO

YES calls the application separately for each target from **INPUT~Target**. This is the default behavior if this option is not provided. If one of the applications detects an error, processing terminates.

NO calls the application once and sends its data about all targets found from **INPUT~Target**.

♦ **ErrorMsg1**

Custom error message to be displayed to the user upon a fail code being returned from the external application. A return status of zero, (0), indicates the application terminated successfully; any other value indicates a failure.

In scripts, this is typically achieved using an exit command, for example, **exit 0** for success, **exit 1** for failure.

A <%formatted string%> can be used with this option, including the <\$SYSTEM_ERROR> variable to display the error code returned by the application. For example:

```
INPUT~ErrorMsg1=BOM checks failed on target
%object_string% with error %$SYSTEM_ERROR%
```

You can use this error message to reflect the type of application, or external checking, that was being performed. If not provided then a default, non-localized, message is returned.

◇ **ErrorMsg2**

Optional custom error message to be displayed to the user upon a fail code being returned from the external application. You can use this message to provide the user a help message, that is, where to look for more information on the problem. For example:

```
INPUT~ErrorMsg2=Please see your e-mail for details.
```

Note

Because error messages are displayed in reverse order, this message appears before **ErrorMsg1**.

◇ **ExportPath**

Defines a directory to export files in datasets. The presence of this option enables the export feature. If the option is not provided, then no files are exported. This option works with the **DATA~DATASETS[=<options>]** described below which creates a data file listing all required datasets. The <options> argument describe the relations, dataset types, and named references required. If **ExportPath** is also defined, then the files from the required name references are exported. For example:

```
INPUT~ExportPath=${TC_TMP_DIR}\WF\Exports
```

The handler does not remove any remaining files from the export path when the external application has terminated. It is the responsibility of the application to remove any remaining files from this directory. If any files being exported already exist in the export directory, then the export fails and the existing file is not overwritten. If this occurs, an error is written to the syslog but not displayed to the user and the handler continues.

◇ **ExportOrigFile**

Exports files with original file name. If this option is not defined, the handler exports files with the name stored in the volume. This option controls the name used for any exported files from datasets when **ExportPath** and **DATA~DATASETS** are defined. This option requires a **YES** value. For example:

```
INPUT~ExportOrigFile=YES
```

◇ **DataPath**

Defines a directory to write data files. This option defines where the configuration file, defined using the **CFG** keyword, and the data files, defined using the **DATA** keyword, are written. For example:

```
INPUT~DataPath=${TC_TMP_DIR}\WF\Data
```

◦ **CFG**

Specifies information to be written to an optional configuration file that can be passed to the external command as an argument. The format is:

CFG~<%formatted string%>

This file name can be extracted in a <%formatted strings%> using the **\$CONFIG_FILE** variable. For example:

```
CFG~JobTag=%$PROCESS.TAG%
CFG~JobName=%$PROCESS .object_name%
CFG~RevID=%$TARGET.item_revision_id%
CFG~ItemID=%$TARGET.item.item_id%
CFG~Project=%$TARGET.IMAN_master_form.project_id%
CFG~OwningUser=%$TARGET.owning_user%
CFG~OwningGroup=%$TARGET.owning_group%
```

The following example writes the following string:

```
JobTag=QmBJ0uKNh9KRfCAAAAAAAAAAAAAA
```

to the configuration file for **000001/A** the workflow process with the **000001/A** target revision owned by **tim** and **Designers** group:

```
JobName=000001/A RevID=A ItemID=000001 Project=Project X
OwningUser=Tim (tim) OwningGroup=Designers
```

- **ARG**

Specifies optional arguments to be sent to the external command. The format is:

ARG~<%formatted string%>

For example:

```
ARG~-cfg=%$CONFIG_FILE%
ARG~-files=%$DATASET_FILE%
ARG~-data=%$DATA_FILE%
```

- **DATA**

Specifies information to be extracted from targets, references, and their related objects. The possible formats are:

- ◊ **DATASETS**

DATA~DATASETS[=<options>]

writes a fixed format data file containing information about attached datasets that can optionally be exported with **INPUT ExportPath**.

This option is used to extract details about datasets attached to the objects specified by **INPUT~Target**. If **INPUT~ExportPath** is defined, then the required files are exported from the required datasets to the export path specified. The properties extracted from the datasets are written to a file with the name **process_tag_datasets.txt** in the current directory or in the directory

specified using **INPUT~DataPath**. This file name can be extracted in a *<%formatted strings%>* using **\$DATASET_FILE**.

Optional filters for relation types, dataset types, and reference types can be supplied. For each filter, an asterisk (*) can be supplied as a wild card to indicate any type. If dataset types are supplied and no reference types, then all references are listed in the data file. If no filters are supplied, then all datasets in all relations and all their references are listed. Any reference files that are exported have their absolute file path listed in the data file. This provides the ability for the external application to perform operations on these files. For example, running checks, printing, converting or to get information about **UGPART** references in **UGMASTER** and **UGPART** datasets in the **IMAN_specification** relation.

```
DATA~DATASETS=IMAN_specification~UGMASTER, UGPART~UGPART
```

The datasets data file is written in a fixed format as follows:

```
item_id~rev_id~relation type~dataset type~dataset
name~dataset_tag~reference type~file name
```

♦ LOV

For an overview of using LOVs in handlers, see [Using lists of values \(LOVs\) in handler arguments](#).

DATA~LOV=<lov-name>

writes a data file containing information about the targets, references and their related objects. A second LOV is used to define all the objects and properties to extract.

Specifies a separate LOV containing a list of alternating lines containing either:

```
OBJECT:multi-level.object.path
```

or

```
PROP:<%formatted string%>
```

The lines beginning with **OBJECT:** are used to find objects using multilevel object paths; lines beginning with **PROP:** specify the properties to extract from these objects and write out to the data file.

The first line in the LOV can be a **PROP:** line, for example, without a preceding **OBJECT:** line, in which case properties are extracted from the main objects found from **INPUT~Target**.

For example:

```
INPUT~<LOV=SYS_EXT_CMD_object_data>
```

where **LOV SYS_EXT_CMD_object_data** can contain:

```
PROP:%item.item_id%~%item_revision_id%~%object_name%~%object_type%
OBJECT:*.IMAN_reference
PROP:REF~%object_string%~%object_type%
OBJECT:*.IMAN_specification.
    UGMASTER, UGPART PROP:UG-HDR~Name~Material
    PROP:UG~%object_string%~%*.
    UGPART-ATTR.material%
```

This example begins by extracting properties from the main objects, then from reference objects attached to the main objects, and finally from the **UGMASTER** and **UGPART** datasets. Notice that there are two **PROP:** lines for the **UGMASTER** and **UGPART** datasets, the first line just has fixed text acting like a header line and the second defines the properties to extract (which includes the material attribute from the **UGPART-ATTR** named reference form).

In the **OBJECT:** lines, a type is required at the start of the multilevel object path to provide more flexibility. An asterisk indicates any type or an asterisk is automatically added within any *<%formatted string%>* for convenience when starting with a \$keyword such as \$TARGET, otherwise an asterisk, or type, is still required, as in the example for the ***.UGPART-ATTR.material** . The output from this example:

```
000001~A~000001~ItemRevision REF~000003/A~ItemRevision
    UG-HDR~Name~Material UG~UGMASTER-000001/A~Steel
```

- **OPTION**

Some keywords have options which can be defined.

- **Value**

You can use any text as a value. However, it is possible to extract values from environment variables within the text using the format:

```
<text>${ENV_VAR}<text>${ENV_VAR}<text>
```

- **%formatted string%**

A *<%formatted string%>* is a string containing alternating fixed text, and object properties defined within a pair of percent characters (%), similar to a batch file statement containing environment variables.

The format is:

```
<text>%<property>%<text>%<property>%<text>
```

where each *<property>* is defined within two percent characters (%) with fixed *<text>* between each property.

A *<property>* to extract relates to a previously defined object, to the workflow process targets or to the current workflow process, depending on the current context where the formatted string is being used

and some optional variables. The *<property>* can be specified as a single Teamcenter property, for an already specified object, or a multilevel object path and property to extract information from another object related to the already defined object target or workflow process.

If a multilevel object path is used within a property field and returns more than one object, then a comma-separated list of the values for the property from each object is given.

A special keyword tag can be used instead of a property name to extract a string representation of an object **PUID**.

- If the defined object is an item revision, then the following example extracts **ItemID/RevID**.

```
%item.item_id%/item_revision_id%
```

where **%item.item_id%** extracts the **item_id** from the revision's item. The **/** is the fixed text and **%item_revision_id%** extracts the revision's id.

- The following example writes the project ID from a target revision's master form as a line in the configuration file.

```
CFG~Project=%$TARGET.IMAN_master_form.project_id%
```

If the project is **Project X**, the configuration file contains the following line:

```
Project=Project X
```

This example uses the **\$TARGET** variable to specify which object the multilevel path starts.

Variables

Values from environment variables can also be extracted within a *<%formatted string%>* using the same format as described for *<Value>*. The *<\${ENV_VAR}>* does not have to be included within the pair of % characters.

There are also some internal variables which can be specified with some options. These are indicated with a \$ character, but without the curly brackets used for environment variables. Also, unlike the environment variables, these must be defined within a pair of percent % characters. For example:

```
ARG~-cfg_file=%$CONFIG_FILE%
```

This example specifies an argument to be sent to the external command. It specifies a *<%formatted string%>* of **cfg_file=%\$CONFIG_FILE%**, so the fixed text is **cfg_file=**, and **%\$CONFIG_FILE%** (between two % signs) extract the name of the configuration file generated by the handler. This option is explained in full detail below under the section for **ARG**, along with other variable.

The following handler variables are available:

\$TARGET

Specifies that a multi level object path should start searching for objects from the current target, as specified with **INPUT~Target=target.path**.

In the main LOV, this is taken as default and so does not have to be specified (except when using **DATA~LOV**), so

%%\$TARGET.item.item_id%

is the same as

%item.item_id%

\$PROCESS

Specifies that a multilevel object path should start searching for objects from the current workflow process.

For example:

%%\$PROCESS.object_name%

extracts the workflow process's name.

This option also provides a path to extract details about objects attached to the workflow process as targets or references.

For example:

%%\$PROCESS.\$REF.object_string%

returns a comma-separated list of the **object_string** property from all references attached to the workflow process, and:

%%\$PROCESS.\$TARGET.object_string%

returns a list of all targets.

\$USER

Can be used to extract information about the current logged in user.

Used on its own will give the full user format person (**user_id**).

Or a path can be used to get other user, person, or group information.

For example:

```
CFG~Person=%%$USER.person%
CFG~UserID=%%$USER.userid%
```

```
CFG~LoginGroup=%$USER.login_group%
CFG~Group=%$USER.group.name%
CFG~Email=%$USER.Person.PA9%
```

\$CONFIG_FILE

Gets the name of the configuration file generated by the handler. The format of the name is:

<DataPath>\<process_tag>_config.txt

or, if **CallPerTarget** is set to **YES**:

<DataPath>\<process_tag><_x>_config.txt

<x> is an incrementing number per target.

\$DATA_FILE

Gets the name of the data file generated by the handler for **DATA~LOV**. The format of the name is:

<DataPath>\<process_tag>_data.txt

or, if **CallPerTarget** is set to **YES**

<DataPath>\<process_tag><_x>_data.txt

Where <x> is an incrementing number per target.

\$DATASET_FILE

Gets the name of the datasets information file generated by the handler for **DATA~DATASETS**. The format of the name is:

<DataPath>\<process_tag>_datasets.txt

or, if **CallPerTarget** is set to **YES**

<DataPath>\<process_tag><_x>_datasets.txt

Where <x> is an incrementing number per target.

\$SYSTEM_ERROR

Gets the error code number returned by the external application. Can be used in the **ErrorMsg1** and **ErrorMsg2** error messages.

Placement

Requires no specific placement, however, do not place on the **Perform** action of the root task.

Restrictions

This handler does not extract data in PLM XML format. The format of the extracted data is defined completely in the LOV using percent (%) formatted strings, except for the file listing the export dataset, which is in a fixed format.

This handler does not have an import feature; however, dataset tags are written to the exported datasets data file and so could be used by a standalone ITK program to import files. Do not use this handler to run an external application that takes a long time to run. It may appear that Teamcenter is unresponsive. If the success or failure of the application is required for process control, it is necessary to wait for the application. In this case, ensure that the workings of the application is visible in a new window to show the user some feedback. Any files exported by the handler are not deleted by the handler after the external application finishes. It is the responsibility of the external application to clean up the export directory.

Examples

- Example 1

The following example calls an application, specified by an environment variable, to perform checks on CAD files. This application requires a configuration file to define various parameters. One of these is an e-mail address so that it can send the user a report. The name of the configuration file is sent to the application as an argument, as is the file name of the data file containing information about the exported dataset files.

Argument	Values
-lov	SYS_EPM_run_cad_checks

The **SYS_EPM_run_cad_checks** LOV contains the following data:

LOV usage

Value	Description
INPUT~Target=\$TARGET.(ItemRevision)	Specifies that the main objects from which data is to be extracted is the job targets which is of class ItemRevision . If multiple targets are found then the application will either be called separately for each target or once with all the data from all targets, depending on the setting CallPerTarget which is defined just below.
INPUT~ErrorMsg1=Cad checks errors (Error % \$SYSTEM_ERROR%)	Defines an error message which is displayed to the user if the application returns an error status.

LOV usage (continued)

Value	Description
INPUT~ErrorMsg2=Please see your e-mail for details	Defines an optional second error message which is displayed to the user as well as ErrorMsg1 .
INPUT~Application= \${CUST_CAD_CHECK_APPLICATION}	Defines the external application which is to be run. This application is defined by a system environment variable, which in this example is CUST_CAD_CHECK_APPLICATION .
INPUT~CallPerTarget=YES	Calls the application for each target.
INPUT~DataPath=C:\WF\Data	Sets a path for data files.
INPUT~ExportPath=C:\WF\Exports	Sets a path for exported dataset files
CFG~JobTag=%\$PROCESS.object_tag%	Writes the process tag (PUID) to the configuration file as JobTag=<Job Tag> .
CFG~JobName=%\$PROCESS .object_name%	Writes the workflow process name to the configuration file as JobName=<Job Name> .
CFG~RevID=%\$TARGET.item_revision_id%	Writes the target object revision ID to the configuration file as RevID=<RevID> .
CFG~ItemID=%\$TARGET.item.item_id%	Writes the target object item ID to the configuration file as ItemID=<ItemID> .
CFG~Project= % \$TARGET.IMAN_master_form.project_id%	Writes the target object Project ID, from the revision master form, to the configuration file as Project=<ProjectID> .
CFG~CadProc=\${CUST_CAD_CHECK_PROC}	Writes the environment variable value to the configuration file as CadProc=<cad_proc> .
CFG~OwningUser=%\$TARGET.owning_user%	Writes the target object owning user to the configuration file as OwningUser=<user> .
CFG~OwningGroup=%\$TARGET.owning_group%	Writes the target object owning group to the configuration file as OwningGroup=<group> .
CFG~Email=%\$USER.E_Mail%	Writes the current user's e-mail to the configuration file, where E_Mail is the label from the person form.
CFG~SMTPServer= \${CUST_RELEASE_SMTP_SERVER}	Writes the environment variable value to the configuration file.

LOV usage (continued)

Value	Description
CFG~FunctionsFile= \${CUST_RELEASE_FUNC_FILE}	Writes the environment variable value to the configuration file.
CFG~SysAdminEmail= \${CUST_RELEASE_SA_MAIL}	Writes the environment variable value to the configuration file.
CFG~AppsArray=Apps1	Writes the value AppsArray=Apps1 to the configuration file.
CFG~WarningDir= \${CUSTOMER_RELEASE_WARNING_DIR}	Writes the environment variable value to the configuration file.
CFG~UPG=\${UPG}	Writes the environment variable value to the configuration file.
CFG~Desc=%\$TARGET.object_desc%	Writes the target object description to the configuration file.
DATA~DATASETS= IMAN_specification~UGMASTER~UGPART	Extracts information about UGPART references in UGMASTER datasets attached to the target revision.
ARG~cfg=%\$CONFIG_FILE%	Sends the configuration file name as an argument.
ARG~files=%\$DATASET_FILE%	Sends the dataset data file name as an argument.

- Example 2

The following example shows the use of **DATA~LOV=lov-name** to extract various details.

Argument	Values
-lov	SYS_EPM_send_ecr_relation_data

when the **SYS_EPM_send_ecr_relation_data** LOV contains the following data:

DATA~LOV=lov-name

Value	Description
INPUT~Target=(ItemRevision)	Specifies that the main object from which data is to be extracted is the job target which is of the ItemRevision class.
INPUT~Application= \${CUST_ECR_EXT_APPLICATION}	Defines the external application that is run. This application is defined by a system environment variable.

DATA~LOV=lov-name (continued)

Value	Description
ARG~-item=%\$TARGET.item.item_id %	Sends the target object's item ID as an argument to the application.
ARG~-rev=%\$TARGET.item_revision_id %	Send the target object's revision ID as an argument to the application.
ARG~-dest=\${CUST_RELEASE_DEST}	Send the environment variable's value as an argument to the application.
ARG~-type=ECR	Sends the value as an argument to the application.
ARG~-data=%\$DATA_FILE%	Sends the name of the data file, to be produced by DATA~LOV , as an argument to the application.
DATA~LOV=<lov-name>	Specifies an LOV containing a list of alternating lines starting with OBJECT: , to specify an object, and then PROP: , to specify the properties to extract from the object to write out to a data file.
DATA~LOV=SYS_EPM_get_ecr_relation_data	This LOV extracts details from the affected item revisions attached to the Mini, Minor, and Major relations in an ECR revision target. The objects are specified using multiple level paths and start from the target objects. The property strings use the %formatting% notation.

- Output in the data file, if the target has two minor relations and one major relation:

```
item-00001~A~Mini
item-00002~B~Mini
item-00005~A~Major
```

LOV SYS_EPM_get_ecr_relation_data

Value	Description
PROP:%item.id%~ECR Started~%creation_date%~%owning_user% ~%IMAN_master_form.ecr_prty%	Extract properties from the target revision.
OBJECT:(ItemRevision).Mini.(ItemRevision)!Buy Revision!Customer Revision!RawMaterial Revision	From any ItemRevision targets, find any ItemRevision objects attached to the Mini relation, except for specific types, for example, Buy Revision.
PROP:%item.item_id%~%item_revision_id%~Mini	Extract properties from any Mini relation revisions.

LOV SYS_EPM_get_ecr_relation_data (continued)

Value	Description
OBJECT:(ItemRevision).Major.(ItemRevision)!Buy Revision!Customer Revision!RawMaterial Revision	From any ItemRevision targets, find any ItemRevision objects attached to the Major relation, except for specific types, for example, Buy Revision .
PROP:% item.item_id %~% item_revision_id %~Major	Extract properties from any Major relation revisions.
OBJECT:(ItemRevision).Minor.(ItemRevision)!Buy Revision!Customer Revision!RawMaterial Revision	From any ItemRevision targets, find any ItemRevision objects attached to the Minor relation, except for specific types, for example, Buy Revision .
PROP:% item.item_id %~% item_revision_id %~Minor	Extract properties from any Minor relation revisions.

EPM-set-condition

Description

Condition tasks have a result attribute that you can set to one of these values: **True**, **False**, or **Unset**. The initial setting of the **Condition** task is **Unset**, until it is either automatically or manually set to **True** or **False**. Successor tasks require the condition result to be set to a value other than its initial unset value.

This handler is used to set a **Condition** task result automatically, without user interaction. Using Business Modeler IDE conditions, the task can evaluate the condition criteria against target objects and user session information.

When queries are used for condition evaluation with this handler, one of the following queries is performed:

- Target query
Performed on workflow process attachments.
- Task query
Performed on the task to which this handler is added.
- Subprocesses query
Performed on the subprocesses that the **Condition** task depends on.

Use **All** | **Any** | **None** to determine whether all, any, or none of the target attachments or subprocesses must meet the query criteria to set the result to **True**; these values apply only to target and subprocess queries.

The **-include_replica** argument queries the **Replica Proposed Targets** along with the targets if the **-query_type** argument is **target**.

Syntax

EPM-set-condition

```
{-condition_name=<condition-name> | { -query=<query-name>
[-query_type=task | target | sub-process] [-log] }}
[-check_targets=all | any | none] [-log] [-reference][-skip_duplicate_references][-include_replica]
```

Arguments

-condition_name

(Optional) Defines the BMIDE condition to be evaluated against target objects. The condition signature accepts a **WorkspaceObject** and **UserSession** in that sequence. The BMIDE condition in the handler argument is evaluated against the target objects based on the value of the **check_targets** argument. The handler decides the true or false path based on the evaluation result of BMIDE condition.

Note

The **-condition_name** and **-query** arguments are mutually exclusive.

-query

(Optional) Defines the query to be run.

Note

The **-condition_name** and **-query** arguments are mutually exclusive.

-query_type

(Optional) Determines the type of query run.

- **task**

Performs a query on the task to which this handler is added.

- **target**

Performs a query on the workflow process attachments.

- **sub-process**

Performs a query on the subprocesses that the **Condition** task depends on.

-check_targets

(Optional) This argument determines the target objects against which to evaluate the BMIDE condition or query.

It determines whether **all**, **any**, or **none** of the target attachments or subprocesses must meet the query criteria to set the result to **True**. This argument applies only to **Target** and **Sub-Processes** queries for the **-query** argument.

When used in conjunction with **-condition_name** argument, the BMIDE condition is evaluated against targets to determine whether **all**, **any** or **none** of the targets meet the condition.

If this argument is not specified and used in conjunction with **-condition_name** argument, the value for this is considered as **all** by default.

-log

(Optional) If a **Condition** task fails, it creates a log file reporting which objects caused the task's query to fail. The header in the log file contains:

- Task name
- Query name
- Date/time stamp

The log file is saved as a dataset and added to the workflow process as a reference attachment. The dataset is stored in the task attachments references folder.

If the **Condition** task does not fail, no log file is created.

-reference

(Optional) Moves target objects not satisfying a **Condition** task's query criteria or BMIDE condition to the task attachments references list.

-skip_duplicate_references

When **-reference** is configured and the object already exists as a reference, then an attempt to add the object as a reference will be skipped. Without this option, an attempt to add an object as a reference when that object is already a reference will result in a duplicate-reference error.

-include_replica

(Optional) Queries the **Replica Proposed Targets** as well as the target objects if the **-query_type** is set to **target**.

Placement

- If the **-query_type** argument is set to **task** or **target**, place on the **Start** action.
- If the **-query_type** argument is set to **sub-process**, place on the **Complete** action.

Restrictions

Typically used for **Condition** tasks only. This handler can also be used with a custom task.

Note

This handler exists as part of the workflow conditional branching functionality. This handler is automatically added to a **Condition** task while creating the workflow process template in Workflow Designer by using the **Query** tab in the **Task Properties** dialog box. Siemens Digital Industries Software recommends that you use this method to configure a **Condition** task, rather than manually configuring and adding this handler to the task using the **Handler** dialog box.

No user interface support is provided to add this handler while using BMIDE conditions with the **-condition_name** argument. The handler must be added manually from the **Handler** dialog box.

Note

Workflow Designer provides a number of prepackaged task templates, such as the **Review** task, **Route** task, and **Acknowledge** task templates. Adding subtasks below any of these specific tasks for the purpose of implementing a branching condition is not recommended, as this may jeopardize the integrity of the task's structure, and doing so may result in unpredictable behavior.

Examples

- In this example, a query is performed on the workflow process attachments. If any of the workflow process attachments meet the criteria defined by the **CM II CN Type** query, the task result on the **Condition** task is set to **True**.

Argument	Values
-query	CM II CN Type
-query_type	target
-check_targets	any

- In this example, an **EPMTask** query, **BM - Has Multiple Targets**, uses the run-time property **num_targets** to count the workflow target objects. If the query result is more than one, the result on the **Condition** task is set to **True**.

Note

The **BM - Has Multiple Targets** query is created using the search class **EPMTask** and is not included in the Teamcenter install.

Argument	Values
-query	BM - Has Multiple Targets
-query_type	task

- In this example, the BMIDE **Fnd0DocRevSubTypes** condition is evaluated against all target attachments one-by-one. The condition evaluation returns **TRUE** if any of the target attachments is a subtype of **Document Revision**, and the workflow takes the **TRUE** path.

Argument	Values
-condition_name	Fnd0DocRevSubTypes
-check_targets	any

Note

The condition used in the handler example above:

```
Fnd0DocRevSubTypes (WorkspaceObject o ,
UserSession u) = ((o != null) AND
u.fnd0ConditionHelper.fnd0isSubTypeOf
(o, "DocumentRevision"))
```

EPM-set-duration

Description

Defines time dependence during process design. The handler is triggered when the task is started. The five handler arguments are the number of years, weeks, days, hours, and minutes of the duration. These arguments are used at execution time to initialize the tasks' duration value and generate the due date when the task is created. The addition of all five arguments determine the total duration time.

Due date calculations based on the duration setting in this handler consider the user's calendar and the value of the **Default_Base_Calendar_Preference** preference.

Syntax

EPM-set-duration -year=<year-value> -week=<week-value> -day=<day-value>
-hour=<hour-value> -minute=<minute-value>

Arguments

At least one or more arguments should be specified.

-year

(Optional) Defines the number of years of the duration.

-week

(Optional) Defines the number of weeks of the duration.

-day

(Optional) Defines the number of days of the duration.

-hour

(Optional) Defines the number of hours of the duration.

-minute

(Optional) Defines the number of minutes of the duration.

Placement

Place on the **Start** action.

Restrictions

Argument values are limited to positive integers. If the **Task Manager** daemon is not running then no late notifications are sent.

The **EPM-set-duration** handler, along with the following calendars and preferences, all work together, and are dependent on each other to define and control time parameters.

- The working time setting in the organization calendar.

- **SiteTimeZone**
- **Default_Base_Calendar_Preference**
- **Schedule Manager** preferences: **SM_Hours_Per_Day_Preference**, **SM_Hours_Per_Week_Preference**, and **SM_Hours_Per_Year_Preference**

Example

The end date is calculated as the sum of duration of the user input multiplied by the preference value.

To calculate time: Year (SM_Hours_Per_Year_Preference) + Week (SM_Hours_Per_Week_Preference) + Day (SM_Hours_Per_Day_Preference) + Hours + Minutes.

For example, the preference settings for a 24-hour duration calendar schedule are:

Year

SM_Hours_Per_Year_Preference=8760 (365 days x 24 hours)

Week

SM_Hours_Per_Week_Preference=168 (7 days x 24 hours)

Day

SM_Hours_Per_Day_Preference=24

Examples

- This example sets the task to be due 5 years, 4 weeks, 3 days, 2 hours, and 1 minute after it is started:

Argument	Values
-year	5
-week	4
-day	3
-hour	2
-minute	1

ERP-set-form-value-AH

Description

Sets a particular field to a given value for all forms of the given type attached as targets of the process, and saves the forms. Use this handler to set a value that depends on the workflow process being used to transfer the data to ERP (for example, for a preproduction transfer process, the BOM usage may be set to **1 = Engineering/Design** and for a production transfer process, it would be set to **2 = Production**).

Note

- This handler overwrites any existing value.
- The user performing the signoff must have write access to the forms whose value is being set.

Syntax

ERP-set-form-value-AH -form_type = <type_name>, -field_name=<field_name>, -field_value=<value>

Arguments

-form_type

Updates any forms of this type attached as targets.

-field_name

Specifies the name of the field to be set.

-field_value

Specifies the value to which to set the field.

Note

These values are all case sensitive. Update the values if the mapping schema changes (for example, new form types or attributes created). The **-field_value** argument should use the whole string defined for the LOV in the mapping file (for example, **1 = Engineering/Design**, **2 = Production**).

Placement

Place on the **Perform Signoff** task.

Restrictions

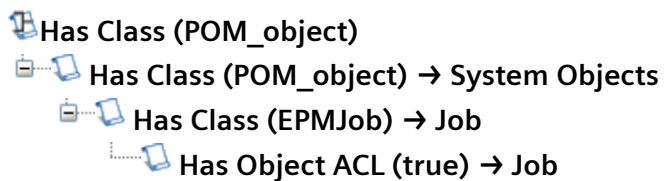
None.

EPM-set-job-protection

Description

Denies the **world:delete** and **world:write** process object protections, allowing an object ACL to be applied to an instance of an **EPMJob** object. This protection prevents the workflow process from being deleted when it completes.

To implement, add the **Has Object ACL (true)→Job** rule under **Has Class (EPMJob)→Job** in Access Manager. For example, the rules needed for this handler should look like the following (for clarity, the other rules are not shown).



Syntax

EPM-set-job-protection

Arguments

None.

Placement

Place on the **Complete** action of a task.

Restrictions

None.

EPM-set-owning-project-to-task

Description

This handler takes the owning project (or program) from the first target object of the workflow and sets it for all Workflow objects (for example, **EPMTask**, and **EPMJob**). The system can restrict access to workflow objects properly since the project is set at the workflow object level. The Access Manager rule tree is also modified to deny general access, but can grant access based on project teams for the workflow (**EPMTask**) objects by adding a new named ACL for tasks (**EPMTask**) in projects. Once the workflow processes are created with these changes, the users from the owning project team of the first target object can access the workflow tasks, whereas other users cannot access them. The process initiator, responsible parties, and reviewers of the workflow are required to be members of the owning project to proceed with the workflow tasks.

Syntax

EPM-set-owning-project-to-task

Arguments

None.

Placement

Place on the Start action of a root task.

Restrictions

Uses only the owning project of first target to set it on workflow objects. It does not consider other assigned projects or the owning project of other targets. If the owning project is not set on first target object, this handler fails to operate.

EPM-set-parent-result

Description

Sets the Boolean condition of its parent task. It is only used when complex compound subtasks are collectively needed to set the parent tasks. This allows for compound/complex combinations of **Condition** tasks.

Syntax

EPM-set-parent-result -value= true | false

Arguments

-value
Set to **true** or **false**.

Placement

Place on the **Start** or **Complete** action.

Restrictions

None.

Note

Placing this handler in a location other than the subtask of a **Condition** task may result in unpredictable behavior.

EPM-set-property

Description

Accepts a list of properties and a list of associated values, and uses those values to set the properties on the specified objects. The properties to be updated are listed in the **-property** argument, and the values are listed in the **-value** argument.

The **EPM-set-property** handler offers two variants. One differentiates between non-array and array properties, and the other distinguishes between values provided in a hard-coded manner versus those sourced from another object.

If you need to configure a value for an array-type property in a hard-coded manner, you can specify only one property name using the **-property** argument. The **-value** argument allows you to set one or more values on the array property.

The value types must be compatible with their associated property types. You can specify the values or obtain them from attachment objects or derived objects.

Note

- This handler overwrites the existing property values with the specified values. For example, in the case of array properties, all existing values are removed from the array, and only the new values are added to the property.
- Workflow handlers such as **EPM-set-property** cannot recognize run-time or compound properties. These handlers only set properties that have a persistent attribute on some object, and they cannot influence the setting of run-time or compound properties.

Property Configuration Combinations

When configuring properties, you can choose from four different combinations based on whether the property is an array and whether the method is hard-coded or not. Combinations can include:

- **Non-Array Property + Hard-Coded Method**

You can set multiple properties at once, each with its corresponding value. It is also possible to set a single property with a single value. Ensure all properties are of the non-array type.

- **Non-Array Property + Non Hard-Coded Method**

Similar to the hard-coded method, this allows setting multiple properties with their values simultaneously or a single property with a single value. All properties must be of the non-array type.

Array Property + Hard-Coded Method

This combination accepts only one property name but allows setting one or more values on that array property.

- **Array Property + Non Hard-Coded Method**

You can set multiple properties at once, with values that reference their source object property. Single property and single value combinations are also supported. All properties must be of the array type.

Caution

Avoid configuring a combination of array and non-array properties, as this is unsupported and may result in unexpected behavior.

Syntax

```
EPM-set-property -property=<list-of-properties> -value=[
<comma-separated-value-list> ] [[-to_attach=<attachment-type> ] [-to_relation=<relation-type>]]
| -to_lov=<lov-name>]] [[[-from_attach=<attachment-type> ] [-from_relation=<relation-type>]]
| -from_lov=<lov-name>]] [-include_type=<comma-separated-type-list> | -exclude_type=<comma-
separated-type-list>] -bypass[-target_task]=<multilevel-task-path>
```

Arguments

-property

Specifies one or more properties to be updated on the specified objects. As mentioned previously, there are two variants with this handler: non-array and array, either provided in a hard-coded way or non-hard coded. In the case of an array property, while setting/resetting a value in a hard-coded manner, only one property can be specified.

Arguments with a **to_** prefix are used to determine the objects to be updated. There should be a one-to-one correspondence between the properties indicated on the **-property** argument and the values indicated on the **-value** argument. The value types should be compatible with the property types. If a property listed on the **-property** argument does not exist for a specified update object, the update for the property is skipped.

Separate multiple properties with commas or the character specified by the **EPM_ARG_target_user_group_list_separator** preference.

-value

Specifies zero or more values to be used to set the associated properties in the **-property** list. You can specify the values, or they may be configured as a property name with a preceding **PROP::** qualifier. If a property name appears on the list, the value is read from an attachment object or a derived object. Arguments with a **from_** prefix are used to identify attachment objects and derived objects. Property types updated using specified values can be integer, Boolean, string, or date types (the date type supports the **\$CURRENT_DATE** keyword, which dynamically obtains the current date). Other property types, such as a tag or tag list, can be updated only if the updating value is obtained from a compatible property type on an attachment object or a derived object.

To retain approvals for a task during rework, specify the integer value representing the decision to be retained. For example, use 89 as the integer value to indicate the decision.

To reset a property value, set an empty value in the handler for the property.

For more information about using empty values, see the *Examples* section.

Acceptable date values are:

- A date in the following format: <yyyy>-<mm>-dd.
- **\$CURRENT_DATE** keyword, which sets the property value to the current date at the time that the handler is run.

Separate multiple values with commas or the character specified by the **EPM_ARG_target_user_group_list_separator** preference.

-to_attach

(Optional) When used by itself, this argument specifies the attachment type objects to be updated. When used in conjunction with the **-to_relation** argument, this argument specifies the attachment type objects to be used as a starting point when locating derived objects to be updated; only the derived objects are updated.

Value	-to_att_type is used by itself	-to_att_type is used with -to_relation
TARGET	Updates target attachments.	Uses target attachments as a starting point when searching for derived objects. Updates only the derived objects.
REFERENCE	Updates reference attachments.	Uses reference attachments as a starting point when searching for derived objects. Updates only the derived objects.
BOTH	Updates both target and reference attachments.	Uses both target attachments and reference attachments as a starting point when searching for derived objects. Updates only the derived objects.
TASK	If there is not a valid target_task argument, the task on which the handler is configured will be used to set the property value.	The task pointed by this target_task will be updated.

Note

Lower case values are also valid.

The **Task** value provided in **-target_task** is optional, and the value is the full path of the task.

To update properties on both attachment objects and derived objects, you must configure two instances of the **EPM-set-property** handler. Configure one instance to update attachments, and configure a second instance to update derived objects.

If a handler instance is configured to update attachment objects and multiple attachment objects exist, all attachment objects are updated. If a handler instance is configured to update derived objects and the handler locates multiple objects, all objects found for all specified attachment objects are updated.

-to_relation

(Optional) Updates objects with the specified relation to the identified attachment type objects.

- For manifestations, use **IMAN_manifestation**.
- For specifications, use **IMAN_specification**.
- For requirements, use **IMAN_requirement**.
- For references, use **IMAN_reference**.
- For BOM views, use **PSBOMViewRevision**.

This argument must be used with the **-to_attach** argument, which identifies attachment types.

-to_attach value	-to_relation behavior
TARGET	Updates objects with the specified relation to the target attachments.
REFERENCE	Updates objects with the specified relation to the reference attachments.
BOTH	Updates objects with the specified relation to both the target and reference attachments.

-to_lov

(Optional) Specifies an LOV to define which objects are to be updated.

For an overview of using LOVs in handlers, see [Using lists of values \(LOVs\) in handler arguments](#).

-from_attach

(Optional) When used by itself, this argument specifies the attachment object used to obtain property values. These values are used to perform updates on the specified update objects (identified by the **-to_attach** and optionally the **-to_relation** arguments). When used in conjunction with the **-from_relation** argument, this argument specifies the attachment objects to be used as a starting point when locating derived objects (the **-from_relation** argument specifies the relationship used to identify derived objects). Property values are obtained from the derived object properties. Only a single object is used to obtain property values. If more than one object is identified, only the first object found is used.

Value	-from_attach is used by itself	-from_attach is used with -from_relation
TARGET	Reads property values from the first target attachment object.	Locates the first object with the specified relation to a target attachment object and reads property values from the related object.
REFERENCE	Reads property values from the first reference attachment object.	Locates the first object with the specified relation to a reference attachment object and reads property values from the related object.

Value	-from_attach is used by itself	-from_attach is used with -from_relation
BOTH	Reads property values from the first target attachment object. If target attachments do not exist, then reads property values from the first reference attachment object if reference attachments exist.	Locates the first object with the specified relation to a target attachment object and reads property values from the related object. If target attachments do not exist or if no object with the specified relation is found, it locates the first object with the specified relation to a reference attachment object and reads property values from the related object.
TASK	Reads property values from a workflow task. If -source_task is not specified, the handler reads from the task on which it is configured.	Reads property values from the task pointed by source_task .

Note

Lower case values are also valid.

The **Task** value provided in **-source_task** is optional, and the value is the full path of the task.

To update properties on both attachment objects and derived objects, you must configure two instances of the **EPM-set-property** handler. Configure one instance to update attachments, and configure a second instance to update derived objects.

If a handler instance is configured to update attachment objects and multiple attachment objects exist, all attachment objects are updated. If a handler instance is configured to update derived objects and the handler locates multiple objects, all objects found for all specified attachment objects are updated.

-source_task

(Optional) The full path to the task from which the property value will be read.

Note the following:

- If **-source_task** is specified, then **-from_attach=TASK** must be specified.
- If **-source_task** is not specified, the property value is read from the task where the handler is located.

-from_relation

(Optional) Specifies the relation used to locate a derived object. The identified derived object is used to obtain property values, which are then used to perform the update.

- For manifestations, use **IMAN_manifestation**.
- For specifications, use **IMAN_specification**.
- For requirements, use **IMAN_requirement**.
- For references, use **IMAN_reference**.

- For BOM views, use **PSBOMViewRevision**.

This argument must be used with the **-from_attach** argument. A derived object is identified by starting with objects of the specified attachment type indicated by the **-from_attach** argument and then locating the first secondary object with the specified relation indicated by the **-relation** argument.

-from_lov

(Optional) Specifies an LOV to obtain an object. Values are read from this object and used to set the properties on the **-property** list.

For an overview of using LOVs in handlers, see [Using lists of values \(LOVs\) in handler arguments](#).

-include_type

(Optional) Updates specified objects only if their type matches one of the types on the list. Do not use this argument with the **-exclude_type** argument.

-exclude_type

(Optional) Updates all specified objects unless their type is one of the types that appears on the **-exclude_type** list. Do not use this argument with the **-include_type** argument.

-bypass

(Optional) Specifies that the user has bypass privileges and allows the property to be set.

-target_task

(Optional) Specifies the multilevel task path identifying the task on which the properties are to be set. The path is from the root task to the subtask with the path levels separated with colons (:).

Note the following:

- If **target_task** is specified, then **-to_attach = task** must be specified.
- If **target_task** is not specified, the property is set on the task where the handler is located.

LOV

For an overview of using LOVs in handlers, see [Using lists of values \(LOVs\) in handler arguments](#).

The LOV can contain multiple optional lines containing filter options followed by multiple lines containing multilevel object paths.

Note

For an overview and examples of multilevel object paths in handlers, see [Defining multilevel object paths](#).

Each multilevel object path line can optionally have a filter option added as a second field after a tilde (~).

OPTION=<value>

{\$TARGET|\$REFERENCE}.<multi.level.object.path>[~ **OPTION=<value>]**

OPTION=<value>

Defines a configurable option to filter object selection.

If you supply an option on an LOV line on its own, it applies to all subsequent lines containing multilevel object paths. The option does not affect any multilevel object paths listed before the option.

If you supply an option on the same line as a multiple level object path, as a second field after a tilde (~) character, it only applies to that line.

Valid values are:

- **RULE={LATEST|<Rule>}**

Specifies the revision rule used to select the revision attached to the workflow process if initiated on an item. Use the keyword **LATEST** to select only the latest revision.

- **INCLUDE PARENTS=YES**

Specifies that all objects found by traversing a multilevel path are attached to the workflow process, not just the last set of objects in a path. For example, when a multilevel path is used to first find items in a workflow process, then find revisions in the item, and then find datasets in the revisions, it is only the datasets that are attached by default. Setting this argument to **YES** causes both the revisions and the datasets to be attached.

This argument reduces the number of lines required in the LOV and improves performance.

\$TARGET|\$REFERENCE

Defines the starting point from which to look for objects. Valid values are:

- **\$TARGET**

Defines the starting point as the workflow process target attachments.

- **\$REFERENCE**

Defines the starting point as the workflow process reference attachments.

multi.level.object.path

Defines a multilevel object path to traverse to find the required objects to attach to the workflow process. For an overview of using multilevel object paths in handlers, see [Defining multilevel object paths](#).

(ItemRevision).IMAN_specification.(Dataset)

Attaches any datasets attached to the specification relation to any revisions found.

For more examples, see the Examples section.

Placement

Requires no specific placement. Proper placement depends on the desired behavior of the workflow process and may require coordination with the placement of other handlers, especially in cases where other handlers depend on the results of **EPM-set-property**. Typical placement might be on the **Start** action or **Complete** action.

Restrictions

- The **-to_relation** argument must be used in conjunction with the **-to_attach** handler.
- The **-from_relation** argument must be used in conjunction with the **-from_attach** handler.
- The **-to_lov** argument is mutually exclusive of the **-to_attach** and **-to_relation** arguments.
For an overview of using LOVs in handlers, see [Using lists of values \(LOVs\) in handler arguments](#).
- The **-from_lov** argument is mutually exclusive of the **-from_attach** and **-from_relation** arguments.
- Do not use the **-include_type** argument and the **-exclude_type** argument together.
- A single instance of this handler cannot update both attachment objects and derived objects. Separate handler instances must be used, where one handler instance updates attachments, and a second instance updates derived objects.
- Due to a potential conflict of interest, you may not want to use this handler with other handlers that also set the same property.

Examples

- Sets the target object's **object_desc** string property to a value of **Component Template**.

Argument	Values
-property	object_desc
-value	Component Template
-to_attach	TARGET
-bypass	

- Sets the target object's **backup_date** date property to a value of **2009-03-01**.

Argument	Values
-property	backup_date
-value	2009-03-01
-to_attach	TARGET
-bypass	

- Sets the target object's **archive_date** date property, **archive_info** string property, and **has_variant_module** Boolean property to the values specified in the example.

Argument	Values
-property	archive_date,archive_info,has_variant_module
-value	\$CURRENT_DATE,Archiving completed process,False

Argument	Values
-to_attach	TARGET
-bypass	

- Uses values from an object with a specifications relation to the reference attachment to set the target objects' properties.

Argument	Values
-property	object_desc
-value	PROP::object_desc
-from_attach	REFERENCE
-from_relation	IMAN_specification
-to_attach	TARGET
-bypass	

- Uses values from an object with a specifications relation to the reference attachment to set properties on objects with a specifications relation to the target attachment.

Argument	Values
-property	object_desc
-value	PROP::object_desc
-from_attach	REFERENCE
-from_relation	IMAN_specification
-to_attach	TARGET
-to_relation	IMAN_specification
-bypass	

- Uses values from an object with a specifications relation to the reference attachment to set properties on **UGMASTER** type objects with a manifestation relation to the target attachments.

Argument	Values
-property	object_desc
-value	PROP::object_desc
-from_attach	REFERENCE
-from_relation	IMAN_specification
-to_attach	TARGET
-to_relation	IMAN_manifestation

Argument	Values
-include_type	UGMASTER
-bypass	

- Uses values from an object with a specifications relation to the reference attachment to set properties on both objects with a specifications relation to the target attachments and objects with a specifications relation to the reference attachments.

Argument	Values
-property	object_desc
-value	PROP::object_desc
-from_attach	REFERENCE
-from_relation	IMAN_specification
-to_attach	BOTH
-to_relation	IMAN_specification
-include_type	UGMASTER
-bypass	

- Uses an LOV to obtain values that are used to update target property values.

Argument	Values
-property	object_desc
-value	PROP::object_desc
-from_lov	SYS_EPM_main_objects
-to_attach	TARGET
-bypass	

- Uses an empty string to reset a property on a **TARGET** object. In this example, the **object_desc** property is reset to "".

Argument	Values
-property	object_desc
-value	
-to_attach	TARGET
-bypass	

- Uses an empty string to reset a property on a **TARGET** object and also sets another property value. In this example, the **object_desc** property is reset to "" and the **sequence_limit** property is set to 6.

Argument	Values
-property	object_desc,sequence_limit
-value	,6
-to_attach	TARGET
-bypass	

- Uses empty strings to reset three properties on a **TARGET** object. In this example, the **object_desc** property is reset to "", the **sequence_limit** property is reset to 0, and the **CUST_text_field** property is reset to "".

Argument	Values
-property	object_desc,sequence_limit,CUST_text_field
-value	""
-to_attach	TARGET
-bypass	

- Adds a property from a target item business object to a target form that is attached to the item revision with a specification relation. To do this, you must omit the **-bypass** argument. This example maps the **item_id** item property to the **prop_soln** CMII CR form property. Both objects have been added to the process as **TARGET** objects.

Argument	Values
-property	prop_soln
-value	PROP::item_id
-from_attach	TARGET
-to_attach	TARGET
-include_type	CMII CR Form
-to_relation	IMAN_specification

- Retain approvals on a task during rework.

Argument	Values
-property	fnd0DecisionsToRetain
-value	89
-to_attach	task
-target_task	This should apply to Review or Acknowledge tasks, especially when dealing with the fnd0DecisionsToRetain property. For other types of tasks, it will do nothing.

- Set an array property (hard-coded method)

Argument	Values
-property	fnd0lcsClassNames
-value	Plastics, hard plastics
-to_attach	target

- Set an array property (non-hard-coded method)

Argument	Values
-property	recipients
-value	PROP::epm_proposed_reviewers
-to_attach	task
-from_attach	target

EPM-set-rule-based-protection

Description

Passes information to Access Manager to determine which named ACL to use while the associated task handler is current or started. "Started" indicates that the start action is completed.

The ACL is applied to the task and all subsequent tasks in the workflow process unless it is changed by another instance of the **EPM-set-rule-based-protection** handler or the process completes. See Executing workflow handlers for more information on how **EPM-set-rule-based-protection** works to accommodate AM functionality.

You can also set workflow ACLs by editing the Named ACL attribute, which automatically updates this handler.

Note

- This handler affects the behavior of the tasks as well the targets. For example, the ACL can grant permission to promote or demote the tasks.
- Accessors, such as approvers or the responsible party, are retrieved from the currently active tasks. So even if the named ACL is the same for two separate tasks, the actual user who gets access for each task could be different. For example, **waynej** is the responsible party for task 1, **bjorn** is the responsible party for task 2, and the ACL grants write access to the responsible party for both tasks. In this case, **waynej** gets write access for duration of task 1 and **bjorn** gets write access for duration of task 2.
- If you have multiple workflow processes in effect at the same time for the same target object, and each process sets its own ACL, a user gets access if any of the ACLs grants that access. To deny access in that situation, all ACLs must deny that access.

Select **Show Task in Process Stage List** to enable the template staging functionality.

- The named ACL defined in this handler becomes the **ACL Name** value in the **Task Attributes Panel** for the task.
- When this handler is applied to a task, the **Show Task in Process Stage List** property on the **Tasks Attributes Panel** is automatically selected. The **Show Task in Process Stage List** displays the task in the **Process Stage List** property for the target object. Tasks in the **Process Stage List** determine the ACL for target objects.

Syntax

EPM-set-rule-based-protection -acl=<named-ACL>

Arguments

-acl

The name of an existing named ACL to be used when the task becomes the current task.

Placement

Place on the **Start** action of any task.

Restrictions

None.

Examples

- This example tells Access Manager to use the **engineering_release_start0** ACL.

Argument	Values
-acl	engineering_release_start0

- This example tells Access Manager to give write access to the responsible party only for the second task in a four-task workflow. The other three tasks are read-only.



- **Task 1**—read-only access for all users.

The **Vault** ACL gives read and copy access to users, but not write access.

Argument	Values
-acl	Vault

- **Task 2**—write access for the responsible party.

The **Grant-Write-to-RP** ACL gives write access only to the responsible party.

Argument	Values
-acl	Grant-Write-to-RP

- **Task 3**—read-only access for all users.

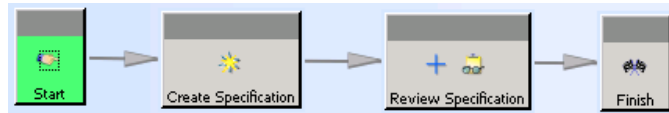
The **Vault** ACL revokes write access starting with this task.

Argument	Values
-acl	Vault

- **Task 4**—read-only access for all users.

No handler is needed because the ACL in **Task 3** still applies.

- This example, when placed on the **Review Specification** task, tells Access Manager to give demote access to only the task's responsible party. Promote access is denied to everybody, including the responsible party.



The screenshot shows the 'Named ACL' dialog box. The 'ACL Name' is 'DemoteReviewTask'. The 'Responsible Party' is 'World'. The 'Assigned ACL Name' is 'DemoteReviewTask'. The 'Assign' button is visible.

Argument	Values
-acl	DemoteReviewTask

EPM-set-status

Description

Applies the appropriate release status to the workflow process targets. This handler gets the release status type that the **EPM-create-status** handler attaches to the root task.

Note

The **EPM-set-status** workflow handler is designed to work on release status effectivity, which is commonly used to express effectivity for item revisions used in a BOMView revision in Structure Manager.

Release status effectivity is not applicable for Product Configurator or 4th Generation Design objects. However, you can use the **CONFMGMT-cut-back-effectivity** workflow handler to propagate the release status effectivity of an engineering change object to configurator and 4GD objects that are attached to the change object as solution items. This translates the release status effectivity to the effectivity model used in Product Configurator and 4th Generation Design.

Note

The **EPM_skip_dataset_purge** preference determines if dataset versions are purged when the **EPM-set-status** workflow handler adds a status.

Note

Configure the **WRKFLW_change_target_lmu** preference to indicate if the **last_mod_user** attribute of a workflow target is changed when the status is applied. Set the preference value to **TRUE** to indicate the attribute value is changed to the user who completes the task, or set the preference value to **FALSE** indicating the attribute value is not changed.

Syntax

```
EPM-set-status -action=append | rename | replace | delete
[-status=<name>]
[-new_status=<new_name>]
[-retain_release_date]
[-set_effectivity]
[-status_not_shared]
```

Arguments

-action

- **append**

Attaches the root-task release status to the targets. Any previous statuses for the same targets are not affected.

- **rename**

Renames the release status from `<name>` to `<new_name>`.

If the `<name>` release status is not found, the handler renames the last status attached to the targets.

- **replace**

Removes all release statuses attached to the targets, and attaches the root task release status to the targets.

Note

If more than one status object exists on the root task, apply the **-status** argument variable `<=status_name>`. If the **-status** argument is not specified then replacement status is not guaranteed.

- **delete**

Removes the release status specified by the **-status** argument from the targets.

- If the **-status** argument is not used, all release statuses are removed from the targets.
- This handler does not remove root-task release statuses that were created in the same workflow as the root task.

This value can also be used to remove release statuses that were applied in other workflows.

-status

Specifies the name of the release status. When used with the **-action** argument, offers additional options to define the status.

Note

Enter the name as defined in the Business Modeler IDE, not the display name.

-action argument value	-status argument result
append	If the specified release status is not attached to the root task, the handler: • Creates a new status with the specified name. • Attaches the new status to the root task.
rename	The handler renames the release status to the value specified in -new_status.
replace	If the specified release status is not attached to the root task, the handler: • Creates a new status with the specified name.

-action argument value	-status argument result
delete	Attaches the new status to the root task. The handler removes the release status from the targets, but does not remove the status from the root task.

-new_status

Specifies the new name for the release status. Use this argument only if you use the **-action** argument's **rename** value.

Enter the name as defined in the Business Modeler IDE, not the display name.

Caution

If the release status type is not defined, effectivity and configuration may be unavailable for the release status.

-retain_release_date

Retains the original release date on the target if it had previously been released. Not valid for **replace**.

-set_effectivity

If used, the handler creates the open-ended date effectivity with release date as start date.

-status_not_shared

Places on each target an individual copy of the root-task release status. By default, all targets share a reference to the release status.

Placement

Place on any action. Typically attached to the **Complete** action.

Restrictions

- By default, the **-action** argument and its **append** value are assumed if no argument is specified, or if an argument other than those specified is supplied to the handler.
- If the root task bears two or more statuses, and if the **-action** argument value is **replace**, the latest status on the root task replaces the status on the targets.

Examples

- This example adds the status object of the root task to the target object:

Argument	Values
-action	append

- This example adds the status object of the root task to the target object and retains the original released date of the target object:

Argument	Values
-action	append
-retain_release_date	

- This example replaces all existing status objects with the status object of the root task:

Argument	Values
-action	replace

- This example replaces existing status objects with the status object of the root task. It also sets an open-ended effectivity with release date as the start date on the new status object:

Argument	Values
-action	replace
-set_effectivity	

- This example renames all the status objects named **pre-released** to the name of the new status object, **released**:

Argument	Values
-action	rename
-status	pre-released
-new_status	released

- This example deletes all status objects from the target object but does not delete it from the root task:

Argument	Values
-action	delete

- This example deletes a status called **released** from the target object, but does not delete it from the root task:

Argument	Values
-action	delete
-status	released

- This example attaches a release status named **released** to the root task:

Argument	Values
-action	append
-status	released

- This example places on each target an individual copy of the root-task release status.

Argument	Values
-action	append
-status_not_shared	

- This example creates a new release status named **released**, attaches that status to the root task. and places an individual copy on each target.

Argument	Values
-action	append
-status_not_shared	
-status	released

EPM-set-task-result-to-property

Description

Reads the specified property from the identified task or target object, and uses that property value to set the result string attribute of the task where this handler is located or on the task specified by the **-target_task** argument. A common use for this handler is to control **Condition** task branching instead of using a more involved scheme that requires a custom handler. Using this handler to set a **Condition** task's result attribute allows the workflow process to branch based on a property of the identified task or target source object.

Syntax

```
EPM-set-task-result-to-property -source=task | target [-source_task=<task-name>] [-include_type=<target-object-type>] [-target_task= $ROOT_TASK | $DEPENDENT_TASK] -property=<property-name>
```

Arguments

-source

Indicates from which source object (**task** or **target**) the identified property should be read. The property is identified by the **-property** argument.

- **task**

Indicates the property should be read from a task. The **-task_name** argument specifies the task to use.

- **target**

Indicates the property should be read from a target object. The **-include_type** argument specifies the target object type to use.

-source_task

Identifies the name of a task from which to read the specified property (the **-property** argument specifies the property). This argument is valid only if **-source=task**. If a valid **-source_task** argument is absent, the property is read from the task where the handler is located.

-include_type

Identifies the target type from which to read the specified property (the **-property** argument specifies the property). This argument is valid only if **-source=target**. If there are more than one target objects of the given type, the first target on the list is used. If a valid **-include_type** argument is absent, the property is read from the first target on the list.

-target_task

Identifies where the result string attribute is set.

This is an optional argument. If **-target_task** is not specified, then the task **result** attribute will be set for the task containing the **EPM-set-task-result-to-property** handler.

- **\$ROOT_TASK**

Sets the result string attribute on the root task of the process.

- **\$DEPENDENT_TASK**

Sets the result string attribute on the parent process task which is dependent on this subprocess. The parent process task should be a **Condition** task.

- **\$PARENT_TASK**

Sets the result string on the immediate parent of the current task.

- **Multi-level task path**

Specifies a colon-delimited path to navigate to a specific descendant task, like the **-source_task** argument.

For example: **-target_task=RootTask:ChildTask:TargetTask**

-property

Specifies the property to be read from the identified source object (**task** or **target**).

Placement

Typically placed on the **Start** action of the specified **Condition** task.

However, this handler can be placed on any task but can set the result only on either the root task or a **Condition** task. The **Condition** task can be the task where the handler is placed or a parent task that is dependent on the task where the handler is placed.

Restrictions

- Do not place this handler on the **Perform** action.
- Do not use this handler in conjunction with other handlers that also set the **result** attribute, such as **EPM-set-condition**, **EPM-set-parent-result**, or a custom handler.
- You can use this handler on the **Complete** action only if a change occurred on the **Perform** action.
- This handler allows you to set the **result** attribute on the root task or any other **Condition** task.

Examples

- This example branches a **Condition** task based on the item revision's revision if a workflow process has an item revision as a target object. The handler is placed on the **Task01 Condition** task.

Argument	Values
-source	target
-include_type	ItemRevision
-property	item_revision_id

You then draw paths from the **Condition** task and assign custom flow path values by right-clicking the path and choosing **Custom**.

- This example branches a **Condition** task based on a task's responsible party. The handler is placed on the **Task02 Condition** task, and the responsible party is read from the **Task01** task.

Argument	Values
-source	task
-source_task	Task01
-property	resp_party

- This example branches a **Condition** task based on a task's responsible party. The handler is placed on the **Task02 Condition** task, but it is not configured with the **-source_task** argument and therefore defaults to reading the responsible party attribute from the **Task02 Condition** task.

Argument	Values
-source	task
-property	resp_party

EPM-start-bypass

Description

EPM-start-bypass enables bypass so that subsequent handlers run with system authority. Bypass is disabled once paired when handler **EPM-end-bypass** executes. See Bypass Handlers for more information.

Note

The **WRKFLW_access_level_for_handlers_execution** preference has precedence over bypass subset handlers when it is set to **system** access. The system setting allows all handlers to execute with system access. Therefore, the **EPM-start-bypass** and **EPM-end-bypass** handlers have no effect.

Syntax

EPM-start-bypass

Arguments

EPM-start-bypass does not accept any arguments.

Placement

There are no restrictions on placement.

Restrictions

EPM-start-bypass must be paired with **EPM-end-bypass** on an action. This is to identify when system authority should start (identified by **EPM-start-bypass**) and when system authority ends (identified by **EPM-end-bypass**).

The system authority granted by the **EPM-start-bypass/EPM-end-bypass** handler pair will not span multiple actions, so you must configure both handlers as a pair on a single action. However, you can configure the handler pair on multiple actions.

Examples

- This example grants bypass for the **EPM-set-property** handler.

Start Action:

EPM-start-bypass

EPM-set-property

- This example will grant bypass for the **EPM-set-property** handler, but not for the **EPM-set-parent-result** handler.

EPM-end-bypass

Complete Action:

EPM-start-bypass

EPM-set-property

EPM-end-bypass

EPM-set-parent-result

- This example shows multiple pairs of **EPM-start-bypass/EPM-end-bypass** within a single action.

Complete Action:

EPM-auto-check-in-out

EPM-start-bypass

EPM-create-sub-process

EPM-end-bypass

EPM-move-attached-objects

EPM-start-bypass

EPM-set-property

EPM-end-bypass

EPM-set-parent-result

EPM-suspend-on-reject

Description

Suspends the task when the approval quorum cannot be met.

Syntax

EPM-suspend-on-reject

Arguments

None.

Placement

Place on the **Perform** action of the **perform-signoffs** task.

Restrictions

Place only on the **perform-signoffs** task.

EPM-system

Description

Runs the first operating system argument passed to it.

The **EPM-system** handler cannot handle run-time command line arguments. For information about addressing such issues, see the [EPM-execute-follow-up](#) action handler. The **EPM-system** handler does not accept return values.

Syntax

EPM-system -command= *<argument>*

Arguments

-command

Operating system command to be run. Define with a standalone program or command. The length is determined by your local system's command line length settings.

Placement

Requires no specific placement.

Restrictions

None.

Examples

- This example sends an e-mail to **smith** with a body from the **/tmp/approval_note.txt** file and the subject **Notification: Task has been approved**:

Argument	Values
-command	mailx -s "Notification: Task has been approved" smith /tmp/approval_note.txt

EPM-trigger-action

Description

Triggers the specified action on the task to which this handler is attached.

Syntax

EPM-trigger-action -action=<action> -comment=<comment>

Arguments

-action

Performs the designated task. Accepts one of these task actions:

- **EPM_assign_action**
- **EPM_start_action**
- **EPM_complete_action**
- **EPM_skip_action**
- **EPM_suspend_action**
- **EPM_resume_action**
- **EPM_undo_action**
- **EPM_abort_action**
- **EPM_perform_action**

-comment

Associates comment with the task action when the action is logged in the workflow audit log file.

Placement

Requires no specific placement.

Note

Infinite loops can occur when **EPM_trigger_action** handler with the task action **EPM_complete_action** is placed on the **Assign**, **Start**, or **Complete** of a task.

Restrictions

None.

Examples

This example performs the **Complete** action, displaying the text **Triggering the Complete action from the EPM-trigger-action handler** when the **Complete** action is logged in the workflow audit log file.

Argument	Values
-action	EPM_complete_action
-comment	Triggering the Complete action from the EPM-trigger-action handler

EPM-trigger-action-on-related-process-task

Description

Triggers an action on a task within a related workflow process.

Workflow processes can be related and/or coupled using reference attachments. Triggered workflow processes can be coupled with the triggering workflow process by:

- Adding triggering workflow process target attachments as reference attachments to the triggered workflow process. For example, the triggering workflow process could be the workflow process for a change object. Each workflow process for the affected item, the problem item, and so on, are then triggered workflow processes. Pasting the change object as a reference attachment to each workflow process for the affected item, the problem item, and so on, establishes a coupling. The change object process can now trigger task actions (such as **Suspend** and **Resume**) in each triggered workflow process.
- Adding triggered workflow process target objects as reference attachments to the triggering workflow process. This example is similar to the previous example. It also uses a coupling, but in the opposite direction: the triggering workflow process could be a review process for a part that is affected by a change. The change object process is then the triggered workflow process. Pasting the change object as a reference attachment to each workflow process for the affected item, the problem item, and so on, establishes a coupling. The part review process can now trigger task actions (such as **Suspend** and **Resume**) in the change object process.
- Adding the triggering workflow process object as a reference to the triggered workflow process. This example uses a coupling achieved by pasting the workflow process object itself, not a target or reference attachment. The triggering workflow process could be the process for a change object. Each process for the affected item, the problem item, and so on, are then triggered processes. Pasting the change process object as a reference attachment to each process for the affected item, the problem item, and so on, establishes a coupling. The change object process can now trigger task actions (such as **Suspend** and **Resume**) in each triggered process.

This handler helps to identify sibling workflow processes (processes that have reference to a higher-level process) and to trigger an action on a task within those processes. For example, you can control the appearance of workflow processes in your inbox by suspending and resuming the workflow processes depending on the reference workflow processes they have.

Syntax

EPM-trigger-action-on-related-process-task

-task=<task-name>

-action=<action-name>

[-active=<ACTION>

[-active=<OTHER-ACTION>]]

[-comment=<comment>]

`[-process_type=Processes_Referencing_Target_Objects | Processes_Referencing_This_Process | Reference_Object_Processes]`

`[-template=<process-template-name>]`

`[-depth=<level>]`

`[-debug]`

Arguments

-task

Name of the task in which the given action needs to be triggered. If the task name is ambiguous (such as **perform-signoffs**), Siemens Digital Industries Software recommends that the task name is qualified with its parent task name (for example, **level2.perform-signoffs** or **conditional branch 2.level2.perform-signoffs**).

-action

Name of the action that needs to be triggered. The following are valid action names: **ASSIGN**, **START**, **PERFORM**, **COMPLETE**, **SUSPEND**, **RESUME**, **SKIP**, **ABORT**, and **UNDO**.

Note

The action cannot succeed if the task is not in the correct state when the action is triggered. For example, the **COMPLETE** action cannot succeed if a **Condition** task result is something other than **Unset**. Therefore, you must set the value before triggering the action. To set the value, write a custom handler that is triggered before this action.

-active

Name of the action for which this handler is valid.

If this argument is used, and the handler is called as part of a trigger to a nonlisted action, the handler silently returns immediately. For more information about valid action names, see the **-action** argument.

This argument can be useful when the handler is used in **Perform** actions. The following actions also automatically run the **Perform** action handlers, raising the potential for infinite loops or unnecessary processing:

- **EPM_add_attachment_action**
- **EPM_remove_attachment_action**
- **EPM_approve_action**
- **EPM_reject_action**
- **EPM_promote_action**
- **EPM_demote_action**
- **EPM_refuse_action**

- **EPM_assign_approver_action**
- **EPM_notify_action**

This argument is optional.

-comment

The comment to be incorporated when the action is triggered.

If this argument is not specified, it defaults to the name of this handler: **EPM-trigger-action-on-related-process-task**.

This argument is optional.

-process_type

The workflow processes to find. It can have one of the following values:

- **Processes_Referencing_Target_Objects**

Finds workflow processes that reference one or more of the target attachments belonging to the current workflow process. The action is initiated for each matching attachment found. For example, if a workflow process references two target attachments belonging to the current workflow process, the action is initiated twice.

This is the default value for this argument

- **Reference_Object_Processes**

Finds workflow processes with target attachments that match reference attachments belonging to the current workflow processes. The action is initiated for each matching attachment found. For example, if the current workflow process reference two target objects of a workflow process, the action is initiated twice.

- **Processes_Referencing_This_Process**

Finds workflow processes that reference the current workflow process.

This argument is optional.

-template

The name of the workflow process template of the workflow process(es) to be triggered.

This argument is useful to save processing time and/or improve robustness. Use this argument to configure this handler to trigger actions on specific workflow processes of a particular workflow process template. This name may contain wildcard characters.

This argument is optional.

-depth

This argument controls the recursion depth.

This argument is useful when the triggering of an action results in another action being triggered (due to the configuration of the **EPM-trigger-action-on-related-process-task** handler, or any other handler placed in that action) and so on.

The recursion depth defaults to 1. If the recursion depth is required, set the depth carefully to avoid infinite loops. If set to zero, make sure that the algorithm converges to a definite end of the recursion.

-debug

This argument writes debug messages to the log file.

This argument is optional.

Placement

Requires no specific placement. Depending on the purpose, may be placed at various tasks and actions. If placed on the **Start** action of the root task, controls whether or not a workflow process can be initiated.

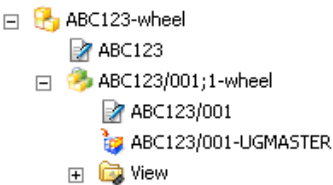
Restrictions

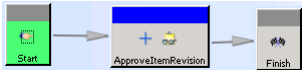
Do not use this handler in a subprocess.

Examples

The following example has two workflow process templates: **Initiate Item Revision** and **Initiate Dataset**. The **EPM-trigger-action-on-related-process-task** handler in the **Initiate Item Revision** process triggers the **Complete** action on the **ApproveDesignWork** task in the **Initiate Dataset** process.

This example uses the following item revision with a **UGMASTER** dataset:



Process Template	Tasks	Steps to follow
Initiate Item Revision	Start →	In the root task in the Start action, add the EPM-trigger-action-on-related-process-task handler with the following arguments: • -task=ApproveDesignWork • -action=COMPLETE • -comment=approved
	ApproveItemRevision (Review task)→	
	Finish	

Process Template	Tasks	Steps to follow
------------------	-------	-----------------

Initiate Dataset

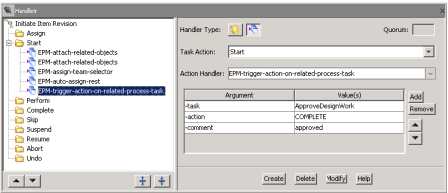


Start →

CreateDesignWork (Review task) →

ApproveDesignWork (Do task) →

Finish



Create an **Initiate Dataset** workflow process for the **ABC123/001-UGMASTER** dataset and paste the **ABC123/001** item revision as the reference attachment.

Sign off the **CreateDesignWork** task, which starts the **ApproveDesignWork** task.

To perform the Do task, select **Task View** then select the **Complete** option.

Note

Do not click **Apply**.

Then, create an **Initiate Item Revision** workflow process for the **ABC123/001** item revision.

Note

Before the **EPM-trigger-action-on-related-process-task** handler is triggered, the **ApproveDesignWork Do** task is in the **Started** state. After the handler executes, the task is in the **Completed** state.

Since **-process_type=Processes_Referencing_Target_Objects** is the default setting, and the **ABC123/001** item revision is a reference attachment of the **Initiate Dataset** process, the **Complete** action of the **ApproveDesignWork** task is triggered.

Note that the **Complete** action is successful only if all conditions for the completing a **Review** task are already met.

ERP-att-logfile-as-dataset-RH

Description

Creates the **ERP_Log_Dataset** text dataset and attaches it as a reference to the process. Through the lifetime of the process, this dataset logs the progress of the ERP-related parts of the process. On completion of the process, the log file is exported to the directory specified by the **Tc_ERP_rellog_file_path** preference.

Syntax

ERP-att-logfile-as-dataset-RH

Arguments

None.

Placement

Place on the **Review** task. Call this handler before any other ERP handler, as other handlers work on the assumption that the ERP logfile dataset exists.

Note

Although not a rule handler, this was made a rule handler that can be placed and run before any other handler.

Restrictions

None.

ERP-attach-targets-AH

Description

Attaches all ERP forms as targets of the process and then creates a transfer folder (of type **ERP_transfer_folder_type**) for each target item revision, which is attached as references to the process. All ERP forms with the relations specified in the **reln_names** argument are pasted into the corresponding transfer folder.

ERP forms are those that are defined in the mapping schema.

Syntax

ERP-attach-targets-AH -reln_names = <reln1>,<reln2>,...

Arguments

-reln_names

A list of the relation types used to relate ERP forms to item revisions.

Separate multiple types with commas or the character specified by the **EPM_ARG_target_user_group_list_separator** preference.

Note

Relation names are case-sensitive and should be named, for example, **tc_specification** not **TC_Specification**.

ERP_Data is the special relation supplied for attaching ERP forms, if these are to be distinguished from other relations. The semantics are as for manifestation:

- The advantage is that ERP forms can be added later in the life cycle without forcing a new revision of the item.
- The disadvantage is that the ERP data is less secure and the forms can be removed or replaced.

Access to the forms is controlled using access rules.

Placement

Place on the **Review** task.

Restrictions

None.

ERP-delete-log-dataset-AH

Description

Cleans up the database by deleting the ERP logfile once the process has successfully completed.

Syntax

ERP-delete-log-dataset-AH

Arguments

None.

Placement

Place this handler on the **Complete** action of the root task.

Restrictions

None.

ERP-download-AH

Description

Extracts attribute values from the Teamcenter database and writes these out to an operating system transfer file. The transfer file is placed in the directory specified by the **Send_file_format** global setting with the name defined by the **Send_file_name** global setting.

The behavior of this handler depends on the **Send_file_format** global setting.

The format of the transfer file can be configured by the mapping file. This is a key feature of the Teamcenter/ERP Connect Toolkit.

This handler also writes the names of the **Send** file and **Response** file paths to the **Description** box of the **ERP_Logfile** dataset, which are required.

Syntax

ERP-download-AH

Arguments

None.

Placement

Place on the **Perform Signoff** task.

Restrictions

None.

ERP-post-upload-AH

Description

Runs after the upload and reads the contents of the ERP logfile dataset. The handler looks in the directory defined in the **Response_file_path** global setting for the **Response** file, with the name defined in the **Description** box of the **ERP_Logfile** dataset. It imports the **Response** file into the latest version of the ERP logfile dataset.

The handler parses the ERP logfile according to the **Send_file_format** global setting as follows:

- If the status is **CREATED** or **CHANGED** and the **set_transfer** argument is set to **YES**, set the **Sent to ERP** box of the respective forms to `<user_id>/<upload_date>`.
- At the end of the logfile, there is a single **UPLOAD_STATUS** parameter. If set to **FAILURE**, the handler returns an error code other than **ITK_ok**, which displays an error message and stalls the process. If set to **SUCCESS**, the handler:
 - Removes transfer folders from the process and delete them.
 - Returns **ITK_ok**, indicating the process/review level is complete.
- The handler parses the ERP logfile for the single overall status of the upload according to the success/error message defined in the **Error_success_message** global setting.

Syntax

ERP-post-upload-AH -set_transfer={YES|NO}

Arguments

-set_transfer

Value must be **YES** or **NO** (case insensitive). If **YES**, the **Sent_to_ERP** fields are set upon successful transfer.

Note

Siemens Digital Industries Software recommends you set the value to **YES**, so it is clear the data is uploaded. If this is only working data, then you can remove the value in the **set_transfer** field to allow data to be resent.

Placement

Place this rule after the **SAP-upload-AH** handler on the **perform-signoff** task.

Restrictions

None.

ERP-set-pathnames-in-logds-AH

Description

Reads the configuration file and sets the path names of the transfer file and response file (listed in the configuration file), in a log dataset property.

Syntax

ERP-set-pathnames-in-logds-AH

Arguments

None.

Placement

Place on the **Complete** action of any task. Apply after the **EPM-set-pathnames-in-logds-AH** handler.

Restrictions

None.

ERP-transform-AI-contents-AH

Description

Reads the PLM XML contents of an AI object attached as reference to the process. It then applies the XSLT transform specified in an input parameter and writes the resulting **.xml** file to the export directory.

Syntax

ERP-transform-AI-contents-AH

Arguments

None.

Placement

Place on the **Complete** action of any task. Apply after the [AI-export-AH](#) handler.

Restrictions

None.

FS0-release-haraanalysis-objects

Description

This handler adds all objects related to the HARA (Hazard Analysis and Risk Assessment) analysis as target attachments to the workflow process and releases them along with the HARA analysis object. The handler traverses the HARA analysis structure, collecting related objects such as Hazard Events, Harms, Potential Effects, Hazards, Safety Goals, and Safe States. It adds these objects to the workflow targets so they are released together. Only objects not already released (no release status) are processed. The handler uses optional arguments to control which related object types are included in the release.

Syntax

FS0-release-haraanalysis-objects

Arguments

-includeHarm

(Optional) When set to **true**, includes Harm objects related to Hazard Events via the **Mq0RiskLosses** relation.

-includePotentialEffect

(Optional) When set to **true**, includes Potential Effect objects related to Hazard Events via the **Mq0RiskToHazardRel** relation.

-includeHazard

(Optional) When set to **true**, includes Hazard objects related to Potential Effects via the **Mq0HazardousSituations** relation.

-includeSafetyGoal

(Optional) When set to **true**, includes Safety Goal objects related to Hazard Events via the **Mq0HazardEventSafetyGoal** relation.

-includeSafeState

(Optional) When set to **true**, includes Safe State objects related to Safety Goals via the **Mq0SafeStates** relation. Only processed if Safety Goal objects are found.

Placement

Place on the Start action of the root process task in the HARA Analysis Release Process workflow template.

Restrictions

Target attachments must include **Mq0HARAAnalysis** objects. Only objects without an existing release status are processed. The handler adds collected objects as both target and reference attachments to the root task.

Examples

This example releases the HARA analysis along with all related safety objects:

Argument	Values
-includeHarm	true
-includePotentialEffect	true
-includeHazard	true
-includeSafetyGoal	true
-includeSafeState	true

GMIMAN-invoke-subscription-event-on-item

Description

Notifies the subscribed user about an event by checking the release status of the item revision with the specified argument.

Syntax

GMIMAN-invoke-subscription-event-on-item -event=<event-type-release-status>

Arguments

-event

Valid event-type release status.

Placement

Add this handler after the **EPM-set-status** handler in the **Complete** action of the release workflow.

Restrictions

This handler can only be used when the GM Overlay is installed. The valid event-type release statuses are limited to the event types that are installed for the Subscription Administration.

HRN-revise

Description

Use the **HRN-revise** handler to revise Teamcenter designs and the corresponding Capital designs in a workflow process. When used in a workflow template, the **HRN-revise** handler collects the Capital designs for the target Teamcenter design through Capital webservices and then revises both the Teamcenter and corresponding Capital designs.

Syntax

HRN-revise

`[-capital_pwd_file = capital_password_file_path]`

`[-capital_user = capital user]`

`[-change_releaselevel_to_draft = true|false]`

`[-ignore_lock = true|false]`

`[-include_children = true|false]`

`[-populate_children = true|false]`

`[-revise_children = true|false]`

`[-include_resolved_comments = true|false]`

`[-include_unresolved_comments = true|false]`

`[-include_pending_checklists = true|false]`

`[-include_completed_checklists = true|false]`

`[-include_notes = true|false]`

`[-include_watchlist = true|false]`

`[-include_links = true|false]`

`[-revise_desc = new revision short description]`

Arguments

-capital_user

The Capital user name used to log on and invoke the Capital webservices.

-capital_pwd_file

The absolute path of the password file used to log on to the Capital webservices.

The password file can be generated through the Teamcenter install utility.

-change_releaselevel_to_draft

If the value is **true**, the release level of the newly created revision is reset to **Draft**.

If the value is **false**, the release level is copied from the source design to the revised design.

Default value is **true**.

-ignore_lock

If the value is **true**, a revision is created even if the design is locked in another session.

If **false**, the system prohibits creating a revision if the design is locked.

Default value is **true**.

-include_children

All child designs are included with the composite revision. Their inclusion is dependent on the **-revise_children** and **-populate_children** arguments.

If **false**, none of the child designs are included in the composite design revision.

Default value is **false**.

-revise_children

Applicable only if **-include_children** argument is **true**.

If set to **true**, a new revision of each child design is created and included with the new composite revision.

If set to **false**, all child designs are included with the composite revision, but a revision is not created for any of the child designs. Instead, the current child revision is associated with the new composite revision for each child design. It revises the composite but maintains a link to the same child designs as in the original composite.

Default value is **false**.

-populate_children

Applicable only if **-include_children** and **-revise_children** arguments are **true**.

If set to **true**, a diagram is created for the revised child design.

If set to **false**, a diagram is not created for the revised child design.

Default value is **false**.

-include_resolved_comments (optional)

Designers and reviewers working on a project can add comments related to the design, diagram, or objects available for the diagram.

If set to **true**, the resolved comments are copied from the source design to the revised design.

If not specified, the Capital preference is honored.

Default value is **true**.

-include_unresolved_comments (optional)

Designers and reviewers working on a project can add comments related to the design, diagram, or objects available for the diagram.

If set to **true**, the unresolved comments are copied from the source design to the revised design.

If not specified, the Capital preference is honored

Default value is **true**.

-include_pending_checklists (optional)

Checklists capture tasks or design checks requiring completion.

If set to **true**, the pending checklists are copied from the source design to the revised design.

If not specified, the Capital preference is honored.

Default value is **true**.

-include_completed_checklists (optional)

Checklists capture tasks or design checks requiring completion.

If set to **true**, the completed checklists are copied from the source design to the revised design.

If not specified, the Capital preference is honored.

Default value is **true**.

-include_notes (optional)

Notes add additional information to a diagram.

If set to **true**, the notes are copied from the source design to the revised design.

If not specified, the Capital preference is honored.

Default value is **true**.

-include_watchlist (optional)

A *watchlist* lists users tagged within comments, notes, or a checklist, who receive an email notification, detailing the content they are associated with.

If set to **true**, the watchlist is copied from the source design to the revised design.

If not specified, the Capital preference is honored.

Default value is **true**.

-include_links (optional)

Links Capital objects to different entities (external or internal to Capital).

Example

A Teamcenter requirement is associated to several Capital objects in various designs. These Capital objects are linked to Teamcenter requirements using links. The link object has a **name** and collection of associated objects. The **name** represents Teamcenter requirement name. Associated objects represent Capital objects associated to that Teamcenter requirement.

If set to **true**, the links are copied from the source design to the revised design.

If not specified, the Capital preference is honored.

Default value is **true**.

-revise_desc

This refers to the new short description for a revised design.

Placement

This is placed on an action, typically on the **Start** or **Perform** action of a **Do** task.

Restrictions

None.

Examples

This collects and revises all the Capital designs for the corresponding target Teamcenter designs.

Arguments	Values
-capital_user	system
-capital_pwd_file	Absolute path of the generated password file.
-change_releaselevel_to_draft	true
-ignore_lock	true
-include_children	false
-populate_children	false
-include_resolved_comments	false
-include_unresolved_comments	false

Arguments	Values
-include_pending_checklist	false
-include_completed_checklist	false
-include_notes	false
-include_watchlist	false
-include_links	false
-revise_children	false
-revise_desc	

HRN-set-reject-state

Description

The **HRN-set-reject-state** handler provides status alignment between Teamcenter designs and **Electrical** designs in Capital.

When used in a Workflow template, the **HRN-set-reject-state** handler fetches all the Capital designs for the target Teamcenter design through Capital webservice. It then assigns the required **Reject** status to the Capital design.

Note

Use this handler in the design approval Workflow process to assign the required status to the electrical design in the Capital tool when the Teamcenter design is rejected.

Syntax

```
HRN-set_reject_state
[-annotate = <true|false>]
[-capital_pwd_file = <capital_password_file_path>]
[-capital_status = <capital reject status>]
[-capital_user = <capital user>]
[-freeze_sharedobjects_used = <{doNotFreeze | freeze | skipIfAllowed}>]
[-run_drcs = <{useCapitalPreference| forceRunDRC| skipIfAlreadyRun}>]
[-treat_warnings_as_errors = <true|false>]
```

Arguments

-annotate

Annotates diagrams after the release of the design.

Valid values are **true** and **false**. The default value is **false**.

-capital_pwd_file

Determines the absolute path of the password file used to log on to the Capital webservice.

The password file can be generated through the Teamcenter install utility.

-capital_status

Defines the **Reject** status name applied on the Capital design.

Defines the release level name and the release transition path in Capital in order to apply the release level on the Capital designs.

-capital_user

Defines the Capital user name used to log on and invoke the Capital webservice.

-freeze_sharedobjects_used

Configures freezing shared objects for a release level in Capital preferences. The possible values are as follows:

- `<doNotFreeze>`
Does not freeze the objects on transitioning to the target release level. If it is configured to freeze shared objects for the target release level in Capital preferences, passing this value results in an error.
- `<freeze>`
Configures whether to freeze shared objects for the target release level in Capital preferences.
- `<skipIfAllowed>`
Skips freezing of shared objects only if it is not configured to freeze shared objects for the target release level in Capital preferences.

The default value is **doNotFreeze**.

-run_drcs

Defines how **Design Rule Checks** (DRCs) perform when the design in Capital is rejected.

- `<useCapitalPreference>`
Runs **Design Rule Checks** (DRCs) configured to run in Capital preferences for a design type and a target release level.
- `<forceRunDRC>`
Enforces **Design Rule Checks** (DRCs) even if they are not configured to run in Capital preferences for a design type and target release level.
- `<skipIfAlreadyRun>`
Skips running **Design Rule Checks** (DRCs) if they have already run.

Example

If a design is already transitioned to a **Release** level of type **Release**, and the transition is happening at another **Release** level of the same type (**Release**), the user may choose to not run them again by passing this value.

The default value is **useCapitalPreference**.

-treat_warnings_as_errors

Changes warnings from **Design Rule Checks** (DRCs) to errors.

Valid values are **true** and **false**. The default value is **false**.

Placement

Place on any action. Typically, place on the **Start** or **Perform** action of a **Do** task.

Restrictions

None.

Examples

This fetches all electrical designs for Capital for the corresponding target Teamcenter designs. It applies **Draft** status to the Capital designs that use the Capital webservice.

Arguments	Values
-annotate	false
-capital_pwd_file	Absolute path of the generated password file
-capital_status	Draft
-capital_user	system
-freeze_sharedobjects_used	doNotFreeze
-run_drcs	useCapitalPreference
-treat_warnings_as_errors	false

HRN-set-release-state

Description

The **HRN-set-release-state** handler provides status alignment between Teamcenter designs and electrical designs in Capital.

When used in a Workflow template, the **HRN-set-release-state** handler fetches all the Capital designs for the target Teamcenter design through Capital webservice. It then assigns the required **Release** status to the Capital design.

Note

Use this handler in the design approval Workflow process to assign the required status to the electrical design in the Capital tool when the Teamcenter design is released.

Syntax

```
HRN-set_release_state
[-annotate = <true|false>]
[-capital_pwd_file = <capital_password_file_path>]
[-capital_status = <capital release status>]
[-capital_user = <capital user>]
[-freeze_sharedobjects_used = <{doNotFreeze | freeze | skipIfAllowed}>]
[-run_drcs = <{useCapitalPreference| forceRunDRC| skipIfAlreadyRun}>]
[-treat_warnings_as_errors = <true|false>]
```

Arguments

-annotate

Annotates diagrams after the release of the design.

Valid values are **true** and **false**. The default value is **false**.

-capital_pwd_file

Determines the absolute path of the password file used to log on to the Capital webservice.

The password file can be generated through the Teamcenter install utility.

-capital_status

Defines the **Release** status name applied on the Capital design.

Define the release level name and the release transition path in Capital in order to apply the release level on the Capital designs.

-capital_user

Defines the Capital user name used to log on to and invoke Capital webservice.

-freeze_sharedobjects_used

Configures freezing shared objects for a release level in Capital preferences. The possible values are as follows:

- `<doNotFreeze>`
Does not freeze the objects on transitioning to the target release level. If it is configured to freeze shared objects for the target release level in Capital preferences, passing this value results in an error.
- `<freeze>`
Configures whether to freeze shared objects for a target release level in Capital preferences.
- `<skipIfAllowed>`
Skips freezing of shared objects only if it is not configured to freeze shared objects for the target release level in Capital preferences.

The default value is **doNotFreeze**.

-run_drcs

Defines how **Design Rule Checks** (DRCs) perform when the design in Capital is released.

- `<useCapitalPreference>`
Runs **Design Rule Checks** (DRCs) configured to run in Capital preferences for a design type and a target release level.
- `<forceRunDRC>`
Enforces **Design Rule Checks** (DRCs) even if they are not configured to run in Capital preferences for a design type and a target release level.
- `<skipIfAlreadyRun>`
Skips running **Design Rule Checks** (DRCs) if they have already run.

Example

If a design is already transitioned to a **Release** level of type **Release**, and the transition is happening at another **Release** level of the same type (**Release**), the user may choose to not run them again by passing this value.

The default value is **useCapitalPreference**.

-treat_warnings_as_errors

Changes warnings from **Design Rule Checks** (DRCs) to errors.

Valid values are **true** and **false**. The default value is **false**.

Placement

Place on any action. Typically, place on the **Start** or **Perform** action of a **Do** task.

Restrictions

None.

Examples

This fetches all electrical designs for Capital for the corresponding target Teamcenter designs. It applies a **Released** status to the Capital designs that use the Capital webservice.

Arguments	Values
-annotate	false
-capital_pwd_file	Absolute path of the generated password file
-capital_status	Released
-capital_user	system
-freeze_sharedobjects_used	doNotFreeze
-run_dracs	useCapitalPreference
-treat_warnings_as_errors	false

IM1-update-session-for-machine-bom

Description

This handler updates the **Fnd0AppSession** associated with the target Machine Revision. It sets the revision rule on all BOM windows in the session to identify the released structure, then persists the session context. The handler finds **Fnd0AppSession** objects related to **Im0MachineRevision** targets via the **IMAN_reference** relation, then applies the specified revision rule to all BOM structures within each session.

Syntax

IM1-update-session-for-machine-bom

Arguments

-revision_rule

(Required) The name of the revision rule to apply to the BOM windows in the session. If the specified rule Machine Builder Clone Revision Rule is not found, it falls back to Industrial Machinery Clone Revision Rule.

Placement

Place on the Start action of the release task in Machinery Release Process and Machinery ConceptDesignRelease Process workflow templates.

Restrictions

Exactly one argument must be provided. If the argument count is not 1, error **EPM_invalid_argument** is thrown. The argument must be named **revision_rule**, otherwise **EPM_invalid_argument** is thrown. Target attachments must include **Im0MachineRevision** objects with associated **Fnd0AppSession** objects.

Examples

This example sets the Machine Builder revision rule on the session:

Argument	Values
-revision_rule	Machine Builder Clone Revision Rule

IMD0-attach-basic-di-to-target

Description

This handler attaches **Imd0EUBasicDIRecord** objects as target attachments to the workflow task. The handler finds Med Device SKU Revision objects from the target attachments, navigates to the related Product Item SKU via **Imd0ProductItemSKUrel**, then finds all EU Basic DI Records related to that Product via **Imd0DIRecordRel** and adds them as targets.

Syntax

IMD0-attach-basic-di-to-target

Arguments

None.

Placement

Place on the Start action of the root process task in the EU DI Review and Submission Process workflow template.

Restrictions

Target attachments must include Med Device SKU Revision objects. At least one EU Basic DI Record must be attached to the Product (error: **IMD0_basic_di_not_attached**).

Examples

No arguments required. The handler traces the chain: Med Device SKU Revision to Product Item SKU (via **Imd0ProductItemSKUrel**) to **Imd0EUBasicDIRecord** (via **Imd0DIRecordRel**) and adds the Basic DI Records as target attachments.

IMD0-attach-itemsku-to-target

Description

This handler attaches the corresponding Item SKU as a target attachment for the Label Design BOM. The handler identifies Label Design Revision objects in the workflow target attachments, finds the primary object related via the **Imd0DILabelDesignRel** relation, retrieves the associated Item SKU, and adds it as a target attachment.

Syntax

IMD0-attach-itemsku-to-target

Arguments

None.

Placement

Place on the Start action of the root process task in the MDLS Label BOM Transfer Process Async workflow template.

Restrictions

Target attachments must include Label Design Revision objects. Each Label Design must relate to exactly one primary object (error: **IMD0_labeldesign_has_multiple_primaryobjs** if multiple found). A primary object must exist (error: **IMD0_labeldesign_without_primaryobj** if none found).

Examples

No arguments required. The handler automatically:

1. Finds Label Design Revision objects in targets
2. Retrieves the primary object via **Imd0DILabelDesignRel**
3. Navigates to the Item SKU and adds it as a target

IMD0-attach-parents-to-target

Description

This handler adds all parent objects of the Med Device SKU Revision as target attachments to the workflow task. The handler finds Med Device SKU Revision objects from the existing target attachments, retrieves all parent items related to each, and adds them as additional targets.

Syntax

IMD0-attach-parents-to-target

Arguments

None.

Placement

Place on the Start action of the Apply Approved Status task in FDA DI Review and Submission Process, NMPA DI Review and Submission Process, IMDRF DI Review and Submission Process, and EU DI Review and Submission Process workflow templates.

Restrictions

Target attachments must include Med Device SKU Revision objects.

Examples

No arguments required. The handler automatically discovers parent objects of the Med Device SKU Revision and adds them as targets.

IMD0-check-di-prop-release

Description

This handler validates properties on Device Identifier (DI) related objects as part of a workflow task. The handler checks that required properties on the specified object type are populated before allowing the workflow to proceed. The handler supports special validation logic for specific DI object types: EU Basic DI Record (**Imd0EUBasicDIRecord**) uses dedicated Basic DI property validation; Med Device SKU Revision (**Imd0MedDevSKURevision**) locates the related Device Certificate object via the **Imd0ProductItemSKUrel** relation and validates its properties; all other types use generic property validation. Conditional property checks include Unit of Use DI Number (required only when Device Count > 1), DM DI Number and DM DI Agency (required only when DM DI Different from PDI flag is true), Sterilization Method (required only when Requires Sterilization Prior to Use flag is true), and FDA Product Code (required unless the SKU is a Kit with a BL-type premarket submission).

Syntax

IMD0-check-di-prop-release

Arguments

-include_type

(Required) The business object type name to validate (for example, **Imd0FDADIRRecord**, **Imd0NMPADIRRecord**, **Imd0IMDRFDIRRecord**, **Imd0EUBasicDIRecord**, **Imd0MedDevIdfrRevision**, **Imd0MedDevSKURevision**).

-properties

(Required) Comma-delimited list of property names to validate on the specified object type.

-from_attach

(Required) Where to find the objects: **Target** for target attachments, **Ref** for reference attachments.

Placement

Place on the Start action of the Validate Props and Check Properties tasks in FDA DI Review and Submission Process, NMPA DI Review and Submission Process, IMDRF DI Review and Submission Process, and EU DI Review and Submission Process workflow templates.

Restrictions

The specified object type must exist in the target or reference attachments, otherwise error **IMD0_no_obj_type_under_target_ref** is thrown. Invalid argument names throw **EPM_invalid_argument**.

Invalid argument values throw **EPM_invalid_argument_value**. Property validation failures throw **IMD0_validation_error** or **IMD0_eudi_validation_error** (for EU Basic DI and SKU types).

Examples

This example validates FDA DI Record properties:

Argument	Values
-include_type	Imd0FDADIRecord
-properties	imd0DINumber,imd0DIAgency
-from_attach	Target

IMD0-export-to-tcxmlfile

Description

This handler exports the target Med Device SKU Revision objects as a Teamcenter XML (TCXML) file during the workflow process. The export uses a specified transfer mode (option set) to control which data is included. The handler can optionally create a dataset from the exported TCXML file and attach it to the workflow as a reference attachment and/or to the target object using a specified relation type.

Syntax

IMD0-export-to-tcxmlfile

Arguments

-transfer_mode

(Required) The transfer mode or option set name used for the TCXML export (for example, **IMD0EUDIEExportOptionSet**, **IMD0FDADIEExportOptionSet**, **IMD0NMPADIEExportOptionSet**, **IMD0IMDRFDIEExportOptionSet**).

-file_prefix

(Optional) Prefix for the exported file name.

-create_dataset

(Optional) Flag to create a TCXML dataset from the exported file and attach it to the target object.

-attach_to_reference

(Optional) Flag to attach the created dataset to the workflow as a reference attachment.

-relation_type

(Optional) Relation type to use when attaching the dataset to the target object.

Placement

Place on the Complete action of the Export Data task in FDA DI Review and Submission Process, NMPA DI Review and Submission Process, IMDRF DI Review and Submission Process, and EU DI Review and Submission Process workflow templates.

Restrictions

Target attachments must include Med Device SKU Revision objects. The specified transfer mode must exist in Teamcenter. Invalid argument names throw **EPM_invalid_argument**.

Examples

This example exports FDA DI data and creates a dataset:

Argument	Values
-transfer_mode	IMD0FDADIEExportOptionSet
-file_prefix	FDA_DI
-create_dataset	true
-attach_to_reference	true

IMD0-validate-eudi-record-prop

Description

This handler validates required properties on EU DI Record (**Imd0EUDiRecord**) and EU Legacy DI Record (**Imd0EULegacyDiRecord**) business objects attached to the workflow task. The handler retrieves objects of the specified types from the target or reference attachments, also locates an EU Basic DI Record, then validates all specified properties using EU DI-specific validation logic.

Syntax

IMD0-validate-eudi-record-prop

Arguments

-include_type

(Required) Comma-separated list of object types to validate (for example, **Imd0EUDiRecord**, **Imd0EULegacyDiRecord**).

-properties

(Required) Comma-separated list of required property names to validate.

-from_attach

(Required) Source of attachments to check: **Target** for target attachments, **Ref** for reference attachments.

Placement

Place on the Start action of the Validate Props and Check Properties tasks in the EU DI Review and Submission Process workflow template.

Restrictions

The specified object types must exist in the target or reference attachments. Invalid argument names throw **EPM_invalid_argument**. Invalid argument values throw **EPM_invalid_argument_value**. Property validation failures throw **IMD0_eudi_validation_error**.

Examples

This example validates EU DI Record properties:

Argument	Values
-include_type	Imd0EUDIRRecord,Imd0EULegacyDIRRecord
-properties	imd0DINumber,imd0DIAgency
-from_attach	Target

Imp0LabelDesignApprove

Description

This handler creates a JSON payload for the **Imp0LabelDesignRevision** object and sends it as an Approval Notification to the Label Management Software via a web service call. The handler constructs a payload containing Label BOM path (from **LBLMGMT_Is_path_property** preference), Label Format (**imp0LabelFormat** property), and Release State (from **last_release_status**).

Syntax

Imp0LabelDesignApprove

Arguments

None. Configuration is driven by Teamcenter preferences:

- **LBLMGMT_Is_path_property**: Property name for the LS path.
- **LBLMGMT_target_software**: Target software identifier.
- **LBLMGMT_lms_url**: Label Management Software endpoint URL.
- **LBLMGMT_tcif_api_key_value**: API key for authentication.

Placement

Place on the Complete action of the Send Notification to Label Mgmt System task in the Label Design Approval Notification Process workflow template.

Restrictions

Requires valid Teamcenter preferences for the Label Management Software connection. Target attachments must include an **Imp0LabelDesignRevision** object. The Label Design must have a release status assigned.

Examples

No arguments required. The handler reads preferences for configuration and processes the **Imp0LabelDesignRevision** from target attachments.

Imp0SendToTcif

Description

This handler creates a JSON payload for **Pka0CopyAssetRevision**, **Pka0DigitalAssetRevision**, or **Imp0LabelDesignRevision** objects and sends it to the Label Management Software via a web service call (TCIF integration). The handler constructs payloads containing: for Copy Assets, LS Path, item ID, TCUID, revision ID, name, locale, master copy flag, and attached text or image content; for Digital Assets, similar to Copy Assets; for Label Design, Label BOM path, label format, and related digital asset information. The payload is sent via an HTTP POST call to the Label Management Software endpoint configured via Teamcenter preferences.

Syntax

Imp0SendToTcif

Arguments

None. Configuration is driven by Teamcenter preferences:

- **LBLMGMT_ls_path_property**: Property name for the LS path.
- **LBLMGMT_target_software**: Target software identifier.
- **LBLMGMT_lms_url**: Label Management Software endpoint URL.
- **LBLMGMT_tcif_api_key_value**: API key for authentication.

Placement

Place on the Complete action of the transfer task in Copy Asset Transfer Process Async (Transfer copy Asset task), Digital Asset Transfer Process Async (Transfer Digital Assets task), and MDLS Label BOM Transfer Process Async (Transfer Label BOM task) workflow templates.

Restrictions

Requires valid Teamcenter preferences for the Label Management Software connection (URL, API key, target software). Target attachments must include **Pka0CopyAssetRevision**, **Pka0DigitalAssetRevision**, or **Imp0LabelDesignRevision** objects.

Examples

No arguments required. The handler reads preferences for configuration and processes all qualifying target attachments.

IPM-Add-Reviewers

Description

This action handler is used to identify the users or roles mentioned Gate assignment list or default reviewer list and assign them a review action.

Syntax

IPM-Add-Reviewers <-from_attach><-reviewer_type><-default_reviewer_property>

Placement

On **Start** of **Select-signoff** task in Review Task.

Arguments

- from_attach** = target | reference
- reviewer_type**= type of reviewer eg .< Senior Management Review Team or Management Review Team >
- default_reviewer_property**= property name <eg ipmOSMEDefault>
- **include_type**= object-type
- **Notification**= <no value>
- **discipline_specification_attachment**=<discipline object>

Restrictions

None.

IPM-attach-objects-from-query

Description

This handler fetches the objects through a saved query using an attribute of target or reference object and attaches.

When executed successfully --> Output: Attaches the objects of Query result to the Target/Reference

Syntax

None.

Placement

No specific placement.

Arguments

-from_attach= target | reference

-attachment= target | reference

-query= Query name from Teamcenter

-property= Property name

-include_type= object-type

Restrictions

None

Example

Arguments	Value
-from_attach	reference
-attachment	reference
-query	getGateApprovalConfigs
-property	prg0EventId
-include_type	Prg0Event

IPM-get-all-related-objects

Description

This handler fetches the Primary or the Secondary objects related to the target or reference object and attaches them to the Target/Reference of the workflow.

When executed successfully --> Output: Attaches the objects of specific type to the Target/Reference.

Syntax

None.

Placement

Generally, Place on **Start** action of the root task, so list of attachments is updated at workflow process initiation.

No specific placement.

Arguments

-from_attach= target | reference

-to_attach= target | reference

-include_type= object-type

-include_related_type= [optional] Specific object type to be attached. They must be valid object types.

-relation= valid relation type eg. IMAN_specification

-get_related_objects= primary | secondary

-depth= Level of depth to travers

Restrictions

None.

Examples

Arguments	Value
-from_attach	target
-to attach	reference
-include_type	Pmg0ItemSKURevision
-include_related_type	Pmg0ProductRevision
-relation	Pmg0ProdItemSKUrel
-get_related_objects	primary
-depth	1

IPM-Get-Event-To-Process

Description

Handler will add Attaches Plan object as attachments.

Syntax

None.

Placement

On **Start** Action of do-task.

Arguments

-from_attach= target | reference

-include_type= object-type

Restrictions

None.

IPM-send-alert

Description

This Action handler send Alert to the assigned user.

Creates createNotificationAlert

Syntax

None.

Placement

On **Start** Action of Perform Signoff.

Arguments

-subject_Key = <ipm0_alert_msg_subject_wf>

-msg_text_Key = <ipm0_alert_msg_body_wf >

-object_from_attach= target | reference

-object_type = object-type

- notifier = TARGET_OWNER | PROCESS_OWNER | PROPOSED_RESPONSIBLE_PARTY | USER | PROJECT_ADMINISTRATOR | PROJECT_AUTHOR | PROJECT_MEMBER etc.

Restrictions

None.

Examples

Arguments	Value
-subject_Key	ipm0_alert_msg_subject_wf
-msg_text_Key	ipm0_alert_msg_body_wf
-object_from_attach	target
-object_type	lcp0Campaign
- notifier	PROCESS_OWNER

IPM-set-completeness

Description

This is Handler Function to set completeness on target or reference based on the value given for object_from_attach. If no value is given it will consider target as default value.

Syntax

None.

Placement

Generally, on **Start** Action of Task.

Arguments

-value= Incomplete, Sufficient

-object_from_attach= target | reference

Restrictions

None.

IPM-set-conditional-status-false

Description

This is Handler Function to set Conditional Release property value to false.

Executed successfully: Conditional Release Property set to false.

Syntax

None.

Placement

No specific placement.

Arguments

-attachment= target <prop is on which object>

-conditional_release_property= <ipm0ConditionalRelease>

Restrictions

None.

IPM-set-conditional-status-true

Description

This is Handler Function to set Conditional Release property value to true.

Executed successfully: Conditional Release Property set to true.

Syntax

None.

Placement

No specific placement.

Arguments

-attachment= target

-conditional_release_property= <ipm0ConditionalRelease>

Restrictions

None.

IPM-set-due-date

Description

This action handler is used to set Due date on target Task.

Due Date is Fetched from Attachments. Prop --> < ipm0DueDate >

Syntax

None.

Placement

No specific placement.

Arguments

No Arguments.

Restrictions

None.

IPM-Update-Event

Description

Handler will use an attribute to filter a set of events related to the plan object and pick the non-started or in-progress event based on the planned date.

Syntax

IPM-Update-Event

Placement

No specific placement.

Arguments

-from_attach = <target / reference>

-include_type = object-type <e.g. – Itemrevision, Prg0ProgramPlan>

-property = <Prop_name>

-value = <Prop_value>

Restrictions

None.

IPM-Update-Maturity

Description

Action handler is used to update maturity of the plan object and discipline specification objects.

If successfully executed: Updates maturity of the plan object and discipline specification objects from the values mentioned in gate approval config object.

Syntax

None.

Placement

On **Start** action of Do-task or empty Task

Arguments

-**discipline_attachment** = target | reference

-**plan_attachment** = target | reference

-**gate_config_attachment** = target | reference

Restrictions

None.

ISSUEMGT-check-review-decision

Description

Checks issue review records for a target issue report revision when the specified review decision is made. If no issue review record is found for the issue report revision contained as a target of the workflow, the signoff decision is reset to **No Decision**. The user is prompted to choose **Tools→Review Issue** to review the issue and record a decision.

Syntax

ISSUEMGT-check-review-decision=<review-decision-type>

Arguments

<review-decision-type>

Specifies which type of signoff decision prompts the system to check the issue review record for the issue report revision. It accepts one of the following values:

- Approve** Issue review records are checked for a target issue report revision when the user approves the signoff.
- Reject** Issue review records are checked for a target issue report revision when the user rejects the signoff.

Placement

Place on the **Perform** action of the **perform-signoffs** task.

Restrictions

None.

Examples

- In this example, issue review records are checked for a target issue report revision when the user approves the signoff. If no issue report revision is found for the target, the signoff is reset to **No Decision**. The user is prompted to choose **Tools→Review Issue** to review the issue and record a decision.

Argument	Values
review-decision-type	-Approve

- In this example, issue review records are checked for a target issue report revision when the user rejects the signoff. If no issue report record is found for the target issue report revision, the signoff is reset to

No Decision. The user is prompted to choose **Tools→Review Issue** to review the issue and record a decision.

Argument	Values
review-decision-type	-Reject

- In this example where no argument is given, issue review records are checked for a target issue report revision when the user performs the signoff, either approving or rejecting it. If no issue report record is found for the target, the signoff is reset to **No Decision**. The user is prompted to choose **Tools→Review Issue** to review the issue and record a decision.

ISSUEMGT-update-issue-status

Description

Counts the issue review decisions from all reviewers and updates the issue status. It takes inputs such as decision type, passing threshold, and the list of issue attribute/value pairs to update when a review decision passes. If you use the **-force_set_properties** argument, the review decision does not need to be passed to update the issue status. You can optionally clean up review records after they are counted and the issue status is updated. It sets a condition when configured with a **Condition** task.

Syntax

```
ISSUEMGT-update-issue-status -review_decision=<decision-string> -threshold=<percentage-passes> -set_condition [-force_set_properties] [-<attribute-name>=<attribute-value>] [-clean_up_review_records]
```

Arguments

-review_decision

Specifies the issue review decision. It accepts one of the following values:

- defer
- reject
- approveFix
- close
- reopen
- approveIssue

-threshold

Sets the percentage required to approve the review decision.

For example, **-threshold=51** means that the review decision passes with a 51 percent majority.

-set_condition

Sets the **Condition** task to **TRUE** if the review decision passes.

-force_set_properties

Forces the issue attributes to be set regardless if review decisions are counted or if review decision passes.

-<attribute-name>

Updates the specified attribute with the specified value when the review decision passes. You can specify more than one attribute and value pair.

-clean_up_review_records

Cleans up review records after they are counted and the issue status is updated.

Placement

Place in any workflow task.

Restrictions

If the **-review_decision** argument is set for this handler and the **-force_set_properties** is not set, Siemens Digital Industries Software recommends placing the **ISSUEMGT-check-review-decision** action handler on a previous **perform-signoffs** task to ensure that review decisions are logged from all reviewers.

LDF-create-object

Description

Creates an object in the remote system and relates it to the workflow attachment.

Syntax

```
LDF-create-object
service_provider
-object_type
[-property::<<oslc-namespace-prefix-url>.property-name>]
[-from_attach]
-attachment_relation
[-remote_user_name]
```

Arguments

-service_provider

Service provider represents the services published by the external application.

Example

Polarion is registered in Teamcenter as a site and service provider under which my objects will be created.

This is a mandatory argument.

The values specified for this argument can be dynamic. Users can configure the handler argument to read the property values from workflow attachments and substitute them as the argument values. For example, **-service_provider=PROP::owning_project** where **owning_project** is the property of the Teamcenter workflow attachment. If corresponding **service_provider** is not found, this handler returns an error.

-object_type

This argument specifies the type of object created in the remote system.

This is a mandatory argument.

-property::<<oslc-namespace-prefix-url>.property-name>

Specifies the property name for the remote object to be created.

Requires a fully qualified property name with a prefix URL prepended to every property in a workflow argument, which is prepended by **-property::**. The OSLC namespace prefix URL must be contained in

angle brackets, < and >, in the **<oslc-namespace-prefix-url><.property-name>** format as shown in the examples section.

The values specified for this argument can be dynamic. User can configure the handler argument to read the property values from workflow attachments and substitute it as the argument value.

For example, **-property::<http://purl.org/dc/terms/>.title =PROP::object_name** where **object_name** is the property of the Teamcenter workflow attachment. User can also configure prefix or suffix.

The dynamic property values can also have prefix or suffix. For example, **-property::<http://purl.org/dc/terms/>.title =ABC PROP::object_name XYZ** ABC is the prefix, < PROP::object_name> is the dynamic value from Teamcenter object, and XYZ is the suffix.

-from_attach

target | reference

(Optional) Specifies which type of attachment (target or reference) to get the property value from when a property is specified in the **-property::<oslc-namespace-prefix-url>.property-name** argument. For example, **-property::<http://purl.org/dc/terms/>.title=PROP::object_name** where **<object_name>** is the property of the Teamcenter workflow attachment.

You can use this argument only when you get the property value from a property of the attachment object.

-attachment_relation

Specifies the relation name linking the remote object with the target. This relation name should match a relation name configured in Linked Data Framework. Refer to *Integrating Applications Using Linked Data Framework > Define the relations to apply when creating remote links*.

This is a mandatory argument.

-remote_user_name

Used by the handler to connect to a remote system like Polarion for sending HTTP requests.

The Restrictions section below describes separate actions required to generate an encrypted password file.

Note

This argument is optional with SSO.

Placement

Place on the **Start** or **Complete** action.

Note

Do not place on a **Perform** action requiring specific user interaction. Placement on the **Perform** action may cause the handler to be triggered multiple times.

Restrictions

Use if you are using the Linked Data Framework for application integrations, and you want Teamcenter workflows to create an object in a remote system.

You must generate an encrypted password file by following these steps in a Teamcenter command shell:

1. Run this command:

```
mkdir %TC_DATA%\polarionconnector
```

2. Run this command:

```
%TC_ROOT%\bin\install -encryptpwf -f=%TC_DATA%\polarionconnector\ <<user name>>
```

Where <<user name>> is user name of remote system such as Polarion ALM. This user name should be configured as a value of the **-remote_user_name** handler.

Examples

- This example shows the **LDF-create-object** handler configuration to create an object in the remote system of type **changerequest**, and attaching the remote link of this object with target by **Lcm0RelatedChangeRequest** relation. Uses service provider and title values from target object properties **object_desc**, **object_name**, respectively.

Argument	Values
-service_provider	PROP::object_desc
-object_type	changerequest
-property:: <http://purl.org/dc/terms/>.title	PROP::object_name
-from_attach	target
-attachment_relation	Lcm0RelatedChangeRequest
-remote_user_name	admin

- This example shows the **LDF-create-object** handler configuration to create an object in the remote system of type **issue**, and attaching the remote link of this object with target by **Lcm0AffectedByDefect** relation. Uses title and description values from target object properties **object_name**, **object_desc** respectively.

Argument	Values
-service_provider	Drive Pilot
-object_type	issue
-property:: <http://purl.org/dc/terms/>.title	PROP::object_name
-property:: <http://purl.org/dc/terms/>.description	PROP::object_desc
-from_attach	target
-attachment_relation	Lcm0AffectedByDefect
-remote_user_name	admin

LDF-set-task-result-to-property

Description

LDF-set-task-result-to-property reads the specified property from the remote object. **LDF-set-task-result-to-property** uses that property value to set the result string attribute of the task where this handler is located, or on the task specified by the **-target_task** argument. A common use for this handler is to control **Condition** task branching instead of using a scheme that requires a custom handler. Using this handler to set the result attribute of a **Condition** task branches the workflow process based on a remote property of the target source object.

Syntax

```
-property
-source
[-attachment_relation]
[ -include_type ]
[ -target_task ]
[ -remote_user_name ]
```

Arguments

-property

Specifies the property to be read from the identified remote object attached to the target with specified relation.

The values specified for this argument require a fully qualified property name with a prefix URL prepended to every property in a workflow argument value. The OSLC namespace prefix URL must be contained in angle brackets in the **<oslc-namespace-prefix-url > -property-name** format as shown in the **Examples** section below.

This is a mandatory argument.

-source

Determines which source object identifies the remote object property. Source object values are either **target** or **reference**. The remote object property is identified in the **-property** argument.

- **target**

Declares that the remote object property is read from a target object. The **-include_type** argument specifies the target object type to use.

- **reference**

Declares that the remote object property is from a reference object. The **-include_type** argument specifies the reference object type to use.

-attachment_relation

Specifies the relation name to expand to get a linked object from a workflow attachment. Linked objects, attached to targets and references in a workflow with the relation specified by **-attachment_relation**, are searched. Linked objects not matching the specified relation criteria are ignored.

This is an optional argument.

-include_type

Identifies the source type to read the specified property of the remote object. The property name is defined in the **-property** argument. If more than one target object of a given type exists, the first target on the list is used. If a valid **-include_type** argument is absent, the property is read from the first target on the list.

-target_task

Identifies where the result string attribute is set. If not specified, then the task result attribute is set for the task containing this handler.

This is an optional argument.

- **\$ROOT_TASK**—Sets the result string attribute on the root task of the process.
- **\$DEPENDENT_TASK**—Sets the result string attribute on the parent process task which is dependent on this subprocess. The parent process task should be a **Condition** task.

-remote_user_name

Used by the handler to connect to a remote system, like Polarion, for sending HTTP requests.

The **Restrictions** section below describes separate actions required to generate an encrypted password file.

Note

This argument is optional with SSO.

Placement

Typically placed on the **Start** action of the specified **Condition** task.

Note

You can apply the **LDF-set-task-result-to-property** handler to *any* task, but it sets the result on either a root or **Condition** task.

The **Condition** task can contain the handler or be a parent of another dependent task that contains the handler.

Restrictions

- Do not place this handler on the **Perform** action.

- Do not use this handler in conjunction with other handlers that set the **result** attribute, such as **EPM-set-condition**, **EPM-set-parent-result**, or a custom handler.
- You can use this handler on the **Complete** action only if a change occurred on the **Perform** action.
- This handler allows you to set the **result** attribute on the root task or any other **Condition** task.
- Use if you are using the Linked Data Framework for application integrations, and you want Teamcenter workflows to create an object in a remote system. You must generate an encrypted password file in a Teamcenter command shell. To create an encrypted password:
 1. Run this command: `mkdir %TC_DATA%\polarionconnector.`
 2. Run this command: `%TC_ROOT%\bin\install -encryptpwf -f=%TC_DATA%\polarionconnector\<user name>.`

Where **<user name>** is the user name of a remote system such as Polarion ALM. Configure this user name as a value of the **-remote_user_name** handler.

Examples

This **LDF_set_task_result_to_property** handler configuration branches a **Condition** task based on the remote object property **Priority**, which is attached to a target change request revision with the relation **Lcm0RelatedChangeRequest**.

Argument	Values
-property	<http://polarion.plm.automation.siemens.com/oslc#>.priority
-source	target
-include_type	ChangeRequestRevision
-attachment_relation	Lcm0RelatedChangeRequest
-remote_user_name	admin

SUBSCMPL-List-Vendor-parts-and-vendors

Description

The **SUBSCMPL-List-Vendor-parts-and-vendors** handler is used by the **Request Substance Declaration** workflow.

For a given item revision or a BOM, the action handler:

- Traverses the item revision or BOM attached to the workflow process and extracts (retrieves) all those vendor parts that are related to the item revision or the BOM using the **VMRepresents** relation. For a conflict mineral declaration request for a particular vendor, the handler retrieves a vendor object.
- The handler then saves the retrieved information in the **Scp0ReqSubsDeclForm** form.

Caution

Do not add this handler to any other workflow process template.

Syntax

SUBSCMPL-List-Vendor-parts-and-vendors

Arguments

None.

Placement

By default, this handler is placed in the correct location of the **Request Substance Declaration** workflow process template. Do not change the placement.

Restrictions

Adding this handler to any other workflow process template may result in errors.

MDL-attach-changes-to-baselines

Description

For all change item revisions that are targets of the root task, this handler finds any baseline revisions in the **Reference Items** folder and attaches the change item revision as a reference to the baseline.

If the attachment fails for any reason, an error is returned.

Syntax

MDL-attach-changes-to-baselines

Arguments

None.

Placement

Requires no specific placement.

Restrictions

None.

MDL-attach-subset-definition-changes

Description

Compares the **mdl0HistorySyncStatus** property for the content of all target subset definitions. Where content is out of synchronization, the handler adds the content to the workflow as a target.

An **Mdl0ModelElement** business object is in sync whenever the **mdl0HistorySyncStatus** property value is empty ("").

Examine both the latest-history and latest configurations for both content and partitions. This is required to get the correct promote-to-history of obsoleted or configured-out content.

Syntax

MDL-attach-subset-definition-changes [-partition=[<scheme1, [scheme2, ...]> | [* | all | any]

Arguments

-partition

(Optional) Attaches the required partitions from the subset definition content up to the root partitions. You can specify multiple partition schemes by name, all partitions, or any partition.

If the **-partition** argument is used, partitions in the specified partition schemes are also attached if the partition is:

- Configured by the subset definition.
- Itself is out of sync.
- Lies on the path from the subset definition content to the root partitions.

Placement

Place before the **MDL-promote-objects-to-history** handler to synchronize the subset definition content with the history.

Restrictions

None.

MDL-promote-objects-to-history

Description

Promotes all targets and any related objects to history. For non-revisable targets, this handler checks the maturity status for object stability.

If the target object is revisable, the logical object is copied to POM history and its references are checked for stability.

If the target object is not revisable, it is checked for stability

Syntax

MDL-promote-objects-to-history

Arguments

None.

Placement

Requires no specific placement.

Restrictions

None.

MDL-snapshot-baseline-revisions

Description

Sets the snapshot date and closure state for all baseline revisions that are targets of the root task.

Deleting and re-adding the baseline snapshot date is an alternative to revising a baseline. It avoids creating additional objects, but does not allow a record to be kept of the failed baseline attempt. The choice between a re-open versus a revise step is a business decision, and it is expected to be formally designed as a workflow.

Siemens Digital Industries Software recommends that you use a workflow action handler to close a baseline before performing signoffs. Once signoffs are complete, we recommend using another workflow action handler to assign a status to the baseline.

Syntax

MDL-snapshot-baseline-revisions -snapshot = add | replace | delete -closure=<name >]

Arguments

-snapshot

Sets the baseline revision snapshot date. The value can be one of the following:

- **add**

Ensure the baseline revision has a snapshot date.

If the baseline revision does not have a snapshot date, it is set to the current date.

If the baseline revision already has a snapshot date, the snapshot date is unchanged.

- **replace**

Sets the baseline revision snapshot date to the current date.

- **delete**

Sets the baseline revision snapshot date to **null**.

-closure

Sets the baseline revision closure property to the specified value.

Placement

Requires no specific placement.

Restrictions

None.

MDT-attach-related-assignment-matrix

Description

Finds and attaches assignment matrix to workflow reference by navigating to parent MDR/Project of deliverables.

Syntax

```
MDT-attach-related-assignment-matrix
[-application_context=<assignment-matrix-app-context>]
[-target_type=<comma-separated-object-type-list>]
```

Arguments

- application_context=<assignment-matrix-app-context>
Specifies the application context of an assignment matrix
- target_type=<comma-separated-object-type-list>
Specifies the real name of the business object types (Sub-type of Workspace Object) on which decisive properties are identified. It supports multiple inputs including list of object types separated by comma.

Placement

No specific placement is required. The handler is typically placed on the **Start** action of the task.

Restrictions

None.

Examples

Example 1

This example identifies assignment matrix with application context MDR&T on the object type Mdt0MDRDelRevision and attaches the assignment matrix into the reference attachment of the workflow.

Argument	Values
-target_type	Mdt0MDRDelRevision
-application_context	MDR&T

MDT-attach-related-deliverables

Description

Assigns tasks in the workflow process to the specified user defined on entries of a related assignment matrix.

Syntax

MDT-attach-related-deliverables
[-attachment=<target | reference>]

Arguments

-attachment=<target | reference>
Specifies the workflow attachment type.

Placement

No specific placement is required. The handler is typically placed on the **Start** action of the task.

Restrictions

None.

Examples

Example 1

This example finds and attaches deliverable revisions from transmittal revision to workflow target.

Argument	Values
- attachment	target

Examples

Example 2

This example finds and attaches deliverable revisions of transmittal revision to workflow reference.

Argument	Values
- attachment	reference

MDT-process-content-load-request

Description

Processes the Content Load Request. Mapping File, which is related to Content Load Request, contains the information of the Primary Objects, Secondary Objects and Datasets. This handler creates primary objects and secondary objects and creates and relates datasets to the secondary object based on the Tc_File_Name column in the mapping file

Syntax

MDT-process-content-load-request

Arguments

None

Placement

No specific placement is required. The handler is typically placed on the **Complete** action of the task.

Restrictions

None.

MDT-set-release-status-on-deliverable-instances

Description

Sets *approval_status* on deliverable revisions and *approval_status* or *rejection_status* on deliverable instances from the transmittal revision based on transmittal response against each deliverable instance.

If response for a deliverable instance is approved, deliverable instance receives status mentioned in the argument *-approval_status* .

If response for a deliverable instance is rejected, deliverable instance receives status mentioned in the argument *-rejection_status*.

Syntax

MDT-set-release-status-on-deliverable-instances

```
[-approval_status=<approval-release-status>]
[-rejection_status=<rejection-release-status>]
[-del_instance_relation=<relation-name>]
[-release_datasets]
[-revise_del_instances]
```

Arguments

-approval_status=<approval-release-status>

Approval status to be applied on deliverables and deliverable instances in case all deliverable instances receive approved response

-rejection_status=<rejection-release-status>=

Rejection status to be applied on deliverables and deliverable instances in case all deliverable instances receive rejected response

-del_instance_relation=<relation-name>

Real name of the relation between deliverables and deliverable instances. It supports multiple values separated by comma.

-release_datasets

(Optional) Specifies whether to apply status on datasets associated with the deliverable instances with *approval_status/rejection_status* This argument does not have any value. The argument itself acts as a flag.

-revise_del_instances

(Optional) Specifies whether to revise rejected deliverable instances and attach them to the respective Deliverable(s) automatically. This argument does not have any value. The argument itself acts as a flag.

Placement

No specific placement is required. The handler is typically placed on the **Complete** action of the task.

Restrictions

None.

Examples

This example sets Approved or Rejected status mentioned in *approval_status* and *rejection_status* on the deliverables and deliverable instances related to deliverables by either relation **Psi0DelInstances**, **Wbs0HasDeliverableRel** based on the transmittal response.

Argument	Values
-approval_status	Approved
-rejection_status	Rejected
-del_instance_relation	Psi0DelInstances, Wbs0HasDeliverableRel

MDT-set-transmittal-sender-property

Description

Set the Initiator of the workflow on the property transmittal sender on transmittal Item of the transmittal revision.

Syntax

MDT-set-transmittal-sender-property

Arguments

None

Placement

No specific placement is required. The handler is typically placed on the **Start** action of the task.

Note

It is recommended to add this handler after attaching the Transmittal Item in target attachments by using handler **EPM-attach-related-objects** to avoid access issues on Transmittal Item.

Restrictions

None.

MDT-update-progress-from-roc

Description

Updates the Internal or External progress values on Deliverable Revisions from the Transmittal using Rules of Credit (RoC) associated with parent MDR or Program or Project or Sub-project. The deliverable progress will only be updated for those deliverables that will have the release status mentioned in argument **-valid_deliverable_status..**

Syntax

MDT-update-progress-from-roc
[-valid_deliverable_status=<release-status-list >

Arguments

-valid_deliverable_status=<release-status-list>

Real name of release status on Deliverable Revisions for which the progress from ROC has to be updated. Only Deliverable Revisions released with one of the valid release statuses will be processed. It supports multiple values separated by comma.

Placement

No specific placement is required. The handler is typically placed on the **Complete** action of the task.

Restrictions

None.

Examples

Example 1

In this example progress will be updated from ROC will only on those Deliverables which have Approved release status

Argument	Values
-target_type	Mdt0MDRDelRevision
-application_context	MDR&T

ME-create-mirror-mbom-AH

Description

Creates a single manufacturing bill of materials (MBOM) master root from a single engineering bill of materials (EBOM) master root. It allows different item types in the MBOM than were in the EBOM for certain nodes based on custom logic. By default, the MEBOM_* preferences listed set the item type to be created.

- MEMBOM_Mirror_MakeRules
- MEMBOM_Mirror_RemoveLineWithIDIC
- MEMBOM_Mirror_ReplaceMakeOnChange
- MEMBOM_Mirror_TypePrefixSuffix

Note

This utility requires a mfg_mbm_author or manufacturing_author license.

You can also customize the item type to be created using the **USER_create_or_ref_item** user exit operation exposed in the Business Modeler IDE through **BMF_ITEM_create_or_ref_id** on the item.

Attachments that are associated with item revisions in the EBOM structure are carried forward. A user exit operation (**USER_sync_item**) must also be available on the item to align any additional attachment information or non-occurrence properties. The user exit is exposed in the Business Modeler IDE through **BMF_ITEM_sync** on the item.

The target must be an item or item revision or a structure context object. The top line of the structure is where the creation starts.

Syntax

ME-create-mirror-AH

```
[-revrule=<revision-rule>]
[-mbomrevrule=<mbom-revision-rule>]
[-depth=<depth>]
[-clientdata=<data>]
[-actiononrelease= {<1> | <2> | <3> | <4>}]
[-mscuid=<UID>]
[-usemfk= {0 | 1}]
[-log=<log-file>]
```

Arguments

-revrule

Specifies the revision rule of the EBOM structure used to traverse. This argument is mandatory only if the target is an item or item revision. Do *not* use this argument if the target is a structure context object.

-mbomrevrule

Specifies the revision rule for the MBOM structure. This argument is mandatory only if the target is a an item or item revision. Do *not* use this argument if the target is a structure context object.

-depth

(Optional) Specifies the depth up to which to create the MBOM nodes.

If you do not specify this value, Teamcenter creates all of the MBOM nodes.

-clientdata

(Optional) Data to be passed to any custom-registered user exit functions defined on the item.

-actiononrelease

(Optional) Specifies a value indicating the action to be taken if an MBOM node already exists (has a computed ID linked to the EBOM) and is released. Possible values are:

- 1 – Skip (the default).
- 2 – Revise and modify,
- 3 – Update properties on the released item.
- 4 – Update properties on the MBOM and its children.

-mscuid

Specifies the UID of the structure context object for the MBOM structure if the **mbomrevrule** is not suitable (for example, it is a private revision rule).

-log

(Optional) Specifies the absolute path and name of the log file to capture details about the nodes created.

-usemfk

(Optional) Specifies whether to include the **MEMBOM_Mirror_TypePrefixSuffix** preference value with the EBOM item identifier as a multifield key that becomes the new MBOM item identifier. Values are:

- | | |
|---|--|
| 0 | Do not include the preference value in the multifield key. This value is the default. |
| 1 | If the preference defines an item type without a prefix or a suffix, include the value in the multifield key. This value is ignored if the preference also defines a prefix or a suffix. |

Placement

Requires no specific placement.

Restrictions

None.

Examples

The following examples of specifying arguments for the **ME-create-mirror-mbom-AH** action handler demonstrate its use and the differences in output caused by changing the arguments. The initial values of the preferences are as follows.

- MEMBOM_Mirror_MakeRules=KEY:Usage_MakeOrPurchase,VALUE:Make
|VALUE:Phantom
- MEMBOM_Mirror_RemoveLineWithIDIC=false
- MEMBOM_Mirror_TypePrefixSuffix=Company,M_
- MEMBOM_Mirror_ReplaceMakeOnChange=false
- Create an MBOM

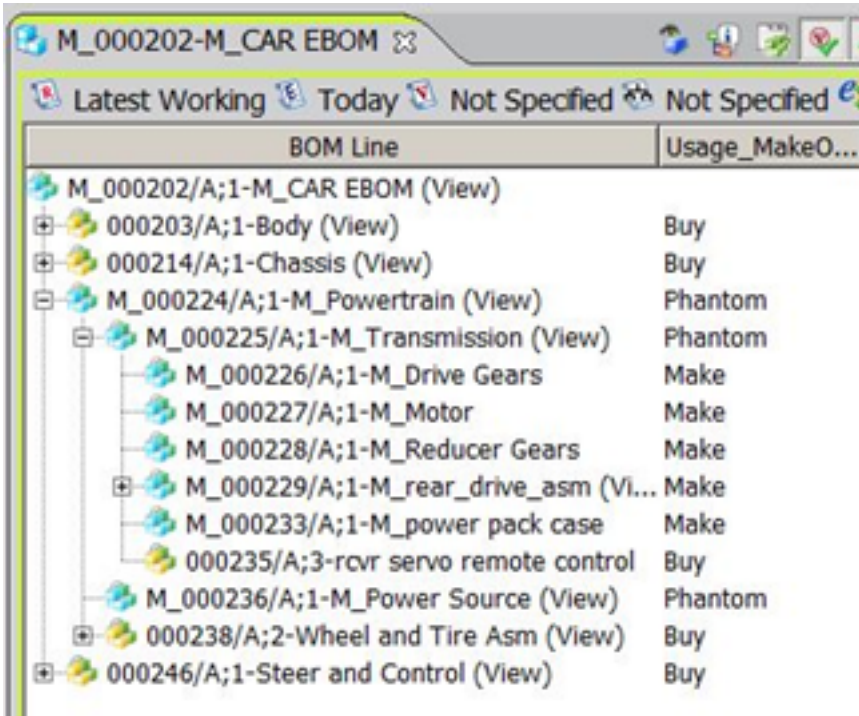
For the following EBOM, which is the target of the workflow:

BOM Line	Usage_MakeOrPurc...
000202/A;1-CAR EBOM (View)	
000203/A;1-Body (View)	Buy
000214/A;1-Chassis (View)	Buy
000224/A;1-Powertrain (View)	Phantom
000225/A;1-Transmission (View)	Phantom
000226/A;3-Drive Gears	Make
000227/A;3-Motor	Make
000228/A;3-Reducer Gears	Make
000229/A;2-rear_drive_asm (View)	Make
000230/A;2-rear_axle	Buy
000231/A;2-rear_drive_housing	Make
000232/A;2-main_drive_gear	Buy
000233/A;3-power pack case	Make
000235/A;3-rcvr servo remote control	Buy
000236/A;1-Power Source (View)	Phantom
000237/A;2-battery pack	Buy
000238/A;2-Wheel and Tire Asm (View)	Buy
000246/A;1-Steer and Control (View)	Buy

The following arguments on **ME-create-mirror-mbom-AH**:

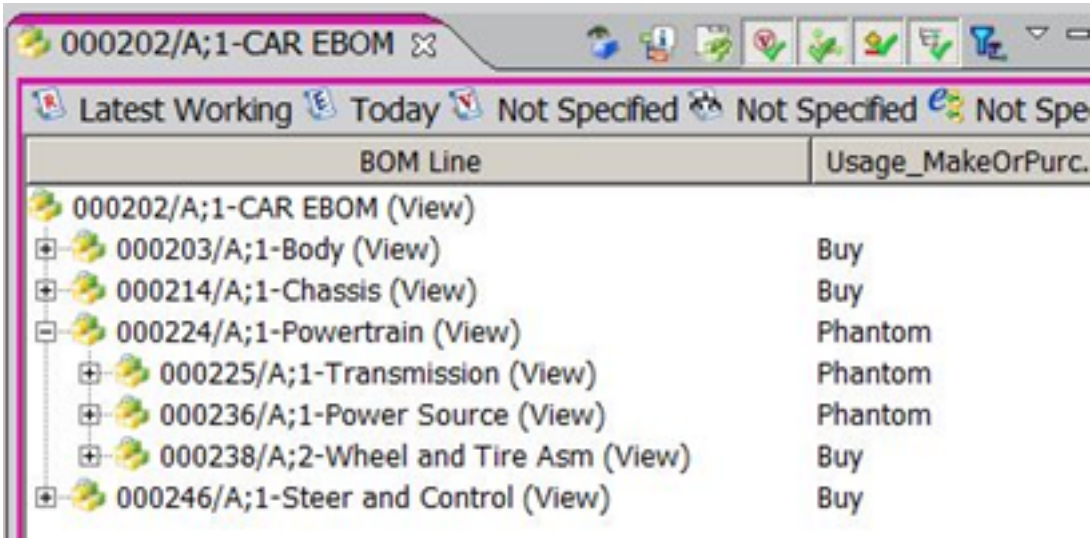
Argument	Values
-revrule=	"Latest Working"
-mbomrevrule=	"Latest Working"

Produce the following MBOM:



- Create the MBOM to a specific level

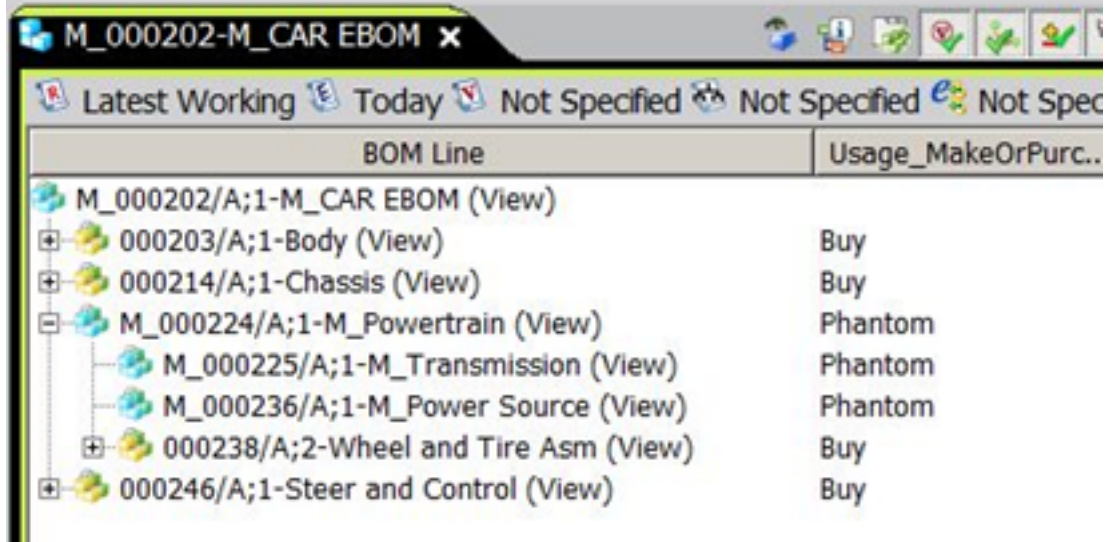
For the following EBOM, which is the target of the workflow:



The following arguments:

Argument	Values
-revrule=	"Latest Working"
-mbomrevrule=	"Latest Working"
-depth=	2

Produce a new MBOM, which contains only two levels of structure. The remaining levels in the EBOM are ignored.



The screenshot shows a software window titled "M_000202-M_CAR EBOM". Below the title bar is a toolbar with icons for "Latest Working", "Today", "Not Specified", and "Not Specified". The main area displays a BOM table with two columns: "BOM Line" and "Usage_MakeOrPurc..". The BOM structure is as follows:

BOM Line	Usage_MakeOrPurc..
M_000202/A;1-M_CAR EBOM (View)	
000203/A;1-Body (View)	Buy
000214/A;1-Chassis (View)	Buy
M_000224/A;1-M_Powertrain (View)	Phantom
M_000225/A;1-M_Transmission (View)	Phantom
M_000236/A;1-M_Power Source (View)	Phantom
000238/A;2-Wheel and Tire Asm (View)	Buy
000246/A;1-Steer and Control (View)	Buy

ME-create-revision-change-XML-AH

Description

Creates a revision change delta XML file. The manufacturing change notice (MCN) revision contains the item revisions to find revision changes. The configuration context object supplies the current configuration, and the MCN can optionally have a **was** configuration set on it. The generated XML file is attached to the request object.

Syntax

ME-create-revision-change-XML-AH
[-filename=<file-name>]

Arguments

(Optional) -filename=<file-name>

When you specify the -filename= argument, the system uses it as a base name; however, the actual filename is **RevisionChangeXML<basename-randomstring>.xml**

References

- (Required) MCN revision object.
- (Required) Configuration context (execution plan type) object.

Targets

(Required) Request object.

Placement

Requires no specific placement.

Restrictions

None.

ME-mbom-resolve-AH

Description

Searches the specified engineering bill of materials (EBOM) for parts that resolve the search recipes defined in the target (root) manufacturing bill of materials (MBOM) and assigns them to the MBOM.

You can choose the scope of the resolution and whether to recursively resolve all nodes underneath the selected scope (**-recurse**) and remove previously assigned parts. Because you most often define the root of the EBOM as the target, be sure to set the **-recurse** argument to **1** to resolve the entire structure.

Syntax

ME-mbom-resolve-AH

```
[-itemid=<UID> | -scuid=<in-context-ID> | -key=<multi-field-key-of-structure-root >]
[-revrule=<revision-rule>]
[-mbomrevrule=<revision-rule>]
[-log=<log-file>]
[-scopeid=<scope-ID> | -scopeidincontext=<scope-in-context-ID> |
-scopekey=<multi-field-key-of-structure-root>]
[-mscopeid=<UID> | -mscopeidincontext=<scope-in-context ID> |
-mscopekey=<multi-field-key-of-structure-root>]
[-recurse=1 | 0]
[-removepreviousresolvednodes=1 | 0]
```

Arguments

-itemid

(Optional) Specifies the root of the EBOM structure to be searched.

One of the **-itemid**, **scuid**, or **key** arguments is mandatory. Therefore, do *not* use if you define a structure context or a key.

-scuid

(Optional) Specifies the structure context capturing the root of the EBOM structure and configuration to be searched.

One of the **-itemid**, **-scuid**, or **-key** arguments is mandatory. Therefore, do *not* use if you define an item or item revision or a key.

-key

(Optional) Specifies the key of the top line of the root EBOM structure to be searched when multiple attributes are used to form the unique item ID. Use the following format:

```
[keyAttr1=keyVal1] [, keyAttr2=keyVal2]...[, keyAttrN=keyValN]
```

One of the **-itemid**, **-scuid**, or **-key** arguments is mandatory. Therefore, do *not* use if you define an item or item revision or structure context.

-revrule

(Optional) Specifies the revision rule of the EBOM structure to be searched. This argument is mandatory only if the EBOM is an item or item revision or key. Do *not* use if the target is a structure context object.

-mbomrevrule

(Optional) Specifies the revision rule for the MBOM structure where the recipes are defined. This argument is mandatory only if the target is *not* a structure context object.

-log

(Optional) Specifies the absolute path and name of the log file to capture details.

-scopeid

(Optional) Specifies the item ID in the EBOM from which to begin the search. This argument cannot be used with **scopeidincontext** or **scopekey**.

If you do not specify this value, Teamcenter begins searching at the top line of the EBOM.

Select one of the **-scopeid**, **-scopeidincontext**, or **-scopekey** arguments. Do *not* use if you define a structure context or a key.

-scopeidincontext

(Optional) Specifies the ID in top level context in the EBOM from which to begin the search. This argument cannot be used with **scopeid**.

If you do not specify this value, Teamcenter begins searching at the top line of the EBOM.

Select one of the **-scopeid**, **-scopeidincontext**, or **-scopekey** arguments. Do *not* use if you define an item or item revision or a key.

-scopekey

(Optional) Specifies the IDIC of the line in the EBOM from which to begin the search. This argument cannot be used with **scopeid**.

If you do not specify this value, Teamcenter begins searching at the top line of the EBOM.

Select one of the **-scopeid**, **-scopeidincontext**, or **-scopekey** arguments. Do *not* use if you define an item or item revision or structure context.

-mscopeid

(Optional) Specifies the item ID in the MBOM to resolve, for example, if you want to resolve for a particular phantom node. This argument cannot be used with **mscopeidincontext**.

If you do not specify this value, Teamcenter resolves at the top line of the MBOM.

Select one of the **-mscopeid**, **-mscopeidincontext**, or **-mscopekey** arguments. Do *not* use if you define a structure context or a key.

-mscopeidincontext

(Optional) Specifies the ID in top level context in the MBOM to resolve, for example, if you want to resolve for a particular phantom node.

If you do not specify this value, Teamcenter resolves the recipes starting at the top line of the MBOM.

Select one of the **-mscopeid**, **-mscopeidincontext**, or **-mscopekey** arguments. Do *not* use if you define an item or item revision or a key.

-mscopekey

(Optional) Specifies the IDIC of the line in the MBOM to resolve, for example, if you want to resolve for a particular phantom node. This argument cannot be used with **mscopeid**.

If you do not specify this value, Teamcenter resolves the recipes starting at the top line of the MBOM.

Select one of the **-mscopeid**, **-mscopeidincontext**, or **-mscopekey** arguments. Do *not* use if you define an item or item revision or structure context.

-recurse

(Optional) Specifies whether to resolve all nodes under the specified scope node. Valid values are **1** and **0**. The default value is **0** meaning Teamcenter only resolves the recipes at the specified scope node.

-removepreviousresolvednodes

(Optional) Specifies whether to remove the previously assigned parts. Valid values are **1** and **0**. The default value is **0** meaning Teamcenter does not remove parts that have already been resolved in the MBOM.

Placement

Requires no specific placement.

Restrictions

None.

Examples

The following arguments search the EBOM (**000202/A;1-CAR_EBOM**) for parts that resolve the recipes defined at node **000224/A** of the target MBOM (**M_000202-M_CAR_MBOM**) and assigns them to the MBOM:

Argument	Values
-itemid=	000202
-revrule=	"Latest Working"
-mbomrevrule=	"Latest Working"
-mscopeid=	000224
-recurse	1

The image shows two SAP BOM views side-by-side. The left view is titled '000202/A;1-CAR EBOM' and the right view is titled 'M_000202-M_CAR MBOM'. Both views have a table with columns 'BOM Line' and 'Usage'. The left view shows a hierarchical structure of assemblies, with '000224/A;1-Powertrain' highlighted by a red box. The right view shows a similar structure, with 'M_000224/A;1-M_Powertrain' highlighted by a red box. A purple arrow points from the left view's red box to the right view's red box, indicating a mapping or resolution.

BOM Line	Usage
000202/A;1-CAR EBOM (View)	
000203/A;1-Body (View)	
000214/A;1-Chassis (View)	
000224/A;1-Powertrain (View)	
000225/A;1-Transmission (View)	
000226/A;3-Drive Gears	
000227/A;3-Motor	
000228/A;3-Reducer Gears	
000229/B;1-rear_drive_asm (View)	
000230/A;2-rear_axle	
000231/A;2-rear_drive_hous	
000232/A;2-main_drive_gear	
000251/A;1-rear_axle_type_	
000233/A;3-power pack case	
000235/A;3-rcvr servo remote c	
000236/A;1-Power Source (View)	
000238/A;2-Wheel and Tire Asm (View)	
000246/A;1-Steer and Control (View)	
000247/A;2-steering pinion	
000248/A;2-Rack	

BOM Line	Usage
M_000202/A;1-M_CAR MBOM (View)	
000203/A;1-Body (View)	Buy
000214/A;1-Chassis (View)	Buy
000215/A;1-Frame (View)	
000219/A;1-Suspension (View)	
M_000225/A;1 M_Transmission (View)	Phantom
M_000224/A;1-M_Powertrain (View)	Phantom
000246/A;1-Steer and Control (View)	Buy

ME-stamp-ids-AH

Description

Traverses a structure according to a closure rule and automatically assigns a value to a specific property based on a recipe determined by the value of the **MEIdGenerationPropertySetting** preference. The workflow targets must be items, item revision, or structure context objects.

The target item or item revision is used as the top line of the BOM window. Normally, the top line of the structure is where the transverse is started. If you need to start at a lower line, use the **-scopeid** or **-scopeidincontext** arguments.

Syntax

ME-stamp-ids-AH

```
[-revrule=<revision-rule>]
[-scopeid=<scope-ID>|-scopeidincontext=<scope-in-context-ID>]
[-closurerule=<closure-rule-name>]
[-preference=<preference-name>]
[-forceupdate=1]
```

Arguments

-revrule

Specifies the revision rule. This argument is mandatory only if the target is an item or item revision to set up the BOM window. Do not use this argument if the target is a structure context object.

-scopeid

(Optional) Specifies the item ID in the manufacturing BOM from which to begin the traversal. This argument cannot be used with **scopeidincontext**.

If you do not specify this value, Teamcenter begins the traversal at the top line in the manufacturing BOM.

-scopeidincontext

(Optional) Specifies the IDIC of the line in the manufacturing BOM from which to begin the traversal. This argument cannot be used with **scopeid**.

If you do not specify this value, Teamcenter begins the traversal at the top line in the manufacturing BOM.

-closurerule

(Optional) Specifies the closure rule that determines which lines in the structure Teamcenter stamps when it traverses the manufacturing BOM structure below the scope line.

If you do not specify a closure rule, every line in the structure below the given scope line is stamped.

-preference

(Optional) Specifies the preference name containing the rules for setting the BOM line property. The default preference is **MEIdGenerationPropertySetting**.

-forceupdate=1

(Optional) Specifies that an existing ID in a **Context** string should be ignored and that a new value is generated. By default, the old value is not overridden.

Placement

Requires no specific placement.

Restrictions

None.

Examples

- This example creates in-context IDs that are based on the **Usage Address** property based on the constituent properties of item ID and item type. To do this:

1. Define the recipe for the IDIC value by setting the **MEIdGenerationPropertySetting** to:

```
type:Item,key: bl_usage_address,prop:bl_item_item_id,
prop:bl_item_object_type
```

2. Do one of the following:

- Create the usage address property on each line under the top line.

Argument	Values
-revrule	Latest Working

- Create the usage address on selected lines specified in a closure rule under a scope line determined by the specified IDIC (top level) value. In other words, the handler begins with a line that you specify by IDIC, traverses the structure from the IDIC line downward using the given closure rule, and stamps the resulting lines with the usage address string.

Argument	Values
-revrule	Latest Working
-scopeidincontext	kJBtMh0hAAbaaA
-closure rule	AccountabilityAll

ME-update-mirror-mbom-AH

Description

Updates a manufacturing bill of materials (MBOM) based on an engineering bill of materials (EBOM) from the top structure level. It allows different item types in the MBOM than were in the EBOM for certain nodes based on custom logic. By default, the MEMBOM_* preferences listed below set the item type to be created.

- MEMBOM_Mirror_MakeRules
- MEMBOM_Mirror_RemoveLineWithIDIC
- MEMBOM_Mirror_ReplaceMakeOnChange
- MEMBOM_Mirror_TypePrefixSuffix

Note

This utility requires a mfg_mbm_author or manufacturing_author license.

You can also customize the item type to be created using the **USER_create_or_ref_item** user exit operation exposed in the Business Modeler IDE through **BMF_ITEM_create_or_ref_id** on the item.

The target must be an item or item revision or a structure context object. The top line of the structure is where the update is started. If you need to start at a lower line, use the **-scopeid** or **-scopeidincontext** arguments.

Syntax

ME-update-mirror-mbom-AH

```
[-revrule=<revision-rule>]
[-mbomrevrule=<mbom-revision-rule>]
[-depth=<depth>]
[-clientdata=<data>]
[-actiononrelease= {<1> | <2> | <3> | <4>}]
[-mscuid=<UID>]
[-mbomroot=<root-itemid>]
[-usemfk= {0 | 1}]
[-log=<log-file>]
```

Arguments

-revrule

Specifies the revision rule of the EBOM structure used to traverse. This argument is mandatory only if the target is an item or item revision. Do *not* use this argument if the target is a structure context object.

-mbomrevrule

Specifies the revision rule for the MBOM structure. This argument is mandatory only if the target is a structure context object. This argument is required if the target is an item revision.

-depth

(Optional) Specifies the depth up to which to create the MBOM nodes.

If you do not specify this value, Teamcenter creates all the MBOM nodes.

-clientdata

(Optional) Data to be passed to any custom-registered user exit functions defined on the item.

-actiononrelease

(Optional) A value indicating the action to be taken if an MBOM node already exists (has a computed ID) and is released. Possible values are:

- 1 – Skip (the default).
- 2 – Revise and modify,
- 3 – Update properties on the released item.
- 4 – Update properties on the MBOM and its children.

-mscuid

The UID of the structure context object for the MBOM structure if the **mbomrevrule** is not suitable (for example, it is a private revision rule). Either this argument or the **mbomroot** argument is mandatory.

-mbomroot

(Optional) The ID of the root of the MBOM structure. Either this argument or the **mscuid** argument is mandatory.

-scopeid

(Optional) Specifies the item ID in the EBOM from which to begin the traversal. This argument cannot be used with **scopeidincontext**.

If you do not specify this value, Teamcenter begins the traversal at the top line in the EBOM.

-scopeidincontext

(Optional) Specifies the IDIC of the line in the EBOM from which to begin the traversal. This argument cannot be used with **scopeid**.

If you do not specify this value, Teamcenter begins the traversal at the top line in the EBOM.

-log

(Optional) Specifies the absolute path and name of the log file to capture details about the nodes created.

-usemfk

(Optional) Specifies whether to include the **MEMBOM_Mirror_TypePrefixSuffix** preference value with the EBOM item identifier as a multifield key that updates the existing MBOM item identifier. Values are:

- 0 Do not include the preference value in the multifield key. This value is the default.
- 1 If the preference defines an item type without a prefix or a suffix, include the value in the multifield key. This value is ignored if the preference also defines a prefix or a suffix.

Placement

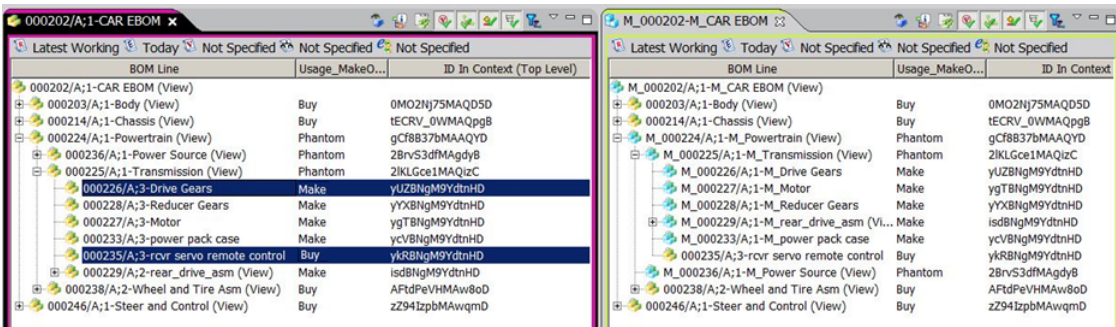
Requires no specific placement.

Restrictions

None.

Examples

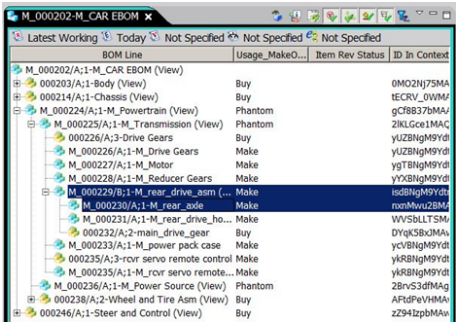
In the following EBOM and MBOM, **M_000229/A;1-M_rear_drive_asm** is released and then the make/buy property on its child, **000230/A;2-rear_axle**, is changed from **Buy** to **Make**.



You revise the MBOM part so you have write access and run the update workflow using the following arguments on **ME-update-mirror-mbom-AH** action handler for the target MBOM:

Argument	Values
-revrule=	"Latest Working"
-mbomrevrule=	"Latest Working"
-actiononrelease=	2

The results are the following:



MES-Update3DPDFReports

Description

Updates all 3DPDF reports attached to selected lines (processes and/or operations), according to the settings on the report creation. If a report update fails, the process continues until all update processes are complete.

The handler creates a dataset with a summary log, detailing for each report whether it successfully updated or not. Also, for each report that has failed to update, the handler creates a dataset with its log.

By default, the datasets are created in the **Newstuff** folder. You can define a different folder with the **MES_3DPDF_UPDATE_WORKFLOW_LOG_FOLDER** preference. If the handler does not complete in 10 minutes, a timeout error message is issued and the task fails. You can change the timeout wait time with the **MES_3DPDF_UPDATE_WORKFLOW_WAIT_TIME** preference.

Syntax

MES-Update3DPDFReports

Arguments

None.

Placement

Requires no specific placement.

Restrictions

Use only on process revision and operation revision business objects.

MultiSite-offline-export

Description

Performs the Multi-Site offline export of specified Teamcenter data for a specified target site.

Syntax

MultiSite-offline-export

```
-dir=<export_path>
{-target_site={<site_names> | ALL [-target_revision_only={TRUE | FALSE}]} | -owning_site=<site-
names>}
[-class=<class_name>]
[-optionset=<optionset_name>]
[-reason=<export_reason>]
[-exclude_files={TRUE | FALSE}]
[-exclude_relation=<relation_names>]
[-latest_ds_version={TRUE | FALSE}]
```

Arguments

-dir

(Required) A string value specifying the path and directory in which the data is exported.

-target_site

(Optional) The name of the site to which the specified objects are transferred. Objects are transferred, but ownership is not transferred. Specify multiple sites using commas.

Specify **ALL** to send the objects to all sites in your Multi-Site federation. "**ALL**" must be specified in all uppercase letters.

-target_site is mutually exclusive with the **-owning_site** argument. One of these arguments must be specified.

-owning_site

(Optional) The name of the site to which ownership of the specified objects is transferred. The specified objects are converted to reference objects before the transfer.

-owning_site is mutually exclusive with the **-target_site** argument. One of these arguments must be specified.

-optionset

(Optional) The option set used for offline export.

-class

(Optional) The class of objects to transfer to the specified site. **-class** can be specified multiple times to transfer more than one class. If **-class** is not given, all specified objects are transferred.

-reason

(Optional) A comment describing the offline data transfer.

-exclude_files

(Optional) Specifies whether to exclude related objects such as Imanfile objects and associated files. A value of **true** excludes files. The default value is **false**.

-exclude_relation

(Optional) Specifies a relation to exclude from the transfer. Specify multiple relations using commas.

-latest_ds_version

(Optional) Specifies that the latest dataset version be included from the transfer. A value of **true** includes the dataset version. The default value is **false**.

-target_revision_only

(Optional) Exports only the released item revision. When this argument is not given, all item revisions are exported.

This argument is only valid when used with **-target_site**, as all revisions are transferred when using the **-owning_site** argument to transfer site ownership.

Placement

Place this handler on the **Complete** action of the root task.

Restrictions

The **-dir** and **-target_site** (or **-owning_site**) arguments must be provided to run this handler. The directory specified by **-dir** must be empty.

Examples

- This example specifies three target sites and stated the data in a directory named *c:\temp\objects*.

Argument	Values
-dir	<i>c:\temp\objects</i>
-target_site	<i>site01,site02,site03</i>

MultiSite-offline-import

Description

Performs the Multi-Site offline import of Teamcenter data to the target site as specified by a related offline export.

Syntax

```
MultiSite_offline_import  
-dir=<import_path>  
[-reason=<import_reason>]
```

Arguments

-dir

(Required) A string value specifying the path and directory from which the offline import is to be performed.

This value is the directory containing the data exported by the related **MultiSite-offline-export** handler.

-reason

(Optional) A comment describing the offline data transfer.

Placement

Place this handler on the **Complete** action of the root task.

Restrictions

The target site must be one of the sites specified by the related export handler.

The imported objects must not already be present at the target site.

Examples

- This example imports objects in the directory *c:\temp\objects*.

Argument	Values
-dir	<i>c:\temp\objects</i>
-reason	This is an example import

OBJIO-restore-target-objects

Description

Restore objects from the archive site to the master site.

The user executing **OBJIO-restore-target-objects** must be a system administrator with DBA privileges.

Syntax

OBJIO-restore-target-objects [-include_bom]

Arguments

-include_bom

Specifies to include assembly components of the BOM at all levels for processing. Caution must be exercised in using this option as all children components of BOM gets archived. This option cannot be used with 4GD target objects.

Placement

No restrictions.

Restrictions

Use in workflow processes with objects belonging to either class or subclass of **PublishedObject** as targets.

OBJIO-archive-target-objects

Description

Archives objects from the master site to the archive site.

The user executing **OBJIO-archive-target-objects** must be a system administrator with DBA privileges.

Syntax

OBJIO-archive-target-objects [-include_bom]

Arguments

-include_bom

Specifies to include assembly components of the BOM at all levels for processing. Caution must be exercised in using this option as all children components of BOM gets archived. This option cannot be used with 4GD target objects.

Placement

No restrictions.

Restrictions

Use in workflow processes with objects belonging to either class or subclass of **Item** or **MdlOApplicationModel** as targets.

OBJIO-release-and-replicate

Description

Supports controlled replication of structure context objects (SCOs).

An SCO represents a virtual product configuration. The assembly for such a configuration might spread across multiple sites. To make the information available as quickly as possible to all sites participating on the assembly, Multi-Site provides *controlled replication*. This functionality replicates these objects to participating sites when the assembly is released.

Note

A *structure context* is a specific configuration of structure representation. A structure context is similar to an occurrence group but contains a configuration context. The configuration context is a persistent object that stores the configuration specified by revision and variant rules. The structure context also contains the root item.

You can use this handler to:

- Configure the target assembly with a specified revision rule or variant rule.
- Perform specified checks against the first level of the target assembly and apply a **Release** status to the target assembly when the checks are successful. You can check that all levels are precise, that no components are stubs, and/or that all components have a **Release** status.

If any check fails, an error appears.

- Initiate additional validation by the **CreateAssemblyPLMXML** Dispatcher task, performed asynchronously.

If the validation fails, a **Release_check_failed** status is applied to the target assembly and an e-mail notification sent to the process initiator

Syntax

```
OBJIO-release-and-replicate [-revision_rule=<revision-rule-to-configure-assembly>]
[-variant_rule=<variant-rule-to-configure-assembly>]
[-check_precise] [-check_no_stubs] [-check_all_released]
```

Arguments

-revision_rule

Specifies the revision rule used to configure the target assembly. If not specified, the **Latest Released** revision rule is used for the BOM configuration.

-variant_rule

Specifies the variant rule used to configure the target assembly. If not specified, the default variant rule is used for the BOM configuration.

-check_precise

Checks that all levels of the assembly are precise. If this check fails, **Release** status is not applied to the assembly.

-check_no_stubs

Checks that no component of the assembly is a stub. If this check fails, **Release** status is not applied to the assembly.

-check_all_released

Checks that each component of the assembly have a **Release** status. If this check fails, **Release** status is not applied to the assembly.

Placement

Requires no specific placement.

Restrictions

Use in workflow processes with SCOs as targets.

OBJIO-send-target-objects

Description

Sends to or synchronizes objects at other Multi-Site Collaboration sites. If the object is not present at the remote site, the object is replicated; otherwise, it is synchronized.

Syntax

```
OBJIO-send-target-objects [-class=<classname>] {-target_site=<site-name> | ALL | $SCHEDULE_SITE |
-owning_site=<site-name> | $SCHEDULE_SITE} [-target_revision_only=<YES>] [-reason=<string>]
```

Arguments

-class

Sends target objects of the specified class to the specified site. You can specify this argument more than once to send different classes of target objects. If this argument is not used, all target objects are sent.

-target_site

Sends the target objects to the specified site, but does not transfer ownership. You can specify multiple sites, separated by a comma or the character specified by the **EPM_ARG_target_user_group_list_separator** preference. Use **ALL** to send the specified target objects to all sites.

Use the **\$SCHEDULE_SITE** keyword to define the target site as the owning site of the schedule task or schedule task proxy link attached to the workflow process as **schedule_task**.

This argument is mutually exclusive with the **-owning_site** argument. One or the other of these two arguments must be specified for the handler to run.

-owning_site

Transfers site ownership of the target objects to the specified site. All target objects are converted to reference objects before the data transfer.

Use the **\$SCHEDULE_SITE** keyword to define the owning site as the owning site of the schedule task or schedule task proxy link attached to the workflow process as **schedule_task**.

This argument is mutually exclusive with the **-target_site** argument. One or the other of these two arguments must be specified for the handler to run.

-target_revision_only

Exports only the released item revision to the remote site. When this argument is not used, all item revisions are exported.

Do not use this argument with the **-owning_site** argument; all revisions must be transferred when transferring site ownership.

-reason

Allows you to enter a string (up to 240 characters) explaining why these objects were sent.

Placement

Requires no specific placement.

Restrictions

- Requires Multi-Site Collaboration to be configured at your site.
- The sending site must own all objects to be sent to other sites.
- When using the **-target_revision_only** argument, the **-class** argument must be set to *<ItemRevision>*.
This argument cannot be used with the **-owning_site** argument; all revisions must be transferred when transferring site ownership.

Examples

- This example shows how to send all item target objects to the **Detroit** and **Tokyo** sites without transferring ownership:

Argument	Values
-class	Item
-target_site	Detroit, Tokyo

- This example shows how to send item and dataset target objects to all sites without transferring ownership:

Argument	Values
-class	Item, Dataset
-target_site	ALL

- This example shows how to transfer site ownership of item and dataset target objects to the **Tokyo** site:

Argument	Values
-class	Item, Dataset
-owning_site	Tokyo

PARTITION-activate-or-inactivate

Description

Marks a partition as active or inactive.

Syntax

PARTITION-activate-or-inactivate -activate={true | false}

Arguments

-activate

Marks the partition as active (**-activate=true**) or inactive (**-activate=false**).

Placement

Place in a new workflow specifically designed to activate or inactivate partitions. The ability to activate partitions must be enabled first by setting the **Ptn0EnableActivationBehavior** business object constant to **true**.

Restrictions

None.

SUBSCMPL-Perform-Compliance-Check

Description

Performs substance compliance check on the target attachment objects. The handler accepts one or more objects.

Syntax

SUBSCMPL-Perform-Compliance-Check

Arguments

None.

Placement

Place on the **Complete** action.

Restrictions

Requires Compliance process manager is configured in the environment.

PIE-export-to-plmxmlfile

Description

Exports targets, references, and/or workflow process information to a PLM XML file. Use this handler to export targets and references data to a PLM XML file during a workflow process. You can also export operation and plant objects or the state of the workflow tasks to the PLM XML file. See *Workflow task actions and states* for more information.

Syntax

```
PIE-export-to-plmxmlfile [-context=<context-string>]
[-attach={target|reference|both}] [-file=<filename>] [-include_process_info] [-revrule]
```

Arguments

-context

Defines the context string, which specifies the transfer mode used for export. If not specified, it uses the default transfer mode.

-attach

Specifies which workflow process attachments are exported. If not specified, only targets are exported.

-file

Specifies the path and file name to which the data is exported. The export file is saved to the server machine.

If the path is not specified, the file is placed in the **TC_TMP_DIR** directory on the server. If this argument is not defined, the workflow process name is used as the file name, and the file is placed in the **TC_TMP_DIR** directory.

-include_process_info

Includes the workflow process information in the PLM XML file.

-revrule

Specifies the revision rule to be applied for the BOM lines while exporting the structure.

This argument applies only when the target object is a single item or item revision. The argument is ignored when the target has multiple objects or when the object is not an item or item revision.

Placement

Requires no specific placement.

Restrictions

None.

Note

Exporting this information may take some time, depending on the export content. Siemens Digital Industries Software recommends using the **-context** and **-file** arguments, which provide better control over the XML file's content and location, respectively.

Examples

This example releases an item revision, exporting the item revision information along with the BOM to a PLM XML file and sending the file to a third-party application. In this example, it is assumed that there is a transfer mode context named **MyApplication** that has a tool attached that connects to the third-party application and process the PLM XML file. Place this handler immediately after you add a release status.

Argument	Values
-context	MyApplication
-attach	target
-file	tceng2myap.xml
-revrule	Latest Working

PMA0_generate_dbom_handler

Description

Generates a design BOM from the selected engineering BOM. The engineering BOM is configured with the **Working; Any Status** revision rule.

In the generated design BOM:

- A (parent) design is created for the (parent) part, and the part is aligned with the design.
- Design occurrences are created below the (parent) design, and are automatically aligned with the corresponding part occurrences. For part occurrences that occur multiple times, the corresponding design occurrences are created as per the grouping logic set by you (administrator).

For each design occurrence, a design is created, and is aligned with the corresponding part.

Additionally, as an administrator, you can configure the generation of the design BOM from an engineering BOM.

Syntax

PMA0_generate_dbom_handler

Arguments

None

Placement

This handler is placed in a standalone workflow and on the **Complete** task action.

Restrictions

The handler can only process one input at a time.

PMA0_generate_part_ebom_handler

Description

Generates an engineering BOM from the selected design BOM. The design BOM is configured with the **Working; Any Status** revision rule.

In the generated engineering BOM:

- A (parent) part is created for the (parent) design, and the design is aligned with the part.
- Part occurrences are created below the (parent) part, and are automatically aligned with the corresponding design occurrences. For design occurrences that occur multiple times, the corresponding part occurrences are created as per the grouping logic set by you (administrator).

For each part occurrence, a part is created, and is aligned with the corresponding design.

Additionally, as an administrator, you can configure the generation of the engineering BOM from a design BOM.

Syntax

PMA0_generate_part_ebom_handler

Arguments

None

Placement

This handler is placed in a standalone workflow and on the **Complete** task action.

Restrictions

The handler can only process one input at a time.

PMA0_generate_product_ebom_handler

Description

Generates a product BOM from the selected design BOM. The design BOM is configured with the **Working; Any Status** revision rule.

In the generated product BOM:

- The topmost node of the design BOM is linked with the product BOM.
- A (parent) part is created for the (parent) design, and the design is aligned with the part.
- Part occurrences are created below the (parent) part, and are automatically aligned with the corresponding design occurrences. For design occurrences that occur multiple times, the corresponding part occurrences are created as per the grouping logic set by you (administrator).

For each part occurrence, a part is created, and is aligned with the corresponding design.

Additionally, as an administrator, you can configure the generation of the engineering BOM from a design BOM.

Syntax

PMA0_generate_product_ebom_handler

Arguments

None

Placement

This handler is placed in a standalone workflow.

Place the following workflow handlers on the **Start** action:

- **EPM-attach-related-objects**

Argument	Values
-relation	IMAN_specification
-attachment	target

- **EPM-attach-related-objects**

Argument	Values
-relation	PSBOMViewRevision
-attachment	target

- [EPM-assign-team-selector](#)

Argument	Values
-assignee	\$PROCESS_OWNER

- [EPM-auto-assign-rest](#)

Restrictions

The handler can only process one input at a time.

PMA0_update_dbom_handler

Description

Updates the design BOM aligned to the selected engineering BOM.

The engineering BOM is configured with the **Working; Any Status** revision rule. For each part occurrence in the engineering BOM, the aligned design occurrence is fetched and updated as per the update rules based on the design maturity. While updating the aligned design BOM, a design occurrence can be added, revised, reused, or removed.

Additionally, as an administrator, you can configure the generation of the design BOM from an engineering BOM.

Syntax

PMA0_update_dbom_handler

Arguments

None

Placement

This handler is placed in a standalone workflow and on the **Complete** task action.

Restrictions

The handler can only process one input at a time.

PMA0_update_ebom_handler

Description

Updates the engineering BOM aligned to the selected design BOM.

The design BOM is configured with the **Working; Any Status** revision rule. For each design occurrence in the design BOM, the aligned part occurrence is fetched and updated as per the update rules based on the part maturity. While updating the aligned engineering BOM, a part occurrence can be added, revised, reused, or removed.

Additionally, as an administrator, you can configure the generation of the engineering BOM from a design BOM.

Syntax

PMA0_update_ebom_handler

Arguments

None

Placement

This handler is placed in a standalone workflow and on the **Complete** task action.

Restrictions

The handler can only process one input at a time.

PMA0_align_dbom_and_ebom_handler

Description

If you select a **Design Revision** and a **Part Revision**, the handler aligns occurrences below them based on the part-design alignment and property matching as defined in the preference **Pma0_Align_DBOM_EBOM_Criteria**.

If you select a **Design Revision**, its related object (part or product) is automatically selected. If there are multiple related objects, an error is displayed.

If you select a **Part Revision** its primary Design Revision is automatically selected for occurrence alignment of their children.

If you select a **Product Revision**, its related design is automatically selected based on the linked design for occurrence alignment of their children. The revision rule for the configuration is taken from the **TC_config_rule_name** preference value.

You can view the report of alignments and failures in the **NewStuff** folder.

For multiple alignments, the **FND0_ENABLE_MULTIPLE_ALIGNMENT_CRITERIA** and **PMA0_Summarized_EBOM_Grouping_Criteria** preferences are used to determine whether the design occurrences can be grouped to align with the matching summarized part occurrence.

Syntax

PMA0_align_dbom_and_ebom_handler

Arguments

None

Placement

This handler is placed in a standalone workflow and on the **Complete** task action.

Restrictions

This handler takes only one EBOM and one design structure as input.

PMA0_align_design_revs_and_part_revs_in_context_handler

Description

When you import Design BOM and Part/Engineering BOM from an external system and want to perform initial Part-Design alignment, submit any part and design subassembly that you want to align through the **TC_Primary_Design_Representation** and **TC_Is_Represented_By** relations. This action handler attaches the part, design, or both item revision objects as target or reference objects to the workflow process. If you submit only a part revision, the system searches for an aligned design revision via CAD-Part alignment and aligns Part-Design Revisions underneath them based on the properties specified in the preference **Pma0_Align_DBOM_EBOM_Criteria**. If no aligned design is found for the input part revision, or if the input is only a design revision, the system throws an error.

You can view the report of alignments and failures in the **NewStuff** folder.

Syntax

PMA0_align_design_revs_and_part_revs_in_context_handler -attachment target | reference

Arguments

-attachment target | reference

You can submit valid item revisions as input to this handler/workflow.

Placement

This handler is placed in a standalone workflow and usually on the **Complete** task action.

Restrictions

This handler takes Part/Design Revision or both as the input. You can not submit both design revisions as part revisions.

SUBSCMPL-Populate-Material-Substance-Declaration-Form

Description

Adds the material and substance details to the declaration form available in the compliance officer's review task list. Details about the materials and substances in the part, including the mass and unit of measure are added to the **Review Substance Declaration Form**.

Caution

Do not add this handler to any other workflow process template.

Syntax

SUBSCMPL-Populate-Material-Substance-Declaration-Form

Arguments

None.

Placement

By default, this handler is placed in the correct location of the **Review Substance Declaration** workflow process template. Do not change the placement.

Restrictions

Adding this handler to any other workflow process template may result in errors.

PROJ-assign-members

Description

Adds members to projects. You can specify the projects and the members using handler arguments only, using properties on a form attached to the workflow template, and using a combination of handler arguments and form properties.

- The list of projects to receive new members is specified directly by projects and indirectly by the **projects_property** argument.
- The list of nonprivileged members to be added to the projects is specified directly by members and indirectly by the **members_property** argument.
- The list of privileged members to be added to the projects is specified directly by privileged_members and indirectly by the **privileged_members_property** argument.

Note

To run this handler, you must be either the project administrator, or the project team administrator of each project receiving new members.

Syntax

```
PROJ-assign-members [-source_task=<task-name.attachment-type>]
[-type=<form_type_name>]
[-projects=<comma_separated_project_list>]
[-members=<comma_separated_member_list>]
[-privileged_members=<comma_separated_member_list>]
[-projects_property=<property_name>]
[-members_property=<property_name>]
[-privileged_members_property=<property_name>]
[-bypass]
```

Arguments

-source_task

Specifies the task-name and attachment-type combination that associates a source form with the EPM task. The default reference attachments are those that are attached to the current task and are of the type specified by the **-type** argument.

<task-name>

Use one of the following values:

- The name of the current task (the default value)
- The **\$ROOT** reserved keyword (the root task)

<attachment-type>

Use one of the following reserved keywords:

- **\$REFERENCE** for reference attachments
- **\$TARGET** for target attachments

-type

Specifies the form type that designates properties to be used as the source of project names and member references.

-projects

Specifies a list of project names to receive new members. The privileged and non-privileged members are added to each project. Members already assigned to a particular project remain assigned.

Separate multiple entries with commas.

-members

Specifies a list of members to be added to the projects as non-privileged members. Each member is of the form group/role/user. An empty value can be specified for group, role, or user when necessary.

Separate multiple members with commas. Separate sub-groups with a period.

-privileged_members

Specifies a list of members to be added to the projects as privileged members. Each member is of the form group/role/user. An empty value can be specified for group, role, or user when necessary.

Separate multiple members with commas.

-projects_property

Specifies the name of a source-form property that designates project names to receive new members. The privileged and non-privileged members are added to each project. Members already assigned to a particular project remain assigned.

If you use this argument, you must use the **-type** argument also.

-members_property

Specifies the name of a source-form property that designates member references to be added to the projects as non-privileged members.

If you use this argument, you must use the **-type** argument also.

-privileged_members_property

Specifies the name of a source-form property that designates member references to be added to the projects as privileged members.

If you use this argument, you must use the **-type** argument also.

-bypass

Specifies that access checks are bypassed for reading the project name and member references from the source form. Otherwise, you must have access to read properties from the source form.

Placement

Place on any task action.

Restrictions

None

Examples

- This example adds members to projects using handler arguments only. In this example, assume the following:
 - The projects to receive members are named **Proj1** and **Proj2**.
 - The user named **john** is to be added to both projects as a non-privileged member. This user has the **Designer** role in the **Engineering** group.
 - The user named **jane** is to be added to both projects as a privileged member. This user has the **Manager** role in the **Engineering** group.

Argument	Values
-projects	Proj1,Proj2
-members	Engineering/Designer/john
-privileged_members	Engineering/Manager/jane

- This example adds members to projects using properties of a form attached to the workflow template. In this example, assume the following:
 - The source form is associated with the root task as a reference attachment.
 - The form type is **Pwf0ProjMemberForm**.
 - The projects to receive members are listed in the value of the **pwf0Projects** form property.
 - The non-privileged members to be added are listed in the value of the **pwf0NonPrivilegedMembers** form property.
 - The privileged members to be added are listed in the value of the **pwf0PrivilegedMembers** form property.

Argument	Values
-source_task	\$ROOT.\$REFERENCE
-type	Pwf0ProjMemberForm
-projects_property	pwf0Projects
-members_property	pwf0NonPrivilegedMembers
-privileged_members_property	pwf0PrivilegedMembers

- This example adds members to a project using a combination of handler arguments and form properties. In this example, assume the following:
 - The source form is associated with the root task as a reference attachment.

- The form type is **Pwf0ProjMemberForm**.
- The projects to receive members are **Proj1** and those that are listed in the value of the **pwf0Projects** form property.
- The non-privileged members to be added are **john**, with the **Designer** role in the **Engineering** group, and those users that are listed in the value of the **pwf0NonPrivilegedMembers** form property.
- The privileged members to be added are **jane**, with the **Manager** role in the **Engineering** group, and those users that are listed in the value of the **pwf0PrivilegedMembers** form property.

Argument	Values
-source_task	\$ROOT.\$REFERENCE
-type	Pwf0ProjMemberForm
-projects	Proj1
-members	Engineering/Designer/john
-privileged_members	Engineering/Manager/jane
-projects_property	pwf0Projects
-members_property	pwf0NonPrivilegedMembers
-privileged_members_property	pwf0PrivilegedMembers

PROJ-update-assigned-projects

Description

Updates the list of projects to which the workflow target objects are assigned. The handler arguments determine project IDs to be assigned to and removed from the targets. You can assign and remove projects using handler arguments only, using properties on a form attached to the workflow template, and using a combination of handler arguments and form properties.

Note

The ability to assign or remove a project is controlled by the following:

- The **TC_project_validate_conditions** preference.
- The Access Manager privileges **Assign to Project** and **Remove from Project**.
- Whether you are a privileged or non-privileged member of the project.

Syntax

```
PROJ-update-assigned-projects [-source_task=<task-name.attachment-type>]
[-type=<form_type_name>]
[-assign_property=<property_name>] [-remove_property=<property_name>]
[-assign_projects=<comma_separated_project_list>]
[-remove_projects=<comma_separated_project_list>]
[-bypass]
```

Arguments

-source_task

Specifies the task-name and attachment-type combination that associates a source form with the EPM task. The default reference attachments are those that are attached to the current task and are of the type specified by the **-type** argument.

<task-name>

Use one of the following values:

- The name of the current task (the default value)
- The **\$ROOT** reserved keyword (the root task)

<attachment-type>

Use one of the following reserved keywords:

- **\$REFERENCE** for reference attachments
- **\$TARGET** for target attachments

-type

Specifies the type name of a form that contains project IDs to assign or remove from the target objects.

-assign_property

Specifies the name of a source-form property that designates projects to assign to the target objects.

If you use this argument, you must use the **-type** argument also.

-remove_property

Specifies the name of a source-form property that designates projects to remove from the target objects.

If you use this argument, you must use the **-type** argument also.

-assign_projects

Specifies a list of projects to assign to the target objects. Projects already assigned to a particular target remain assigned.

Separate multiple entries with commas.

-remove_projects

Specifies a list of projects to remove from the target objects. Projects not already assigned to a particular target remain unassigned.

Separate multiple entries with commas.

-bypass

Specifies that access checks are bypassed for reading the source form and for writing the target objects. Otherwise, you must have both read access to the source form and write access to the target objects.

Note

If you use this argument, you must have the Access Manager privileges **Assign to Project** and **Remove from Project** for each project assigned to or removed from the target objects.

Placement

Place on any task action.

Restrictions

None

Examples

- This example assigns and removes projects from the target objects using handler arguments only. In this example, assume that the projects to be assigned are **Proj1** and **Proj2**, and that the projects to be removed are **Proj3** and **Proj4**.

Argument	Values
-assign_projects	Proj1,Proj2
-remove_projects	Proj3,Proj4

- This example assigns and removes projects from the target objects using properties of a form attached to the workflow template. In this example, assume the following:
 - The source form is associated with the root task as a reference attachment.
 - The form type is **Pwf0AssignProjForm**.
 - The projects to be assigned are listed in the value of the **pwf0AssignProjects** form property.
 - The projects to be removed are listed in the value of the **pwf0RemoveProjects** form property.

Argument	Values
-source_task	\$ROOT.\$REFERENCE
-type	Pwf0AssignProjForm
-assign_property	pwf0AssignProjects
-remove_property	pwf0RemoveProjects

- This example assigns and removes projects from the target objects using a combination of handler arguments and form properties. In this example, assume the following:
 - The source form is associated with the root task as a reference attachment.
 - The form type is **Pwf0AssignProjForm**.
 - The projects to be assigned are **Proj2** and those that are listed in the value of the **pwf0AssignProjects** form property.
 - The projects to be removed are **Proj4** and those projects that are listed in the value of the **pwf0RemoveProjects** form property.

Argument	Values
-source_task	\$ROOT.\$REFERENCE
-type	Pwf0AssignProjForm
-assign_projects	Proj2
-remove_projects	Proj4
-assign_property	pwf0AssignProjects
-remove_property	pwf0RemoveProjects

PS-attach-assembly-components

Description

Attaches all the components of the target assembly as the targets of the same workflow process. This handler is intended for use only with item revisions.

When a workflow process is initiated for an item revision, this handler derives the components of the targeted item revision by traversing item revisions attached BOM.

By default, the handler traverses only one level deep. Set the **-depth** argument to **all** to traverse all levels. In this case, if any of the derived objects are subassemblies, they are also traversed and their component item revisions are also added as targets to the workflow process. If any remote item revisions are encountered, a warning is displayed and the remote item revisions are attached as references to the workflow process.

By default, all component item revisions currently in workflow process are ignored. If the **EPM_multiple_processes_targets** preference is set to **ON**, you can use the **-include_in_process_targets** argument to attach components that are currently in workflow process.

Note

If the **WRKFLW_allow_replica_targets** preference is set to **true** and if any replica object qualifies to be attached as a workflow target, that object is attached as a **Replica Proposed Target** to the workflow process.

If the preference is set to **false** or is undefined, the handler attaches replica objects as references instead of targets.

Note

If the target item revision contains attachments such as BOM view revisions, datasets should be released along with the assembly, the **EPM-attach-related-objects** handler should be used in conjunction with this handler.

Syntax

```
PS-attach-assembly-components [-depth=<depth-of-traversal>]
[-owned_by_initiator][owned_by_initiator_group] [-initiator_has_write_prev]
[[exclude_released [-traverse_released_component]]] [-rev_rule=<revision-rule>]
[-saved_var_rule=<saved-variant-rule> ]
[[exclude_related_type=<types-to-be-excluded>] |
[include_related_type=<types-to-be-included>]] [-add_excluded_as_ref]
[-include_in_process_targets]
```

Arguments

-depth

Defines the depth to which the traversal should take place. Specify **1** to traverse one level deep. Specify **all** to traverse all levels.

If not specified, traverses one level deep.

-owned_by_initiator

Adds all the component item revisions owned by the initiator as targets to the workflow process.

-owned_by_initiator_group

Adds all the component item revisions owned by the initiator's group as targets to the workflow process.

-initiator_has_write_prev

Adds all the component item revisions to which the initiator has write access as targets to the workflow process.

-exclude_released [-traverse_released_component]

Excludes released component item revisions from being added as targets. If the released component is a subassembly, the handler does not traverse the components of the released component unless **traverse_released_component** is also specified. The **traverse_released_component** argument can only be used in conjunction with the **exclude_released** argument.

The **-traverse_released_component** argument can only be used in conjunction with the **-exclude_released** argument.

If the **-traverse_released_component** is used, the handler traverses the structure of the released component, and adds the components as targets to the workflow process.

If the **-depth** argument is set to **1**, **-traverse_released_component** only traverses one level deep.

If the **-depth** argument is set to **all**, the **-traverse_released_component** traverses all levels of the subassembly.

-rev_rule

Defines the name of the revision rule to be applied for BOM traversal. If not supplied, the default revision rule is used.

-saved_var_rule

Defines the name of the saved variant rule to be applied on BOM window for BOM traversal.

-exclude_related_type

Defines the types to be excluded from being added as targets.

The **-exclude_related_type** and **-include_related_type** arguments are mutually exclusive. Only one of these can be specified as arguments to the handler. If both arguments are specified, an error is displayed when running a workflow process using this handler.

-include_related_type

Defines the types to be included as targets.

The **-exclude_related_type** and **-include_related_type** arguments are mutually exclusive. Only one of these can be specified as arguments to the handler. If both arguments are specified, an error is displayed when running workflow process using this handler.

-add_excluded_as_ref

Adds components that are not included as targets as reference to the workflow process.

-include_in_process_targets

Can be used only if the preference **EPM_multiple_processes_targets** is set to **ON**. In this case, this argument attaches components that are currently in process as targets.

Placement

Can place on any action. Typically placed on the **Start** action of the root task so that the initial list is expanded at the start of the workflow process.

Restrictions

Do not place the **disallow_adding_targets** handler before this handler or it fails. The **disallow_adding_targets** handler can be used after the placement of this handler.

Examples

- This example releases an assembly when only one level of traversal is required. Only the components of the top-level assembly are released, not the components of any subassemblies:

Argument	Values
-depth	1

- This example releases an assembly using a specific revision rule and a saved variant rule. For this example, the **Working** revision rule and the **GMC 300 Rule** variant rule are used:

Argument	Values
-rev_rule	Working
-saved_var_rule	GMC 300 Rule

- This example releases an assembly using the default revision rule and the default saved variant rule, releasing only the components owned by the workflow process initiator:

Argument	Values
-owned_by_initiator	

- This example releases an assembly using the default revision rule and the default saved variant rule, releasing only the components owned by the group to which the workflow process initiator belongs:

Argument	Values
-owned_by_initiator_group	

- This example releases an assembly using the default revision rule and the default saved variant rule, releasing only the components to which the workflow process initiator has write access:

Argument	Values
-initiator_has_write_prev	

- This example releases an assembly, including all components traversed to all depths, using the **Latest Released** revision rule, excluding released components from the assembly but attaching them as references:

Argument	Values
-depth	all
-rev_rule	Latest Released
-exclude_released	
-add_excluded_as_ref	

- This example releases an assembly, including all components traversed to all depths using the **Latest Released** revision rule, excluding released components from the assembly but attaching them as references, yet traversing the excluded released components to all depths for subcomponents to be added as targets:

Argument	Values
-depth	all
-rev_rule	Latest Released
-exclude_released	
-traverse_released_component	
-add_excluded_as_ref	

- In this example, consider an assembly containing these revisions: **CORP_Part**, **CORP_Tool**, **CORP_Vehicle**, **CORP_Product**, **CORP_Analysis**, **CORP_Proc_Plan**, **CORP_Facility**, and **CORP_Build**.

To release the top-level assembly, excluding all the **CORP_Build** revisions, define the arguments:

Argument	Values
-exclude_related_type	CORP_Build

- In this example, consider an assembly containing the revisions: **CORP_Part**, **CORP_Tool**, **CORP_Vehicle**, **CORP_Product**, **CORP_Analysis**, **CORP_Proc_Plan**, **CORP_Facility**, and **CORP_Build**.

To release the top-level assembly, including only the **CORP_Build** revisions, define the arguments:

Argument	Values
-include_related_type	CORP_Build

- This example releases an assembly containing targets already in process. This argument can only be used if the **EPM_multiple_processes_targets** preference is set to **ON**.

Argument	Values
-include_in_process_targets	

- This example releases an assembly, including all components traversed to all depths using the **Latest Released** revision rule, excluding released components from the assembly but attaching them as references, yet traversing the excluded released components to all depths for subcomponents to be added as targets, and all **CORP_Build** item revisions must be excluded:

Argument	Values
-depth	all
-rev_rule	Latest Released
-exclude_released	
-traverse_released_component	
-add_excluded_as_ref	
-exclude_related_type	CORP_Build

Additional Information

This handler attaches component item revisions of the assembly to the workflow process. Therefore, you should not place the **EPM-disallow-adding-targets** handler before this handler.

Care should be taken when using this handler in conjunction with the **EPM-check-status-progression** and **PS-check-assembly-status-progression** handlers; possible placement conflicts could arise, including:

- If you place the above rule handlers in a **Task** action ahead of this handler, there is a possibility that the assembly may never be released, as some business rules may fail, and the rule handlers may return an **EPM_nogo**.
- If you place this handler in a **Task** action ahead of the above rule handlers, there is a possibility that the assembly may be released, but may not follow the business rules. For example, the assembly may have a status which may not follow the progression path.

Teamcenter provides another method of releasing an entire assembly. You can use the **Advanced Paste** button to compile a list of objects to be pasted into the assembly. These objects can be appended to the list from multiple sources, including query results, active rich client applications, and BOM views.

PS-make-mature-design-primary

Description

Sets the item revision as the primary representation of the associated part revision. This handler checks if the input item revision is mature. If it is, all part revisions for the design revision are found and the item revision is set as the primary representation.

Syntax

PS-make-mature-design-primary

Arguments

None.

Placement

Preferably placed on the **Complete** action.

Restrictions

Considers only item revisions or a subclass of them.

PS-occ-effectivity-cutback

Description

Process Occurrence Effectivity Cutbacks associated with a **BOMViewRevision** object. The **BOMViewRevision** object needs to be a target to the workflow. This workflow handler currently supports only **Execute**.

Syntax

PS-occ-eff-cutback [-mode = { EXECUTE }]

Arguments

- mode** Sets the handler to operate in a specified mode. Currently supported mode is to search and apply active cutbacks related to BVRs attached as targets to workflow process using the **Execute** flag.

Placement

The handler should be used in the **Complete task** folder.

Restrictions

Must have write access to **BVR** in order to apply occurrence effectivity cutback updates.

PS_attach_occurrence_revisions

Description

When you release a collaborative product or part item rev and you also want to release corresponding usages. You can use this workflow handler to release all the corresponding usages.

Syntax

PS_attach_occurrence_revisions

`[-include_all_working_revisions = true|false]`

Arguments

If no argument is provided then by default is **include_all_working_revisions=false**.

-include_resolved_comments (optional)	If set to <i><true></i> , all working revisions of each usage under given part revision must be released along with parent part revision.
	If set to <i><false></i> , only latest working revision of each usage under given part revision must be released along with parent part revision.

Placement

None

Restrictions

None

PUBR-publish-target-objects

Description

Publishes target objects (that is, enters them) in the Object Directory Services (ODS) database.

Syntax

```
PUBR-publish-target-objects [-class=<classname>] [-site=<site-ID>]
```

Arguments

- class**
Class of the target objects being published. This argument can be supplied more than once to publish multiple classes of target objects. If not supplied, all target objects are published. See the second item in the [Restrictions](#) section.
- site**
ODS sites that publish the objects. This argument can be supplied more than once to publish the objects to multiple ODS sites. If not supplied, the default ODS is used.

Placement

Requires no specific placement.

Restrictions

- Requires Multi-Site Collaboration to be configured at your site.
- The class must be defined by the **TC_publishable_classes** preference, or it cannot be published.
- You can control the publication behavior of item revision objects by changing the setting of the **TC_publish_item_or_itemrev** preference. You can publish only the item revision object, only its parent item object, or both.

Examples

This example shows how to publish all item revision target objects to **Detroit** and **Tokyo** ODSs:

Argument	Values
-class	ItemRevision
-site	Detroit, Tokyo

PUBR-unpublish-target-objects

Description

Unpublishes target objects (removes them) from the ODS.

Syntax

PUBR-unpublish-target-objects [-class=<classname>] [-site=<site-ID>]

Arguments

- class**
Teamcenter <classname> of the target objects being unpublished. This argument can be supplied more than once to unpublish multiple classes of target objects. If not supplied, all target objects are unpublished.
- site**
Teamcenter ODS <site-ID>s that unpublishes the objects. This argument can be supplied more than once to unpublish the objects to multiple ODS sites. If not supplied, the default ODS is used.

Placement

Place on any task where a demotion or cancellation is performed.

Restrictions

Do not place this handler on the **Perform** action, or any other action that is called multiple times. Place on an action that is only called once, such as **Start**, **Complete**, or **Undo**.

Examples

This example shows how to unpublish all item and dataset target objects from the default ODS:

Argument	Values
-class	Item, Dataset

QC-Release-Checklist-Specification

Description

Set a release status on a quality checklist template.

Syntax

QC-Release-Checklist-Specification -release_status=<<release_status_you_want_to_set>> -attachment=target -include_type=Qc0ChecklistSpecification

Arguments

-release_status

Specifies the release status that should be set on the quality checklist template.

-attachment

Attachment types that are inherited from the tasks specified in the **-target** argument. Choices are **task**, **reference**, or **signoffs**.

-include_type

Defines the type of target object to be checked.

Placement

Placed on the **Completed** action.

Restrictions

None.

QFM0FMEAREALIZATION-release-fmea-structure

Description

This workflow action handler releases the Failure Mode and Effects Analysis (FMEA) structure which is under the given FMEA root node.

Syntax

QFM0FMEAREALIZATION-release-fmea-structure [-retain_release_date=true/false][-exclude_released=true/false]

Arguments

-retain_release_date

Indicates whether to keep the released date or not while adding the release status on the FMEA object.

-exclude_released

Does not release an already released FMEA object.

Placement

Requires no specific placement.

Restrictions

None.

Note

For more information about workflow handlers, see [Workflow](#).

RDV-delete-ugcgm-markup

Description

Attaches all the drawing sheets as a target object for a **UGMASTER/UGPART** dataset in the selected workflow process, so the **DrawingSheet** dataset also attains a release status once the workflow process is approved. If the **DrawingSheet** dataset names are the same as for the previous item revisions, all **DirectModelMarkup** datasets are deleted if the **UGMASTER/UGPART** dataset names are also the same as in the previous revision.

Syntax

RDV-delete-ugcgm-markup [-type=<valid-dataset-type>, [<valid-dataset-type>]]

Arguments

-type
The valid dataset types for this handler are **UGMASTER** and **UGPART**. A user can specify more than one dataset type separated by a comma or the character specified by the **EPM_ARG_target_user_group_list_separator** preference. If the user does not specify any dataset type, this handler assumes **UGPART** as the dataset type.

Placement

Place on the **Start** action of the root task.

Restrictions

None.

Examples

Argument	Values
-type	UGMASTER, UGPART

RDV-generate-image

Description

Generates NX part images for display by Web Reviewer. This handler calls an external NX UFUNC (no license required) to accomplish this. The generated images are stored as named references to the **UGMASTER** dataset; image types and sizes are specified in the preference XML file.

Syntax

RDV-generate-image [-stop] [-continue]

Arguments

-stop

Halts the process if image generation is unsuccessful.

-continue

For noncritical image generation, continues the process regardless of unsuccessful image generation.

Placement

Place at a point in the workflow process where the initiator has write and copy access to the **UGMASTER** dataset (that is, before object protections are locked down). Siemens Digital Industries Software recommends that this handler have its own **Review** task at the beginning of the workflow process.

Restrictions

- Parts requiring images must be **UGMASTER** dataset targets of the workflow process.
- The **ugimg** executable must be located in the **\$UGII_BASEDIR/ugmanager** directory.

Note

Part files are automatically updated to the current NX version.

RDV-generate-ugcgm-drawing

Description

Generates drawing sheet datasets (CGM images) of NX drawings for display in Lifecycle Visualization. You must add this handler to a release procedure as an action handler. You should initiate the release procedure containing this action handler by selecting the **UGPART/UGMASTER** dataset. The **UGMGR_DELIMITER** preference must be added as a preference. This handler calls an external NX UFUNC program to generate the CGM images of the drawing sheets in the part. The generated images are stored as named references to the **DrawingSheet** dataset that is attached to the **UGMASTER/UGPART** dataset with an **IMAN_Drawing** relationship.

This handler requires NX to be installed on all systems on which the handler runs. In a 2-tier environment, NX must be installed on all clients that run this workflow handler. In a four-tier environment, handlers run in the **tcserver** process, so NX must also be installed onto the enterprise tier servers (pool servers). The environment variables **UGII_BASE_DIR** and **UGII_ROOT_DIR** (normally set by the NX installation) are used to determine the location of the NX software. This example depicts the two environment variables set to NX on a Windows platform.

```
set UGII_BASE_DIR = c:\apps\nx75
set UGII_ROOT_DIR = c:\apps\nx75\ugii\
```

Syntax

RDV-generate-ugcgm-drawing [-type=<valid-dataset-type>] [-text= text|polylines]

Arguments

-type

The valid dataset types for this handler are **UGMASTER** and **UGPART**. You can specify more than one dataset type separated by a comma or the character specified by the **EPM_ARG_target_user_group_list_separator** preference. If you do not specify any dataset type, this handler assumes **UGPART** as the dataset type.

-text

Specifies whether the text in your file is converted into searchable, standard font text or records text as CGM polyline elements, each of which is a collection of line segments. The valid values are **text** or **polylines**.

Placement

Place on the **Start** action of the root task.

Restrictions

If you are using Teamcenter Integration for NX, this handler may require the external NX program **export_ugdwgimages** to be copied from **\$TC_BIN\ugcgm_images** to **\$TC_BIN** or **UGII_BASE_DIR\ugmanager** directory.

The release procedure script **start_ugdwgimages** looks for the UFUNC program in the **UGII_BASE_DIR\ugmanager** directory first, then in the **\$TC_BIN** directory.

Examples

Argument	Values
-type	UGMASTER, UGPART
-text	text

RDV-tessellation-handler

Description

Tessellates NX datasets. It identifies which datasets to tessellate by reading the targets set in the **EPM_tessellation_target_type** preference and comparing them against the targets identified for the workflow process. Datasets identified as targets in both the workflow process and the preferences are tessellated. Targets are objects such as **UGMASTER** and **UGALTREP** datasets.

This handler can be run in the background or foreground. The background mode can be configured to act in:

- **Synchronous mode**

The workflow process waits for the tessellation to complete.

- **Asynchronous mode**

The workflow process continues after the tessellation is initiated.

Syntax

```
RDV-tessellation-handler -continue | {-signoff | -background |
-status=<status-type>}
```

Arguments

-continue

Continues the review process, even when tessellation is unsuccessful. Use for noncritical tessellation processes.

-signoff

Completes the **perform-signoffs** task if the handler was placed on the **Complete** action of the **perform-signoffs** task. Completes the process if the handler was placed on the **Complete** action of the root task.

-background

Runs tessellation in the background.

-status

Status type to be applied to a rendered child.

Placement

- In the foreground mode, it requires no specific placement.
- For background tessellation, do the following:
 - For asynchronous background tessellation, use the **-background** argument and place on the **Complete** action of the root task after the **EPM-set-status** handler.
 - For synchronous background tessellation, use the **-signoff** argument and place on the **Complete** action of the **perform-signoffs** task.

Restrictions

NX datasets must be included as targets of the process.

Preferences

You must set the following preferences before running the tessellation process with this action handler:

- **EPM_tessellation_target_type**
Defines the NX dataset types requiring tessellation. Only targets matching these types are tessellated.
- **EPM_tessellation_servers=<hostname>:<port-number>**
Defines the host name and port number of the tessellation server. The value **None** indicates that the tessellation is performed on the client side only.

Environment Variables

You must set the following environment variables before running the tessellation process with this action handler:

- <UGII_ROOT_DIR >
- <UGII_BASE_DIR >

Examples

If a business process required that **UGMASTER** and **UGALTREP** datasets are tessellated when they are released, the tessellation can be performed in the modes:

- **Foreground mode**
Include the handler in the workflow process template.
- **Background/Synchronous mode**
Set the **-background** and **-signoff** arguments for the handler, and place the handler in the **Complete** action of the **perform-signoffs** task of the Review task. The workflow process waits for tessellation to complete before continuing.
- **Background/Asynchronous mode**
Set the **-background** argument for the handler, and place the handler in the **Complete** action of the root task.

Define the tessellation server by setting this preference in the **preference** XML file:

EPM_tessellation_server=<hostname>:<port>

Define the NX datasets that can be tessellated by listing the required NX datasets as values in the following preference in the **preference** XML file:

EPM_tessellation_target_type=
UGMASTER
UGALTREP

SUBSCMPL-Review-Substance-Declaration

Description

Reviews the declaration sent by the supplier to ensure that the information is correct and complete. Verifies that all stakeholders have signed off on the declaration. Once the review is complete:

- It adds the property and relation values from the part declaration and part declaration revision to the vendor part in Teamcenter.
- If auto compliance check is set to **true**, it sends the part for a compliance check.
- It adds a release status of approved or rejected to the part declaration revision in Teamcenter.
- It sends an approval status email to stakeholders if the **SUBSCMPL_mail_approval_status** preference is set to **true**.

Caution

Do not add this handler to any other workflow process template.

Syntax

SUBSCMPL-Review-Substance-Declaration

Arguments

None.

Placement

By default, this handler is placed in the correct location of the **Review Substance Declaration** workflow process template. Do not change the placement.

Restrictions

Adding this handler to any other workflow process template may result in errors.

RM-attach-SM-tracelink-requirement

Description

Sends requirements tracelinked to Schedule Manager tasks to the specified folder in the task assignee's worklist.

This action handler is implemented to attach defining or complying objects using the trace links on predecessor tasks.

Syntax

RM-attach-SM-tracelink-requirement
 [-defining_complying_type=defining | complying]
 [-folder_type=target | reference] [-tracelink_subtype=<subtype>]

Arguments

- defining_complying_type**
Specifies if the **defining** or **complying** requirement is sent. If this argument is not specified, **defining** is the default.
- folder_type**
Specifies if the requirement is placed in the task's **target** or **reference** folder in the worklist. If this argument is not specified, **target** is the default.
- tracelink_subtype**
Sends only the specified subtype of the tracelink object.

Placement

Place on the **Start** action of the root task of the workflow process.

Restrictions

This handler is implemented only for **RequirementRevision**, **ParagraphRevison**, and **RequirementSpecRevision** and its subtypes.

Examples

- This example sends a Schedule Manager task linked to a requirement with a tracelink to the **Tasks to Perform** folder of the assignee's worklist and places the defining requirement object in the task's **Targets** folder.

Argument	Values
-defining_complying_type	defining
-folder_type	target

- This example sends a Schedule Manager task linked to a requirement with a tracelink to the **Tasks to Perform** folder of the assignee's worklist and places the complying requirement object in the task's **References** folder.

Argument	Values
-defining_complying_type	complying
-folder_type	reference

RM-attach-tracelink-requirement

Description

Sends requirements tracelinked to Teamcenter objects in the **Targets** folder to the specified folder in the workflow assignee's worklist.

Syntax

```
RM-attach-tracelink-requirement
[-defining_complying_type=defining | complying]
[-folder_type=target | reference] [-tracelink_subtype=<subtype>]
```

Arguments

- defining_complying_type**
Specifies if the **defining** or **complying** requirement is sent.
- folder_type**
Specifies if the requirement is placed in the task's **target** or **reference** folder in the worklist.
- tracelink_subtype**
Sends only the specified subtype of the tracelink object.

Placement

Place on the **Start** action of the root task of the workflow process.

Restrictions

None.

Examples

- This example sends the defining requirement linked to Teamcenter objects in the **Targets** folder with a tracelink to the **Targets** folder of the **Tasks to Perform** folder of the assignee's worklist.

Argument	Values
-defining_complying_type	defining
-folder_type	target

- This example sends the defining requirement linked to Teamcenter objects in the **Targets** folder with a tracelink to the **References** folder of the **Tasks to Perform** folder of the assignee's worklist.

Argument	Values
-defining_complying_type	complying
-folder_type	reference

SAP-set-valid-date-AH

Description

Copies the **Effect In** date from the release status object attached to the process and adds it to the **valid_from** box of all **BOMHeader** forms attached to the process using transfer folders. This handler is only required if you want to store the **Effect In** date persistently on the form. Use the special **effect_in_date** keyword to obtain the value for the transfer.

If the date is not set or there is no release status attached to the process, today's date is used.

Note

This handler requires the **valid_from** attribute to exist in the form type with **erp_object ="BOMHeader"**.

Syntax

SAP-set-valid-date-AH

Arguments

None.

Placement

Place on the **Perform Signoff** task.

Restrictions

None.

SAP-upload-AH

Description

Calls the script defined in the **Transfer_script** global setting. This script calls a third-party upload program to update the ERP system.

This action handler depends on the **Send_file_format** global setting.

The upload program reads the data from the transfer file and updates the ERP database. The action handler passes the following arguments to the upload program:

- **Transfer file path**/*<name>*
Set by the **Send_file_path** global setting.
- **Response file path**/*<name>*
Set by the **Response_file_path** global setting.

Note

This handler invokes the upload program and exits with success status, regardless of the success or otherwise of the upload itself. Success or failure of upload is logged in the ERP logfile dataset. The [ERP-post-upload-AH](#) handler must then be called to process the outcome of the upload.

Syntax

SAP-upload-AH

Arguments

None.

Placement

Place on the **Perform Signoff** task.

Restrictions

None.

SCHMGT-approve-timesheetentries

Description

Retrieves the target objects, the scheduled task, and the corresponding schedule, in the approve branch of the **TimeSheetApproval** workflow process. The minutes from the time sheet entry are updated in the scheduled task.

The **TimeSheetApproval** workflow is run from Schedule Manager. This handler can only be used within the **TimeSheetApproval** workflow process template. Do not add this handler to any other workflow process template.

Syntax

SCHMGT-approve-timesheetentries

Arguments

None.

Placement

By default, this handler is placed in the correct location of the **TimeSheetApproval** workflow process template. Do not change the placement.

Restrictions

This handler can only be used within the **TimeSheetApproval** workflow process template along the approval path. Adding this handler to any other workflow process template causes the workflow process to fail.

SCHMGT-reject-timesheetentries

Description

Retrieves the target objects, the scheduled task, and the corresponding schedule, in the reject branch of the **TimeSheetApproval** workflow process. The minutes from the time sheet entry are updated in the scheduled task.

The **TimeSheetApproval** workflow is run from Schedule Manager. This handler can only be used within the **TimeSheetApproval** workflow process template. Do not add this handler to any other workflow process template.

Syntax

SCHMGT-reject-timesheetentries

Arguments

None.

Placement

By default, this handler is placed in the correct location of the **TimeSheetApproval** workflow process template. Do not change the placement.

Restrictions

This handler can only be used within the **TimeSheetApproval** workflow process template along the reject path. Adding this handler to any other workflow process template causes the workflow process to fail.

SCHMGT-revise-timesheetentries

Description

Retrieves the target objects, the scheduled task, and the corresponding schedule, for the **TimeSheetRevise** workflow process. The minutes from the time sheet entry are updated in the scheduled task.

The **TimeSheetRevise** workflow is run from Schedule Manager. This handler can only be used within the **TimeSheetRevise** workflow process template. Do not add this handler to any other workflow process template.

Syntax

SCHMGT-revise-timesheetentries

Arguments

None.

Placement

By default, this handler is placed in the correct location of the **TimeSheetRevise** workflow process template. Do not change the placement.

Restrictions

This handler can only be used within the **TimeSheetRevise** workflow process template. Adding this handler to any other workflow process template causes the workflow process to fail.

SCHMGT-sync-schedule-attachments

Description

Synchronizes the change attachments of the parent schedule task with the workflow's change attachments. The change attachments of the schedule tasks are the same as that of the workflow after executing this handler if no error is encountered during the operation.

This handler works with remote schedule tasks only. The workflow does not inherit the change relations for local schedule tasks.

Syntax

SCHMGT-sync-schedule-attachments [-attachment= <attachment-types>]

Arguments

-attachment

(Optional) Specify one or more of the following change attachment types to synchronize.

- **problem_item**
- **solution_item**
- **impacted_item**

Separate multiple attachment types with commas or the character specified by the **EPM_ARG_target_user_group_list_separator** preference.

If this argument is not specified, all three change attachments types are synchronized.

Placement

Place on the **Start** or **Complete** action of any task. Do not place on the **Perform** action.

Because this handler invokes Multi-Site operations, Siemens Digital Industries Software recommends that you place this handler on a task marked for background processing.

Restrictions

None.

SERVICEFORECASTING-approve-ma-extension

Description

Approves a change in a maintenance action due date in Service Scheduler.

Syntax

SERVICEFORECASTING-approve-ma-extension -prop=ssf0ExtensionApproval -value=Approved

Arguments

- prop**
Specifies the property to be updated. The only valid property for this handler is **ssf0ExtensionApproval**.
- value**
Specifies the value for the property. The only valid value for this handler is **Approved**.

Placement

Place on the **Start** action of a task that follows the approval path of a **Review** task.

Restrictions

None.

Examples

- Approves the request to change a maintenance action due date.

Argument	Values
-prop	ssf0ExtensionApproval
-value	Approved

SERVICEPROCESSING-approve-service-structure

Description

Runs an approval process for SLM service structures.

Syntax

SERVICEPROCESSING-approve-service-structure

Arguments

None.

Placement

Requires no specific placement.

Restrictions

Use only for approval of SLM service structures inheriting from a transaction element.

SFD0-add-mpwshuttle-objects-as-targets

Description

This handler adds objects related to MPW (Multi-Project Wafer) Shuttle as target attachments to the workflow process. The handler processes the MPW Shuttle objects in the workflow target attachments and fetches related Intake Objects, MaskSet Objects, Tapeout Objects, and FloorPlan Objects. All found related objects are added as target attachments to the workflow root task. If certain related objects (for example, MaskSet, FloorPlan) are not present, no error is thrown; the target will contain only the MPW Shuttle and whichever related objects exist. Only MPW Shuttle objects without an existing release status are processed.

Syntax

SFD0-add-mpwshuttle-objects-as-targets

Arguments

None.

Placement

Place on the Start action of the root process task in the MPW Shuttle Workflow Process template.

Restrictions

Target attachments must include **Sfd0MPWShuttle** objects. Only Shuttle objects without an existing release status are processed. Objects that are not of type **Sfd0MPWShuttle** are skipped.

Examples

No arguments are required. The handler automatically discovers and adds:

- Intake Objects (via **Sfd0MPWShuttleIntakeRel** relation)
- MaskSet Objects (via **Sfd0MPWShuttleMaskSetRel** relation)
- Tapeout Objects (via **sfd0Tapeout** property on MaskSet)
- FloorPlan Objects (via saved query)

SFD0-check-mpwmaskset-attached

Description

This handler checks if a Mask Set with the MPW flag (**sfd0IsMPW=True**) is attached to the Die Design associated with an Intake object. If an MPW Mask Set is found, an error is thrown to prevent further processing. The handler first checks if any **Sfd0MPWShuttle** objects are present in the target attachments. If a Shuttle object is found, the handler exits immediately without performing any check (the check only applies when Intake objects alone are being processed). For Intake objects, it retrieves the attached Die Design, then checks all Mask Sets on that Die Design for the MPW flag.

Syntax

SFD0-check-mpwmaskset-attached

Arguments

None.

Placement

Place on the Complete action of the Make Cancel Ready task in the Cancelled Review Workflow template.

Restrictions

If **Sfd0MPWShuttle** objects are present in targets, the handler exits immediately (returns **ITK_ok**). Only checks Intake objects for MPW Mask Set attachment on their Die Design. Throws error **SFD0_MASKSET_ATTACHED** if an MPW Mask Set is found.

Examples

No arguments are required. The handler automatically:

1. Checks targets for Shuttle objects (skips if found)
2. For each Intake object, reads the **sfd0DieDesign** property
3. Finds Mask Sets via **Sfd0MaskSetRel** relation on the Die Design
4. Checks the **sfd0IsMPW** flag on each Mask Set
5. Throws error if any MPW Mask Set is attached

SMC0-create-solution-variants

Description

Creates solution variants for item revisions and variant rules attached as target and reference respectively to the root task.

Solution variants can be created for multiple item revisions and variant rules. Ensure that each item revision and variant rule have a one to one correspondence. In other words, when creating solution variants through a workflow, Teamcenter uses a *1:1 mapping* between the target item revision (source structure) and the reference variant rule.

You can provide the following optional arguments. They are applicable to the creation of all solution variants and cannot be provided individually for each input Item revision.

- Revision rule
- Solution variant category
- Multilevel boolean parameters

The output is:

- Item revision attached as a target.
- Solution variants attached as a target.

Both are attached as target and added in that order. The source item revision is attached first and then its associated solution variants. Users of the workflow handler can check its type. The source item revision is of the item revision type and the solution variant is the item type.

Any input item revision for which the handler fails to create a solution variant is added as a reference attachment to the root task.

Syntax

SMC0-create-solution-variants -revision_rule -sv_multi_level -sv_category

Arguments

-revision_rule

Defines the name of revision rule to be applied for BOMWindow configuration.

-sv_multi_level

Specify **0** or **1**. A multilevel solution variant is created if the input is **true**. A single level is created if **false** is provided as the input.

-sv_category

Specify one of the following values:

- **0** for unmanaged solution variant category.
- **1** for managed solution variant category.
- **2** for reuse solution variant category.

Placement

This handler can be placed on any action. It is typically placed at the **Start** action of the root task so that the initial list is expanded at the start of the workflow process.

Restrictions

None.

Examples

Use the following arguments and values to create a multilevel reused solution variant:

Argument	Values
Target Attachment	Item revision for which you want to create a new reused solution variant.
Reference Attachment	Variant rule acts a recipe with which a solution variant is to be created.
Revision_rule	Latest Working
sv_category	2

SMC0-create-solution-variants2

Create solution variants for a source structure using variant rules.

Description

Both the source structure, which is attached as a target, and the resulting solution variants are item revisions. The specific revision of the variant rule is attached as a reference to the root task. You can specify one or more variant rules, with one solution variant created per rule.

If you specify an engineering change with an effectivity in the target, the solution variants with the effectivity mentioned in the engineering change are created.

You can provide the following optional arguments. They are applicable to the creation of all solution variants and cannot be provided individually for each input source structure.

- Revision rule
- Solution variant category
- Multilevel boolean parameters

The output contains the following:

- Source structure is attached as a target.
- Solution variants are attached as a target.
- The creation report is attached as a reference.

The source structure and the resulting solution variants are attached as targets. The source structure is attached first and then its associated solution variants. Users of the workflow handler can check its type. The source structure and the solution variants are of the item revision type.

Syntax

SMC0-create-solution-variants2 -revision_rule -sv_multi_level -sv_category

Arguments

-revision_rule

Defines the name of the revision rule to be applied for the BOMWindow configuration. This is a required argument.

-sv_multi_level

Specify **0** or **1**. A multilevel solution variant is created if the input is **true**. A single level is created if **false** is provided as the input. If no input is provided, a multilevel solution variant is created by default.

-sv_category

Specify one of the following values:

- **0** for the unmanaged solution variant category.
- **1** for the managed solution variant category.
- **2** for the reuse solution variant category.

Placement

This handler can be placed on any action. It is typically placed at the **Start** action of the root task so that the initial list is expanded at the start of the workflow process.

Restrictions

None.

Examples

Use the following arguments and values to create a multilevel reused solution variant:

Argument	Values
Target Attachment	Item revision for which you want to create a new reused solution variant.
Reference Attachment	Variant rule acts as a recipe which is used to create a solution variant.
Revision_rule	Latest Working
sv_category	2

SMCO-update-solution-variants

Description

Updates solution variants that are attached in targets as items or item revisions.

When a structure (item revision) is modified both within and outside a change context, all the Reuse type solution variants that are associated with impacted structure (item revisions) are updated.

The default configuration for updating the solution variants is:

- Revision rule – Use the **Latest Working** revision rule.
- Effectivity – Use the effectivity of the active change. If an active change context is not set, effectivity is ignored.

Syntax

SMCO-update-solution-variants

Arguments

None

Placement

This handler can be placed on any action. It is typically placed at the **Complete** action of the root task so that the initial list is expanded at the start of the workflow process.

Restrictions

None.

Examples

Use the following arguments and values to create a multilevel reused solution variant:

Argument	Values
Target Attachment	Item revision for which you want to update the associated reused solution variant.

SMC0-update-solution-variants2

Description

Updates solution variants for the source structure that are attached as a target to the root task. The source structure and the solution variant are of the item revision type.

When a source structure is modified, both within and outside of a change context, all solution variants that are of the Reuse type associated with the impacted solution variant are updated. The Managed and Unmanaged type of solution variants are not updated.

If you specify a structure and one or more solution variants in the target, all solution variants are updated.

If you specify a structure and a solution variant in the target, and then you specify a different revision of the variant criteria in the reference, the specified solution variant is updated with the revision of the variant criteria. In this case, you must specify only one solution variant in the target.

If you also specify an engineering change with an effectivity in the target, the solution variants with the effectivity that is mentioned in the engineering change are updated.

The default configuration for updating solution variants is:

- Revision rule – Use the **Latest Working** revision rule.
- Effectivity – Use the effectivity of the active change. If an active change context is not set, effectivity is ignored.

The output is:

- Source structure is attached as a target.
- Updated solution variants are attached as a target.
- The update report is attached as a reference.

The source structure and solution variants are attached as targets. The source structure is attached first and then the updated solution variants are attached.

Syntax

SMC0-update-solution-variants2 -revision_rule

Arguments

-revision_rule

Defines the name of the revision rule to be applied for the BOMWindow configuration.

Placement

This handler can be placed on any action. It is typically placed at the **Complete** action of the root task so that the initial list is expanded at the start of the workflow process.

Restrictions

None.

Examples

Use the following argument and value to update a multilevel reused solution variant.

Argument	Values
Target Attachment	Source structure for which you want to update the associated reused solution variant.

Use the following arguments to update the solution variants.

Argument	Values
Target Attachment	Source structure for which you want to update the solution variants. List of solution variants that need be updated.

Use the following arguments to update a solution variant with a revision of the variant criteria.

Argument	Values
Target Attachment	Source structure for which you want to update the solution variant. The solution variant that must be updated.
Reference Attachment	The revision of the variant criteria to be used for updating.

SMP-auto-relocate-file

Description

Relocates all released datasets of a job to a specified directory. Teamcenter does not automatically register this handler. Users have to register and modify the handler code to suit their requirements, using the sample code provided. For more information about using this handler and to reference the sample code, see *Teamcenter Server Customization*.

TCRS-auto-approve-first-step

Description

Automatically approves the first task with this handler attached. Use this handler only when the **TCRS-trigger-approve-first-step** handler is placed on the root task. This handler only works the first time. If the task starts again, Teamcenter Rapid Start does not approve the first task automatically.

Syntax

TCRS-auto-approve-first-step

Arguments

None.

Placement

Place on the **Started** action of the **perform-signoffs** task.

Restrictions

None.

TCRS-create-form

Description

Creates a new form and attaches it to the item revision for all the target revisions. You can specify the form type and the type of relation that is used to attach the form to the item revision.

Syntax

TCRS-create-form -form_type=form [-rev_type=item_rev_type]
[-description=description] [-name=name] [-relation]

[-separator=separator]

Arguments

Parameter	Description	Default	Required
-form_type	Valid type of form.	None	Yes
-rev_type	Determines the item revision type under which the form is to be created. This item revision type must relate to one of the defined item types.	ItemRevision	No
-description	Description of the form.	Empty string	No
-name	Name of the form to be created. If this parameter is not specified, the default form name is used.	Item_id + separator + Rev_ID	No
-relation	Relation used to attach the form to the item revision. This must be a valid relation type between a form and a revision.	IMAN_reference	No
-separator	Separator between the item ID and revision ID if the parameter name was not indicated.	Minus sign (-)	No

Placement

Must be set in the **Started** or **Completed** action.

Restrictions

All item revisions must have write privileges at the level at which the handler is used.

Only one form of a **form_type** can be created and attached to the target revision by this handler. If the item revision already a form of the specified **form_type** attached, a new form of that **form_type** cannot be added.

Example

This example shows you how to create a form **My Form Type**. The form name is **MyForm**. The form is attached to target item revision using the **EPM_reference** relation.

Argument	Values
-form_type	My Form Type
-name	MyForm
-rev_type	ItemRevision
-relation	IMAN_reference

TCRS-Create-Print-Requests

Description

For Teamcenter Rapid Start, prints datasets at the server installation. It can be used on items, item revisions, or datasets. When used on items or item revisions, it prints all the datasets that are attached to them. This handler comes with the **Server Print** feature.

Syntax

TCRS-Create-Print-Requests [-prINTERNAME] [-watermark]

Arguments

Parameter	Value	Default	Required
-prINTERNAME	Defines the name and path to the printer.	The default printer name from the TcX_Server_Printers preference.	No
-watermark	Specifies the watermark text for the printed output.		No

Placement

Place on the **Complete** action.

Restrictions

None.

Examples

This example shows how to print the datasets that are attached to the target object of workflow with the watermark "My water mark."

Argument	Values
-prINTERNAME	My printer
-watermark	My water mark

Note

The workflow handler TCRS-Create-Print-Requests uses the translator 'visprint' to work. However, this translator has been deprecated in Teamcenter Rapid Start since Tc 11.x.

Accordingly, this workflow handler has been deprecated from Tc 11.x onwards and will be removed in a future version of Teamcenter Rapid Start.

TCRS-create-snapshot

Description

Creates a snapshot of the target BOM view revision. It adds a folder as a reference under the job and as a reference under the item revision.

You can view the folder that is created while capturing the snapshot in the **Where Used** subtab of the item revision.

Syntax

```
TCRS-create-snapshot [-revision_rule=<rule-name>][-name=<snapshot-name>] [-description=<snapshot-description>]
```

Arguments

Parameter	Value	Required
-revision_rule	Defines the name of the revision rule to be applied for BOM traversal.	No
-name	Defines the name of snapshot.	No
-description	Defines the description of snapshot.	No

Placement

Requires no specific placement.

Restrictions

All item revisions must have write privileges at the level at which the handler is used.

Examples

This example shows how to create snapshot with the name **'MySnapshot'**, using the **Precise** or **Latest Working** revision rule.

Argument	Values
-revision_rule	Precise; Latest Working
-name	MySnapshot

TCRS-Create-Translation-Request

Description

For Teamcenter Rapid Start, creates a translation request. All datasets attached to the item revision are translated into a printer-friendly format (PDF, HPGL, or TIFF). The translated datasets are then attached to the item revision.

Syntax

**TCRS-Create-Translation-Request [-pr] [-tr=ONDEMAND | CHECKIN | IMPORT]
[-ms=<msoffice-translation>] [-ug=<nix-translation>] [-se=<solid-edge-translation>]**

Arguments

Parameter	Value	Default	Nullable	Required
-pr	Specifies a between 1 and 5 . The greater the number, the higher the priority in the translation schedule.	3	Yes	No
-tr [ONDEMAND CHECKIN IMPORT]	Categorizes the reason for the translation request.	ONDEMAND	Yes	No
-ms	Defines Microsoft Office translations. For example, to convert Microsoft Office to PDF, the value should be pdf .	pdf	Yes	No
-ug	Defines NX translations. For example, to convert NX to PDF, the value must be pdf . Valid values are as follows: • cgm • hpg • jt • pdf • tif	hpg	Yes	No
-se	Used to define Solid Edge translations. For example, to convert Solid Edge to PDF, the value should be pdf . Valid values are:	jt	Yes	No

Parameter	Value	Default	Nullable	Required
	• bmp • dwg • dxf • emf • igs • jpg • jt • pdf • plmxml • sat • step • stl • tif • xgl • xt			

Placement

Place on the **Started** action of the root task.

Restrictions

None.

Examples

This example shows how to create a translation request for all the Solid Edge datasets attached to the item revision to a PDF file.

Argument	Values
-se	pdf

TCRS-delete-dataset

Description

Allows you to delete a dataset attached to an item revision. You can also delete the named reference of any target dataset.

Syntax

TCRS

```
-delete-dataset -dataset_type=<dataset-type>
[-relation=<relation-type>]
[-reference=<named-reference-type>]
```

Arguments

Parameter	Value	Required
-dataset_type	Defines the dataset type to be deleted.	Yes
-relation	Defines the relation between the target object and the specified dataset type.	No
-reference	Defines the named reference type of the dataset to be deleted.	No

To delete the dataset and the attached named reference, use the **-dataset_type** and **-relation** arguments only.

To delete only the named reference, use all three arguments.

Note

If you delete a dataset, it must be added as a workflow target.

If you delete a named reference, the dataset containing the reference must be added as a workflow target.

Add the action handler **EPM-auto-check-in-out** before and after the existing **TCRS-delete-dataset**

Example:

```
EPM-auto-check-in-out (-assignee=$RESPONSIBLE_PARTY -include_related_type=UGPART -action=check-out)
```

```
TCRS-delete-dataset
```

```
EPM-auto-check-in-out (-assignee=$RESPONSIBLE_PARTY -include_related_type=UGPART -action=check-in )
```

Placement

Must be set in the **Started** or **Completed** action.

Restrictions

All item revisions must have write privileges at the level at which the handler is used.

Examples

This example shows how to delete the **Text** named reference of the **Text** dataset with the **IMAN_reference** relation to the target object.

Argument	Values
-dataset_type	Text
-relation	IMAN_reference
-reference	Text

TCRS-delete-log-datasets

Description

Deletes all datasets with a given name or the description attached to the root task as references. This handler is mainly used for deleting unnecessary log datasets previously created by other rule handlers.

The **TCRS-delete-log-datasets** handler first removes the attachment from the reference folder of the root task.

Note

The removal of attachment from the reference is required to for deletion of datasets.

The handler then tries to delete the datasets. If it fails during deletion, it provides a relevant message indicating the failure. An example prohibiting the deletion would be if a dataset is checked out or referenced.

Syntax

TCRS-delete-log-datasets [-name=<name>] [-desc=<description>]

Arguments

Parameter	Value	Required
-name	Name of the dataset.	Not required if -desc is provided.
-desc	Description of the dataset.	Not required if -name is provided.

Placement

Place the handler on an action of the root task that occurs after the datasets are created.

Restrictions

None.

Examples

Syntax	Description
TCRS-delete-log-datasets -name=CheckBomChildStatus	This example shows how to delete all log datasets with the name CheckBomChildStatus .
TCRS-delete-log-datasets -desc= HANDLER_LOG	This example shows how to delete all log datasets with the description HANDLER_LOG .
TCRS-delete-log-datasets -name=CheckBomChildStatus -desc= HANDLER_LOG	This example shows how to delete all log datasets with the name CheckBomChildStatus and the description HANDLER_LOG .

TCRS-export-signoff-data

Description

Maps the workflow signoff information, such as the name of the approver and the approval date in the title block of a 2D drawing dataset. Once the signoff information is mapped on the 2D CAD file, this handler converts the native CAD file into a PDF dataset using a conversion utility. The PDF dataset is an exact copy of the 2D CAD drawing file.

Note

Solid Edge must be installed on the same computer where this workflow will be executed. The conversion utility starts Solid Edge in the background.

Note

Currently, this handler only supports Solid Edge draft files. This handler is dependent on a Solid Edge conversion utility, which is delivered with the core Solid Edge software installation.

This utility is located in the Solid Edge installation at the following location:

<Solid Edge install directory>\Program\SEEC_Workflow_PDF_Generation.exe

Use the short path convention in the values of **SE_TO_PDF_WORKING_DIR** and **SE_TO_PDF_EXECUTABLE_DIR**.

This workflow handler can be used in both Teamcenter and Teamcenter Rapid Start workflows.

The **TCRS-export-signoff-data** handler retrieves the signoff user name and date from the *Workflow Audit Log* and invokes **SEEC_Workflow_PDF_Generation.exe**.

The **TCRS-export-signoff-data** handler maps the following signoff user and date to the Solid Edge custom properties automatically, as defined in the *Define_SolidEdge_Properties_ForWorkflow_ToSync.ini* mapping file.

u1	User name who performed the first signoff.
d1	Date when first user performed the signoff.
u2	Name of the second user who performed the signoff.
d2	Date on which the second user performed the signoff.

The Solid Edge custom properties are created by the utility and do not need to be created beforehand.

This updates the Solid Edge property texts on the drawing, saves it as a PDF, and uploads it to Teamcenter.

Syntax

TCRS-export-signoff-data [-person] [-tif] [-replace]

Arguments

Parameter	Description	Required	Nullable
-person	Prints the person name on the PDF file instead of the user name.	No	Yes
-tif	Generates a TIFF dataset instead of a PDF dataset.	No	Yes
-replace	Replaces any existing PDF dataset that may have been created when this handler was executed previously.	No	Yes

Placement

This handler is placed on the **Complete** action after a **Release** task template. The item revision must be released before this handler can be run in order to gather the signoff information for the workflow.

Note

At least one **Review** task is required to release the Solid Edge drawing. The **TCRS-export-signoff-data** handler uses this to retrieve the required information from the *Workflow Audit Log*.

Restrictions

- All item revisions must have a release status before this handler can be run.
- Create the following preferences before using this handler:
 - **SE_TO_PDF_WORKING_DIR**
Specifies the staging location where the **PDF-generation** utility is run.
For example:
SE_TO_PDF_EXECUTABLE_DIR=C:\PDF_Staging_Folder
 - **SE_TO_PDF_EXECUTABLE_DIR**
Specifies the location of the *SEEC_WorkFlow_PDF_Generation* conversion utility.
For example:
SE_TO_PDF_EXECUTABLE_DIR=C:\Progra~1\Solide~1\Program\SEEC_WorkFlow_PDF_Generation.exe

Note

Use the short path convention in the values of *SE_TO_PDF_WORKING_DIR* and *SE_TO_PDF_EXECUTABLE_DIR*.

- **SE_PDF_GEN_WAITING_PERIOD**

Specifies the number of seconds the handler should wait for the PDF file to be generated.

For example:

SE_PDF_GEN_WAITING_PERIOD=20

Note

The value for this preference should be of type 'integer' while creation.

- Set the following preferences to the indicated values:

- **TC_audit_manager=ON**
- **TC_audit_manager_version=3**

Examples

In the following example **-person** and **-replace** arguments are specified. This converts the attached 2D draft dataset into PDF. It also retrieves the workflow signoff information, such as the signoff user name and signoff date. The signoff user name is replaced by the person name of the signoff user and the generated PDF has the person name in the title block.

Argument

-person

-replace

TCRS-export-to-tcxmlfile

Description

Exports targets and references to a TC XML file.

Syntax

TCRS-export-to-tcxmlfile -option_set=<export-option-set >-filename=<export-file-name >[-attach= target | reference | both]

Arguments

Parameter	Description	Required
-option_set	Name of the transfer option set when exporting the objects.	Yes
-filename	TC XML file name for the exported objects. The file name path can be absolute or relative. If the file name is relative, the exported file is created relative to where the tcserver process is running. For example, if you specify -filename=myTCXML , the <i>myTCXML</i> file is created in the directory where the tcserver is running. If you specify -filename=D:\tcxmlfiles\myTCXML , the <i>myTCXML</i> file is created in the <i>D:\tcxmlfiles</i> directory.	Yes
-attach	Object attachments to be exported from the process target and the reference folder. target reference both	No

Placement

Does not require specific placement.

Restrictions

None.

Examples

This example shows how to export targets of the workflow to TC XML file with the name "ExportFileName", using the **TIEConfiguredExportDefault** transfer option set.

Argument	Values
-option_set	TIEConfiguredExportDefault
-filename	ExportFileName
-attach	target

TCRS-IRM-cleanfields

Description

Allows you to delete the values of item revision master form attributes.

The attribute names must be defined as a Teamcenter preference. Create a Teamcenter preference called **EXPRESS_IRM_cleanfields<release>**, where <release> is the value defined in the **-block** parameter. For example, define the **EXPRESS_IRM_cleanfields<release>** preference values as follows:

- TCX_Rel_No
- TCX_Rel_Txt

The field names must match the real attribute name and not the display names.

When the handler is run, the values stored in the **Release No** and **Release text** fields of the item revision master form are deleted.

Syntax

TCRS-IRM-cleanfields -block=<blockname>

Arguments

Parameter	Value	Required
-block	Any value.	Yes

Placement

Requires no specific placement.

Restrictions

All item revisions must have write privileges at the level at which the handler is used.

Examples

Argument	Values
-block	release

TCRS-purge-dataset

Description

Allows you to purge all previous versions of a dataset. All datasets that are a target of the EPM task are purged.

Syntax

TCRS-purge-dataset

Arguments

None.

Placement

Placement of this handler should be after the **Do Task**.

Restrictions

All datasets that require purging must be a target of the EPM task.

TCRS-release-previous-itemrevs

Description

Sets a status on the current revision's preceding item revisions, dependent on their current status. Subsequently, the specified item revisions can optionally be sent into a workflow.

Note

The status is also applied to the datasets attached to the ItemRevision with the Specifications (IMAN_specification) relation (and only with this relation).

Syntax

```
TCRS-release-previous-itemrevs -status= <status-name>
[-rev_status=<status-name>][ -latest] [-proc_name=<workflow-process-name>] [-
job_name=<workflow-job-name>] [-job_desc=<workflow-job-description>]
```

Arguments

Parameter	Description	Default	Required
-status	Assigns a release status. If this parameter is set to \$NONE , you can start a workflow on the previous revision without assigning a status.	None	Yes
-rev_status	Use commas or the character specified by the EPM_ARG_target_user_group_list_separator preference to separate the list of valid status names. Use any to use all status names or none to leave all item revisions without a status.	None	No
-latest	If this parameter is specified, the handler evaluates the rev_status list against the most relevant release status (latest) of each previous item revision. If left unspecified, the handler evaluates the rev_status list against all available release statuses of each previous item revision.	None	No

Note

This parameter is optional and can be left blank.

Parameter	Description	Default	Required
-proc_name	Name of the workflow that will start according to the item revisions.	None	No
-job_name	Job name for this workflow.	<job_timestamp>	No
-job_desc	Job description for this workflow.	Empty string	No

Placement

Place before the **ADD** status (preferably in the **Start** action of the **Add Status** task).

Restrictions

None

Examples

This example releases all previous item revision with a status of 60.

Argument	Values
-status	60

This example releases all the previous item revision which has a latest status of 30, with a status of 60.

Argument	Values
-status	60
-rev_status	30
-latest	

This example releases all the previous item revision which has a status of 60 and a process name of <New Process 1>.

Argument	Values
-status	60
-proc_name	New Process 1

This example releases all the previous item revisions which have a status of 30 irrespective of the latest status of each previous revision, with a status of 60.

Argument	Values
-status	60
-rev_status	30

TCRS-remove-targets-with-status

Description

Allows you to remove target objects with specified status from the workflow process.

Syntax

TCRS-remove-targets-with-status -status= *<status-name>*

Arguments

Parameter	Value	Required
-status	Status of objects to remove.	Yes

Placement

Place on the **Started** action of the root task.

Restrictions

None.

Examples

This example removes all objects with a status of 60 from the workflow process.

Argument	Values
-status	60

TCRS-set-bom-precise

Description

Switches all target BOM view revisions to precise or imprecise.

Syntax

TCRS-set-bom-precise [-revision_rule=<config-rule>] [-precise=true|false]

Arguments

Parameter	Description	Default	Required
-revision_rule	Name of the configuration rule. Examples: • Latest Released • Latest by Creation Date • Precise; Working	Default configuration rule of the user.	No
-precise	Set to true for precise BOM view revisions or false for imprecise BOM view revisions	true	No

Placement

Must be set in the **Started** or **Completed** action.

Restrictions

All BOM view revisions must have write privileges at the level that the handler is used.

Examples

This example sets the target BOM view revisions to **-precise** with a revision rule of **Latest Released**.

Argument	Values
-revision_rule	Latest Released
-precise	true

TCRS-setstatus-EngOrder-folder

Description

For Teamcenter Rapid Start, releases the contents of a specific engineering order folder. It is commonly used to assign the obsolete status to an obsolete item revision during an engineering order process.

This handler sets the status for relation type **IMAN_specification** only.

For information for applying a status to other objects, see the [EPM-set-status](#) handler.

Syntax

TCRS-setstatus-EO-folder **-eo_folder=<relation-name>** **[-status=<status-name>]** **[-type=<EO-revision-type>]**

Arguments

Parameter	Description	Default	Required
-eo_folder	Relation name of the engineering order pseudofolder. For example, the relation name of the New Parts folder is TCX_New_Parts .	TCX_Obsolete_Parts	Yes
-status	Status for the engineering order.	90	No
-type	Type of the engineering order revision (for example, Eng_Order Revisions).	Revision	No

Placement

Before setting the status on the engineering order.

Restrictions

None.

Examples

This example sets a release status of 90 to the item revisions in the obsolete folder.

Argument	Values
-eo_folder	TCX_Obsolete_Parts
-status	90
-type	Eng_Order Revisions

TCRS-store-review-data

Description

Stores the workflow approver’s information (logon ID) and the approval date of the workflow task into the item revision master form or the **UGPartAttribute** form.

Syntax

TCRS-store-review-data -name=<attribute-name> -date=<attribute-name-date> [-mode=signoff | owner | modifier | delete] [-dest=IRM | UGPartAttr] [-person]

Arguments

Parameter	Description	Required
-name	Stores the approver’s logon name. This attribute should be of type string and should have sufficient length. Note This field gets auto populated from the workflow.	Yes
-date	Stores the approval date of the task. This attribute must be of type string and should have sufficient length. Note This field gets auto populated from the workflow.	Yes
-mode	Valid values are: • signoff: Approver and date approved of the current level. This is the default value. • owner: Owners and date created. • modifier: Last modified user and modification date. • delete: Previous attribute contents will be deleted.	No
-dest	Defines the destination form type. Valid values are IRM (item revision master form)	No

Parameter	Description	Required
	and UGPartAttr (UGPartAttribute form). IRM is the default value.	
-person	If this parameter is used, the actual person name of the signoff person is used instead of the user ID. The values for the argument are any of the person's attributes, such as address, city, state, zip code, or country.	No

Placement

Set in the **Completed** action. If **-mode=signoff**, set in the **Completed** action of the **perform-signoffs** task.

Set in the **Undo** or **Started** action. If **-mode=delete**, set in the **Undo** or **Started** action of the **perform-signoffs** task. Placement on the **Undo** action is done on a **Review** task.

Note

- The workflow can be designed so when a task is demoted on the **Reject** action, this handler is called. This can be achieved by placing **EPM-demote-on-reject** handler on the **Perform** action of the **perform-signoffs** task. A **Reject** action causes a demotion to the previous task, invoking the **Undo** action.
- To run this handler on an Item Revision which is owned by another user, use this handler at **START** with argument **-mode=delete** and at **COMPLETE** with argument **-mode=modifier of Do Task**.

Restrictions

All item revisions must have write privileges at the level that the handler is used.

Note

To run the **TCRS-store-review-data** workflow handler on an Item Revision which is owned by another user, use this handler at **START** with argument **-mode=delete** and at **COMPLETE** with argument **-mode=modifier**.

Examples

Argument	Values
-name	approver_name
-date	approval_date
-mode	signoff

This example shows how to store the workflow approver's information into the item revision master form.

1. Consider a custom item having custom properties **approver_name String[128]** and **approval_date String[128]**.
2. Log in as an admin, and create a new workflow with an **Acknowledgment** task and place the action handler **TCRS-store-review-data** in the **Complete** action of **perform-signoffs** task.
3. Add the following parameters to the action handler:
 - -name=approver_name
 - -date=approval_date
4. Publish the workflow.
5. Log in as a user and submit an item of type custom object to the workflow. In **Worklist > Tasks to Perform > Select Reviewers**, assign a user as a reviewer to perform the task.
6. Now, in **Worklist > Tasks to Perform**, select the **perform-signoffs** task. Approve the task.
7. Check the **approver_name** and **approval_date** properties on the **ItemRevisionMaster** Form of the item. These fields will be populated with the workflow approver's name logon information and the date on which it was approved.

TCRS-generate-pdf

Description

Converts the attached dataset of a specified type and relation to the PDF format and reattaches the generated PDF as a dataset with a specified relation.

Syntax

TCRS-generate-pdf

-exe=<PDF convertor tool executable path>

[-input_key=<Input parameter name>]

[-output_key=<Output parameter name>]

[-additional_args=<Additional parameters which are expected by the tool>]

-input_dataset_type=<The type of Dataset, (Word, cgm, etc.), which is to be converted to PDF>

-input_dataset_relation=<Relation of Dataset which is to be converted to PDF>

-output_relation=<Relation with which the newly generated PDF will be attached>

[-extension_required=<If extension of input file is required by the third party PDF converter tool (y/n)>]

Arguments

Arguments	Values	Required
-exe	PDF converter tool executable path	Yes
-input_key	Specifies the input parameter name that accepts the input file path of the PDF converter tool. If the tool does not accept the input key, this will be an optional argument. For example, --in_file .	No
-output_key	Specifies the output parameter name which accepts the output file path of the PDF converter tool where the new file will be placed. If the tool does not accept the output file path then this will be an optional argument. For example, --out_file	No
-additional_args	Specifies any additional parameters expected by the tool with the name value pair. If the tool does not accept any additional parameters then this will be an optional argument.	No
-input_dataset_type	Specifies the type of dataset that must be converted to PDF. You can specify multiple dataset types using comma-separated values.	Yes
-input_dataset_relation	Specifies the relation of dataset that must be converted to PDF. You can specify multiple dataset types using comma-separated values.	Yes

Arguments	Values	Required
-output_relation	Specifies the relation with which the newly generated PDF is attached.	Yes
-extension_required	Specifies if the extension of the input file is required by the third party PDF converter tool (y/n).	No

Placement

Requires no specific placement.

Restrictions

User must install a third party PDF convertor tool on the server as a prerequisite for this handler. All item revisions must have *write* privileges at the level that the handler is used.

Notes

This handler accepts comma separated input dataset types and relations. For example, **-input_dataset_type=MSWord,Bitmap**, or **-input_dataset_relation=IMAN_specification,IMAN_reference**. Consider that the above parameters are provided with comma-separated values to the handler and an item revision has 6 datasets of following types and relations attached:

- MSWord Dataset1 (IMAN_specification)
- MSWord Dataset2 (IMAN_reference)
- Bitmap Dataset3 (IMAN_specification)
- Bitmap Dataset4 (IMAN_reference)
- MSWord dataset5 (IMAN_rendering)
- Text Dataset6 (IMAN_specification)

In this case, the handler converts the datasets that are matched with the specified types and relations. Therefore only **Dataset1**, **Dataset2**, **Dataset3** and **Dataset4** will be converted to PDF. These four new PDF datasets will be attached as a datasets to the item revision while retaining the old ones as they are.

Therefore, **Dataset5** and **Dataset6** are not converted to the PDF format (as the **IMAN_rendering** relation and **Text** type are not specified as an input).

Example

Consider that **PDF Editor**, a third-party PDF generation tool is installed on the server at *C:\Program Files (x86)\PlotSoft\PDFill\PDFill.exe*. To convert an image file of the type Bitmap (.bmp) to PDF and save it at *C:\out.pdf*, enter the following command:

```
C:\Program Files (x86)\PlotSoft\PDFill\PDFill.exe OCR C:\image.bmp C:\out.pdf
```

In this example, the tool accepts **OCR** as an input parameter key. The following table lists the arguments that you must provide.

Argument	Value
-exe	C:\Program Files (x86)\PlotSoft\PDFill\PDFill.exe
-input_key	OCR
-input_dataset_type	bitmap
-input_dataset_relation	IMAN_specification
-output_relation	IMAN_Rendering

The **input_key**, **output_key**, and **additional_args** parameters vary depending on the PDF generation tool. In this example, the **output_key** and **additional_args** parameters are not provided as these are not required by the **PDF Editor** tool.

Once the handler with the above parameters is executed, the input dataset of type **Bitmap** with the relation **Specifications** is converted to PDF. This generated PDF is reattached as a dataset with the **Rendering** relation.

TCRS-bom-plmxml-export

Description

Exports targets and references information to an XML file. Use this handler to export targets and references data to an XML file. You can use the **TCRS-bom-plmxml-export** workflow handler to export all files attached to the item revision.

Syntax

```
TCRS-bom-plmxml-export-context=<transfer-mode>[
-viewtype=<view-type-to-export>][-attach=[target|
reference]-revrule=<revision-rule>][-path=<data-export-path>]-prefix=<filename-prefix>]
-postfix=<filename-prefix>]
```

Arguments

Arguments	Definition	Required	Value/Default Value
-context	Defines the context string, which specifies the transfer mode used for export.	Yes	transfer_mode
-context	Defines the context string, which specifies the transfer mode used for export.	Yes	transfer_mode
-viewtype	Specifies the view type that you want to export.	No	view
-attach	Specifies which workflow process attachments are exported. If not specified, only targets are exported.	No	target
-revrule	Specifies the revision rule to be applied for the BOM lines while exporting the structure. If you do not specify a value, the latest revision is used.	No	Latest Working
-path	Specifies the path where you want to export the	No	TC_TMP_DIR

Arguments	Definition	Required	Value/Default Value
	data. The exported file is saved to the server machine. If not defined, the data is exported to <i>TC_TMP_DIR</i> .		
-prefix	Specifies the prefix text for the exported file name.	No	None
-postfix	Specifies the post-fix text for the exported file name.	No	None

Placement

Requires no specific placement.

Restrictions

None.

Example

- This example shows how to export the targets in the workflow to an XML file using the **ConfiguredDataExportDefault** transfer mode and the **Latest Working** revision rule.

Argument	Value
-context	ConfiguredDataExportDefault
-attach	target
-revrule	Latest Working

- This example shows how to export all the files attached to the target object revision in a workflow. The context value is set to **ConfiguredDataFilesExportDefault**, and path value is set to an existing folder path. On completion of workflow, the attachments get downloaded to the path on the server machine, along with the xml file.

Argument	Value
-context	ConfiguredDataFilesExportDefault
-attach	target
-revrule	Latest Working

TCRS-trigger-approve-first-step

Description

Initiates the approval after an **auto-approve-first** step is done, so that it is done only when the workflow is started (not after a reject).

Syntax

TCRS-trigger-approve-first-step

Arguments

None.

Placement

Place on the **Started** action of the root task.

Restrictions

None.

TSTK-CreateTranslationRequest

Description

Creates a new translation request for all datasets matching the type specified using the translator specified with the provider and service name. If more than one dataset exists in the item revision, multiple translation requests are created.

This handler does not create translation requests for custom types.

Note

The target of the handler must be an item revision. Any other target that is not an item revision is not supported. The handler traverses the item revision to look for the dataset that was specified in the handler definition.

NX datasets containing drawing sheets must be pasted into the **Target** folder for **nxtocgmdirect** to create CGM files.

Syntax

```
TSTK-CreateTranslationRequest -ProviderName= <UGS> -ServiceName=  
<nxtopvdirect >-Priority=<1> -DatasetTypeName=<UGPART>
```

Arguments

-ProviderName

Creates a new translation request for all datasets with the specified translator provider name.

-ServiceName

Creates a new translation request for all datasets with the specified service name.

-Priority

Defines the priority assigned to the new translation request.

-DatasetTypeName

Specifies the dataset name for the selected workflow and item revision. Custom types cannot be specified.

Placement

The **Start** or **Complete** action.

Restrictions

None.

SUBSCMPL-Update_Compliance_Result

Description

Updates the substance compliance result status and the exemption remark on the compliance result object in Teamcenter and displays the compliance result and the applied exemption in the **Compliance** tab. Additionally, the handler also relates the applied exemption to the compliance result for the selected regulation.

Caution

Do not add this handler to any other workflow process template.

Syntax

SUBSCMPL-Update_Compliance_Result

Arguments

None.

Placement

By default, this handler is placed in the correct location of the **Apply Exemptions** workflow process template. Do not change the placement.

Restrictions

Adding this handler to any other workflow process template may result in errors.

VAL-approve-result-overrides

Description

Sets all requested result overrides to the **Approved** state for the workflow targets when the **perform-signoffs** task is approved.

Syntax

VAL-approve-result-overrides

Arguments

None.

Placement

Place on the **Perform** action of the **perform-signoffs** subtask of a **Review** task.

Restrictions

This handler should be used with the **perform-signoffs** task of the **OverrideReviewTask** template. This handler assumes that all target objects, reference objects, and status types are attached to the root task.

VAL-reject-result-overrides

Description

Sets all requested result overrides to the **Rejected** state for the workflow targets when the **perform-signoffs** task is approved.

Syntax

VAL-reject-result-overrides

Arguments

None.

Placement

Place on the **Perform** action of the **perform-signoffs** subtask of a **Review** task.

Restrictions

This handler should be used with the **perform-signoffs** task of the **OverrideReviewTask** template. This handler assumes that all target objects, reference objects, and status types are attached to the root task.

VAL-set-condition-by-check-validation-result

Description

This action handler can be configured to set the **Condition** task result status using **Validation Rule** and **Validation Object** applications a from workflow process. It can also check target NX datasets validation result status. To add this handler to a workflow process template, the user must have a well-defined **Validation Rule set file** that best describes the business process in terms of which NX datasets should run checks at certain times and the conditions that the check must meet.

The handler sets the **Condition** task result based on the overall result status of the verification (true when all target NX datasets satisfy all rules defined in the **Validation Rule set file**). The handler logs validation rules and validation result checks. The format of the log file name is *<First-target-name_Time-stamp>*. The log file is stored in the directory specified by the **TC_TMP_DIR** environment variable. If **TC_TMP_DIR** is not defined, it is stored in the **%TEMP%** directory (Windows) or **/tmp** directory (Linux).

When a **Condition** task template is configured with this action handler, no other saved queries or handlers should be added to the task template. The logic that this handler uses to check validation results is the same logic used by the **VAL-check-validation-result-with-rules** rule handler.

Syntax

VAL-set-condition-by-check-validation-result

-rule_item_revision=<item-revision-id> [**-current_event**=<value>]

[**-pass_item_revision_only**] [**-ref_log**]

Arguments

-rule_item_revision

The item revision ID that the validation rule set dataset is attached under.

-current_event

A value that is used to select validation rules from the rule file by comparing with the event values list of each rule. When this argument is not provided, all rules from the rule file are selected at the first step. When a rule is defined without the event values list, then the rule is selected at the first step. The event values list of a rule can contain an asterisk (*) as a wildcard. The event values list also can be marked as exclusive (it is inclusive by default).

-pass_item_revision_only

When this argument is added to the input list, only item revision targets are passed to the handler. NX datasets are searched from each item revision and verified according to rules.

-ref_log

If this argument is present and the validation fails, the validation results log is created and the log is attached, but no warning is displayed.

If this argument is not present and the validation fails, the validation results log is created, the log is *not* attached, and no warning message is displayed.

If the validation passes, the validation results log is not created and no message is displayed.

Placement

Place under the **Complete** action.

Restrictions

- **-rule_item_revision** cannot be NULL.
- You cannot customize the path names that branch from the **Condition** task. They must be either **T** or **F**.

VAL-set-condition-result-overrides

Description

If there are unapproved result override requests for the workflow targets, sets the condition to **EPM_RESULT_True**. If there are no unapproved result override requests, sets the condition to **EPM_RESULT_False**.

Syntax

VAL-set-condition-result-overrides

Arguments

None.

Placement

Place on the **Start** action of a **Condition** task.

Restrictions

This handler assumes that all target objects, reference objects, and status types are attached to the root task.

Rule handlers

Rule Handlers

Rule handlers integrate workflow business rules into EPM workflow processes at the task level. They attach conditions to an action. Rule handlers confirm that a defined rule has been satisfied. If the rule is met, the handler returns the **EPM_go** command, allowing the task to continue. If the rule is not met, it returns the **EPM_nogo** command, preventing the task from continuing. If there are multiple targets for a single rule handler, all targets must satisfy the rule for **EPM_go** to be returned (**AND** condition).

Many conditions defined by a rule handler are binary (that is, they are either true or false). However, some conditions are neither true nor false. EPM allows two or more rule handlers to be combined using logical **AND/OR** conditions. When several rule handlers are combined using a logical **Or** condition, rule handler quorums specify the number of rule handlers that must return **EPM_go** for the action to complete.

ASBUILT-validate-for-checkedout-physicalpartrevision

Description

Validates that the as-built structure does not contain any checked-out physical parts by any user other than the one submitting the physical part to a workflow.

Syntax

ASBUILT-validate-for-checkedout-physicalpartrevision

Arguments

None.

Placement

Place at the entry of the workflow to validate that the target structure does not contain any checked out physical part revisions.

Restrictions

This handler is available only when Teamcenter service lifecycle management Service Manager or As-Built Manager is licensed and installed.

ASBUILT-validate-for-physicalpartrevision

Description

Validates that the submitted object is a physical part revision before traversing the as-built structure and releasing each of the physical part revisions.

Syntax

ASBUILT-validate-for-physicalpartrevision

Arguments

None.

Placement

Place at the entry of the workflow to validate that the target object is a physical part revision for as-built structure traversal.

Restrictions

This handler is available only when Teamcenter service lifecycle management Service Manager or As-Built Manager is licensed and installed.

ASBUILT-validate-missing-structure

Description

Validates the as-built structure does not contain any missing or unidentified physical parts.

Syntax

ASBUILT-validate-missing-structure

Arguments

None.

Placement

Place at the entry of the workflow to validate that the target structure does not contain any missing physical parts.

Restrictions

This handler is available only when Teamcenter service lifecycle management Service Manager or As-Built Manager is licensed and installed.

ASMAINTAINED-validate-for-checkedout-physicalpartrevision

Description

Checks if any physical parts are checked out in the as-maintained structure by a user other than the creator or submitter of the workflow process.

Syntax

ASMAINTAINED-validate-for-checkedout-physicalpartrevision

Arguments

None.

Placement

Place at the entry of the workflow to validate that the target structure does not contain any checked out physical parts.

Restrictions

This handler is available only when Teamcenter service lifecycle management Service Manager is licensed and installed.

ASMAINTAINED-validate-for-latest-asmphysicalpartrevision

Description

Checks if the target physical part revision is the latest revision.

Syntax

ASMAINTAINED-validate-for-latest-asmphysicalpartrevision

Arguments

None.

Placement

Place at the entry of the workflow to validate that the target physical part revision is the latest one.

Restrictions

This handler is available only when Teamcenter service lifecycle management Service Manager is licensed and installed.

ASMAINTAINED-validate-for-unserviceable-physicalpartrevision

Description

Checks the as-maintained structure for any unserviceable physical parts.

Syntax

ASMAINTAINED-validate-for-unserviceable-physicalpartrevision

Arguments

None.

Placement

Place at the entry of the workflow to validate that the target structure does not contain any unserviceable physical parts.

Restrictions

This handler is available only when Teamcenter service lifecycle management Service Manager is licensed and installed.

ASMAINTAINED-validate-missing-asmaintained-structure

Description

Validates the as-maintained structure does not contain any missing or unidentified physical parts.

Syntax

ASMAINTAINED-validate-missing-asmaintained-structure

Arguments

None.

Placement

Place at the entry of the workflow to validate that the target structure does not contain any missing physical parts.

Restrictions

This handler is available only when Teamcenter service lifecycle management Service Manager is licensed and installed.

AUTOSCHEDULING-person-reassign-validate

Description

Verifies that when a workflow task with an attached job card or job task is reassigned to another user, that user has the discipline (skill) and qualifications specified on the job card or job task.

Syntax

AUTOSCHEDULING-person-reassign-validate

Arguments

None.

Placement

Place on the **Start** action of the **perform-signoffs** task.

Restrictions

None.

CM-validate-concurrent-change-exists

Description

Validates the presence of a concurrent change and issues a warning if detected. Typically used in a Validate task to help automatically route the workflow.

Syntax

CM_validate_concurrent_change_exists

Arguments

None.

Placement

Place on the Start or Complete action of a task.

Restrictions

None.

EPM-assert-signoffs-target-read-access

Description

Checks if all the selected reviewers have read access to the attached target attachments.

Syntax

EPM-assert-signoffs-target-read-access [-check_assignee=\$RESOURCE_POOL_ALL]

Arguments

-check_assignee

(Optional) If the selected reviewer is a resource pool, checks if all members of the resource pool have read access to the attached targets.

The only valid value is **\$RESOURCE_POOL_ALL**.

Placement

Place on the **Complete** action of a **select-signoff-team** task.

Restrictions

None.

EPM-assert-targets-checked-in

Description

Verifies that all target objects in this workflow process are checked in.

Note

EPM-assert-targets-checked-in will not execute on a fail path. Target objects in the workflow are only verified as checked in when a success path is taken.

Syntax

EPM-assert-targets-checked-in

Arguments

None.

Placement

Requires no specific placement.

Restrictions

None.

EPM-check-action-performer-role

Description

Checks whether the user performing this action matches the criteria specified in the handler arguments.

Syntax

```
EPM-check-action-performer-role -responsible=[owner|$OWNER] |
[group|$GROUP] | [$RESPONSIBLE_PARTY] | [privileged | $PRIVILEGED] |
[<group>::[*|<role>]] | [<role>]
[-mode=recipient | performer]
```

Arguments

-responsible

(Required) Checks if the user matches the specified value. Valid values are:

- **owner | \$OWNER**
Specifies the owner of the task.
- **group | \$GROUP**
Specifies that the current user's logged-on group be the same as one of the groups of the task's responsible party.
- **\$RESPONSIBLE_PARTY**
Specifies the responsible party of the task.
- **privileged | \$PRIVILEGED**
Specifies the responsible party of the task and the owner of the workflow process. If the task does not have a responsible party, the handler ascends the hierarchy of tasks to find the first assigned responsible party.
- **<group>::[*|<role>]**
Specifies a group name and role name to match.
- **<role>**
Specifies a role name to match.

-mode

(Optional) Valid values are:

- **recipient**
Checks if the assignee of the task matches the profiles.

This argument is only effective when the handler is under the **Assign** action and will be ignored when the handler is under the **non-Assign** action.

If present, it must be the first argument. If it is not defined, the default value is **performer**.

- **performer**

Checks if the current login user matches the profiles.

Placement

Requires no specific placement. Typically place on the **Assign**, **Skip**, or **Undo** actions to control access to those actions.

Restrictions

There must be no role in the database with the name **privileged**.

Examples

- This example allows the owner of the workflow process and the responsible party to trigger the action.

Argument	Values
-responsible	privileged

- This example allows any member of the **engineering** group to trigger the action.

Argument	Values
-responsible	engineering::*

- This example allows any user with the role of **manager** to trigger the action.

Argument	Values
-responsible	::manager

- This example allows any user with the role of **designer** in the **engineering** group or the **Project Administrator** role in the **Project Administration** to trigger the action.

Argument	Values
-responsible	Project Administration::Project Administrator, engineering::designer

- This example allows any user with the role of **designer** in the **structure** subgroup of the **engineering** group to trigger the action.

Argument	Values
-responsible	structure.engineering::designer

- This example allows only the **performer** of the task who has a **manager** role to trigger the action.

Argument	Values
-mode	recipient
-responsible	::manager

EPM-check-condition

Description

By default, this handler is placed on the **Complete** action of the **Condition** task, and on the successor tasks of the **Validate** task. When placed on these tasks, no arguments should be used. When placed on the **Complete** action of the **Condition** task, the handler confirms the result of the **Condition** task is either **true** or **false** or the specified custom result. The handler prevents the **Condition** task from completing until the default setting of **unset** has been modified to **true** or **false**. When placed on the successor tasks of the **Validate** task, the handler confirms whether errors occurred (either any error, or the specified errors.)

This handler can also be placed on the **Start** action of all tasks immediately succeeding the **Condition** task. Use the **-source_task** argument to specify the name of the preceding **Condition** task and the **-decision** argument to specify the result (**true**, **false**, or specified custom result) that must be met. (This value is defined during the workflow process template design, when the two or more flow paths that branch from the **Condition** task are created.) The handler returns **EPM_go** when the value matches or **EPM_nogo** when the value does not match. The immediately succeeding tasks only start if they match the required value, resulting in the conditional branching of the workflow process flow.

This handler exists as part of the workflow conditional branching functionality. Manually adding this handler to a task other than a **Condition** task, a task succeeding a **Condition** task, or the successor task of a **Validate** task has no advantage and is not recommended.

Syntax

```
EPM-check-condition -source_task= <task-name>-decision= {true | false |<custom-result> | ANY |
<error-code>}
```

Arguments

-source_task

(Optional) Specifies the name of the preceding **Condition** task. This argument is required if you place the handler on the **Start** action of a task succeeding a **Condition** task.

You must omit this argument if you place the handler on the **Complete** action of a **Condition** task.

-decision

(Optional) Specifies the result that must be met. Use this argument in conjunction with a **Condition** task, placing this handler on a successor task. Valid values are the following:

- *<custom-result>*

Valid values are any string. When the **Condition** task's task results return a value matching the value defined for this argument, the successor task starts when the **Condition** task completes. Multiple values are accepted, separated by commas or the character specified by the **EPM_ARG_target_user_group_list_separator** preference.

Note

This value is automatically set when you use the **Set Custom Result** option to configure the flow path from the **Condition** task to the successor task.

- **ANY**

Use this value in conjunction with a **Validate** task, placing this handler on a successor task. Indicates that if *any* error occurs on the **Validate** task, the workflow process starts the successor task.

Note

This value is automatically set when you use the **Set to Error Path** option to configure a failure path from the **Validate** task to the successor task.

- `<error-code>`

Use this value in conjunction with a **Validate** task, placing this handler on a successor task. Indicates that if the specified error codes occur on the **Validate** task, the workflow process starts the successor task.

Note

This value is automatically set when you use the **Set Error Codes** option to configure a failure path from the **Validate** task to the successor task.

Placement

Place on the **Complete** action of a **Condition** task, the **Start** action of any successor tasks of a **Condition** task, or the successor tasks of a **Validate** task.

Restrictions

None.

Note

Workflow Designer provides a number of prepackaged task templates, such as the **Review** task, **Route** task, and **Acknowledge** task templates. Adding subtasks below any of these tasks to implement a branching condition is not recommended as this may jeopardize the integrity of the task's structure, and doing so may result in unpredictable behavior.

EPM-check-item-status

Description

Verifies that all secondary relations connected by **ImanRelations** of a target item or item revision have been released or that these secondary objects are also target objects in this workflow process. If the target object is an item, this handler checks the item's **Requirements** folder; if the target object is an item revision, this handler checks the item revision's **Specification** folder. All objects in these folders must satisfy these requirements for the handler to return **EPM_go**. The relation, type, and status arguments verify their relation, type, and status, respectively.

Syntax

```
EPM-check-item-status [-relation=<relation-name>] [-include_related_type=<object-type>]
[-allowed_status=<status-name-to-check>]
```

Arguments

- relation**
(Optional) Relation name.
- include_related_type**
(Optional) Object type.
- allowed_status**
(Optional) Status to check.

Placement

Requires no specific placement.

Restrictions

None.

Examples

- This example verifies the text datasets in the **Requirements** folder of a target object have the status of **X**:

Argument	Values
-relation	IMAN_requirement
-include_related_type	Text
-allowed_status	X

- This example verifies all the **UGPART** datasets of a target object have been assigned status. For example, that the datasets are released, or are the target object of the present job:

Argument	Values
-include_related_type	UGPART

EPM-check-object-properties

Description

Checks that a required or non-null value has been entered for the specified properties of the specified object type that is attached to the current workflow process. If any specified properties do not have the required values, an error message lists those properties.

If the specified object type is a form, this handler also checks for form attributes. If the **-check_first_object_only** argument is specified, it only checks the property on the first attached target type. You can use this handler to ensure that you are not releasing the form without defining the mandatory attributes.

In case there are multiple instances of this handler in the same workflow task and multiple workflow targets, each handler instance will make the entire workflow task fail, even if another instance has its criteria fulfilled. So although the Rule Quorum is **OR** for all these instances, the total result will be **false**.

Syntax

```
EPM-check-object-properties -include_type=<object-type>
-property=<property-names>
[-value=<required-values>]
[-attachment=<attachment-type>]
[-check_first_object_only] [-include_replica]
```

Arguments

Note

To check for a single property value that is not null, omit the **-value** argument.

-include_type

(Required) Specifies the type of the workflow target/reference attachments to be checked. Workflow attachments not matching the specified type are not checked.

This argument is used in cases where the check is used only on a specific type subset of workflow attachments, particularly if that property is specific to that type and not found on others.

Note

Multiple values can be added to **-include_type** by using a comma-separated list.

Note

An error does not occur if target/reference objects do not match the **-include_type** value.

-property

(Required) Specifies the properties to be checked. Enter a list separated by commas or the character specified by the **EPM_ARG_target_user_group_list_separator** preference.

Note

If the handler uses a property that references a group member and its value is being checked, then the value should be specified as: **group/role/person name (user id)**.

Caution

If you specify a property of the **Reference** type, the handler checks the referenced object, not the workflow attachment.

-value

(Optional) Specifies the required real values to be checked. Enter real values as defined in Business Modeler IDE.

Caution

Do not enter localized values.

Enter a list separated by commas or the character specified by the **EPM_ARG_target_user_group_list_separator** preference. The order of these values must match the order of properties listed in the **-property** argument.

Note

If **-value** is not specified, then any populated value will be accepted.

-attachment

(Optional) Specifies the type of attachment to be checked.

- **target**
Checks the targets attachment.
- **reference**
Checks the reference attachment.
- **schedule_task**
Checks the schedule task attachment.
- **both**
Checks **target** and **reference** types of attachments.

If this argument is not used, the target attachment is checked.

-check_first_object_only

(Optional) If specified, only the first object of type specified by type is considered. This argument is optional.

-include_replica

(Optional) Checks the **Replica Proposed Targets** as well as the target objects if the **-attachment=target** argument is also specified.

If the **-attachment=schedule_task** argument is specified with this argument, it ignores the attached schedule object if it is a proxy link of schedule task.

Placement

Place on any action except the **Perform** action.

Restrictions

None. Both empty and null values are treated as null values.

Examples

- This example checks the target **CMII CR Form** for nonempty values for **cr_priority** and **prop_soln** properties:

Argument	Values
-include_type	CMII CR Form
-property	cr_priority,prop_soln
-attachment	target

- This example checks the target **CMII CR Form** for the specific value **1 = High** for the **cr_priority** property, and the specific value **Corrective Action** for the **cr_type** property:

Argument	Values
-include_type	CMII CR Form
-property	cr_priority,cr_type
-value	1 = High,Corrective Action
-attachment	target

- This example checks the target **CMII CR Form** for the specific value **1 = High** for the **cr_priority** property, and the specific value **Corrective Action** for the **cr_type** property, and any nonempty value for the **prop_soln** property:

Argument	Values
-property	cr_priority,prop_soln,cr_type
-value	1 = High,,Corrective Action
-include_type	CMII CR Form
-attachment	target

Note

Not placing a value between two commas instructs the system to check for any non-null values for the corresponding property. In the previous example, the second of the three properties to be checked, the **prop_soln** property, corresponds to the empty value. Therefore, any non-null values for this property are checked.

- This example checks the target **CMII CR Form** for the specific value **1 = High** for the **cr_priority** property, and the specific value **Corrective Action** for the **cr_type** property, and any nonempty value for the **prop_soln** property:

Argument	Values
-include_type	CMII CR Form
-property	cr_priority,cr_type,prop_soln
-value	1 = High,Corrective Action
-attachment	target

Note

An alternative method of checking for nonvalues as illustrated in example 3 is to place the property that needs to be checked for nonvalues at the end of the properties list, as in the previous example. This also instructs the system to check for any non-null values for the corresponding property.

- This example checks the target *and* reference **CMII CR Form** for the specific value **1 = High** for the **cr_priority** property, and the specific value **Corrective Action** for the **cr_type** property and any nonempty value for the **prop_soln** property:

Argument	Values
-include_type	CMII CR Form, CMII CN Form
-property	cr_priority,prop_soln,cr_type
-value	1 = High,,Corrective Action
-attachment	both
-check_first_object_only	

EPM-check-related-objects

Description

Checks whether the specified target object contains the required secondary related objects, and whether those objects are in process or have achieved a valid status. You can check only one type of target object per handler. You can check for either a primary or secondary attachment type; the validation confirms the attachment is the specified type and specified relation.

Note

If this handler is checking multiple objects, all objects must meet the criteria to satisfy this handler.

Syntax

EPM-check-related-objects

[include_type=<type-of-target-object>]

**{-primary_type=<type-of-target-object>
|-secondary_type=<secondary-object-type>}**

[-relation=<relation-type>]

**[-allowed_status=<status-names>
| ANY | NONE |
IN_PROCESS]**

[-check_first_object_only]

**[-check_only_for_assembly]
[-check_only_for_component]
[-ignore_empty_bom]
[-negate_return_result]**

Arguments

-include_type

(Optional) Specifies the type of the target object.

-primary_type

(Optional) Specifies the type of the primary attachment.

This argument is mutually exclusive of the **-secondary_type** argument. You may specify only one of these arguments.

-secondary_type

(Optional) Specifies the type of the secondary attachment. This argument is mutually exclusive of the **-primary_type** argument. You may specify only one of these arguments.

-relation

(Required) Specifies the relation to be checked. The relation is between the specified target object and the specified attachment (either the primary attachment or the secondary attachment).

- Specify verification of a manifestation relationship with **IMAN_manifestation**.
- Specify verification of a specification relationship with **IMAN_specification**.
- Specify verification of a requirement relationship with **IMAN_requirement**.
- Specify verification of a reference relationship with **IMAN_reference**.
- Specify verification of a BOM view attachment with **PSBOMViewRevision**.
- Specify verification of an impacted item of a change object with **CMHasImpactedItem**.
- Specify verification of a solution item of a change object with **CMHasSolutionItem**.
- Specify verification of a problem item of a change object with **CMHasProblemItem**.
- Specify verification of a reference item of a change object with **CMReferences**.
- Specify verification of a change object that implements another change object with **CMImplements**.

-allowed_status

(Optional) Specifies the target object status to be verified:

- Specify any Teamcenter status with **ANY**.
- Specify no status, or working status, with **NONE**.
- Specify in process with **IN_PROCESS**.

-check_first_object_only

(Optional) If specified, only the first object of type specified by **-include_type** is considered.

-check_only_for_assembly

(Optional) If specified, the checks specified in the handler are made only on targets that have an assembly (BOM) structure associated with it.

-check_only_for_component

If specified, the checks specified in the handler are only made for non-assembly target type.

-ignore_empty_bom

(Optional) If used, this argument must be specified in combination with the argument **[-check_only_for_assembly]**. Specifying this argument makes the checks applicable only on a non-empty BOM target. Empty BOMs/leaf nodes of BOM that do not have any children are excluded from the check.

-negate_return_result

(Optional) If specified, checks that the primary or secondary attachment type is not present on the target.

Placement

Requires no specific placement.

Restrictions

None.

Examples

- This example checks for a secondary attachment of type **xyz**, with a release status of **Released**, with an **IMAN_specification** relation to the target item revision:

Argument	Values
-include_type	ItemRevision
-secondary_type	xyz
-relation	IMAN_specification
-allowed_status	Released

- This example checks for a primary attachment that is a **ChangelItemRevision**, currently in process, and attached to the target item revision with a **CMHasImpactedItem** relation:

Argument	Values
-include_type	ItemRevision
-primary_type	ChangelItemRevision
-relation	CMHasImpactedItem
-allowed_status	IN_PROCESS

- This example checks for a primary **ChangelItemRevision** attachment that is either a change request (ECR) or change notification (ECN), that is in process, and attached to the target item revision with a **CMHasImpactedItem** relation. This checks for both **ChangeRequestRevision** and **ChangeNoticeRevision** **ChangelItemRevisions**, whether in process or not:

Argument	Values
-include_type	ItemRevision
-primary_type	ChangelItemRevision:: ChangeRequestRevision~ ChangeNoticeRevision
-relation	CMHasImpactedItem
-allowed_status	IN_PROCESS

- This example checks for any released secondary **xyz** attachment with an **IMAN_specification** relation to the **type1** target object:

Argument	Values
-include_type	type1
-secondary_type	xyz
-relation	IMAN_specification
-allowed_status	ANY

- This example checks for a secondary **xyz** attachment with no status in the **Impacted Items** folder of the target change object revision:

Argument	Values
-include_type	ChangeItemRevision
-secondary_type	xyz
-relation	CMHasImpactedItem
-allowed_status	NONE

- This example checks for a secondary dataset attachment with a working status attached to the target item revision. Defining the **secondary_type** as **Dataset** checks for all dataset types of the defined relation:

Argument	Values
-include_type	ItemRevision
-secondary_type	Dataset
-relation	IMAN_specification
-allowed_status	NONE

- This example checks for a secondary attachment of type **xyz**, with a release status of **Released**, with an **IMAN_specification** relation to the target item revision only:

Argument	Values
-include_type	ItemRevision
-secondary_type	xyz
-relation	IMAN_specification
-allowed_status	Released

EPM-check-responsible-party

Description

Verifies that the current user is the responsible party for the task (every task has a default responsible party). If not, it verifies whether the current user meets the criteria specified in the argument of the handler.

Syntax

```
EPM-check-responsible-party [-responsible={User|Group|Role}:<value>]
```

Arguments

-responsible
(Optional) Defines an additional responsible party.

Placement

Place on the **Perform** action of the task.

Restrictions

This handler cannot be placed on the **Perform** action of the *root* task.

Examples

This example shows user **george**, members of group **dba**, and the responsible party being allowed to perform the action associated with this handler.

Argument	Values
-responsible	User:george, Group:dba

EPM-check-signoff

Description

Checks decisions of all the signoffs attached to this task. If the number of approvals is greater than, or equal to, the quorum, then **EPM_go** is returned. If it is possible to obtain enough approvals from those signoffs without a decision, **EPM_undecided** is returned. Otherwise, there are too many rejections and the function **EPM_nogo** is returned.

Syntax

EPM-check-signoff -quorum=<n>

Arguments

-quorum

(Required) Specifies the approval quorum, where <n> is an integer specifying the quorum. A value of -1 sets the quorum equal to the total number of signoffs; in other words, a unanimous decision is required.

Placement

Requires no specific placement.

Restrictions

None.

EPM-check-status-progression

Description

Checks the complete release status progression of a specific object. For example, this handler identifies the last status added on any item revision because the handler considers that the latest status for that item revision.

- This handler can also check whether the object follows a nonlinear progression. A nonlinear progression does not require every subsequent release status of an object to follow the progression path in the same order, though the latest release status must always be greater than the previous release status. For example, if the progression path is **Experimental**, **Quote**, **Design**, **Manufacture**, **Production**, the object can achieve **Experimental**, **Quote**, and then **Production** release statuses, skipping **Design** and **Manufacture**.
- If the workflow process contains several **Condition** tasks that apply different release statuses at different levels, the value provided in the **-status** argument can be used. If this argument is not used in this situation, the status applied to the target object is applied to the object.

Syntax

EPM-check-status-progression

[-status=<status-being-applied-to-the-target-object>]

[-rev=current_rev|previous_rev|latest_rev|greatest_released_rev]

Arguments

-status

(Optional) Derives the status being applied to the target object.

-rev

(Optional) Checks for one of the following:

- Only the current revision, use **current_rev**. Even if the previous revision is released to a production status, the current revision is released to a lesser status than production.
- The latest release status of the immediately previous revision, use **previous_rev**.
- The greatest release status of all the revisions of the target, use **latest_rev**.

For example: An object has revisions **A**, **B**, and **C**. Revision **A** is released later than revision **B**, and **C** is not released. The **latest_rev** option returns **A**.

- The latest release status of the greatest release status of the target object, use **greatest_released_rev**.

For example: An object has revisions **A**, **B**, and **C**. Revision **A** is released later than revision **B**, and **C** is not released. The **greatest_released_rev** option returns **B**.

Note

The **EPM-check-status-progression** rule handler first identifies the last status added on an item revision. The handler considers that the latest status for that item revision. Then this handler looks at the various **-rev** arguments to determine which revision to use.

When checking the last status added to each revision, status maturity is established by the release status order in the **ProgressionPath.plmxml** file.

Placement

Place on any task action. Typically placed on the **Complete** action of the **perform-signoffs** task.

Restrictions

None.

Examples

- This example checks the status of design against the progression path when the workflow process contains several **Condition** tasks, which apply different release statuses at different levels:

Argument	Values
-status	Design

- In this example, consider the scenario:
 - Progression path: **Quote, Experimental, Development, Design, Manufacturing, Production**
 - IR ABC123
 - IR ABC123/001 has **Experimental** status
 - IR ABC123/002 in **Working** state
 - IR ABC123/003 status not yet applied

To release IR ABC123/003 based on the current revision status only, define the following arguments. Previous revision statuses are not checked. Even if the previous revision was released to a **Production** status the current revision can be released to a lesser status than **Production**. In this scenario, IR ABC123/003 can be released to **Quote** status or upward, even though IR ABC123/001 is released to **Experimental** status.

Argument	Values
-rev	current_rev

- In this example, consider the previous scenario. To release IR ABC123/003 based on the latest release status of its immediate previous revision, define the following arguments. The previous revision is IR ABC123/002, which is in **Working** state and does not have a status applied. In this case, IR ABC123/003 can be released to **Quote** status or upward.

Argument	Values
-rev	previous_rev

- In this example, consider the previous scenario. To release IR ABC123/003 based on the last status of the latest released revision, define the following arguments. The latest released revision is IR ABC123/001, its last status was **Experimental**. In this case, IR ABC123/003 can be released only to **Experimental** status or upward.

Argument	Values
-rev	latest_rev

- In this example, consider the progression path and values:
 - Progression path: **Quote, Experimental, Development, Design, Manufacturing, Production**.
 - IR XYZ123
 - IR XYZ123/001 has **Design** status
 - IR XYZ123/002 has **Experimental** status
 - IR XYZ123/003 has **Development** status
 - IR XYZ123/004 status not yet applied

To release IR XYZ123/004 based on the greatest release status among all the revisions of the target object, define the following arguments. IR XYZ123/004 releases as **Design**.

Argument	Values
-rev	greatest_released_rev

Additional Information

The progression path must be manually defined in the **ProgressionPath.plmxml** file before the handler can reference the path. The file is stored in the <TC_DATA> directory. Create a backup copy of this file before editing it.

All target types that you want to follow the progression path must be set in this file. A **UserData** block must be created for each type that follows a progression path. For example, to define the progression path for the **ItemRevision**, **PSBOMView**, and **MSWord** types, the **UserData** blocks can be defined as follows:

```
<UserData id="id1">
  <UserValue title="Type" value="ItemRevision"/>
  <UserValue title="ReleaseProgressionList"
    value="Quote, Development, Prototype, Production">
  </UserValue>
</UserData>
<UserData id="id2">
  <UserValue title="Type" value="PSBOMView"/>
  <UserValue title="ReleaseProgressionList"
    value="Quote1, Development1, Prototype1, Production1">
  </UserValue>
</UserData>
```

```
</UserValue>
</UserData>
<UserData id="id3">
  <UserValue title="Type" value="MSWord"/>
  <UserValue title="ReleaseProgressionList"
    value="Quote2, Development2, Prototype2, Production2">
  </UserValue>
</UserData>
```

Note

- Add the **UserData** blocks between the **<PLMXML>** and **</PLMXML>** tags.
- Ensure you increment the **UserData id** value when you add a new entry.
- After adding a new **UserData** block, change the value for **Type** to a type you are defining.
- You can modify the value of the release status to meet your requirements.

EPM-check-target-attachments

Description

Checks that the specified target object contains the required attachment with the required status or statuses. You can provide the target object type, relation type, attached object type, and valid statuses as handler arguments.

This handler can be used with an LOV to specify different types of targets and attachments to be checked, requiring just one occurrence of the handler. For an overview of using LOVs in handlers, see [Using lists of values \(LOVs\) in handler arguments](#).

Note

Enable [debugging functionality](#) for this handler with the `TC_HANDLERS_DEBUG` environment variable.

Syntax

EPM-check-target-attachments { **-include_type**=<target-object-type>

-include_related_type=<attached-object-type>

-relation=<relation-type>} | **-lov**=<lov-name>}

[**-allowed_status**=<valid-status-names> | **ANY** | **NONE**]

Arguments

-include_type

(Optional) Defines the type of target object to be checked.

Note

To check multiple values for a single argument, separate the values with commas or the character specified by the `EPM_ARG_target_user_group_list_separator` preference.

Example

The following example checks if the **UGMASTER** or **UGPART** dataset exists in the **ItemRevision** type with the <IMAN> specification relation **EPM-check-target-attachments**:

```
-include_type<=ItemRevision >
-include_related_type<= UGMASTER,UGPART>
-relation<=IMAN specification >
-allowed_status<=NONE>
```

-include_related_type

(Optional) Defines the type of attachment to be checked.

-relation

(Optional) Specifies the relation between the target object and the attachment:

- Specify a manifestation relationship with **IMAN_manifestation**.
- Specify a specification relationship with **IMAN_specification**.
- Specify a requirement relationship with **IMAN_requirement**.
- Specify a reference relationship with **IMAN_reference**.
- Specify a BOM view attachment with **PSBOMViewRevision**.
- Specify an impacted item of a change object with **CMHasImpactedItem**.
- Specify a solution item of a change object with **CMHasSolutionItem**.
- Specify a problem item of a change object with **CMHasProblemItem**.
- Specify a reference item of a change object with **CMReferences**.
- Specify a change object that implements another change object with **CMImplements**.

-allowed_status

(Optional) Specifies the required status of the attachment. Multiple statuses can be checked by listing valid Teamcenter statuses separated by commas or the character specified by the **EPM_ARG_target_user_group_list_separator** preference.

ANY checks for any status. **NONE** checks for working status.

-lov

(Optional) Specifies the list of values (LOVs) used to define which objects are attached to which target objects.

This argument is mutually exclusive of the **-include_type**, **-include_related_type**, and **-relation** arguments. It can be used with the **-allowed_status** argument to check relation status.

See the LOV row, for the required LOV format.

LOV

For an overview of using LOVs in handlers, see [Using lists of values \(LOVs\) in handler arguments](#).

The LOV can contain multiple optional lines: a line for each type of target to check, followed by one or more multilevel object path lines specifying the relations required for that target type.

For an overview of using multilevel object paths in handlers, see [Defining multilevel object paths](#).

If the system does not find any targets for one of the target types, it checks the next target type line.

When a target exists for the specified type, then each relation listed must exist. An error is reported for each relation type missing.

[\$TARGET.]<target-(class)-or-type-1>

<relation1>.<sec-obj-(class)-or-type-in-target-1>

<relation2>.<sec-obj-(class)-or-type-in-target-1>

[\$TARGET.]<target-(class)-or-type-2>

<relation1>.<sec-obj-(class)-or-type-in-target-2>

<relation2>.<sec-obj-(class)-or-type-in-target-2>

...

Note

When using a LOV with this handler, you can improve readability and clarity by indenting the relation lines with spaces. You can also add line numbers in square brackets.

[\$TARGET.]<target-(class)-or-type-1>

Defines the type/class of target to check, using a comma-separated list of types/classes in the format shown next.

Target lines are prefixed with **\$TARGET** or identified by their lack of dots (.).

[(Class)[!Type1][,(Class2)[,Type1[,...]]]]

For example, to specify that all item revisions are checked except software revision:

(ItemRevision)!Software Revision

<relation1>.<sec-obj-(class)-of-type-in-target-1>

A multilevel object path that must start with a relation (such as **IMAN_specification**). Defines a secondary object that must exist in the specified relation for the target line.

Relation lines always contain a dot (.).

For example, to check that a **UGMASTER** and **UGPART** dataset exist in all revision targets of the design revision type:

\$TARGET.Design Revision

IMAN_specification.UGMASTER

IMAN_specification.UGPART

Placement

Requires no specific placement.

Restrictions

If checking multiple statuses through LOVs, this handler must be used once for each status.

Examples

- This example checks the targeted change revision for an item revision with any status in the **Problem Items** folder:

Argument	Values
-include_type	ChangeItemRevision
-include_related_type	ItemRevision
-relation	CMHasProblemItem
-allowed_status	ANY

- This example checks the targeted change revision for an item revision with no status in the **Impacted Items** folder:

Argument	Values
-include_type	ChangeItemRevision
-include_related_type	ItemRevision
-relation	CMHasImpactedItem
-allowed_status	NONE

- This example checks the targeted change revision for the **CORP_Part** revision with a released status in the **Solution Items** folder:

Argument	Values
-include_type	ChangeItemRevision
-include_related_type	CORP_PartRevision
-relation	CMHasSolutionItem
-allowed_status	Released

Alternatively, you can use these LOV settings:

Argument	Values
-lov	SYS_EPM_check_target_attachments
-allowed_status	Released

where the **SYS_EPM_check_target_attachments** LOV contains this data:

\$TARGET.ChangelItemRevision
CMHasSolutionItem.CORP_PartRevision

- This example checks the targeted change revision for an item revision for any status of the following statuses (**Concept Approval**, **Funding Approval**, **Design Approval**) in the **Solution Items** folder:

Argument	Values
-include_type	ChangelItemRevision
-include_related_type	ItemRevision
-relation	CMHasSolutionItem
-allowed_status	Concept Approval,Funding Approval,Design Approval

- This example checks the targeted change revision for an item revision in the **Solution Items** folder, irrespective of status:

Argument	Values
-include_type	ChangelItemRevision
-include_related_type	ItemRevision
-relation	CMHasSolutionItem

- This example performs specific relation checks for particular revision type targets and other relation checks for the remaining revision types all with no status:

Argument	Values
-lov	SYS_EPM_check_target_attachments
-allowed_status	NONE

where the **SYS_EPM_check_target_attachments** LOV contains this data:

Value	Description
Software Revision, DocumentRevision	Check that any software and document revision targets have a text dataset attached in the IMAN_specification relation.

Value	Description
IMAN_specification.Text	
DocumentRevision	
IMAN_specification.Word, Excel, PowerPoint	Check that any DocumentRevision targets also have a Word, Excel OR PowerPoint dataset attached in the IMAN_specification relation.
(ItemRevision)!Software Revision! DocumentRevision	Check that any other targets of class ItemRevision , (in other words, that are not SoftwareRevision or DocumentRevision) have a UGMASTER and UGPART attached in the IMAN_specification relation.
IMAN_specification.UGMASTER	
IMAN_specification.UGPART	
(ItemRevision)	
Proj.Project	Check that any revision targets also have a project item attached to the custom Proj relation.

Note

The relation lines are indented for clarity.

EPM-check-target-object

Description

Checks the status of the object to determine whether to allow the action.

Note

Enable debugging functionality for this handler with the **TC_HANDLERS_DEBUG** environment variable.

Syntax

EPM-check-target-object -allowed_status=
<status-name>| -disallowed_status=<status-name>

Arguments

-allowed_status

(Optional) Defines statuses to check against target objects. If a potential target matches any of the statuses defined with this argument, paste is available.

Accepts one or more valid Teamcenter status names.

Indicate *any* status with one of the following:

***|all|ALL|any|ANY**

Indicate *no* status with one of the following:

null|NULL|none|NONE

Indicate *in process* status:

IN_PROCESS

-disallowed_status

(Optional) Defines statuses to check against target objects. If a potential target matches any of the statuses defined with this argument, paste is unavailable. Can use in place of **-status** for clarity. A warning message is displayed indicating noncompliance to the business rule when you click **OK**. Additionally, if the argument passed to the handler is incorrect, this warning message is also displayed when you click **OK**.

Accepts one or more valid Teamcenter status names.

Indicate *any* status with one of the following:

***|all|ALL|any|ANY**

Indicate *no* status with one of the following:

null|NULL|none|NONE

Indicate *in process* status:

IN_PROCESS

Placement

Place on the **Perform** action of the root task.

Restrictions

None.

Examples

- This example allows any target to be attached with a status of **Pending** or with no status (work in progress):

Argument	Values
-allowed_status	Pending, NONE

- This example disallows any targets from being attached with a status of **Released** or **Obsolete**:

Argument	Values
-disallowed_status	Released, Obsolete

EPM-debug-rule

Description

Notifies a user that an action is executing. Attaching **EPM-debug-rule** to any EPM action notifies the user when that task action runs by printing that action name to the standard output device.

Syntax

EPM-debug-rule -comment=<string>

Arguments

-comment

(Optional) Additional descriptive string appended to the action name.

Placement

Requires no specific placement.

Restrictions

None.

Examples

This example notifies the user when the **Complete** action runs by printing **Complete, action is executing** to the standard output device.

Argument	Values
-comment	action is executing

Note

This example assumes you have attached this handler to a **Complete** action.

EPM-disallow-adding-targets

Description

Disallows adding targets interactively after a workflow process is initiated. A switch can be used to specify the types of objects to be excluded. If you configure other handlers to add targets programmatically, they are added during the workflow process even if this handler is used.

It is good practice to add this handler to the root task **Perform** action to ensure that target objects are not added from a workflow process once it is started. If you want to allow the addition of objects of all types as targets, this handler should be removed from the respective workflow process template, and you must ensure that the desired users have change access to the workflow process (job) object. You may need to use the **EPM-set-rule-based-protection** handler to ensure that the required change access is asserted.

Note

The **EPM-attach-related-objects** and **PS-attach-assembly-components** handlers are dependent on this handler.

Syntax

EPM-disallow-adding-targets [-exclude_type=<type-of-object [, type-of-object2]>]

Arguments

-exclude_type=<type-of-object [, type-of-object2]>
(Optional) Types of objects that are allowed to be added as targets after the workflow process is initiated.

Placement

Place on the **Perform** action of the root task.

Restrictions

None.

Examples

This example allows only BOM view revisions to be added interactively as targets after the workflow process is initiated.

Argument	Values
-exclude_type	BOMView Revision

EPM-disallow-removing-targets

Description

Prevents targets from being removed from a workflow process after the workflow process has been started.

It is good practice to add this handler to the root task of the **Perform** action. This prevents target objects from being removed from a workflow process once it is started. To allow the removal of targets, verify that this handler has been removed from the respective workflow process template (if it has not been removed, do so) and ensure that the desired users have *change* access to the workflow process object. You may need to use the **EPM-set-rule-based-protection** handler to ensure that the required *change* access is asserted.

Note

The named ACL must have *change* access to provide the proper protection.

Syntax

EPM-disallow-removing-targets

Arguments

None.

Placement

Place on the **Perform** action of the root task.

Restrictions

None.

EPM-disallow-reviewers

Description

Prevents specified users, the workflow process owner, reviewers for a specified task, reviewers from all tasks, or a combination of them from being added to a signoff team in a **Review** task.

Syntax

```
EPM-disallow-reviewers -assignee=user:[<user-name-1>] [,user:<user-name-2>,...] |
[user:$PROCESS_OWNER] -task=[<parent-task-name>:<sub-task-name> | ALL]
```

Arguments

-assignee

(Optional) Specifies the user IDs and/or the workflow process owner that are not allowed as reviewers.

Any Teamcenter users or **\$PROCESS_OWNER** are specified in the following format:

user:<user-name-1>, **user:**<user-name-2>, ...

You must use either the **-assignee** or the **-task** argument. You can optionally use both.

-task

(Optional) Specifies the parent task and subtask names, separated by a colon (:), for an existing **select-signoff-team** task in the workflow process. Reviewers for this task are not allowed as reviewers for the task with this handler. You can specify all tasks in the workflow process with the **ALL** keyword.

You must use either the **-assignee** or the **-task** argument. You can optionally use both.

Placement

Place *only* on the **Complete** action of the **select-signoff-team** task.

Restrictions

None.

Examples

- This example prevents the user **Smith** from being a reviewer:

Argument	Values
-assignee	user:Smith

- This example prevents the workflow process owner and user **Smith** from being reviewers:

Argument	Values
-assignee	user:\$PROCESS_OWNER, user:Smith

- This example prevents the existing reviewers on the **Review1:SST1** task from being reviewers:

Argument	Values
-task	Review1:SST1

- This example prevents the existing reviewers on all other **select-signoff-team** tasks within the workflow process from being the reviewers:

Argument	Values
-task	ALL

- This example prevents the process owner and existing reviewers on the **Review1:SST1** task from being reviewers:

Argument	Values
-assignee	user:\$PROCESS_OWNER
-task	Review1:SST1

EPM-hold

Description

Pauses the task, requiring the user to perform an action on the task before the task can complete. Typically, a task completes automatically once started. **EPM-hold** prevents this automatic completion.

Use this rule handler with custom tasks that require customized **Perform** actions, or to require the user to manually perform a **Complete** action to complete the task.

This handler checks the **task_result** property of the task to which it is attached. If this property is not set to **Completed**, this handler pauses the task. If the value is set to **Completed**, the task progresses normally.

In addition, in case of **Notify** tasks that are sub-tasks of **Route** tasks, this handler checks whether the reviewers are completely assigned to the **Route** task. If the reviewers' assignment is complete, then it allows the **Notify** task to proceed even if the value of **task_result** property of the **Notify** task is not set to **Completed**.

Configuring a task to display forms using EPM-display-form, EPM-hold, and EPM-create-form

To configure a task to display a form when a user performs a specified action, use the **EPM-hold** handler. This handler pauses the task, requiring the user to perform an action on the task before the task can complete. Without the use of this handler, a task completes automatically once started.

To create an instance of a specified form and attach the form to the specified task, use the **EPM-create-form** handler.

Therefore, the **EPM-create-form** handler creates the form when the **Start** action is initiated, the **EPM-display-form** handler displays the form when the **Perform** action is initiated, and the **EPM-hold** handler prevents the task from automatically completing, allowing the form to be completed by the user.

Variations on the above example may be required for a more sophisticated interaction when it is required that the task not complete until required fields are entered in the form. This type of configuration requires the creation of customized rule handlers.

Syntax

EPM-hold

Arguments

None.

Placement

Place on the **Complete** action of any task with which you want the user to interact before the task completes.

Restrictions

None.

Additional Information

- By default, this handler is placed in the **Do** task template, pausing the task to allow the **Do Task** dialog box to display when the user performs the **Perform** action on a selected **Do** task.
- Use this handler with custom tasks that present custom forms when the user performs the **Perform** action.

For information about configuring custom tasks to present custom forms when the **Perform** action is invoked, see the description of the [EPM-display-form](#) handler.

EPM-invoke-system-rule

Description

Runs an external command (specified with the **-command** argument) such as Perl scripts, shell scripts, or external ITK programs, then continues or halts the workflow process based on the return code of the external command.

Use this handler for increased control of the workflow process. For example, to synchronize NX attributes and structure with Teamcenter, or to generate JT tessellation from CAD files.

This handler writes process-related information to an XML file. The file is passed to the external script or program as **-f <XML-file-name>**. APIs are provided (in the form of Perl modules) to read the XML file and perform functions on its data objects. The APIs are located in the **Workflow.pm** file in the **<TC_ROOT>/bin/tc** directory.

Write Perl scripts (for example, **<TC_ROOT>/bin/iman_check_renderings.pl** for background tessellation of CAD data) using the provided APIs to read the XML file and perform required functions on its data objects. Then use the Perl script as the value of the **-command** argument in the workflow process template.

Note

Siemens Digital Industries Software recommends you place the Perl scripts in the **<TC_ROOT>/bin** folder.

Alternatively, you can place the script in an alternate location and provide an absolute path to the location (for example, **c:\temp\test.bat**). However, using an absolute path requires that you update the template if there are any changes. In the previous example, **c:\temp\test.bat** is a path on a Windows platform. If you were to change to a Linux platform, the template would need to be updated. This second method is not recommended.

The handler returns a code that is mapped to:

- **EPM_go** when the external script returns **0** or **EPM_go** and no other errors are returned
- **EPM_nogo** when the external script/program returns error or **EPM_nogo**
- **EPM_undecided** when the external script/program returns **EPM_undecided**

Syntax

```
EPM-invoke-system-rule -command=<name-of-the-external-program>
[-trigger_on_go= [<task>:]<action>]
[-trigger_on_nogo= [<task>:]<action>]
[-trigger_on_undecided= [<task>:]<action>] [-skip_unreadable_objs]
[-change_status_on_go= [<old-status-name>:][<new-status-name>]]
[-change_status_on_nogo= [<old-status-name>:][<new-status-name>]]
[-change_status_on_undecided= [<old-status-name>:][<new-status-name>]]
[-add_occurrence_notes] [-comment=<signoff-comment>]
```

```
[-responsible_party= [User:<responsible-party>; Task:<task-name>]]
[-reviewer= [User:<user-id>] [; Group:<group>] [; Role:<role>] [; Level:<level>]]
[-send_mail=<user-ids>] [-initiate_process] [-where_used=<item-revision-type>]
[-expand=<item-revision-type>] [-list_sibling_processes=<wildcarded-procname>]
[-depth=<maximum-recursion-depth>] [-debug]
```

Arguments

-command

(Required) Name of the external executable. This executable can be an external Perl script that reads and modifies the XML file that this handler writes, or an ITK program to perform specific functionality.

-trigger_on_go

(Optional) Triggers an action in the same workflow process when **EPM_go** is returned.

Trigger an action in another task by specifying an action and task name. If another task name is unspecified, the specified action in the current task is triggered.

The system supports the following actions:

ASSIGN, START, PERFORM, COMPLETE, SUSPEND, RESUME, SKIP, ABORT, REFUSE, UNDO, REJECT, APPROVE, PROMOTE, DEMOTE.

Action names are not case sensitive.

Task names cannot contain a colon or a period. If the task name is ambiguous (for example, **select-signoff-team**), hierarchical notation is required.

-trigger_on_nogo

(Optional) Triggers an action in the same workflow process when **EPM_nogo** is returned. Trigger an action in another task by specifying an action and task name. If another task name is unspecified, the specified action in the current task is triggered.

The system supports the following actions:

ASSIGN, START, PERFORM, COMPLETE, SUSPEND, RESUME, SKIP, ABORT, REFUSE, UNDO, REJECT, APPROVE, PROMOTE, DEMOTE.

Action names are not case sensitive.

Task names cannot contain a colon or period. If the task name is ambiguous (for example, **select-signoff-team**), hierarchical notation is required.

-trigger_on_undecided

(Optional) Triggers an action in the same workflow process when **EPM_undecided** is returned.

Trigger an action in another task by specifying an action and task name. If another task name is unspecified, the specified action in the current task is triggered.

The system supports the following actions:

ASSIGN, START, PERFORM, COMPLETE, SUSPEND, RESUME, SKIP, ABORT, REFUSE, UNDO, REJECT, APPROVE, PROMOTE, DEMOTE.

Action names are not case sensitive.

Task names cannot contain a colon or period. If the task name is ambiguous (for example, **select-signoff-team**), hierarchical notation is required.

-skip_unreadable_objs

(Optional) Unreadable objects are not processed. The handler does not attempt to write information about unreadable objects into the XML file; the objects are skipped.

If this argument is not specified, the handler displays an error when a failure occurs when there is no read access.

-change_status_on_go

(Optional) Adds, removes, or changes the status of attachments when **EPM_go** is returned.

Both the old and new status names are optional.

- If both status names are specified, the new status name replaces the old status name.
- If only the new status name is specified, the corresponding status is added.
- If only the old status name is specified, the corresponding status name is removed.
- If neither status name is specified, no action is taken.

If a value is not provided for this argument, the value set by the external Perl script is read.

-change_status_on_nogo

(Optional) Adds, removes, or changes the status of attachments when **EPM_nogo** is returned.

Both the old and new status names are optional.

- If both status names are specified, the new status name replaces the old status name.
- If only the new status name is specified, the corresponding status is added.
- If only the old status name is specified, the corresponding status name is removed.
- If neither status name is specified, no action is taken.

If a value is not provided for this argument, the value set by the external Perl script is read.

-change_status_on_undecided

(Optional) Adds, removes, or changes the status of attachments when **EPM_undecided** is returned.

Both the old and new status names are optional.

- If both status names are specified, the new status name replaces the old status name.

- If only the new status name is specified, the corresponding status is added.
- If only the old status name is specified, the corresponding status name is removed.
- If neither status name is specified, no action is taken.

If a value is not provided for this argument, the value set by the external Perl script is read.

-add_occurrence_notes

Sets occurrence notes of target assemblies. Can be used in combination with the **-expand** argument to set **OccurrenceNotes** for components of assembly structures.

-comment

(Optional) The signoff decision is set depending on the return code of the external program:

- 0=Approve
- 1=Reject
- 2=No Decision

If a value is not provided for this argument, the value set by the external Perl script is read.

-responsible_party

Assigns a responsible party. If no user ID is specified for this argument, the value set by the external Perl script is read.

-reviewer

(Optional) Assigns a reviewer for a release level. If no reviewer is specified for this argument, the value set by the external Perl script is read.

-send_mail

(Optional) Sends target, reference, or sibling objects through program mail. If one or more user IDs are defined for this argument, the workflow process is sent to the specified users through program mail.

Separate multiple user IDs with a space, a comma, or the character specified by the **EPM_ARG_target_user_group_list_separator** preference.

If no user IDs are defined for this argument, the recipients and the contents of the envelope set by the external Perl script is read.

-initiate_process

(Optional) Initiates a workflow process for another object. Target objects are defined by the values set by the external Perl script.

-where_used

(Optional) Reports the where-used of item and item revision target attachments by writing the hierarchy of all parent and grandparent assemblies of item and item revision target attachments into the XML file to allow the external Perl script to perform required functions. If an **ItemRevision** type is specified, the type argument is compared to the corresponding item revision type. For example, **ItemRevision** matches objects of the **Item** type. If an item revision type is specified, the parent assemblies of only those target attachments that match this type are listed.

-expand

(Optional) Reports the assembly of item and item revision target attachments by writing the hierarchy of all child and grandchild components of item and item revision target attachments into the XML file to allow the external Perl script to perform required functions.

If an **ItemRevision** type is specified, the type argument is compared to the corresponding item revision type. For example, **ItemRevision** matches objects of the **Item** type. The assembly structure is expanded for all item revision of all matching item target attachments.

If an item revision is specified, the child components of only those target attachments are listed that match this type.

-list_sibling_processes

(Optional) Writes information regarding processes that belong to the same **Change** item into the XML file to allow the external Perl script to perform required functions. The information concerns processes sharing the same **Change** item as a reference attachment.

If a process template name is specified in the procedure definition, only the processes that match the procedure name are included.

-depth

(Optional) Increases the maximum incursion depth. The **-trigger_on_go** or **-initiate_process** arguments could cause the triggered action to use the same handler in a deeper level of recursion. If this is intended, the maximum level of recursion must be set to the desired number. If necessary, it can be disabled by setting it to 0. The default is set to 1, to avoid infinite loops.

-debug

(Optional) Enables debugging. Each occurrence of this argument increases the debug level by one. Debug messages are written to the Teamcenter error stack for display in the rich client user interface, as well as written to the syslog file.

Placement

Place on the **Start** or **Complete** action of any task. If this handler is configured to set the signoff decisions on a **perform-signoffs** task (for example, if the **-comment** argument is specified), then place on the **Complete** action of the **perform-signoffs** task.

Restrictions

Do not add to a workflow process containing *any* handler using resource pools.

Examples

This example shows how to run the **iman_check_renderings_pl** script using the **-command** argument. Do not list the file extension in the value. This value runs either the **iman_check_renderings_pl.bat** (Windows) or **iman_check_renderings_pl** (Linux) script, depending on which platform the server is running.

Note

The script should be placed in the `<TC_ROOT>/bin` directory.

Argument	Values
-command	iman_check_renderings_pl

EPM-signoff-team-validation

Description

Checks to ensure the minimum number of reviewers specified by the **-num_reviewers** argument is assigned to the **select-signoff-team** task. If no argument is provided, the handler checks for at least one reviewer.

If the number of reviewers assigned to the **select-signoff-team** task is less than the minimum reviewers required, then **EPM_nogo** is returned.

Syntax

EPM-signoff-team-validation [-num_reviewers= <minimum-number>]

Arguments

-num_reviewers

(Optional) Minimum number of reviewers required for the **select-signoff-team** task.

Placement

Place *only* on the **Complete** action of the **select-signoff-team** task.

Restrictions

None.

Examples

This example checks to see if at least 2 reviewers are assigned to the **select-signoff-team** task.

Argument	Values
-num_reviewers	2

EPM-validate-target-objects

Description

Restricts the types of objects that can be added as target objects. It always prevents the **Home**, **Newstuff**, and **MailBox** folders from being added as target objects.

Note

Enable **debugging functionality** for this handler with the **TC_HANDLERS_DEBUG** environment variable.

Syntax

EPM-validate-target-objects

```
[-include_type =<type-of-workspace-object[, type-of-workspace-object2,..]>]
[-exclude_type =<type-of-workspace-object[, type-of-workspace-object2,..]>]
[-latest_rev]
```

Arguments

-include_type

(Optional) Defines the type of objects that can be added as target objects to a workflow process. You can define more than one type by using commas or the character specified by the **EPM_ARG_target_user_group_list_separator** preference between the types.

Accepts valid Teamcenter object types, such as **ItemRevision**, **UGMASTER**, and **UGPART**.

When you add any object type or class as a target, all its subtypes are also included. To explicitly exclude any subtypes, use the **-exclude_type** argument.

For example, if this argument is specified as **ItemRevision**, any type of item revision (for example, **DocumentRevision**, and so on, and any custom item revision types) is allowed.

Does not accept bracketed () class notation to distinguish between classes and types.

-exclude_type

(Optional) Defines the type of objects that cannot be added as target objects to a workflow process. You can define more than one type by using commas or the character specified by the **EPM_ARG_target_user_group_list_separator** preference between the types.

Accepts valid Teamcenter object types, such as **ItemRevision**, **UGMASTER**, and **UGPART**.

If this argument is specified as **ItemRevision**, any type of item revision (for example, **DocumentRevision**, and so on, and any custom item revision types) is disallowed.

-latest_rev

(Optional) Ensures any revisions added to the workflow process are the latest revision within their owning item.

Placement

Place on any action in any task.

Restrictions

None.

Examples

- This example allows only item revisions as targets:

Argument	Values
-include_type	ItemRevision

- This example allows **MEOPRevision** objects as the targets and disallows **MENCMachining Revision** and **METurningRevision** objects:

Argument	Values
-include_type	MEOPRevision
-exclude_type	MENCMachining Revision, METurningRevision

Note

MEOPRevision is the parent type (class) for **MENCMachining Revision** and **METurningRevision**. In this example, all **MEOPRevision** subtypes are allowed as targets except for **MENCMachining Revision** and **METurningRevision**.

- This example allows only the latest item revisions as targets:

Argument	Values
-include_type	ItemRevision
-latest_rev	

EPM-verify-digital-signature

Description

Verifies if the target objects and, optionally, the schedule task have a valid digital signature.

Syntax

EPM-verify-digital-signature [-include_schedule_task] [-quorum=<size>] [-no_void]

Arguments

-include_schedule_task

(Optional) Verifies the digital signature on the schedule task and all target objects of the workflow. If this argument is not provided, the digital signature is verified only on the target objects of the workflow.

-quorum

(Optional) Specifies the minimum number of valid digital signatures each target must have, where <size> is a positive integer specifying the quorum. If this argument is not specified, all digital signatures on all targets must be valid.

-no_void

(Optional) Checks each target object in the workflow for a void digital signature. If the target object has one or more void digital signatures, the handler fails with an error indicating the failure, even if the quorum in the **-quorum** argument for valid digital signatures is met.

Placement

Place on any action on any task.

Restrictions

None.

ERP-check-effective-date-RH

Description

Checks the **Effect In** date on the release status attached to the process does not have a value before the current date.

Syntax

ERP-check-effective-date-RH

Arguments

None.

Placement

Place on the **perform-signoff** task.

Restrictions

None.

ERP-check-target-status-RH

Description

Checks that the release status for target item revisions is specified.

Syntax

ERP-check-target-status-RH -status_name=<name>

Arguments

-status_name

Specifies the name of the release status.

Restrictions

None.

ERP-validate-data-RH

Description

Applies the validation criteria specified in the mapping schema on all forms attached to the process's transfer folders and related **BOMComponent** data. The following validations are performed:

- For each attribute:
 - If the attribute parameter is required, the field must have a value.
 - If the attribute definition has an LOV, the value in the field must match one in the list. Although this is checked at entry time, this allows for LOVs that changed in the mapping since the data was originally entered.

For an overview of using LOVs in handlers, see [Using lists of values \(LOVs\) in handler arguments](#).

 - For string attributes, the length of string entered must be no more than that defined in the schema.
 - If there is a custom validation function defined using the **custom_check** attribute parameter, call the function.
- For each **BOMHeader** to be sent to ERP:
 - Check a corresponding BOMView revision of the correct type exists, as described for the [SAP-check-forms-attached-RH](#) handler.
 - Check all components with the same item ID have the same attribute values (for those attributes specified in the mapping schema, except quantity).
 - Check component attribute values conform to parameters in the mapping schema (mandatory, LOV, length). Although LOVs cannot be presented to the user for Structure Manager notes, values can still be validated with this handler.

Syntax

ERP-validate-data-RH

Arguments

None.

Placement

Call this handler after you attach data with [ERP-attach-targets-AH](#). Place this handler on the **perform-signoff** task.

Restrictions

None.

ICS-assert-target-classified

Description

Checks whether an item is classified by verifying that target objects of the specified types in this workflow process are classified. If the item is classified, the rule handler returns **EPM_go**. If the item is not classified, it returns **EPM_nogo**. The user then has the option of associating this rule handler with the selected workflow completion process, therefore, preventing the state transition if the item does not comply with the classified business rule.

Syntax

ICS-assert-target-classified -allowed_type =<type-of-workspace-object
[, type-of-workspace-object2,..]>

Arguments

-allowed_type
Must be valid workspace object types. For example: **ItemRevision** and **ITEM**

If this argument is specified as **Dataset**, any type of dataset (**UGMASTER**, **UGPART**, **Text**, and so on) is considered.

If this argument is specified as **ItemRevision**, any type of item revision (**DocumentRevision**, and so on, and any custom item revision types) is considered.

Placement

Place on any action and on any task.

Restrictions

None.

Examples

This example checks item revisions as targets:

Argument	Values
-allowed_type	ItemRevision

This handler is very useful in restricting unclassified items and item revisions from being released.

IM1-check-bomchild-release-status

Description

This handler checks all components (children) of a target assembly in a BOM view revision for a valid release status. If any child component does not have a release status and is not also included as a target of the workflow, the handler fails the workflow. For **Im0MachineRevision** targets, the handler also checks the associated **Fnd0AppSession** objects. It retrieves all BOM structures from the session and verifies that the top-line item revision of each structure has the required release status (default: **Im0EngineeringReleased**, configurable via the **IMS_release_status_name** preference). If validation fails, the handler removes all attachments from the task and aborts the workflow process.

Syntax

IM1-check-bomchild-release-status

Arguments

None. The expected release status type is configurable via the Teamcenter preference **IMS_release_status_name** (default: **Im0EngineeringReleased**).

Placement

Place as a business rule on the Start action in Machinery Release Process, RequirementSpec Release Process, MachinePart MachineDesign Release Process, InstallAssembly Release Process, and ECAD Release Process workflow templates.

Restrictions

Targets must be ItemRevision subtypes. All BOM children must have a release status, unless they are also included as workflow targets (error: **IM1_child_not_released_error**). For Machine Revisions, the associated session objects must also have the required release status (error: **IM1_BOMLine_not_released_error**). On failure, the handler removes all attachments and aborts the workflow process.

Examples

No arguments required. The handler:

1. Opens BOM windows for each target Item Revision
2. Checks each child line for release status
3. For Machine Revisions, also validates session structure releases
4. Returns **EPM_nogo** and aborts if any child is not released

IMD0-check-related-eudi-record

Description

This handler validates whether the Med Device SKU Revision in the workflow targets has a related EU DI Record or EU Legacy DI Record via the specified relation type. Returns **EPM_go** if a valid EUDI Record is found, **EPM_nogo** otherwise.

Syntax

IMD0-check-related-eudi-record

Arguments

-relation

(Required) The relation type to check for EUDI records (for example, **ImdODIRecordRel**).

Placement

Place as a business rule on the Start action of the root process task in the EU DI Review and Submission Process workflow template.

Restrictions

Target attachments must include Med Device SKU Revision objects. A valid EUDI Record must exist via the specified relation. Returns **EPM_nogo** if no matching record is found.

Examples

Argument	Values
-relation	ImdODIRecordRel

IMD0-Validate-DI-Record

Description

This handler validates the DI Record associated with a Label Design BOM. The handler finds Label Design Revision objects in the target attachments, navigates to their primary objects via the DI Label Design relation, retrieves the associated Item SKU, and verifies that a DI Record exists on that Item SKU via the **Imd0DIRecordRel** relation.

Syntax

IMD0-Validate-DI-Record

Arguments

None.

Placement

Place as a business rule on the Start action of the root process task in the MDLS Label BOM Transfer Process Async workflow template.

Restrictions

Target attachments must include Label Design Revision objects. Each Label Design must relate to exactly one primary object (error: **IMD0_labeldesign_has_multiple_primaryobjs** if multiple found). Each Label Design must have a primary object (error: **IMD0_labeldesign_without_primaryobj** if none found). The Item SKU must have a DI Record attached (error: **IMD0_no_di_record** if not found).

Examples

No arguments required. The handler validates the chain: Label Design to Primary Object (via **Imd0DILabelDesignRel**) to Item SKU to DI Record (via **Imd0DIRecordRel**). Returns **EPM_go** if DI Record exists, **EPM_nogo** otherwise.

IPM-check-attachments

Description

This handler checks if any specified object type is present in the target or reference. If object is found sets the task result to True.

Syntax

None.

Placement

On **Complete** Action of task.

Arguments

-from_attach= target | reference

-include_type= object-type

Restrictions

None.

IPM-check-duplicate-objects

Description

This **RULE HANDLER** checks if the objects in target or reference has duplicates in reference or target partially or completely.

Syntax

None.

Placement

No specific placement.

Arguments

-compare= target | reference

-compare_with= object-type

content= all | partial

Restrictions

None.

IPM-compare-object-properties

Description

This Rule Handler compares the 2 different properties of 2 different objects. If they are equal, it will return EPM_go ,else EPM_nogo is returned.

Also if the argument set_conditional_result is provided, it will set the conditional task result to True (EPM_go)or False (EPM_nogo) based on the handler return value.

Syntax

None.

Placement

On **Start** Action of Condition Task.

Arguments

-from_object= target | reference

Attachment type of object whether object is attached as target or reference, which is to be considered for property comparison.

-from_object_type= object-type

Object type to consider for property comparison.

-to_object= target | reference | job

Attachment type of object whether object is attached as target or reference, which is to be considered for property comparison.

-to_object_type= object-type

Object type to consider for property comparison.

-from_obj_prop_name= Property name to be compared <Prop 1>

-to_obj_prop_name= Property name to be compared <Prop 2>

-set_conditional_result=

Restrictions

None.

Example

Consider example where target object of type limOIddea property "object_name" is to be compared with workspaceobject in job property "owning_user" and if values are equal result should be EPM_go.

Arguments	Value
-from_object	target
-from_object_type	limOIddea
-to_object	job
-to_object_type	workspaceobject
-from_obj_prop_name	object_name
-to_obj_prop_name	owning_user
-set_conditional_result	<NO VALUE>

LDF-sync-ldf-status

Description

Queries the remote Linked Data Framework (LDF) integrated systems, such as Polarion, for properties, and checks their values against the expected values configured.

- If the values match, the handler applies the configured status to the target(s) and allows the task to continue processing.
- If the expected values do not match, the handler does not allow a task to continue processing.

Querying a remote system like Polarion is accomplished through APIs against LDF objects attached to the root task by target or reference relations, or attached to a target or reference by a specified relation or property.

Note

Arguments specific to applying release status are the same as the **EPM-set-status** handler. Any added, modified, or deleted **EPM-set-status** handler arguments apply to the **LDF-sync-ldf-status** handler arguments.

Syntax

```
LDF-sync-ldf-status --property=<<oslc-namespace-prefix-url>.property-name>
[-remote_user_name=<user_name>]
[-attachment={<target / reference / both>}] [-attachment_property=<property-name>]
[-attachment_relation=<relation-name>]
] [-include_type=<include-type>]
[-include_related_type=<include_related_type>] [-check_first_object_only]
[-[action={<append/replace/delete>}]]
[-status=<name>]
[-new_status=<new-status>]
[-retain_release_date] [-set_effectivity]
```

Arguments

Parameter	Description	Default	Req
-property:: <<oslc-namespace-prefix-url> . property-name>	Specifies the remote property or properties check. Requires a fully qualified property name with a prefix URL prepended to every property in a workflow argument, which is prepended by -property:: . The OSLC namespace prefix URL must be contained		Yes

Parameter	Description	Default	Req
	in angle brackets, < and >, in the <<oslc-namespace-prefix-url >.property-name> format as shown in the Examples section.		
	Enter a list separated by commas or the character specified by the EPM_ARG_target_user_group_list_separator preference.		
-remote_user_name	Used by the handler to connect to a remote system like Polarion for sending HTTP requests.		No
	The Restrictions section describes separate actions required to generate an encrypted password file.		
-attachment	Specifies the type of attachment to be checked:	target	No
	target		
	Checks the target attachments		
	reference		
	Checks the reference attachment		
	both		
	Checks target and reference types of attachments.		
-attachment_property	Property of the attachment to derive the linked object.		No
-attachment_relation	Specifies the relation name to expand to get the linked object from workflow attachment. Linked objects attached to targets and references of a workflow with the relation specified by attachment_relation are searched. Linked objects not matching the specified relation are not checked.	Lcm0AffectedByDefect	No
-include_type	Specifies the type of workflow target and reference attachments to be checked.	target	No

Parameter	Description	Default	Req
	Workflow attachments not matching the specified type are not checked.		
-include_related_type	Specifies the type of linked object to retrieve that is related to the workflow attachment using the attachment_relation value. This argument should be used in conjunction with the attachment_relation or attachment_property arguments.	target	No
-check_first_object_only	If specified, only the first object of the type specified by include_type is considered. This argument is optional.	true	No
-status	When the check is satisfied, a new milestone with the name specified by this argument is added to targets and references of the workflow.	task-name	No
	If you do not provide this argument, the workflow will not advance even after the Notify Teamcenter function is run and you must then manually complete the workflow from Teamcenter.		
-action	Specifies an action:	append	No
	append		
	Attaches the status objects from the root task to the target objects, with no impact to any previous status objects applied to the same targets.		
	replace		
	Deletes all existing status objects attached to target objects and attaches the status objects from the root task to the target objects.		
	rename		
	Renames an existing status object attached to the target objects from old_name to new_name .		

Parameter	Description	Default	Req
	If a status object with the old_name status is not found, it renames the last status object attached to the target objects. If the target object has an existing status, the status object is renamed from old_name to new_name. delete Deletes the status status_name specified by the status argument from the target object. If the delete argument is not used in combination with the status argument, all status objects are removed from the target objects. If the status objects being removed from the target objects were created in the same workflow, they are attached to the root task upon creation and are not removed from the root task by this handler.		
-new_status	Specifies the new name for the status object.		No
	Use in conjunction with rename and replace actions.		
-retain_release_status	Retains the original release date on the target object if it had previously been released. Not valid for replace .	false	No
-set_effectivity	If used, the system creates the open-ended date effectivity with the release date as the start date.	false	No

Placement

Because this is a rule handler with some action handler behavior, place it as the last rule handler in the rule handler list for the task **Complete** action.

Restrictions

Use if you are using the LDF framework for application integrations and you want Teamcenter workflows to apply status based on LDF linked property values.

You must generate an encrypted password file by following these steps in a Teamcenter command shell:

1. Run this command:

```
mkdir %TC_DATA%\polarionconnector
```

2. Run this command:

```
%TC_ROOT%\bin\install -encryptpwf -f=%TC_DATA%\polarionconnector\<<user name>>
```

Where <<user name>> is user name of remote system such as Polarion ALM. This user name should be configured as a value of the **-remote_user_name** handler.

Examples

- The following example checks the status property of linked objects on the remote system.

Argument	Values
-property:: <<http://polarion.plm.automation.siemens.com/oslc#>> .priority	Low, Medium
-attachment	target
-attachment_relation	Lcm0RelatedChangeRequest
-include_type	ChangeRequestRevision
-status	Synced
-action	append
-remote_user_name	admin

MDT-deliverable-validation-for-instances

Description

Validates if a deliverable instance is not attached to more than one deliverable and attaches the deliverables as target or reference attachments of the workflow.

Syntax

MDT-deliverable-validation-for-instances

-relation = *relation-name*

-primary_type = *object-type*

-attachment= = *target* | *reference*

Arguments

-relation*relation-name*

Real name of the relation between Deliverable and Deliverable Instances

-primary_type = *object-type*

Real Name of the Business Object Type (Sub-type of Workspace Object) on which the instances are to be validated.

-attachment= = *target* | *reference*

Attachment type for Deliverables. target or reference.

Placement

Requires no specific placement. Typically placed on the **Start** or **Complete** action of the task.

Restrictions

None.

Examples

Example

In this example deliverables of type **Mdt0MDRDelRevision** will be verified for only one Deliverable Instance related with relation **Psi0DelInstances** and the deliverables will be attached to the reference attachments of workflow.

Argument	Values
-relation	Psi0DelInstances
-primary_type	Mdt0MDRDelRevision
-attachment	reference

MDT-hard-errors-validation

Description

Validate the hard errors in mapping file, including but not limited to, mandatory column names in mapping file and corresponding entries in the preferences for the BO Type, etc.

Syntax

MDT-hard-errors-validation

Arguments

None.

Placement

Requires no specific placement. Typically placed on the **Start** action of the task.

Restrictions

None.

MDT-reference-deliverables-validation

Description

Validate if a Deliverables for the Deliverable Instances are available as reference attachments of the workflow.

Syntax

MDT-reference-deliverables-validation

-relation = *relation-name*

-primary_type = *object-type*

Arguments

-relation*relation-name*

Real name of the relation between Deliverable and Deliverable Instances

-primary_type = ***object-type***

Real Name of the Business Object Type (Sub-type of Workspace Object) on which the instances are to be validated.

Placement

Requires no specific placement. Typically placed on the **Complete** action of the task.

Restrictions

None.

Examples

Example 1

In this example the handler will validate if the deliverables of type **Mdt0MDRDelRevision** will be verified for only one deliverable instance related with relation **Psi0DelInstances** are available in the reference attachments of the workflow.

Argument	Values
-relation	Psi0DelInstances
-primary_type	Mdt0MDRDelRevision

MDT-validate-transmittal-response-for-del-instances

Description

Validates if all the deliverable instances from the transmittal have received a response. If a response is not available for any of the deliverable instances, the process is not taken further.

Syntax

MDT-validate-transmittal-response-for-del-instances

Arguments

None.

Placement

Requires no specific placement. Typically placed on the **Perform** action of the task

Restrictions

This handler is only valid for **Approve** action of **Acknowledge and Review** task.

MESINTEG_ValidateReleaseAndExport

Description

Performs customized validation checks for Manufacturing Execution System Integration. This handler does the following:

- Takes the **CC** object and creates BOP windows.
- Configures all windows with the configuration rule.
- Calls the validation checks for any BOP window.

If a validation check fails or there is an error or warning, it is returned within the **validationError** structure and added to the log in the handler or in the user interface.

Syntax

MESINTEG_ValidateReleaseAndExport -Type = <callback-type-1, callback-type-2, ...>

Arguments

-Type

Specifies the callback type, for example, **MFG_ValidationChecksCallback** or **MESINTEG_ValidationChecksCallback**. Each **-Type** value is paired with the **-Name** value, separated by commas or the character specified by the **EPM_ARG_target_user_group_list_separator** preference. You can have more than one type/name pair.

-Name

Specifies the callback name, for example, **ValidationCheck1**.

Each **-Type** value is paired with the **-Name** value, separated by commas or the character specified in the **EPM_ARG_target_user_group_list_separator** preference. You can have more than one type/name pair.

-perform

Specifies the list of operations to be performed by the action handler.

Values include **Validate**, **Release**, **GenerateMESWIRep**, **Export**, and **modifyscope**.

Note

Specify these values without spaces and separated by commas or the character specified in the **EPM_ARG_target_user_group_list_separator** preference

-fullexport

Indicates whether it is a full export or a delta export.

-ContinueOnFail

(Optional) Specifies whether to continue checking if the previous check fails. The default value is **False**. You can use multiple values, separated by commas or the character specified by the

EPM_ARG_target_user_group_list_separator preference. Specify one value less than the number of type/name pairs, because if the last check fails, there is no check to continue.

-export_as_fai

Specifies whether to consider the work package as part of the **Send to MES** command.

If set to **True**, the work package is considered as a part of the **Send to MES** command.

Placement

Place this handler on any workflow that eventually creates a BOP window from the **VisStructureContext**, exports the data, and updates the release status.

Restrictions

None.

Examples

Arguments used in the **ReleaseToMES**, **Send**, and **ReleaseUpdateToMES** workflows.

Note

Specify values without spaces and separated by commas or the character specified in the **EPM_ARG_target_user_group_list_separator** preference

Argument	Values
-Type	MFG_ValidationChecksCallback, MFG_ValidationChecksCallback, MFG_ValidationChecksCallback
-callback_name	Release Status Validation, Workarea Assigned Validation, Process Hierarchy Validation, Workarea Name Validation
-perform	Validate, Release, GenerateMESWIRep, Export
-fullexport	True
-ContinueOnFail	True or False
-export_as_fai	True, False
	If this property is set to True , the work package is considered as a part of the Send to MES command.

Arguments used in the **ReleaseToProduction** workflow.

Note

Specify values without spaces and separated by commas or the character specified in the **EPM_ARG_target_user_group_list_separator** preference

Argument	Values
-Type	MFG_ValidationChecksCallback
-callback_name	Change Object Validation
-target	production
-perform	Validate, Pending, Export, exportdelta, Release, modifyscope
	The modifyscope value is specific to ReleaseToProduction workflow. If you want to use this value, you must register this callback using the following command: <pre>install_callback -u=<Tc-admin-user> -p=<password> -g=<group> -mode=<create> -type=<MFG_ModifyScopeCallback> -library=<library> -function=<function> -name=<Modify Export Scope></pre>
	Note Do not use modifyscope, Pending, or exportdelta values for MES Integration.
-fullexport	TRUE
-ContinueOnFail	TRUE
-export_as_fai	True, False

If this property is set to **True**, the work package is considered as a part of the **Send to MES** command.

MFG-invoke-customized-validations

Description

Performs customized validation checks for Manufacturing Execution System Integration. This handler does the following:

- Takes the **CC** object and create BOP windows.
- Configure all windows with the configuration rule.
- Calls the validation checks for any BOP window.

If a validation check fails or there is an error or warning, it is returned within the **validationError** structure and added to the log in the handler or in the user interface.

Syntax

MFG-invoke-customized-validations -Type = <callback-type-1, callback-type-2, ...> -Name =<callback-name-1, callback-name-2, ...> [-ContinueOnFail = True|False, True|False, ...]

Arguments

-Type

The callback type; for example, **MFG_ValidationChecksCallback** or **MESINTEG_ValidationChecksCallback**. Each -Type value is paired with the -Name value, separated by commas or the character specified by the **EPM_ARG_target_user_group_list_separator** preference. You can have more than one type/name pair.

-Name

The callback name; for example, **ValidationCheck1**. Each -Type value is paired with the -Name value, separated by commas or the character specified by the **EPM_ARG_target_user_group_list_separator** preference. You can have more than one type/name pair.

-ContinueOnFail

(Optional) Whether or not to continue checking if the previous check failed. The default is **False**. You can use multiple values, separated by commas or the character specified by the **EPM_ARG_target_user_group_list_separator** preference. There should be one less value than the number of type/name pairs, because if the last check fails, there is not another check to continue to.

Placement

Place this handler on any workflow that transfers a **CC** object to a BOP window.

Restrictions

None.

Examples

- This example runs three different validation checks, **ValidationCheck1**, **ValidationCheck2**, and **ValidationCheck3**. If **ValidationCheck1** fails, the handler runs **ValidationCheck2** anyway. If **ValidationCheck2** fails, the handler does not run **ValidationCheck3**.

Argument	Values
-Type	MFG_ValidationChecksCallback, MFG_ValidationChecksCallback, MFG_ValidationChecksCallback
-Name	ValidationCheck1, ValidationCheck2, ValidationCheck3
-ContinueOnFail	True, False

MROCORE-validate-for-class

Description

Validates that the item revision submitted to the workflow is a physical part revision. If it is a physical part revision, the handlers returns **EPM_go**. If it is not a physical part revision, the handler displays an error, returns the decision as **EPM_nogo**, and stops further processing.

Syntax

MROCORE-validate-for-class -class name=<class-name>

Arguments

-class name
Specifies the class name to validate.

Placement

Place at the entry of the workflow to validate that the target object is the physical part revision for the as-built structure traversal.

Restrictions

This handler is available only when Teamcenter service lifecycle management Service Manager or As-Built Manager is licensed and installed.

PMA0_validate_change

Description

Validates if a change is valid to be released by evaluating the default and custom business validation rules defined in Teamcenter.

This handler validates one engineering change notice at a time and evaluates all the business rules applicable for the solution items in that change notice. It returns **EPM_go** if the change is valid and returns **EPM_nogo** if the change validation fails.

Syntax

PMA0_validate_change

Arguments

NA

Placement

Placed on a **Complete** action.

Restrictions

None.

PS-check-assembly-status-progression

Description

Enforces status value progression for BOM assemblies. When an assembly is selected for release to a specific status, this handler checks if all its components are at or above the status of the assembly.

An item revision is required as the target of the workflow process. Additional targets are derived by traversing the BOM attached to the target item revision. The handler then compares the targeted release status to the release status of its components. The latest release status of the components must be the same or later in the status progress, in relationship to the targeted release status of the assembly.

This handler traverses only one level. If every subassembly of the target were previously released by this handler, all subassemblies would have been forced to align to the progression path.

Note

If the target release status of the assembly must be checked against the latest release status of its own preceding revisions, use the [EPM-check-status-progression](#) handler before using this handler.

If the workflow process contains several **Condition** tasks that apply different release statuses at different levels, the value provided in the **-status** argument can be used. If this argument is not used in this situation, the status applied to the target object is applied to the object. There is no validation ensuring the value provided by this argument is a valid status being applied by the current release procedure.

You can check the BOM components for a specific status, rather than for any status. In this case, the handler traverses the BOM, checking for the specific release status of each individual component, rather than any status; the progression path is not read.

Syntax

```
PS-check-assembly-status-progression [-rev_rule=<revision-rule>]
[-saved_var_rule=<saved-variant-rule>] [-status=
<status-being-applied-to-the-target-object>] [-check_component_status=
<component-status-to-be-checked-against>] [-check_unconfigured]
```

Arguments

-rev_rule

Specifies the name of the revision rule to be applied for BOM traversal. If not supplied, the default revision rule is used.

-saved_var_rule

Specifies the name of the saved variant rule to be applied on BOM window for BOM traversal.

-status

Defines the status being applied to the target object. This argument can also be used to check the status of component objects. When used in conjunction with the **ReleaseProgressionList** defined in

the *ProgressionPath.plmxml* file, this argument allows the handler to verify that component objects have at least the specified status or any status later in the progression path. This enables assemblies with components having different statuses (for example, Lock, Check, or Release) to be submitted to a workflow, as long as each component has reached at least the minimum status defined in the progression path.

If you do not specify **-status**, you must specify **-check_component_status**. The name provided should be the name of a status type already defined in the Business Modeler IDE, not the display name.

-check_component_status

Checks if all the components have this status. If you do not specify **-check_component_status**, you must specify **-status**. The name provided should be the name of a status type already defined in the Business Modeler IDE, not the display name.

-check_unconfigured

Returns **NO-GO** in case the applied revision rule on the assembly results in unconfigured children.

When specified, all the components are checked to see if they are configured. While the components are checked, if any component has an invalid status, the status is ignored for the time being and the rest of the components are continued to be checked. After all the components are checked:

- If one of the components is unconfigured, **NO-GO** is returned regardless of whether any other component has an invalid status.
- If all the components are configured then if one of them has an invalid status, the invalid status error is displayed and **NO-GO** is returned.

Placement

Place on any task action. However, if the target assembly is very large, placing it on the **Start** action of the root task could affect performance. With this placement, the **Create Process** dialog box does not close until the entire assembly is traversed.

Restrictions

If there are separate release progression tables for assemblies and for components, there must be common statuses between these two tables. If there are no common statuses between these two tables, this handler returns an **EPM_nogo** and aborts the release process of the assembly when the workflow process is initiated. See the fourth example below.

Examples

- In this example, assume that the revision rule is **Working** and the variant rule is **GMC 300 Rule**.

If an assembly target object has to be checked against the status of its components, using a specific revision rule and saved variant rule to configure the assembly, define the arguments:

Argument	Values
-rev_rule	Working
-saved_var_rule	GMC 300 Rule
-status	Design

- In this example, if the assembly target object being released has to check if each of its components are at exactly **Design** status (not any other status in the progression path), define the following argument. When using the **-check_component_status** argument, the progression path is not read and all components must have the exact same status specified:

Argument	Values
-check_component_status	Design

- In this example, assume a workflow process contains several **Condition** tasks, which apply different release statuses at different levels, and **Design** is a status at one of the levels. To check the status of **Design** against the progression path, rather than deriving the status being applied to the target object, define the following argument:

Argument	Values
-status	Design

- In this example, to configure the handler to verify that component objects have at least a minimum status in the progression path, allowing components with different statuses to be submitted to a workflow:

- Edit the *ProgressionPath.plmxml* file in the *TC_DATA* directory (create a backup before editing).
- Add the following lines to define the progression path for the target type:

```
<UserValue title="Type" value="ItemRevision"/>
<UserValue title="ReleaseProgressionList"
value="plm4lock,plm4check,plm4production">
</UserValue>
```

- In Workflow Designer, add the **PS-check-assembly-status-progression** handler to the **Start** action of the root task.
- Define the following arguments for the handler:

Argument	Values
-rev_rule	Latest Working
-status	plm4lock

With this configuration, the handler verifies that all component objects have at least the **plm4lock** status or any status later in the progression path (such as **plm4check** or **plm4production**). This allows an assembly with components having different statuses (for example, one component with **Lock** status and

another with **Check** status) to be submitted to the workflow, as long as each component has reached at least the minimum status defined.

In this example, consider the scenario:

- Assy1/A is a **CORP_Product** item revision, at **Design** status
- 002/A is a **CORP_Part** item revision, at **Design** status
- 003/A is a **CORP_Part** item revision, at **Design** status
- **CORP_Product** progression path: **Assembly Quote, Experimental, Development, Design, Prototype, Manufacturing, Production**
- **CORP_Part** progression path: **Quote, Experimental, Development, Design, Manufacturing, Production**

If Assy1/A is now being released to **Prototype** status, the handler returns an **EPM_nogo** because the component's progression path (and therefore the component progression table) does not contain the **Prototype** status. The assembly process would be aborted.

Additional Information

- If the target release status of the assembly has to be checked against the latest release status of its own preceding revisions, the best practice is to use the **EPM-check-status-progression** handler before this handler.
- The progression path must be manually defined in the **ProgressionPath.plmxml** file before the handler can reference the path. The file is stored in the <TC_DATA> directory. Create a backup copy of this file before editing it.

All target types that you want to follow the progression path must be set in this file. A **UserData** block must be created for each type that follows a progression path. For example, to define the progression path for the **ItemRevision**, **PSBOMView**, and **MSWord** types, the **UserData** blocks can be defined as follows:

```
<UserData id="id1">
  <UserValue title="Type" value="ItemRevision"/>
  <UserValue title="ReleaseProgressionList"
    value="Quote, Development, Prototype, Production">
  </UserValue>
</UserData>
<UserData id="id2">
  <UserValue title="Type" value="PSBOMView"/>
  <UserValue title="ReleaseProgressionList"
    value="Quote1, Development1, Prototype1, Production1">
  </UserValue>
</UserData>
<UserData id="id3">
  <UserValue title="Type" value="MSWord"/>
  <UserValue title="ReleaseProgressionList"
    value="Quote2, Development2, Prototype2, Production2">
  </UserValue>
</UserData>
```

```
</UserValue>  
</UserData>
```

Note

- Add the **UserData** blocks between the **<PLMXML>** and **</PLMXML>** tags.
- Ensure you increment the **UserData id** value when you add a new entry.
- The **value** attribute of the **UserValue** tag must contain the name of the status type already defined in the Business Modeler IDE, not the display name.
- After adding a new **UserData** block, change the value for **Type** to a type you are defining.
- You can modify the value of the release status to meet your requirements.

PS-check-occ-notes

Description

Checks whether a value has been entered for the specified occurrence note types on the occurrences of a given assembly.

Syntax

PS-check-occ-notes -note_types=<occurrence-note-type-names>

Arguments

-note_types
Defines the occurrence note types to be validated.

Placement

Requires no specific placement.

Restrictions

None.

Examples

This example checks if the given assembly has the **Torque** and **Power** occurrence note types defined in all its BOM lines:

Argument	Values
-note_types	Torque,Power

QFM0FMEAREALIZATION-validate-for-class

Description

This workflow rule handler validates the type of object submitted to workflow.

Syntax

QFM0FMEAREALIZATION-validate-for-class -className=<*[BusinessObject class name]*>

Arguments

-className

Business object class name.

Placement

Place on the **Start** action of a **Do** task.

Restrictions

None.

Note

For more information about workflow handlers, see [Workflow Handlers](#).

SAP-check-forms-attached-RH

Description

Makes the following checks:

- For each BOM, check that the master data for each component and the assembly itself is created in ERP at the plant specified in the associated **BOMHeader** form or is a target of the current process. This prevents the upload failing, which it would if the component data did not already exist. This handler does not make any calls to ERP; it simply checks the **Sent to ERP** box.

Note

If the process has both component and assembly item revisions, the material data is created first, and then the BOMs.

- For each **BOMHeader** form, there must be a corresponding BOM view revision with the view type specified by the **TC_view_type** attribute in the form.
- Complete sets of ERP forms are attached to each item revision as a target of the process. The mapping schema allows data for an **erp_object**, typically plant-specific, to be split across several form types. As the upload is expecting a complete set of attribute values for an **erp_object**, a complete set of forms must be transferred (for example, an instance of each form type defined for the **erp_object**).
- For a BOM, check that the parent and all components have had their master data **Sent to ERP** for the plant in which the BOM is created or are part of the process.

Note

If the **erp_object** defines a key field with the **is_key_flg** parameter, the value in this field is used to distinguish between different instances of data for the same **erp_object**. For example, all forms having value 1000 in the **plant** field for form types with **erp_object PlantSpecific** constitute the set of forms defining the plant-specific data for plant 1000.

This handler only searches for ERP forms defined in the mapping schema attached by the relation types listed by the **-reln_names** argument. This list should be consistent with that used in the **ERP-attach-targets-AH**. Only those forms whose state has not yet been transferred to ERP (for example, those for which the **Sent_to_ERP** field is empty) are checked.

Syntax

SAP-check-forms-attached-RH -reln_names = <reln1>,<reln2>,...

Arguments

-reln_names

A list of the relation types used to relate ERP forms to item revisions.

Separate multiple types with commas or the character specified by the **EPM_ARG_target_user_group_list_separator** preference.

Note

Relation names are case sensitive and should be named, for example, **tc_specification** not **TC_Specification**.

ERP_Data is the special relation supplied for attaching ERP forms.

Placement

Place this handler on the **Review** task.

Restrictions

None.

SAP-check-forms-to-download-RH

Description

Checks to make certain all form sets in transfer folders are valid, with the same rules as the [SAP-check-forms-attached-RH](#) rule handler. However, the **SAP-check-forms-to-download-RH** handler is intended for final checking of the form sets to be sent, rather than an initial input validation set.

Syntax

SAP-check-forms-to-download-RH

Arguments

None.

Placement

Call this handler after data is attached using the [ERP-attach-targets-AH](#) handler. Place this handler on the **Perform Signoff** task.

Restrictions

None.

SFD0-check-violations-exists

Description

This handler checks if any Design violations exist on the DRC (Design Rule Check) Execution object attached to a Tapeout or PostTapeout object. If violations are found, an error is thrown and the handler returns **EPM_nogo** to prevent the workflow from proceeding. The handler iterates through all target attachments, finds DRC Execution objects via the **Sfd0DRCExecutionRel** relation, and then checks each DRC Execution for design violations via the **sfd0DesignViolation** property. In both workflows, this handler acts as a decision gate: if no violations exist (**EPM_go**), the workflow proceeds to set the DRC status to DRC Clean; if violations exist (**EPM_nogo**), the workflow routes to violation review and waiver handling.

Syntax

SFD0-check-violations-exists

Arguments

None.

Placement

Place as a business rule on the Start action of the Any Violations validate task in Tapeout Process Workflow and Post Tapeout Process Workflow.

Restrictions

Target attachments should be Tapeout or PostTapeout objects. If any DRC Execution related to a target has design violations (**sfd0DesignViolation** count > 0), error **SFD0_VIOLATION_EXISTS** is thrown. Objects without DRC Executions are silently skipped.

Examples

No arguments are required. The handler automatically:

1. Finds all DRC Execution objects related to each target via **Sfd0DRCExecutionRel**
2. Checks the **sfd0DesignViolation** property on each DRC Execution
3. Returns **EPM_go** if no violations exist (workflow sets DRC Clean)
4. Returns **EPM_nogo** if violations exist (workflow routes to waiver handling)

SFD0-mpwshuttle-objects-commit-review

Description

This handler validates MPW Shuttle objects during a commit review. It verifies that all Intake Objects attached to the Shuttle have a valid maturity state for the Committed milestone. The handler fetches all Intake Objects related to the MPW Shuttle and checks their maturity state. For the commit review, Intake Objects must have a maturity state of 20 (Committed) or 30 (Production). If any Intake Object has an invalid maturity state, the handler returns **EPM_nogo** and throws an error listing the offending objects. The handler also refreshes the target attachment list by removing non-Shuttle objects and re-adding the latest related objects (Intake, MaskSet, Tapeout, FloorPlan).

Syntax

SFD0-mpwshuttle-objects-commit-review

Arguments

None.

Placement

Place on the Start action of the perform-signoffs task within the Commit Review task in the MPW Shuttle Workflow Process.

Restrictions

Target attachments must include **Sfd0MPWShuttle** objects. All Intake Objects must have a maturity state of 20 (Committed) or 30 (Production). If any Intake Object has an invalid maturity state, the handler returns **EPM_nogo**.

Examples

No arguments are required. The handler automatically validates Intake Object maturity states and refreshes target attachments.

SFD0-mpwshuttle-objects-production-review

Description

This handler validates MPW Shuttle objects during a production review. This is a stricter validation than the commit review. The handler verifies that all Intake Objects must be in Production state (maturity = 30), at least one MaskSet must be attached to the Shuttle, each MaskSet must have a Tapeout object attached, and at least one FloorPlan must be associated with the Shuttle. If any of these conditions fail, an appropriate error is thrown and the handler returns **EPM_nogo**. The handler also refreshes the target attachment list by removing non-Shuttle objects and re-adding all validated related objects.

Syntax

SFD0-mpwshuttle-objects-production-review

Arguments

None.

Placement

Place on the Start action of the perform-signoffs task within the Production Review task (Make Production Ready path) in the MPW Shuttle Workflow Process.

Restrictions

Target attachments must include **Sfd0MPWShuttle** objects. All Intake Objects must be in Production state (maturity = 30). At least one MaskSet must be attached to the Shuttle. Each MaskSet must have a Tapeout object attached. At least one FloorPlan must be associated with the Shuttle.

Examples

No arguments are required. The handler automatically validates all production review requirements and refreshes target attachments.

TCRS-check-approver

Description

Compares the lists of assigned users for two specified tasks. If the same user is assigned to both tasks, the handler displays a warning message or stops the task, depending on the value you enter for the **-stop** argument. You specify the tasks with the **-a_task** and **-b_task** arguments, and the user with the **-a_user** and **-b_user** arguments.

Syntax

```
TCRS-check-approver [-a_task=<task-name>|$PREVIOUS|$NEXT] [
-a_user=<userid>|$USER] [-a_jobowner] [-b_task=<task-name>|$PREVIOUS|
$NEXT] [-b_user=<userid>|$USER] [-b_jobowner] [-stop=[Y|N]]
```

Arguments

Parameter	Description	Required	Value/Default Value
-a_task	All signoffs of this task are assigned to set A .	No	<task-name>
-a_user	The user specified or the current user is assigned to set A .	No	<userid>
-a_jobowner	The owner of the job is added the quantity of A .	No	None
-b_task	All signoffs of this task are assigned to set B .	No	<task-name>
-b_user	The user specified or the current user is assigned to set B .	No	<userid>
-b_jobowner	The owner of the job is added the quantity of B .	No	None
-stop	The job stops if a signoff is found. Default is Y .	No	Y

Placement

Place on the **Completed** action of the **select-signoff-team** task.

Tip

You can also place the handler on the **Completed** action of the **perform-signoffs** task.

Restrictions

None

Examples

- If the same user is assigned to task **R1** and task **R2**, the handler returns **EPM_nogo** and stops the task.

Argument	Value
-a_task	\$PREVIOUS
-b_task	\$NEXT
-stop	Y

- If the user who is currently logged on is assigned to the **R2** task, the handler displays a warning message but does not stop the task.

Argument	Value
-a_user	\$USER
-b_task	\$NEXT
-stop	N

- If the same user is assigned to both task **R1** and task **R2**, or if the **R2** signoff list includes the job owner, the handler returns **EPM_nogo** and stops the task.

Argument	Value
-a_task	\$PREVIOUS
-a_jobowner	n/a
-b_task	\$NEXT
-stop	Y

TCRS-check-bom-precise

Description

Checks whether all BOM view revisions are precise.

Syntax

TCRS-check-bom-precise [-stop=[y|n]] [-maxdepth=<depth>]

Arguments

Parameter	Description	Optional/ Mandatory	Value/Default Value
-stop=n	Specifies if workflow should stop or not on error. The default value is y.	Optional	y.
-maxdepth	Levels in the assembly to be checked. The value 0 corresponds to all levels. The default value is 1.	Optional	<depth>

Placement

Must be set in the **Completed** action of the **perform-signoffs** task.

Restrictions

None.

Examples

Checks whether all BOM view revisions are precise on all levels.

Argument	Value
-stop	y
-maxdepth	0

TCRS-check-bomchild-statuslist

Description

Checks all components of a target assembly in a BOM view revision for a valid status.

Syntax

TCRS-check-bomchild-statuslist [-rule=<configurationrule>]

-statelist=<status>

[-check_job=[y|n]] [-log=[error<all>]] [-stop=[y|n]] [-maxdepth=[<depth>]]

Arguments

Parameter	Description	Optional/ Mandatory	Value/Default Value
-rule	Configuration rule.	Optional	<configurationrule>
-statelist	List of valid status names. Separate multiple names with commas or the character specified by the EPM_ARG_target_user_group_list_separator preference.	Mandatory	<status>
-check_job	Defines the terms of the component status.	Optional	n = All components must possess a correct status or be target objects in the same workflow. y = All components must possess a correct status and be target objects in the same or another workflow. If no value is provided the default the value is y.

Parameter	Description	Optional/ Mandatory	Value/Default Value
-log	Log data record.	Optional	• error = Record incorrect components only. • all = Record all component examinations. If no value is provided the records are not stored in the log.
-stop	Warning in the event of an error (=n) or Workflow with error stop (=y)	Optional	Warning in the event of an error (=n) or Workflow with error stop (=y) If no value is provided the handler stops in the event of an error.
-maxdepth	Level in the assembly to be checked.	Optional	• 1 = First level • 2 = Second level • 0 = All levels If no value is provided the default value is 1.

Placement

Must be set in the **Completed** action of the **perform-signoffs** task. After this handler is used, no changes should be made to the BOM view revisions.

Restrictions

None.

Examples

1. Checks all components of a target assembly in a BOM view revision for a valid status.

Argument	Value
-stop	y
-maxdepth	0

2. Checks first level components of a target assembly in a BOM view revision for a valid status and log error.

Argument	Value
-log	error
-stop	y
-maxdepth	1

Note

Use argument -log = error to get the all item with invalid status in file "vpdg_rh_cbcsl.txt" in temp directory.

TCRS-check-comps-against-pattern

Description

Checks the components against a specified pattern, where components include **Item**, **ItemRevision**, **Dataset**, **BOMView**, and **BOMViewRevision**. The pattern is as follows: the item ID should be eight characters and all characters should be digits. In addition, all of the target components should not have a status attached to it.

Syntax

```
TCRS-check-comps-against-pattern -mode=[list|check_only] -file=<dataset-name>
```

Arguments

Parameter	Description	Optional/ Mandatory	Value/Default Value
-mode	Defines how the check should be performed. Valid values are: • list Lists all the components in the newly created dataset defined by the file argument. The dataset is attached as a reference to the workflow process. • check_only The dataset named reference is replaced with the latest information.	Mandatory	list or check_only
-file	Specifies the name of the dataset that should be attached as a reference to the workflow process.	Mandatory	<dataset-name>

Placement

Must be set in the **Started** action.

Restrictions

Handler should not be put after the **Completed** action.

Examples

Lists all the components in the newly created dataset defined by the file argument. The dataset is attached as a reference to the workflow process.

Argument	Value
-mode	list
-file	<dataset_name>

TCRS-check-datasets

Description

Checks that datasets attached to the target item revision are of the specified type. This handler also checks that the dataset name matches the specified pattern.

Syntax

TCRS-check-datasets **-type**=<dataset-type> **-search_dataset_name**=|<Pattern1>;<Pattern2>|
-check_include_dataset_name=<name-of-dataset> [**-stop**=y|n] [**-dataset_to**=target|job|item]

Arguments

Parameter	Description	Optional/ Mandatory	Value/Default Value
-type	The type of dataset for the item revision.	Mandatory	<dataset-type>
-search_dataset_name	The names of the datasets to be searched. Separate multiple names with a semicolon (;).	Mandatory	<Pattern1>;<Pattern2>
-check_include_dataset_name	Identifies the dataset. This argument accepts a single value only.	Mandatory	<name-of-dataset>
-dataset_to	Defines the location of the text dataset which contains the errors in case of failure. This text dataset is stored at the location specified in this argument. Valid values are target object (target), an attachment to the item revision (item), or an attachment to the job (!).	Optional	target
-stop	Determines whether or not to stop the workflow when the attached dataset names do not match the name in the -search_dataset_name argument.	Optional	y

Placement

Requires no specific placement.

Restrictions

All item revisions must have write privileges at the level that the handler is used.

Examples

- The following example checks all UGPART datasets with an **EZ** or **GZ** prefix in their names and **ANT** as postfix.

Argument	Values
-type	UGPART
-search_dataset_name	EZ;GZ
-check_include_dataset_name	EZ
-dataset_to	item

- In the following example, the handler checks whether a PDF dataset named **testDoc** is attached to the target item revision. Also, the error log is attached to the item if the dataset is not found.

Argument	Values
-type	PDF
-search_dataset_name	testDoc
-check_include_dataset_name	testDoc
-stop	y
-dataset_to	item

TCRS-check-itemrev-status

Description

Checks the latest status of target objects.

Syntax

TCRS-check-itemrev-status [-status=<status-type>] [-unreleased] [-latest] [-targetstatus]

Arguments

Parameter	Description	Optional/ Mandatory	Value/Default Value
-status	All target objects must be released with this status type.	Optional	<status-type>
-unreleased	All target objects should be without a release status.	Optional	None
-latest	Validates that the status of the target object is the latest.	Optional	None
-targetstatus	Stops the handler if any of the target objects are released	Optional	None

Dependent arguments

If you specify the **-latest** argument, you must specify the **-status** argument also.

If you specify the **-targetstatus** argument, you must specify the **-status** argument also.

Placement

Requires no specific placement.

Restrictions

None.

Examples

- Checks the status of unreleased target objects.

Argument	Value
-unreleased	None

- Stops the handler if any of the target objects are released.

Argument	Value
-status	<status_name>
-targetstatus	None

TCRS-check-jobowner

Description

Checks that the owner of a certain stage (task) of a release process cannot delegate approval.

Syntax

TCRS-check-jobowner [-who=jobowner] [-task=this]

Arguments

Parameter	Description	Optional/ Mandatory	Value/Default Value
-who	User ID to examine. Currently, jobowner is the only valid value. This parameter is reserved for possible future extensions.	Optional	jobowner
-task	Task to examine. Currently, this is the only valid value. This parameter is reserved for possible future extensions.	Optional	this

Placement

Must be set in the **Completed** action of the **select-signoff-team** task.

Restrictions

The current default behavior allows the user to delegate their approval after the **select-signoff-team** task completes. The **Completed** action of the **select-signoff-team** task does not get called again; therefore, the newly assigned user is not validated. To allow this validation, Siemens Digital Industries Software recommends that you include this check in the **Finish** action of the **select-signoff-team** task. Because the handler reports an error only after the user has approved, and a delegation at this point is not possible, the release process must be deleted and restarted.

Examples

Checks that the owner of a certain stage (task) of a release process cannot delegate approval. Currently, these two values are supported.

Argument	Value
-who	jobowner
-task	this

TCRS-check-prev-itemrev-status

Description

This handler checks whether a release process is valid for an item revision, based on its current status, and checks whether the target item revisions are released with specified status. You can also check whether the target item revision is unreleased and whether it currently has no status.

Syntax

```
TCRS-check-prev-itemrev-status [-status=<status-name>][ -unreleased] [ -latest ]
```

Arguments

- status**
A valid status name to be checked.
- unreleased**
Ensures that all target item revisions have no status.
- latest**
 - When this parameter is specified, the handler validates the most current status of the target item revision and ensures the latest status on target item revision is same as status specified in **-status** parameter.
 - When this parameter is not specified, the handler validates all statuses attached to target item revisions.

Parameter	Description	Required	Value/Default Value
-status	A valid status name to be checked.	No	<status_name>
-unreleased	Ensures that all target item revisions have no status.	No	None
-latest	- When this parameter is specified, the handler validates the most current status of the target item revision and ensures the latest status on target item revision is same as status specified in -status parameter. - When this parameter is not specified, the handler validates all statuses attached to target item revisions.	No	None

Placement

Place on the **Started** action of the root task of a release process. If an error occurs, this rule handler prevents the start of the release process.

Restrictions

The **latest**, **unreleased** and **targetstatus** parameters are mutually exclusive.

Examples

- In this example the handler ensures that target item revisions are released with status **Approved**.

Argument	Value
-status	Approved

- Use the **-unreleased** argument to ensure that all target item revisions are not yet released and have no attached status.

Argument	Value
-unreleased	

- In this example, the handler ensures that all target item revisions have **90** as their last status.

Argument	Value
-status	90
-latest	

- In this example, the workflow stops if any of the target item revisions are already released with status **60**.

Argument	Value
-status	60
-targetstatus	

TCRS-check-signoff

Description

Checks the signoff users against signoffs from other task.

Syntax

TCRS-check-signoff [-task=[\$PREVIOUS|\$NEXT]]

Arguments

-task =\$PREVIOUS | \$NEXT

When the argument is set to **\$PREVIOUS**, the handler checks the task before the current task. If set to **\$NEXT**, the handler checks the task after the current task. The default is **\$PREVIOUS**. This is an optional parameter.

Placement

Place on the **Completed** action of the **select-signoff-team** task.

Restrictions

None.

Examples

In this example:

- The workflow uses two **Review** tasks, **Task 1** and **Task 2**.
- The **TCRS-check-signoff** handler is placed on the **Complete** action of the **select-signoff-team** task for **Task 2**, with the following argument:

Argument	Values
-task	\$PREVIOUS

The handler compares the **Task 2** signoff team with that of **Task 1**, and displays an error message if it finds a reviewer who is a member of both teams.

TCRS-check-status

Description

Initiates a workflow process if the current and the previous revisions have a valid release status.

Syntax

```
TCRS-check-status [-previous_status={any|none|<status-list>}]  
[-previous_check=all|last] [-current_status={any|none|<status-list>}] [-current_check=all|last] [-  
stop={Y|N}]
```

Arguments

Parameter	Description	Optional/ Mandatory	Value/Default Value
-previous_status	Status on the predecessor revision to be tested.	Optional	any/none/<status-list>
-previous_check	For the previous revision, the entire list for valid status on the predecessor revision only (not on all revisions) or check either the last release status.	Optional	all/last
-current_status	Status on the target revision to be checked.	Optional	any/none/<status-list>
-current_check	For the target revision, the entire list for valid status or check the last release status.	Optional	all/last
-stop	Set to N to continue with warning or Y to stop the process.	Optional	N/Y

Parameter	Description
-previous_status	Status on the predecessor revision to be tested.
-previous_check	For the previous revision, the entire list for valid status on the predecessor revision only (not on all revisions) or check the last release status.
-current_status	Status on the target revision to be checked.
-current_check	For the target revision, the entire list for valid status or check either the last release status.
-stop	Set to Y to stop the process or N to continue with warning.

Placement

Place on the **Started** action of a root task.

Restrictions

None.

Note

The argument **-previous_status** checks only the status or status list of the predecessor revision of the targeted item revision, not all item revisions. Similarly, the argument **-current_status** checks only the status or status list of the targeted item revision only.

Examples

- In this example, the handler ensures that previous revision of the target item revision have any status type. If not, the handler stops and an error message is displayed.

Argument	Values
-previous_status	any
-stop	Y

- In this example, the handler ensures that previous revision of the target item revision have no status type. If any status type is found, the handler stops and an error message is displayed.

Argument	Values
-previous_status	none
-stop	Y

- In this example, the handler ensures that previous revision of the target item revision have **TCM Released** as their last status type. If not, the handler stops and an error message is displayed.

Argument	Values
-previous_status	TCM Released
-previous_check	last
-stop	Y

- In this example, the handler ensures that previous revision of the target item revision have had **TCM Released** as their status type at any time. If not, the handler stops and an error message is displayed.

Argument	Values
-previous_status	TCM Released
-previous_check	all
-stop	Y

- In this example, the handler ensures that target item revision have any status type. If not, the handler stops and an error message is displayed.

Argument	Values
-current_status	any
-stop	Y

- In this example, the handler ensures that target item revision have no status type. If any status type is found, the handler stops and an error message is displayed.

Argument	Values
-current_status	none
-stop	Y

- In this example, the handler ensures that target item revision have **TCM Released** as their last status type. If not, the handler stops and an error message is displayed.

Argument	Values
-current_status	TCM Released
-current_check	last
-stop	Y

TCRS-has-target-drawing

Description

Checks that the target item revisions have a CAD dataset associated with it. If the item revisions do not have an attached dataset, the handler returns an error.

Syntax

TCRS-has-target-drawing

Arguments

None.

Placement

Requires no specific placement.

Restrictions

None.

VAL-check-validation-result

Description

Evaluates the validation result of each target before releasing the object. The handler first looks for all results relative to all targets. If no validation result is found, or all results are outdated or failed, the handler reports the corresponding error message and returns an **EPM_nogo** and the workflow is cancelled. If at least one validation result is successful and current, the handler returns an **EPM_go** and the workflow proceeds.

There are five situations in which validation results are checked:

- If the target object is an item revision, the handler finds all the validation targets by the closure rule specified in the **NX Agent** and then finds all the results relative to these validation targets.
- If the target object is an item, the handler runs on the latest revision, searching for validation results as specified in the previous situation. You may also supply a handler specifying the item revisions. After the first handler runs, the second handler runs on the specified item revisions as specified in the previous situation.
- If the target object is a dataset, the handler finds the validation results relative to the dataset.
- If the target object is a folder, the handler includes all secondary objects under the folder in its search for validation results.
- If there are multiple objects as targets, (for example, if multiple item revisions are selected as targets of a workflow), the handler finds all the validation results relative to all the validation targets by closure rule.

Syntax

VAL-check-validation-result [-each_validation_target]

Arguments

-each_validation_target

(Optional) At least one validation result must exist for each NX dataset for the workflow to proceed.

If this argument is not used, the workflow proceeds if there is a successful result on one NX dataset.

Placement

Place on the **Start** action of the root task. The workflow process is aborted if a target is not validated, or if its validation result is not **Pass**.

An alternative is to place on the **Complete** action of the root task. The release status is not added to a target if it is not validated, or if its validation result is not **Pass**.

Restrictions

None.

VAL-check-validation-result-with-rules

Description

Leverages validation rule and validation object applications from the workflow process and checks target NX datasets validation result status. To add this handler to a workflow process template, the user must have a well-defined validation rule set file that best describes the user's business process in terms of what NX datasets should run what checks at what time and what conditions that the check must meet. The handler returns a **EPM_go** or **EPM_nogo** decision based on overall result status of the verification (**EPM_go** is returned only when all target NX datasets satisfy all rules defined in validation rule set file).

The handler logs validation rules and validation result checks. The format of the log file name is *<First-target-name_Time-stamp>*. The log file is stored in the directory specified by the **TC_TMP_DIR** environment variable. If **TC_TMP_DIR** is not defined, it is stored in the **%TEMP%** directory (Windows) or **/tmp** directory (Linux).

Note

The system will not process a log file name longer than 32 characters when the **TC_Allow_Longer_ID_Name** preference is set to **false**. In this situation, if the log file name is longer than 32 characters, the log file name is automatically truncated.

Syntax

VAL-check-validation-result-with-rules -rule_item_revision=<item-rev-id> [-current_event=<event-value>] [-pass_item_revision_only] [-ref_log]

Arguments

-rule_item_revision

The item revision ID that the validation rule set dataset is attached under.

-current_event

A value that is used to select validation rules from the rule file by comparing with the event values list of each rule. When **-current_event** is not provided, all rules from the rule file are selected at the first step. When a rule is defined without the event values list, the rule is also selected at the first step. The event values list can contain a wildcard (* only). The event values list also can be marked as exclusive (inclusive by default).

-pass_item_revision_only

When this argument is added to an input list, only item revision targets are passed to the handler. NX datasets are searched from each item revision and verified according to rules.

-ref_log

If this argument is present and the validation fails, the validation results log is created, a warning message is displayed, and the log is attached.

If this argument is not present and the validation fails, the validation results log is created, a warning message is displayed, but the log is *not* attached.

If the validation passes, the validation results log is not created and no message is displayed.

Placement

Do not place this handler on the root task. Place it on the **Start** action of a subsequent task after a target is attached.

Note

If the handler is placed on the root task, and the handler fails to complete, the workflow process itself is not created. No log file under the **Newstuff** folder is created.

Restrictions

-**rule_item_revision** cannot be NULL.

validateIPObjct

Description

This handler validates IP Object Revisions before transfer to an external system (Methodics). The handler checks that the IP BOM structure has precise assemblies (non-precise assemblies are not allowed), the Library Name property is not empty and does not contain whitespace, and the Object Name does not contain whitespace characters. The handler traverses the entire IP BOM hierarchy recursively, validating each child line. Objects that have already been transferred (non-empty IPVExternal ID) are skipped along with their subtrees.

Syntax

validateIPObjct

Arguments

None.

Placement

Place as a business rule on the Start action of the root process task in the Transfer IP to Methodics workflow template.

Restrictions

Target attachments must include IP Object Revision objects. The IP BOM structure must have precise assemblies (non-precise assemblies are not allowed). The Library Name property must not be empty and must not contain whitespace. The Object Name must not contain whitespace characters.

Examples

No arguments are required. The handler automatically validates the IP BOM structure and properties.